

关于本文档

文档名称：《Kubernetes数据存储和配置管理》

使用协议：《知识共享公共许可协议(CCPL)》

贡献者

贡献者名称	贡献度	文档变更记录	个人主页
马哥（马永亮）	主编		http://github.com/iKubernetes/
王晓春	作者	kubernetes-v1.32.3	http://www.wangxiaochun.com

文档协议

署名要求：使用本系列文档，您必须保留本页中的文档来源信息，具体请参考《知识共享 (Creative Commons) 署名4.0公共许可协议国际版》。

非商业化使用：遵循《知识共享公共许可协议(CCPL)》，并且您不能将本文档用于马哥教育相关业务之外的其他任何商业用途。

您的权利：遵循本协议后，在马哥教育相关业务之外的领域，您将有以下使用权限：

共享 — 允许以非商业性质复制本作品。

改编 — 在原基础上修改、转换或以本作品为基础进行重新编辑并用于个人非商业使用。

致谢

本文档中，部分素材参考了相关项目的文档，以及通过搜索引擎获得的内容，这里先一并向相关的贡献者表示感谢。

Kubernetes 数据存储和配置管理

本章内容

- 数据存储
- 配置管理

1 数据存储

1.1 存储机制

1.1.1 Docker 存储

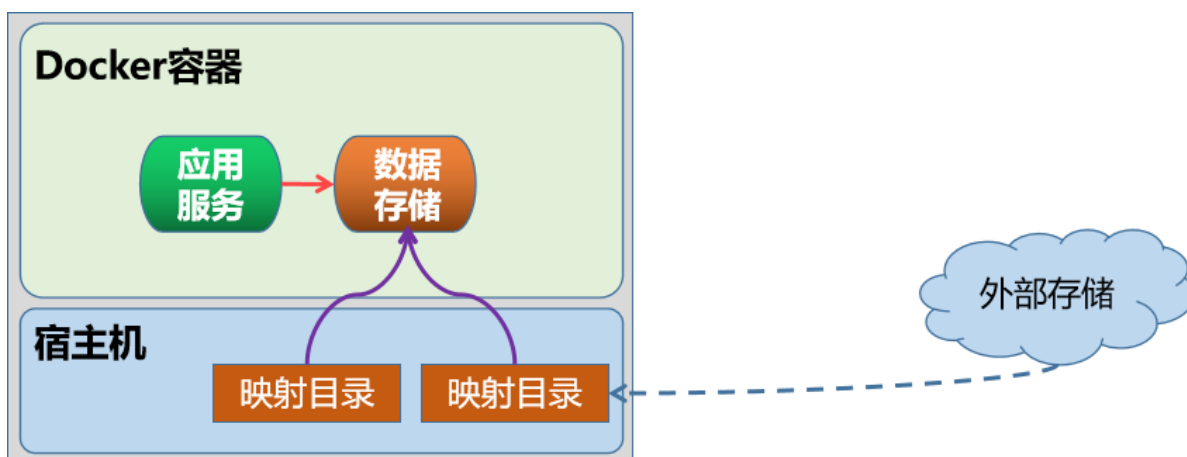
Docker的文件系统 与Docker容器具有相同的生命周期，但是Docker容器可能会遇到同时运行到多节点场景中，此时会因为节点崩溃、服务崩溃、网络原因，导致容器异常退出，所以一旦将数据存储到容器内部，会导致数据丢失。

Docker镜像是只读的文件，Docker容器可读可写，但是不能够数据持久化。

所以为了避免这种数据异常丢失的情况，一般会将容器的数据(程序状态数据、应用存储数据等)，存储在容器的文件系统之外。即通过引入外部的存储空间来实现 数据的可持久化效果。

Docker的两种文件存储机制：

- host机制
通过数据卷或者数据卷容器，将当前宿主机上面的文件系统目录与容器里面的工作目录进行一一的关联。
即使我们对本地是磁盘做 raid等冗余功能，但是这种等级的安全并不理想。
- 网络存储机制
通过网络的方式，将外部的存储空间挂载到当前宿主机，然后借助于host机制实现容器数据的可持久化。



1.1.2 Kubernetes 存储机制

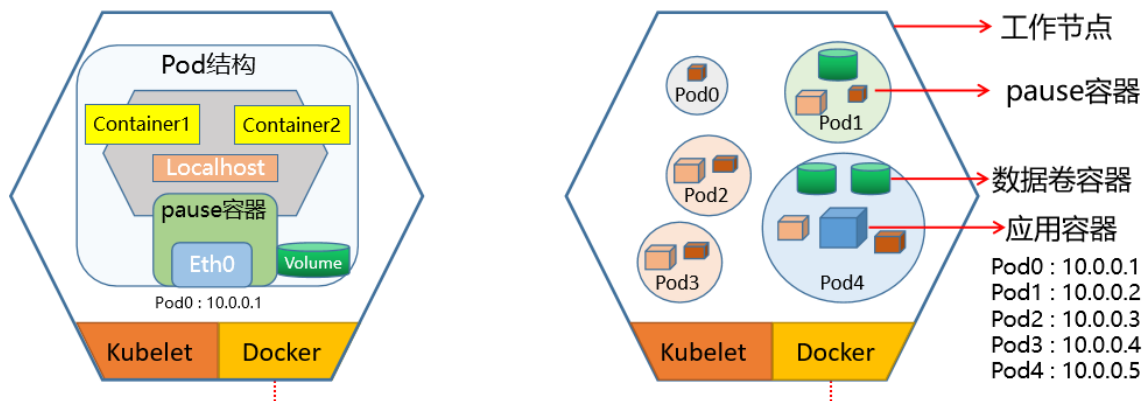
Container 中的文件在磁盘上是临时存放的，这给 Container 中运行的较重要的应用程序带来一些问题。

- 当容器崩溃时。 kubelet 可能会重新创建容器，可能会导致容器漂移至新的宿主机，容器会以干净的状态重建。导致数据丢失
- 在同一 Pod 中运行多个容器需要共享数据

Kubernetes 卷 (Volume) 这一抽象概念能够解决这两个问题。

<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/>

Kubernetes 集群中的容器数据存储



Kubernetes支持的存储类型

<https://kubernetes.io/zh-cn/docs/concepts/storage/>

Kubernetes支持丰富的存储类型，可以分为树内和树外两种

树内 In-Tree 存储卷插件

Kubernetes 内置的插件,可以直接支持,无需安装

类型	举例
临时存储卷	emptyDir
本地数据卷	hostPath、local
文件系统	NFS、CephFS、GlusterFS、fastdfs、Cinder、gitRepo(DEPRECATED)
块设备	iSCSI、FC、rdb(块设备)、vSphereVolume
存储平台	Quobyte、PortworxVolume、StorageOS、ScaleIO
云存储数据卷	Aliyun OSS、Amazon S3、AWS Elastic Block Store、Google gcePersistentDisk等
特殊存储卷	ConfigMap、Secret、DownwardAPI、Projectd、flocker

树外 Out-of-Tree 存储卷插件

经由容器存储接口CSI或FlexVolume接口扩展出的外部的存储系统称为Out-of-Tree类的存储插件

以前，所有卷插件都是“树内（In-Tree）”的。“树内”插件是与 Kubernetes 的核心组件一同构建、链接、编译和交付的。

这意味着向 Kubernetes 添加新的存储系统（卷插件）需要将代码合并到 Kubernetes 核心代码库中。

后来出现的CSI 和 FlexVolume 都允许独立于 Kubernetes 代码库开发卷插件，并作为扩展部署（安装）在 Kubernetes 集群上。

- Flexvolume

Kubernetes社区较早实现的存储卷扩展机制，当前已淘汰

Flexvolume运行在host空间，不能使用RBAC授权机制访问Kubernetes API,导致其功能极大的受限。

Flexvolume 以脚本实现，性能不佳

- CSI 插件

Container Storage Interface 是当前Kubernetes社区推荐的插件实现方案

CSI 不仅支持Kubernetes平台存储插件接口，而且也作为云原生生态中容器存储接口的标准,公用云对其有更好的支持

Kubernetes 支持 CSI 的接口方式实现更大范围的存储功能扩展,更为推荐使用

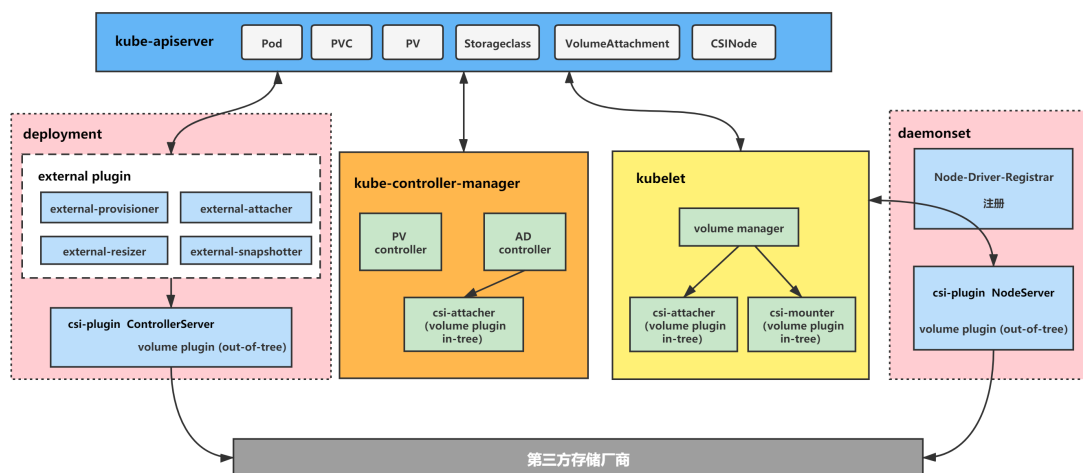
<https://github.com/container-storage-interface/spec/blob/master/spec.md>

CSI 有多种实现,比如:

Rancher 是为使用容器的公司打造的容器管理平台。Rancher 简化了使用 Kubernetes 的流程，开发者可以随处运行 Kubernetes，满足 IT 需求规范，赋能 DevOps 团队。这个团队研发的Longhorn就是要给非常好的存储平台。

Longhorn是一个轻量级且功能强大的云原生Kubernetes分布式存储平台，可以在任意基础设施上运行。Longhorn与Rancher结合使用，将帮助用户在Kubernetes环境中轻松、快速和可靠地部署高可用性持久化块存储。

CSI 主要包含两个部分：CSI Controller Server 与 CSI Node Server，分别对应Controller Server Pod和 Node Server Pod



- Controller Server

也称为CSI Controller

在集群中只需要部署一个 Controller Server，以 deployment 或者 StatefulSet 的形式运行
主要负责与存储服务API通信完成后端存储的管理操作，比如 provision 和 attach 工作。

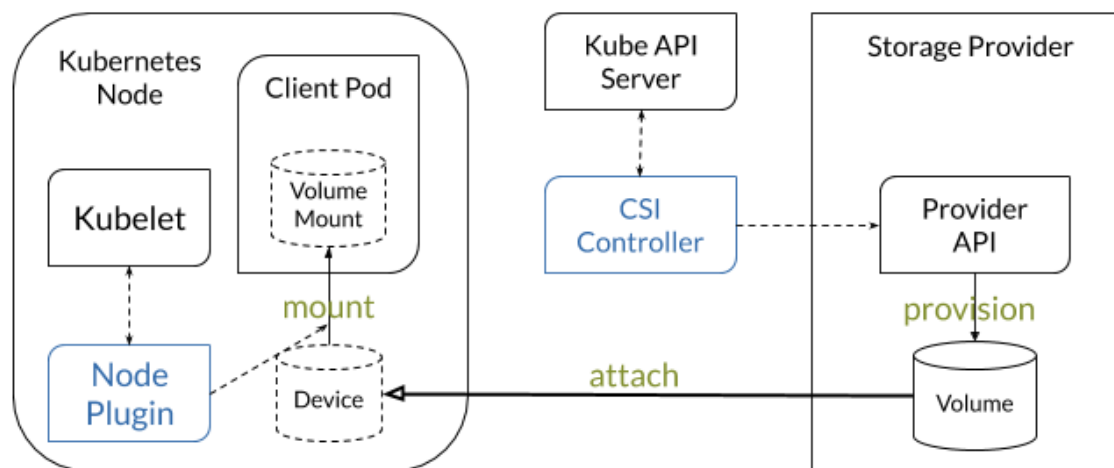
- Node Server

也称为CSI Node 或 Node Plugin

保证每一个节点会有一个 Pod 部署出来，负责在节点级别完成存储卷管理，和 CSI Controller 一起完成 volume 的 mount 操作。

Node Server Pod 是个 DaemonSet，它会在每个节点上进行注册。

Kubelet 会直接通过 Socket 的方式直接和 CSI Node Server 进行通信、调用 Attach/Detach/Mount/Unmount 等。



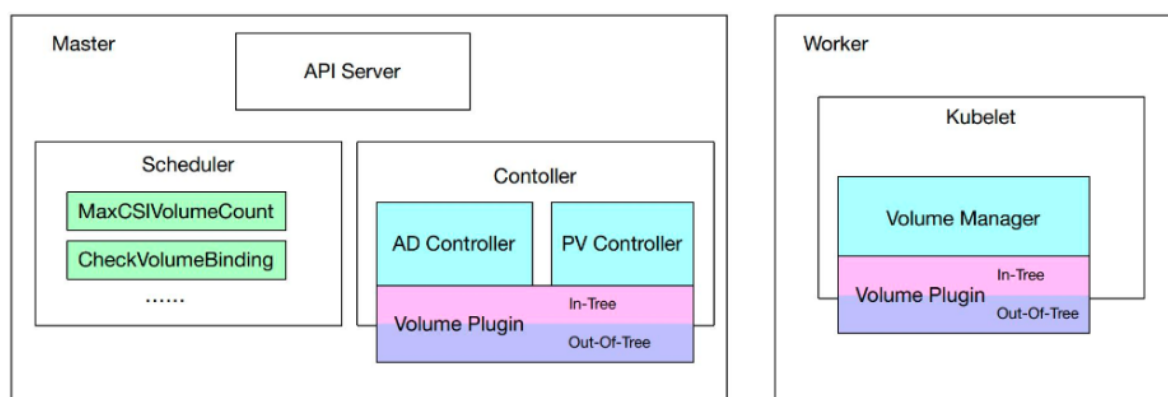
CSI 插件包括以下两部分:

- CSI-Plugin: 实现数据卷的挂载、卸载功能。
- CSI-Provisioner: 制备器 (Provisioner) 实现数据卷的自动创建管理能力, 即驱动程序, 比如: 支持云盘、NAS等存储卷创建能力

Kubernetes 存储架构

存储的组件主要有: attach/detach controller、pv controller、volume manager、volume plugins、scheduler

每个组件分工明确

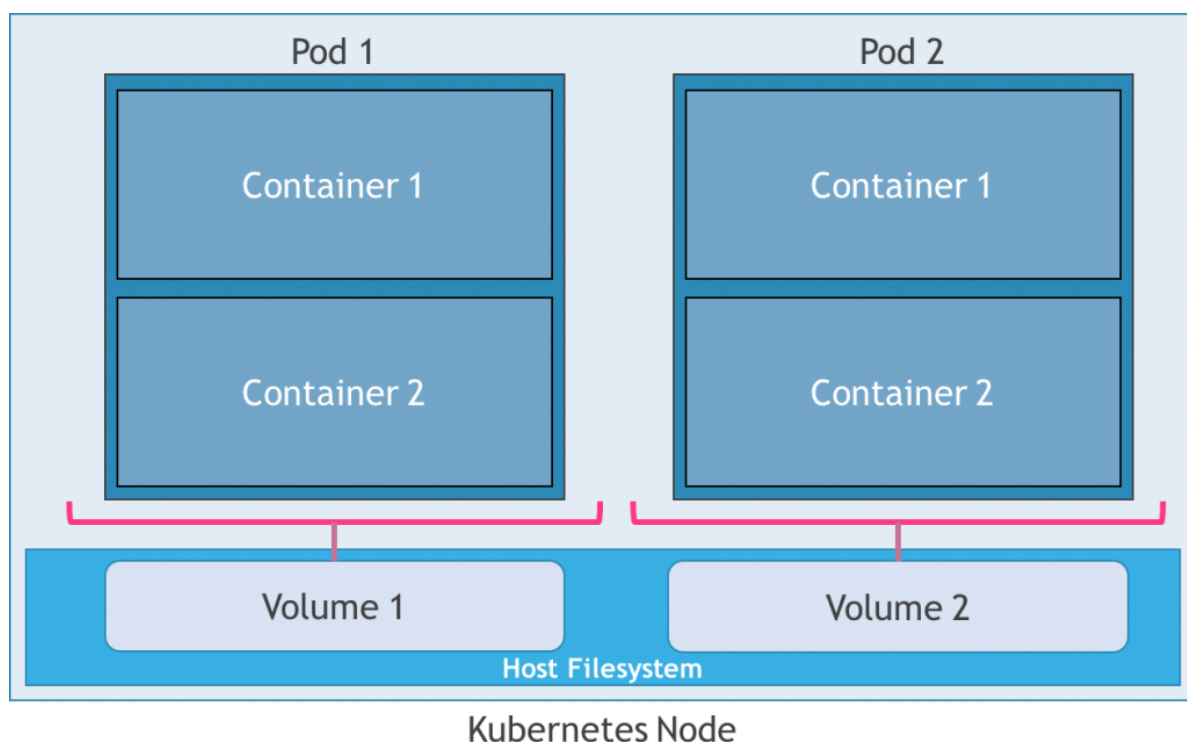


- AD控制器: 负责存储设备的Attach/Detach操作
 - Attach: 将设备附加到目标节点
 - Detach: 将设备从目标节点上卸载
- Volume Manager: 存储卷管理器, 负责完成卷的Mount/Umount操作, 以及设备的格式化操作等
- PV Controller: 负责PV/PVC的绑定、生命周期管理, 以及存储卷的Provision/Delete操作
- volume plugins: 包含k8s原生的和各厂商的存储插件, 扩展各种存储类型的卷管理能力
 - 原生的包括: emptydir、hostpath、csi等
 - 各厂商的包括: aws-ebs、azure等
- scheduler: 实现Pod的调度, 涉及到volume的调度。比如ebs、csi关于单node最大可attach磁盘数量的predicate策略, scheduler的调度至哪个指定目标节点也会受到存储插件的影响,

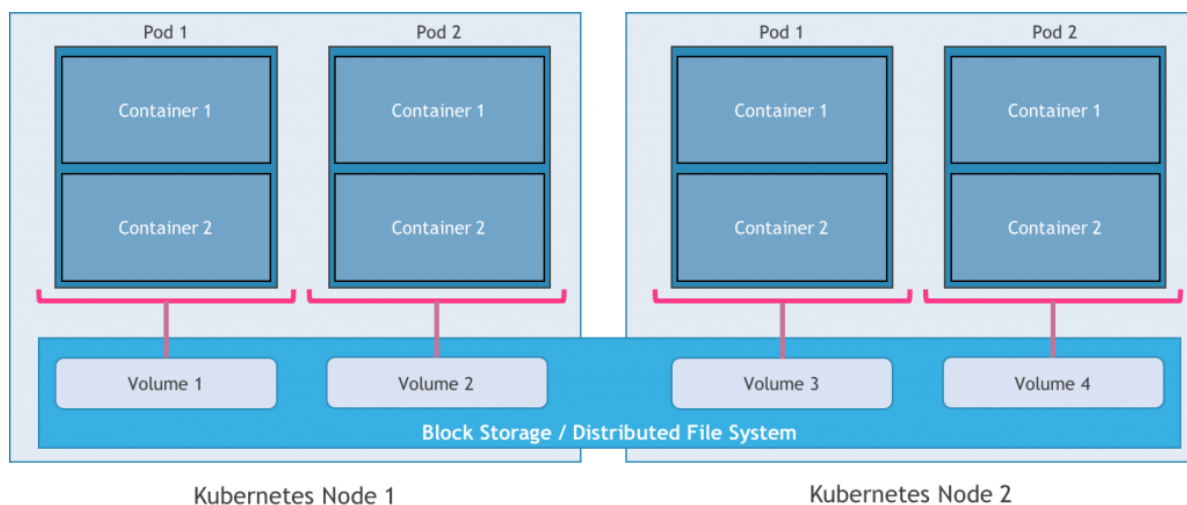
1.1.3 Pod的存储卷 volume

Kubernetes 支持在Pod上创建不同类型的任意数量的卷来实现不同数据的存储

单节点存储



多节点存储



存储卷本质上表现为 Pod中所有容器共享访问的目录

而此目录的创建方式、使用的存储介质以及目录的初始内容是由Pod规范中声明的存储卷类型来源决定

kubelet内置支持多种存储卷插件，存储卷是由各种存储插件(存储驱动)来提供存储服务

存储卷插件(存储驱动)决定了支持的后端存储介质或存储服务，例如hostPath插件使用宿主机文件系统，而nfs插件则对接指定的NFS存储服务

Pod在规范中需要指定其包含的卷以及这些卷在容器中的挂载路径

存储卷对象并非都是 Kubernetes系统一等公民，即有些卷不是独立的资源类型api-resources

存储卷需要定义在指定的Pod之上

有些卷本身的生命周期与Pod相同，但其后端的存储及相关数据的生命周期通常要取决于存储介质

存储卷可以分为：临时卷和持久卷

- [临时卷](#)类型的生命周期与 Pod 相同，当 Pod 不再存在时，Kubernetes 也会销毁临时卷
- [持久卷](#)可以比 Pod 的存活期长。当 Pod 不再存在时，Kubernetes 不会销毁持久卷。
- 但对于给定 Pod 中任何类型的卷，在容器重启期间数据都不会丢失。

范例: 查看Pod 支持的卷

```
[root@master1 ~]#kubectl explain deployment.spec.template.spec.volumes
[root@master1 ~]#kubectl explain pod.spec.volumes
KIND:      Pod
VERSION:   v1

RESOURCE: volumes <[]Object>

DESCRIPTION:
  List of volumes that can be mounted by containers belonging to the pod.
  More info: https://kubernetes.io/docs/concepts/storage/volumes

  Volume represents a named volume in a pod that may be accessed by any
  container in the pod.

FIELDS:
  awsElasticBlockStore <Object>
    awsElasticBlockStore represents an AWS Disk resource that is attached to a
    kubelet's host machine and then exposed to the pod. More info:
    https://kubernetes.io/docs/concepts/storage/volumes#awselasticsearchblockstore

  azureDisk <Object>
    azureDisk represents an Azure Data Disk mount on the host and bind mount to
    the pod.

  azureFile <Object>
    azureFile represents an Azure File Service mount on the host and bind mount
    to the pod.

  cephfs <Object>
    cephFS represents a Ceph FS mount on the host that shares a pod's lifetime

  cinder <Object>
    cinder represents a cinder volume attached and mounted on kubelets host
    machine. More info: https://examples.kubernetes.io/mysql-cinder-
    pd/README.md

  configMap <Object>
    configMap represents a configMap that should populate this volume

  csi <Object>
    csi (Container Storage Interface) represents ephemeral storage that is
    handled by certain external CSI drivers (Beta feature).

  downwardAPI <Object>
    downwardAPI represents downward API about the pod that should populate this
    volume

  emptyDir <Object>
    emptyDir represents a temporary directory that shares a pod's lifetime.
    More info: https://kubernetes.io/docs/concepts/storage/volumes#emptydir
```

ephemeral <Object>

ephemeral represents a volume that is handled by a cluster storage driver. The volume's lifecycle is tied to the pod that defines it - it will be created before the pod starts, and deleted when the pod is removed.

Use this **if**: a) the volume is only needed **while** the pod runs, b) features of normal volumes like restoring from snapshot or capacity tracking are needed, c) the storage driver is specified through a storage class, and d) the storage driver supports dynamic volume provisioning through a PersistentVolumeClaim (see EphemeralVolumeSource **for** more information on the connection between this volume type and PersistentVolumeClaim).

Use PersistentVolumeClaim or one of the vendor-specific APIs **for** volumes that persist **for** longer than the lifecycle of an individual pod.

Use CSI **for** light-weight local ephemeral volumes **if** the CSI driver is meant to be used that way - see the documentation of the driver **for** more information.

A pod can use both types of ephemeral volumes and persistent volumes at the same time.

fc <Object>

fc represents a Fibre Channel resource that is attached to a kubelet's host machine and **then** exposed to the pod.

flexvolume <Object>

flexvolume represents a generic volume resource that is provisioned/attached using an exec based plugin.

flocker <Object>

flocker represents a Flocker volume attached to a kubelet's host machine. This depends on the Flocker control **service** being running

gcePersistentDisk <Object>

gcePersistentDisk represents a GCE Disk resource that is attached to a kubelet's host machine and then exposed to the pod. More info:
<https://kubernetes.io/docs/concepts/storage/volumes#gcepersistentdisk>

gitRepo <Object>

gitRepo represents a **git** repository at a particular revision. DEPRECATED: GitRepo is deprecated. To provision a container with a **git** repo, mount an EmptyDir into an InitContainer that clones the repo using **git**, **then** mount the EmptyDir into the Pod's container.

glusterfs <Object>

glusterfs represents a Glusterfs mount on the host that shares a pod's lifetime. More info:

<https://examples.kubernetes.io/volumes/glusterfs/README.md>

hostPath <Object>

hostPath represents a pre-existing file or directory on the host machine that is directly exposed to the container. This is generally used **for** system agents or other privileged things that are allowed to see the host machine. Most containers will NOT need this. More info:
<https://kubernetes.io/docs/concepts/storage/volumes#hostpath>

iscsi <Object>
iscsi represents an iSCSI Disk resource that is attached to a kubelet's host machine and then exposed to the pod. More info: <https://examples.kubernetes.io/volumes/iscsi/README.md>

name <string> **-required-**
name of the volume. Must be a DNS_LABEL and unique within the pod. More info: <https://kubernetes.io/docs/concepts/overview/working-with-objects/names/#names>

nfs <Object>
nfs represents an NFS mount on the host that shares a pod's lifetime More info: <https://kubernetes.io/docs/concepts/storage/volumes#nfs>

persistentVolumeClaim <Object>
persistentVolumeClaimVolumeSource represents a reference to a PersistentVolumeClaim in the same namespace. More info: <https://kubernetes.io/docs/concepts/storage/persistent-volumes#persistentvolumeclaims>

photonPersistentDisk <Object>
photonPersistentDisk represents a PhotonController persistent disk attached and mounted on kubelets host machine

portworxVolume <Object>
portworxVolume represents a portworx volume attached and mounted on kubelets host machine

projected <Object>
projected items for all in one resources secrets, configmaps, and downward API

quobyte <Object>
quobyte represents a Quobyte mount on the host that shares a pod's lifetime

rbd <Object>
rbd represents a Rados Block Device mount on the host that shares a pod's lifetime. More info: <https://examples.kubernetes.io/volumes/rbd/README.md>

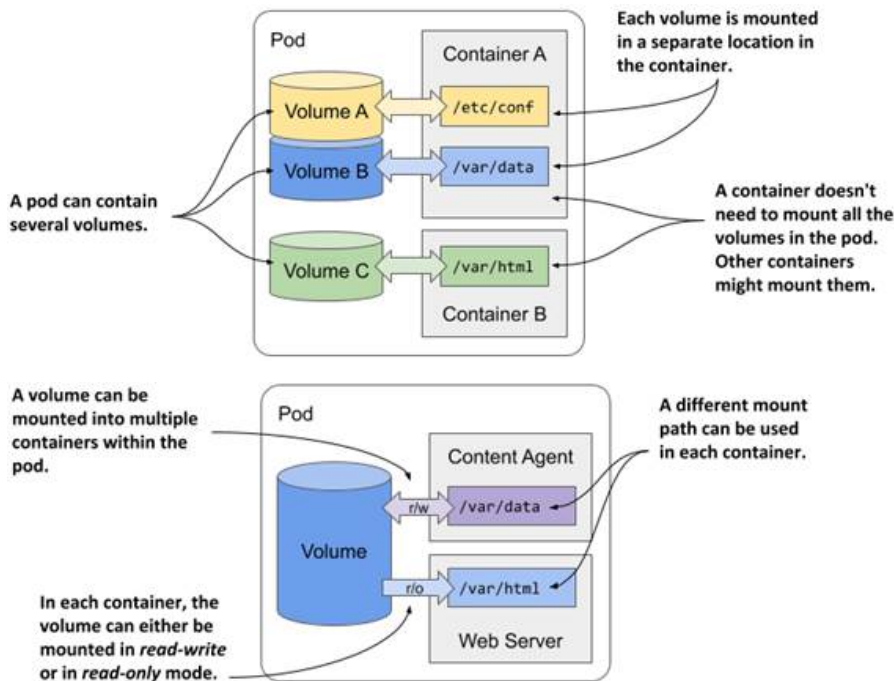
scaleIO <Object>
scaleIO represents a ScaleIO persistent volume attached and mounted on Kubernetes nodes.

secret <Object>
secret represents a secret that should populate this volume. More info: <https://kubernetes.io/docs/concepts/storage/volumes#secret>

storageos <Object>
storageOS represents a StorageOS volume attached and mounted on Kubernetes nodes.

vsphereVolume <Object>
vsphereVolume represents a vsphere volume attached and mounted on kubelets host machine

- 一个Pod可以添加任意个卷
- 同一个Pod内每个容器可以在不同位置按需挂载Pod上的任意个卷，或者不挂载任何卷
- 同一个Pod上的某个卷，也可以同时被该Pod内的多个容器同时挂载，以共享数据
- 如果支持，多个Pod也可以通过卷接口访问同一个后端存储单元



存储卷的配置由两部分组成

- 通过pod.spec.volumes字段定义在Pod之上的存储卷列表，它经由特定的存储卷插件并结合特定的存储供给方的访问接口进行定义
- 嵌套定义在容器的pod.spec.containers.volumeMounts字段上的存储卷挂载列表，它只能挂载当前Pod对象中定义的存储卷

Pod 内部容器使用存储卷有两步：

- 在Pod上定义存储卷，并关联至目标存储服务上
volumes
- 在需要用到存储卷的容器上，挂载其所属的Pod中pause的存储卷
volumeMount

容器引擎对共享式存储设备的支持类型：

- 单路读写 - 多个容器内可以通过同一个中间容器对同一个存储设备进行读写操作，比如：本地磁盘文件
- 多路并行读写 - 多个容器内可以同时为同一个存储设备进行读写操作，比如：NFS
- 多路只读 - 多个容器内可以同时为同一个存储设备进行只读操作，比如：CDROM

Pod的卷资源对象属性

```
apiVersion: v1
kind: Pod
metadata:
  name: <string>
  namespace: <string>
spec:
  volumes:
    - name <string>
```

定义卷
存储卷名称标识，仅可使用DNS标签格式的字符，在当前Pod中必须唯一

```

VOL_TYPE <Object>                                # 存储卷插件及具体的目标存储供给方的相关配置
containers:
- name: ...
  image: ...
  volumeMounts:
    # 挂载卷
    - name <string>                                # 要挂载的存储卷的名称，必须匹配存储卷列表中某项的定义
      mountPath <string>                           # 容器文件系统上的挂载点路径
      readOnly <boolean>                            # 是否挂载为只读模式，默认为“否”，即可读可写
      subPath <string>                              # 挂载存储卷上的一个子目录至指定的挂载点
      subPathExpr <string>                          # 挂载由指定的模式匹配到的存储卷的文件或目录至挂载点

```

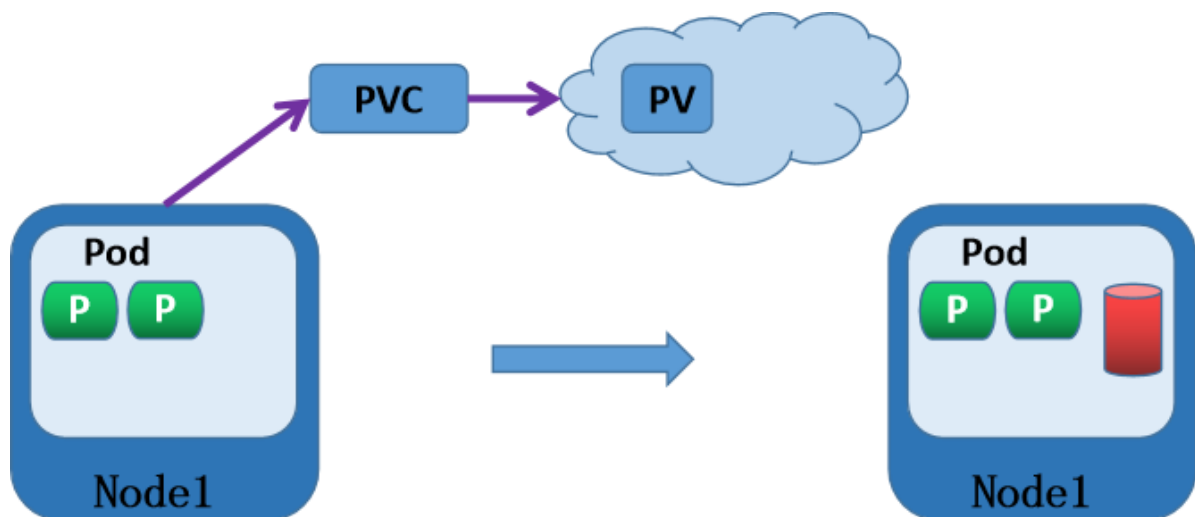
虽然这种借助于Kubernetes存储插件的方式，为pod应用引入各种存储机制直接挂载使用，但是这种方案有天然的劣势：

- 要保证所有的pod节点支持后端的指定类型的存储服务。
- 所有的后端指定服务的配置都要在pod上自定义，不方便其它Pod复用
- 这些卷插件将pod定制的配置属性传递给kubelet，然后实现对应的效果。

以上说明，意味着一个开发者要开发一个pod对象，必须首先是一个存储专家，这使得Kubernetes的使用门槛过高。

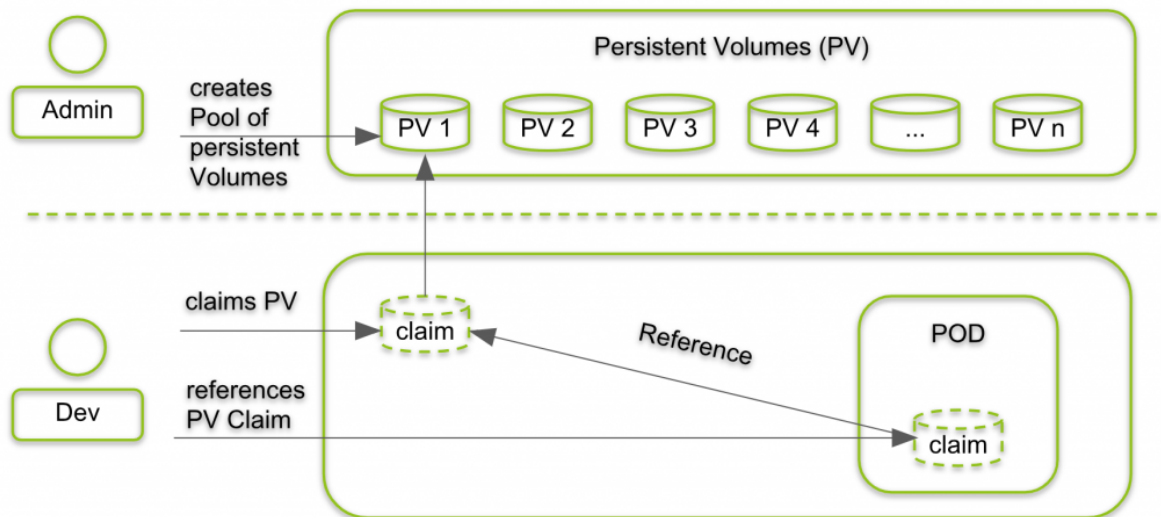
Kubernetes为了解决上述问题，引入了两个非常重要的存储资源类型: PV 和 PVC

1.1.4 PV和PVC



- 由专业的存储管理员来管理所有的存储后端：
 - 在专用的存储设备上，创建各种类型级别的PV(Persistent Volume)
 - 或者通过存储模板文件SC(storageclasses)来自动创建大量不同类型的PV对象。
- 由开发人员定制需要的PVC(Persistent Volume Claim)，然后关联到pod上
- Pod通过PVC到PV上请求一块独立大小的网络存储空间，然后直接使用

通过这种分层的管理机制，让开发人员和存储人员专人做专事，大大的减轻工作的负载。

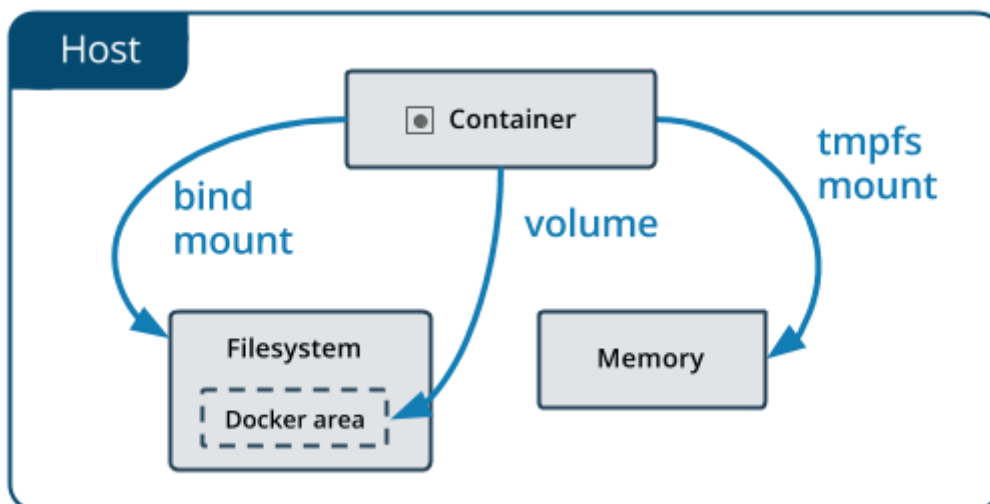


1.2 emptyDir

<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#emptydir>

一个emptyDir volume在pod被调度到某个Node时候自动创建的，无需指定宿主机上对应的目录。

适用于在一个Pod中不同容器间的临时数据的共享



emptyDir 数据默认存放在宿主机的路径如下

```
/var/lib/kubelet/pods/<pod_id>/volumes/kubernetes.io~empty-dir/<volume_name>/<FILE>
```

#注意：此目录随着Pod删除，也会随之删除，不能实现持久化

emptyDir 特点如下：

- 此为默认存储类型
- 此方式只能临时存放数据，不能实现数据持久化
- 跟随Pod初始化而来，开始是空数据卷
- Pod 被删除，emptyDir对应的宿主机目录也被删除，当然目录内的数据随之永久消除
- emptyDir 数据卷介质种类跟当前主机的磁盘一样。
- emptyDir 主机可以为同一个Pod内多个容器共享
- emptyDir 容器数据的临时存储目录主要用于数据缓存和同一个Pod内的多个容器共享使用

emptyDir 属性解析

```
#定义卷
kubectl explain pod.spec.volumes.emptyDir
    medium          #指定媒介类型，主要有 default(默认为宿主机的本地磁盘，The default is ""
which means to use the node's default medium)和Memory两种,默认情况下，emptyDir卷支持
节点上的任何介质，SSD、磁盘或网络存储，具体取决于自身的环境。也可以将emptyDir.medium,字段设置
为Memory，让 Kubernetes使用tmpfs(内存支持的文件系统)，虽然tmpfs非常快，但是tmpfs在节点重启
时，数据同样会被清除，并且设置的大小会被计入到Container的内存限制当中。
    sizeLimit      #当前存储卷的空间限额，默认值为nil表示不限制

#使用卷
kubectl explain pod.spec.containers.volumeMounts
    mountPath      #挂载到容器中的路径,此目录会自动生成
    name           #指定挂载的volumes名称
    readOnly       #是否只读挂载
    subPath        #指定挂载卷的子目录路径,默认挂载卷的根目录
```

配置示例

```
# volume配置格式
volumes:
- name: volume_name
  emptyDir: {}

# volume使用格式
containers:
- volumeMounts:
  - name: volume_name
    mountPath: /path/to/container/ #容器内路径

#注意：关于volume和容器的关系，每个容器都可以单独挂载多个volume，每种volume都可以被多个容器挂
载

#示例1
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir: {}

#示例2
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
```

```

- image: registry.k8s.io/test-webserver
  name: test-container
  volumeMounts:
    - mountPath: /cache
      name: cache-volume
volumes:
- name: cache-volume
  emptyDir:
    medium: Memory
    sizeLimit: 500Mi

```

范例: emptydir 使用

```

#清单文件
[root@master1 ~]#cat storage-emptydir-1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-emptydir
  labels:
    name: storage-emptydir
spec:
  volumes:
    - name: empty-volume
      emptyDir: {} #默认磁盘存储
      #emptyDir:
      #  medium: Memory
      #  sizeLimit: 500Mi

  containers:
    - name: storage-emptydir-container
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      volumeMounts:
        - name: empty-volume
          mountPath: /nginx/www/empty/

#创建资源
[root@master1 ~]#kubectl apply -f storage-emptydir-1.yaml

#查看资源性信息
[root@master1 ~]# kubectl describe pod
Name:          storage-emptydir
...
Mounts:
  /nginx/www/empty/ from empty-volume (rw)
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-sbgjs
(ro)
Conditions:
  Type             Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  empty-volume:
    Type:          EmptyDir (a temporary directory that shares a pod's lifetime)

```

```
Medium:
SizeLimit: <unset>
```

.....

#确认目录自动生成

```
[root@master1 ~]#kubectl exec volume-nginx -- ls -ld /nginx/www/empty
drwxrwxrwx 2 root root 4096 Mar 22 06:33 /nginx/www/empty
```

#写文件

```
[root@master1 ~]#kubectl exec -it storage-emptydir -- sh
# echo empty-volume > /nginx/www/empty/test.txt
# cat /nginx/www/empty/test.txt
empty-volume
```

#确定pod所有在主机

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS	GATES		
volume-nginx	1/1	Running	0	3m14s	172.16.3.56	node1.wang.org
	<none>	<none>				

#在pod所在主机上查找文件

```
[root@node1 ~]#find / -name test.txt
/var/lib/kubelet/pods/1888534d-074b-4644-9ad7-96eba12883da/volumes/kubernetes.io~empty-dir/empty-volume/test.txt
```

```
[root@node1 ~]#find / -name test.txt | xargs cat
empty-volume
```

```
[root@node1 ~]#cd /var/lib/kubelet/pods/dfd8ef27-e3d9-486b-ad11-03f45a328f75/
[root@node1 dfd8ef27-e3d9-486b-ad11-03f45a328f75]#tree
```

```
.
├── containers
│   ├── volume-nginx-container
│   │   └── 063a281e
│   └── etc-hosts
├── plugins
│   ├── kubernetes.io~empty-dir
│   │   ├── empty-volume
│   │   │   └── ready
│   │   └── wrapped_kube-api-access-rbv6t
│   │       └── ready
│   └── volumes
│       ├── kubernetes.io~empty-dir
│       │   └── empty-volume
│       └── kubernetes.io~projected
│           ├── kube-api-access-rbv6t
│           │   ├── ca.crt -> ..data/ca.crt
│           │   ├── namespace -> ..data/namespace
│           └── token -> ..data/token
```

11 directories, 7 files

#删除pod

```
[root@master1 ~]#kubectl delete -f storage-emptydir-1.yaml
pod "storage-emptydir" deleted
```

#pod对应的目录也被删除

```
[root@node1 ~]#ls /var/lib/kubelet/pods
413120f6-6f80-47dd-969f-0c6185b65423  5412e06e-78bd-457b-8da3-600f7371da9b
ff394444-be24-4ff8-b99e-10ba0deaafdd2
```

范例: 在一个Pod中定义多个容器通过 emptyDir 共享数据

```
[root@master1 ~]#cat storage-emptydir-2.yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-emptydir
spec:
  volumes:
  - name: nginx-data
    emptyDir: {}
  containers:
  - name: storage-emptydir-nginx
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    volumeMounts:
    - name: nginx-data
      mountPath: /usr/share/nginx/html/
  - name: storage-emptydir-busybox
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    volumeMounts:
    - name: nginx-data
      mountPath: /data/
  command:
  - "/bin/sh"
  - "-c"
  - "while true; do date > /data/index.html; sleep 1; done"
```

#初始化对象

```
[root@master1 ~]#kubectl apply -f storage-emptydir-2.yaml
```

#查看pod对象

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS GATES			
storage-emptydir	2/2	Running	0	22s	10.244.3.6	
node1.wang.org	<none>		<none>			

#查看两个容器内的数据

```
[root@master1 ~]#kubectl exec -it storage-emptydir -c storage-emptydir-busybox -
- cat /data/index.html
```

```
Mon Sep 27 03:16:05 UTC 2021
```

```
[root@master1 ~]#kubectl exec -it storage-emptydir -c storage-emptydir-nginx --
cat /usr/share/nginx/html/index.html
```

```
Mon Sep 27 03:16:05 UTC 2021
```

#测试pod的web效果

```
[root@master1 ~]#curl 10.244.3.6
```

```
Mon Sep 27 03:17:03 UTC 2021
```

```
[root@master1 ~]#curl 10.244.3.6
```

```
Mon Sep 27 03:17:04 UTC 2021
```

```
[root@master1 ~]#curl 10.244.3.6
```

```
Mon Sep 27 03:17:05 UTC 2021
```

#结果显示：两个容器间实现了数据卷的共享操作，在**busybox**中定制信息，然后在**nginx**容器中获取信息。

1.3 hostPath

参考资料：

<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#hostpath>

hostPath 可以将宿主机上的目录挂载到 Pod 中作为数据的存储目录

hostPath 一般用在如下场景：

- 容器应用程序中某些文件需要永久保存
- Pod删除，hostPath数据对应应在宿主机文件不受影响,即hostPath的生命周期和Pod不同,而和节点相同
- 宿主机和容器的目录都会自动创建
- 某些容器应用需要用到容器的自身的内部数据，可将宿主机的/var/lib/[docker|containerd]挂载到Pod中

hostPath 使用注意事项：

- 不支持数据的漫游，hostpath只有在其所在宿主机可用，其它节点不可用
- 不同宿主机的目录和文件内容不一定完全相同，所以Pod迁移前后的访问效果不一样
- 不适合Deployment这种分布式的资源，更适合于DaemonSet
- 宿主机的目录不属于独立的资源对象的资源，所以对资源设置的资源配额限制对hostPath目录无效

配置属性

#配置属性

kubectl explain pod.spec.volumes.hostPath

path #指定哪个宿主机的目录或文件将被共享给Pod使用

type #指定路径的类型，一共有7种，默认的类型是没有指定

空字符串 #默认配置，在关联**hostPath**存储卷之前不进行任何检查,如果宿主机没有对应的目录，会自动创建

DirectoryOrCreate #宿主机上不存在**path**，就自动创建**0755**权限的空目录

Directory #**path**必须存在,挂载已存在的目录

FileOrCreate #宿主机上不存在挂载文件，就创建**0644**权限的空文件

File #必须存在的文件

Socket #事先必须存在的**Socket**文件路径

CharDevice #事先必须存在的字符设备文件路径

BlockDevice #事先必须存在的块设备文件路径

#配置格式：

volumes:

- name: volume_name

hostPath:

path: /path/to/host

#示例

apiVersion: v1

kind: Pod

metadata:

name: test-pod

spec:

containers:

- image: registry.k8s.io/test-webserver

```

name: test-container
volumeMounts:
- mountPath: /test-pod
  name: test-volume
volumes:
- name: test-volume
  hostPath:
    # 宿主上目录位置
    path: /data
    # 此字段为可选
    type: Directory

```

范例: 使用主机的时区配置

```

[root@master1 ~]#cat storage-hostpath-timezone.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-hostpath-timezone
spec:
  volumes:
  - name: timezone
    hostPath:
      path: /etc/timezone      #此文件是影响时区
      type: File
  - name: localtime          #此文件可选，此处挂载失败，不影响时区
    hostPath:
      path: /etc/localtime
      type: File
  containers:
  - name: c01
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    #command: ["sh","-c","sleep 3600"]
  - name: c02
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    command: ["sh","-c","sleep 3600"]
    volumeMounts:
    - name: timezone
      mountPath: /etc/timezone #容器此为文件是普通文件，挂载节点目录成功
    - name: localtime
      mountPath: /etc/localtime #容器此为文件是软链接，此处挂载失败

[root@master1 ~]#kubectl apply -f storage-hostpath-timezone.yaml

[root@master1 ~]#kubectl get pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE
pod-hostpath-test 2/2     Running   0          3s    10.244.2.42   node2.wang.org
<none>         <none>

```

#查看宿主机的配置

```

[root@node2 ~]#cat /etc/timezone
Asia/Shanghai

```

#查看结果

```

[root@master1 ~]#kubectl exec pod-hostpath-timezone -c c01 -- date

```

```

Wed May 8 03:22:53 UTC 2024
[root@master1 ~]#kubectl exec pod-hostpath-timezone -c c01 -- cat /etc/timezone
Etc/UTC

[root@master1 ~]#kubectl exec pod-hostpath-timezone -c c02 -- date
Wed May 8 11:23:55 CST 2024

[root@master1 ~]#kubectl exec pod-hostpath-timezone -c c02 -- cat /etc/timezone
Asia/Shanghai

```

范例: 实现 Redis 数据的持久化

```

[root@master1 ~]#cat storage-hostpath-redis.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-redis
spec:
  nodeName: node1.wang.org      #指定运行在指定worker主机上
  volumes:
  - name: redis-backup
    hostPath:
      path: /backup/redis      #宿主机的目录会自动创建
  containers:
  - name: hostpath-redis
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/redis:6.2.5
    volumeMounts:
    - name: redis-backup
      mountPath: /data

```

#关键点: spec.containers.volumeMounts的name属性和spec.volumes的那么属性完全一致,因为他们是基于name来关联的。

#注意: redis的镜像将数据保存到了容器的/data 目录下。

#创建资源对象

```

[root@master1 ~]#kubectl apply -f storage-hostpath-redis.yaml

```

#查看Pod 状态

```

[root@master1 ~]#kubectl get pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
hostpath-redis	1/1	Running	0	4s	172.16.3.59	node1.wang.org

<none> <none>

#查看pod信息

```

[root@master1 ~]#kubectl describe pod hostpath-redis

```

```

Mounts:
  /data from redis-backup (rw)
.....
volumes:
  redis-backup:
    Type:          HostPath (bare host directory volume)
    Path:          /backup/redis
    HostPathType:

```

#生成数据

```

[root@master1 ~]#kubectl exec -it hostpath-redis -- bash
root@hostpath-redis:/data# pwd

```

```

/data
root@hostpath-redis:/data# redis-cli
127.0.0.1:6379> set name wangxiaochun
OK
127.0.0.1:6379> bgsave
Background saving started

#在对应的worker节点主机查看数据目录自动生成
[root@node1 ~]#tree /backup/
/backup/
├── redis
│   └── dump.rdb

1 directory, 1 file
[root@node1 ~]#ll /backup/redis/dump.rdb
-rw-r--r-- 1 systemd-coredump systemd-coredump 116 Mar 22 15:58
/backup/redis/dump.rdb

#删除pod, 重新创建
[root@master1 ~]#kubectl delete -f storage-hostpath-redis.yaml

#重新创建Pod, 验证数据是否还在
[root@master1 ~]#kubectl apply -f storage-hostpath-redis.yaml

#确认数据还存在
[root@master1 ~]#kubectl exec -it hostpath-redis -- redis-cli keys '*'
1) "name"
[root@master1 ~]#kubectl exec -it hostpath-redis -- redis-cli get name
"wangxiaochun"

```

范例: 实现 NFS 服务

<https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/deploy/example/nfs-provisioner/README.md>

```

[root@master1 ~]#cat storage-nfs-server.yaml
---
kind: Service
apiVersion: v1
metadata:
  name: nfs-server
  labels:
    app: nfs-server
spec:
  type: ClusterIP # use "LoadBalancer" to get a public ip
  selector:
    app: nfs-server
  ports:
    - name: tcp-2049
      port: 2049
      protocol: TCP
    - name: udp-111
      port: 111
      protocol: UDP
---
kind: Deployment

```

```

apiVersion: apps/v1
metadata:
  name: nfs-server
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nfs-server
  template:
    metadata:
      name: nfs-server
      labels:
        app: nfs-server
    spec:
      nodeSelector:
        "kubernetes.io/os": linux
        "server": nfs
      containers:
        - name: nfs-server
          #image: itsthenetwork/nfs-server-alpine:12
          image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nfs-server-
alpine:12
          env:
            - name: SHARED_DIRECTORY
              value: "/exports"
          volumeMounts:
            - mountPath: /exports
              name: nfs-vol
          securityContext:
            privileged: true
          ports:
            - name: tcp-2049
              containerPort: 2049
              protocol: TCP
            - name: udp-111
              containerPort: 111
              protocol: UDP
          volumes:
            - name: nfs-vol
              hostPath:
                path: /nfs-vol # modify this to specify another path to store nfs
share data
                type: DirectoryOrCreate

```

```
[root@master1 ~]#kubectl create ns nfs
```

#给NFS运行的节点指定标签

```
[root@master1 ~]#kubectl label nodes node1.wang.org server=nfs
node/node1.wang.org labeled
```

```
[root@master1 ~]#kubectl apply -f storage-nfs-server.yaml -n nfs
service/nfs-server created
deployment.apps/nfs-server created
```

```
[root@master1 ~]#kubectl get -n nfs all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nfs-server-857f859f57-hgf29	1/1	Running	0	12s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
service/nfs-server	ClusterIP	10.99.201.239	<none>	2049/TCP,111/UDP
12s				

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nfs-server	1/1	1	1	12s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nfs-server-857f859f57	1	1	1	12s

#查看NFS服务的Pod所有的主机

```
[root@master1 ~]#kubectl get pod -n nfs -o wide
nfs-server-857f859f57-hgf29 1/1 Running 4 (122m ago) 22d
10.244.1.10 node1.wang.org <none> <none>
```

#在NFS主机节点上自动生成在NFS共享目录

```
[root@node1 ~]#ll -d /nfs-vol/
drwxr-xr-x 2 root root 4096 5月 9 10:35 /nfs-vol//
```

#查看NFS共享配置

```
[root@master1 ~]#kubectl exec -n nfs nfs-server-857f859f57-hgf29 -- exportfs -v
/exportfs <world>
(async,wdelay,hide,no_subtree_check,insecure_locks,fsid=0,sec=sys,rw,insecure,no_root_squash,no_all_squash)
```

#NFS客户端安装包并挂载，注意：NFS-v4.0不支持showmount查看

```
[root@node3 ~]#apt update && apt -y install nfs-common
[root@node3 ~]#showmount -e 10.99.201.239
```

#但是支持挂载NFS服务的伪根

```
[root@node3 ~]#mount 10.99.201.239:/ /mnt
```

#查看挂载属性

```
[root@node3 ~]#grep nfs /proc/mounts |
10.99.201.239:/ /mnt nfs4
rw,relatime,vers=4.2,rsize=262144,wsiz=262144,namlen=255,hard,proto=tcp,timeo=600,retrans=2,sec=sys,clientaddr=10.0.0.203,local_lock=none,addr=10.99.201.239 0
0
```

#创建文件

```
[root@node3 ~]#ls /mnt
[root@node3 ~]#touch /mnt/wangxiaochun.txt
```

#在NFS服务所在节点查看文件生成

```
[root@node1 ~]#ls /nfs-vol
wangxiaochun.txt
```

1.4 网络共享存储

虽然hostpath可以根据pod的配置自动创建关联的宿主机目录，但是因为pod通常会运行在不同的宿主机上，因此每个宿主机目录下面可能没有想需要的相同的数据文件

在集群场景中，想要实现比基于hostpath方式实现更强大所有的数据文件进行同步功能。

基于此，需要一种可以实现网络共享的存储方式。

Kubernetes 支持多种网络共享的存储方法

1.4.1 实现NFS的网络共享存储

1.4.1.1 NFS的网络共享存储配置说明

```
https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#nfs
https://github.com/kubernetes/examples/tree/master/staging/volumes/nfs
```

和传统的方式一样, 通过 NFS 网络文件系统可以实现Kubernetes数据的网络存储共享

使用NFS提供的共享目录存储数据时, 需要在系统中部署一个NFS环境, 通过volume的配置, 实现pod内的容器间共享NFS目录。

属性解析

```
#配置属性:
kubectl explain pod.spec.volumes.nfs
server      #指定nfs服务器的地址
path        #指定nfs服务器暴露的共享地址
readOnly    #是否只能读, 默认false

#配置格式:
volumes:
- name: <卷名称>
  nfs:
    server: nfs_server_address #指定NFS服务器地址, 如果是域名, 需要宿主机节点能解析此域名
    path: "共享目录"           #指定NFS共享目录
    readOnly: false            #指定权限

#注意事项: 要求Kubernetes所有集群都必须支持nfs客户端命令, 即所有节点都必须执行 apt -y
install nfs-common|nfs-client

#示例
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: registry.k8s.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /my-nfs-data
      name: test-volume
  volumes:
  - name: test-volume
    nfs:
      server: my-nfs-server.example.com
      path: /my-nfs-volume
      readOnly: true
```

1.4.1.2 案例

范例: 使用集群外的NFS存储

#NFS服务器软件安装

```
[root@master1 ~]#apt update && apt install -y nfs-kernel-server 或者 nfs-server
[root@master1 ~]#systemctl enable --now nfs-server.service
```

#配置共享目录

```
[root@master1 ~]#mkdir /nfsdata
```

#指定权限

```
[root@master1 ~]#echo '/nfsdata *(rw,all_squash,anonuid=0,anongid=0)' >> /etc/exports
```

#如果使用默认权限, 会提示Pod下面错误

```
[root@master1 ~]#echo '/nfsdata *(rw)' >> /etc/exports
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
volumes-nfs	0/1	CrashLoopBackOff	2 (14s ago)	30s

```
[root@master1 ~]#kubectl logs volumes-nfs
```

```
chown: changing ownership of '.': operation not permitted
```

#说明:

*: 表示允许所有连接, 该值可以是一个网段|IP|域名的形式

rw 表示读写共享权限

no_root_squash 表示root登录nfs资源的时候, 不压缩其权限

sync 所有操作数据会同时写入硬盘和内存

#重启服务

```
[root@master1 ~]#exportfs -r
```

```
[root@master1 ~]#exportfs -v
```

```
/nfsdata <world>
```

```
(sync,wdelay,hide,no_subtree_check,anonuid=0,anongid=0,sec=sys,rw,secure,no_root_squash,all_squash)
```

#在所有kubernetes的worker节点充当NFS客户端, 都需要安装NFS客户端软件

```
[root@nodex ~]#apt update && apt -y install nfs-common 或者 nfs-client
```

#测试访问

```
[root@node1 ~]#showmount -e 10.0.0.101
```

```
Export list for 10.0.0.101:
```

```
/nfsdata *
```

#注意: 如果所有客户端想要使用nfs的功能, 必须提前安装软件, 否则会发生找不到资源的报错。

#编写资源配置文件

```
[root@master1 ~]#cat storage-nfs-1.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: volumes-nfs
```

```
spec:
```

```
  nodeName: node1.wang.org #指定在node1节点运行pod, 节点的/etc/hosts文件或DNS解析此域名
```

```
  volumes:
```

```
    - name: redisdatapath
```

```
      nfs:
```

```

server: nfs.wang.org      #注意:需要宿主机节点/etc/hosts文件或DNS解析此域名,而不是由
Pod通过coreDNS完成解析
  path: /nfsdata
containers:
- name: redis
  #image: redis:6.2.5
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/redis:6.2.5
  volumeMounts:
  - name: redisdatapath
    mountPath: /data

#在worker节点做域名解析nfs.wang.org
[root@node1 ~]#vim /etc/hosts
10.0.0.200 nfs.wang.org

#创建pod
[root@master1 ~]#kubectl apply -f storage-nfs-1.yaml

#写入数据
[root@master1 ~]#kubectl exec -it volumes-nfs -- bash
root@volumes-nfs:/data# redis-cli set name wangxiaochun
OK
root@volumes-nfs:/data# redis-cli get name
"wangxiaochun"
root@volumes-nfs:/data# redis-cli bgsave
Background saving started
root@volumes-nfs:/data#

#验证数据生成
[root@master1 ~]#ls /nfsdata/ -l
total 4
-rw-r--r-- 1 systemd-coredump systemd-coredump 116 Mar 23 22:05 dump.rdb

#创建另一个pod
[root@master1 ~]#cat storage-nfs-2.yaml
apiVersion: v1
kind: Pod
metadata:
  name: volumes-nfs-2
spec:
  nodeName: node2.wang.org  #指定另一台主机
  volumes:
  - name: redisdatapath
    nfs:
      server: nfs.wang.org  #注意:需要做名称解析
      path: /nfsdata
  containers:
  - name: redis
    image: redis:6.2.5
    volumeMounts:
    - name: redisdatapath
      mountPath: /data

#再次创建pod
[root@master1 ~]#kubectl apply -f storage-nfs-2.yaml

[root@master1 ~]#kubectl get pod

```

```
volumes-nfs      1/1      Running    0          3m44s
volumes-nfs-2    1/1      Running    0          21s
```

#验证数据生成

```
[root@master1 ~]#kubectl exec -it volumes-nfs-2 -- bash
root@volumes-nfs-2:/data# redis-cli get name
"wangxiaochun"
root@volumes-nfs-2:/data#
```

#结果显示：基于nfs网络文件存储实现了更强大的文件共享效果

范例: 利用NFS共享实现nginx页面同步

```
[root@master1 ~]#mkdir -p /nfsdata/nginx
[root@master1 ~]#vim /etc/exports
/nfsdata          10.0.0.0/24(rw,no_root_squash)
[root@master1 ~]#exportfs -r

[root@master1 ~]#cat storage-nfs-nginx.yaml
---
apiVersion: v1
kind: Namespace
metadata:
  name: storage

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-nfs
  namespace: storage
  labels:
    app: nginx-nfs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-nfs
  template:
    metadata:
      labels:
        app: nginx-nfs
    spec:
      volumes:
        - name: html
          nfs:
            server: nfs.wang.org    #注意:需要做名称解析
            path: /nfsdata/nginx
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
          name: nginx
          volumeMounts:
            - name: html
              mountPath: /usr/share/nginx/html

---
apiVersion: v1
kind: Service
```

```

metadata:
  name: service-nginx-nfs
  namespace: storage
spec:
  ports:
    - name: http
      port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: nginx-nfs
  type: LoadBalancer

```

```
[root@master1 ~]#kubectl apply -f storage-nfs-nginx.yaml
```

```
[root@master1 ~]#kubectl get all -n storage
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-nfs-6f6cd9577b-m6zks	1/1	Running	0	2m39s
pod/nginx-nfs-6f6cd9577b-sh8wl	1/1	Running	0	2m39s
pod/nginx-nfs-6f6cd9577b-xbz9z	1/1	Running	0	2m39s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx-nfs	3/3	3	3	2m39s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nginx-nfs-6f6cd9577b	3	3	3	2m39s

1.4.2 实现其他的网络共享存储

1.4.2.1 CephFS 配置

```

apiVersion: v1
kind: Pod
metadata:
  name: volumes-cephfs-demo
spec:
  containers:
    - name: redis
      image: redis:alpine
      volumeMounts:
        - mountPath: "/data"
          name: redis-cephfs-vol
  volumes:
    - name: redis-cephfs-vol
      cephfs:
        monitors:
          - 10.0.0.201:6789
          - 10.0.0.202:6789
          - 10.0.0.203:6789
        path: /kube/namespaces/default/redis1
        user: fsclient
        secretFile: "/etc/ceph/fsclient.key"

```

1.4.2.2 GlusterFS 配置

```
apiVersion: v1
kind: Pod
metadata:
  name: volumes-glusterfs-demo
  labels:
    app: redis
spec:
  containers:
  - name: redis
    image: redis:alpine
    ports:
    - containerPort: 6379
      name: redisport
    volumeMounts:
    - mountPath: /data
      name: redisdata
  volumes:
  - name: redisdata
    glusterfs:
      endpoints: glusterfs-endpoints
      path: kube-redis
```

1.4.2.3 Cinder 配置

2012年OpenStack Folsom的发布引入了Cinder，VM镜像是短暂的，它们的存储只维持在那台虚拟机的生命周期。

Cinder提供对块存储的管理支持，通过使用iSCSI, 光纤通道或者NFS协议，以及若干私有协议提供后端连接，展现给计算层。

Cinder接口提供了一些标准功能，允许创建和附加块设备到虚拟机，如“创建卷”，“删除卷”和“附加卷”。

还有更多高级的功能，支持扩展容量的能力，快照和创建虚拟机镜像克隆。

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-cinder-demo
spec:
  containers:
  - image: mysql
    name: mysql
    args:
    - "--ignore-db-dir"
    - "lost+found"
    env:
    - name: MYSQL_ROOT_PASSWORD
      value: YOUR_PASS
    ports:
    - containerPort: 3306
      name: mysqlport
    volumeMounts:
    - name: mysqldata
      mountPath: /var/lib/mysql
  volumes:
```

```
- name: mysqldata
  cinder:
    volumeID: e2b8d2f7-wece-90d1-a505-4acf607a90bc
    fsType: ext4
```

1.5 PV 和 PVC

1.5.1 PV 和 PVC 说明

<https://kubernetes.io/zh-cn/docs/concepts/storage/persistent-volumes/>
<https://github.com/kubernetes/examples/tree/master/volumes/storageos>
<https://github.com/kubernetes/examples/tree/master/staging/volumes/nfs>

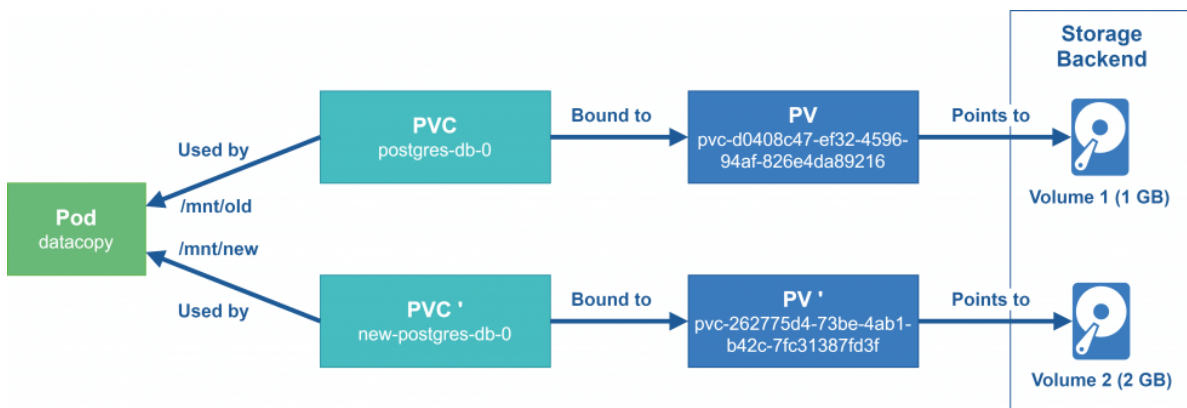
之前提到的Volume可以提供多种类型的资源存储(可持久或不持久),但是它定义在Pod上的,并不是一个独立的API资源类型,是隶属于"资源对象"的组成部分。

这导致用户在创建Pod时还需要关注存储的具体实现,这种方式造成Pod和存储的耦合度很高,无疑是很不方便管理的

由于每个Pod对存储的要求不同,比如大小,读写权限等,所以还需要Pod用户提前自行准备存储,对用户有更高的存储方面技术的要求

此外Pod 使用的存储空间并没有加以限制,所以很可能会导致某个Pod占有过多的存储空间

Kubernetes-v1.0开始即支持PV和PVC 来解决上述问题



PV Persistent Volume

工作中的存储资源一般都是独立于Pod的,将之称为资源对象Persistent Volume(PV),是由管理员设置的存储,它是kubernetes集群的一部分, PV 是 Volume 之类的卷插件,但具有独立于使用 PV 的 Pod 的生命周期。

Persistent Volume 跟 Volume类似, 区别就是:

- PV 是集群级别的资源,负责将存储空间引入到集群中,通常由管理员定义
- PV 就是Kubernetes集群中的网络存储,不属于Namespace、Node、Pod等资源,但可以被它们访问
- PV 属于Kubernetes 整个集群,即可以被所有集群的Pod访问
- PV是独立的网络存储资源对象,有自己的生命周期
- PV 支持很多种volume类型,PV对象可以有很多常见的类型:本地磁盘、NFS、分布式文件系统...

<https://kubernetes.io/zh-cn/docs/concepts/storage/persistent-volumes/#persistent-volumes>

持久卷的类型 [↗](#)

PV 持久卷是用插件的形式来实现的。Kubernetes 目前支持以下插件：

- `cephfs` - CephFS volume
- `csi` - 容器存储接口 (CSI)
- `fc` - Fibre Channel (FC) 存储
- `hostPath` - HostPath 卷 (仅供单节点测试使用；不适用于多节点集群；请尝试使用 `local` 卷作为替代)
- `iscsi` - iSCSI (SCSI over IP) 存储
- `local` - 节点上挂载的本地存储设备
- `nfs` - 网络文件系统 (NFS) 存储
- `rbd` - Rados 块设备 (RBD) 卷

以下的持久卷已被弃用。这意味着当前仍是支持的，但是 Kubernetes 将来的发行版会将其移除。

- `awsElasticBlockStore` - AWS 弹性块存储 (EBS) (于 v1.17 弃用)
- `azureDisk` - Azure Disk (于 v1.19 弃用)
- `azureFile` - Azure File (于 v1.21 弃用)
- `cinder` - Cinder (OpenStack 块存储) (于 v1.18 弃用)
- `flexVolume` - FlexVolume (于 v1.23 弃用)
- `gcePersistentDisk` - GCE Persistent Disk (于 v1.17 弃用)
- `glusterfs` - Glusterfs 卷 (于 v1.25 弃用)
- `portworxVolume` - Portworx 卷 (于 v1.25 弃用)
- `vsphereVolume` - vSphere VMDK 卷 (于 v1.19 弃用)

旧版本的 Kubernetes 仍支持这些“树内 (In-Tree)”持久卷类型：

- `photonPersistentDisk` - Photon 控制器持久化盘。(从 v1.15 版本开始将不可用)
- `scaleIO` - ScaleIO 卷 (v1.21 之后不可用)
- `flocker` - Flocker 存储 (v1.25 之后不可用)
- `quobyte` - Quobyte 卷 (v1.25 之后不可用)
- `storageos` - StorageOS 卷 (v1.25 之后不可用)

PVC Persistent Volume Claim

Persistent Volume Claim(PVC) 是一个网络存储服务的请求。

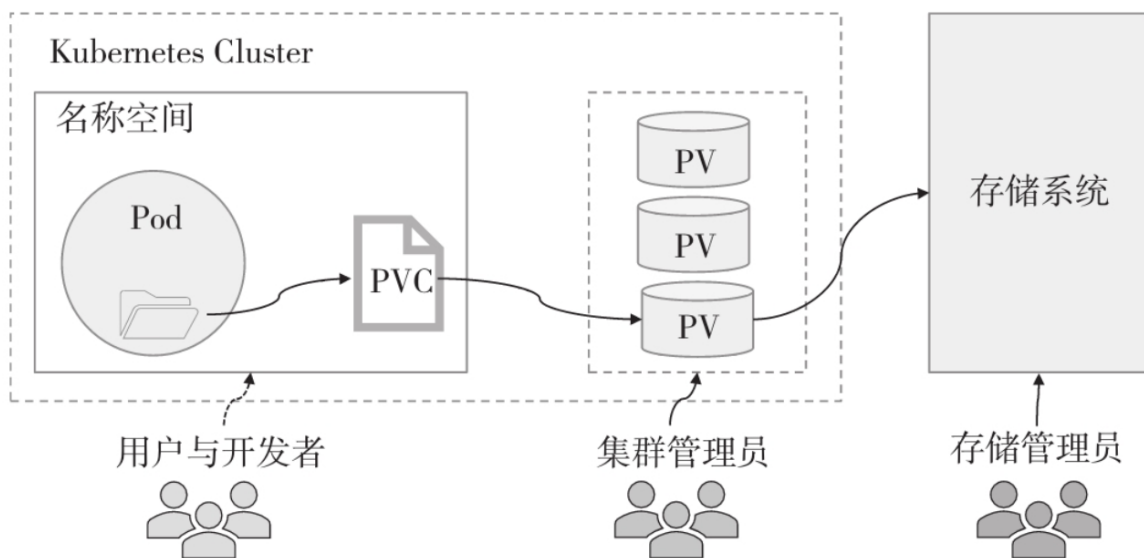
PVC 属于名称空间级别的资源，只能被同一个名称空间的Pod引用

由用户定义，用于在空闲的PV中申请使用符合过滤条件的PV之一，与选定的PV是“一对一”的关系

用户在Pod上通过pvc插件请求绑定使用定义好的PVC资源

Pod能够申请特定的CPU和MEM资源，但是Pod只能通过PVC到PV上请求一块独立大小的网络存储空间，而PVC 可以动态的根据用户请求去申请PV资源，不仅仅涉及到存储空间，还有对应资源的访问模式，对于真正使用存储的用户不需要关心底层的存储实现细节，只需要直接使用 PVC 即可。

Pod、PV、PVC 关系



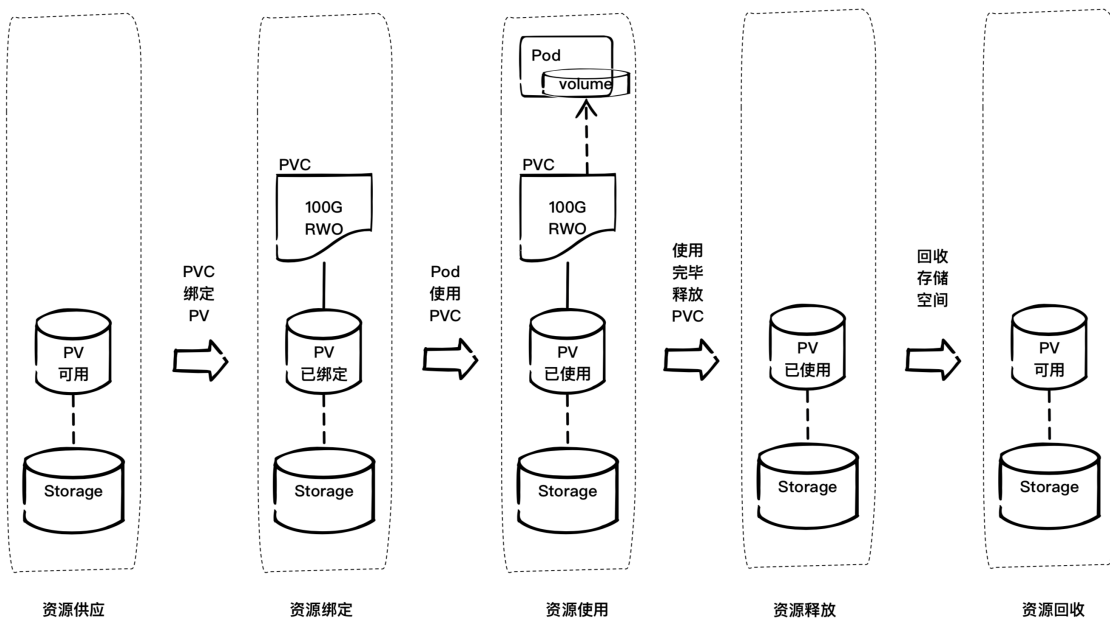
前提:

- 存储管理员配置各种类型的PV对象
- Pod、PVC 必须在同一个命名空间

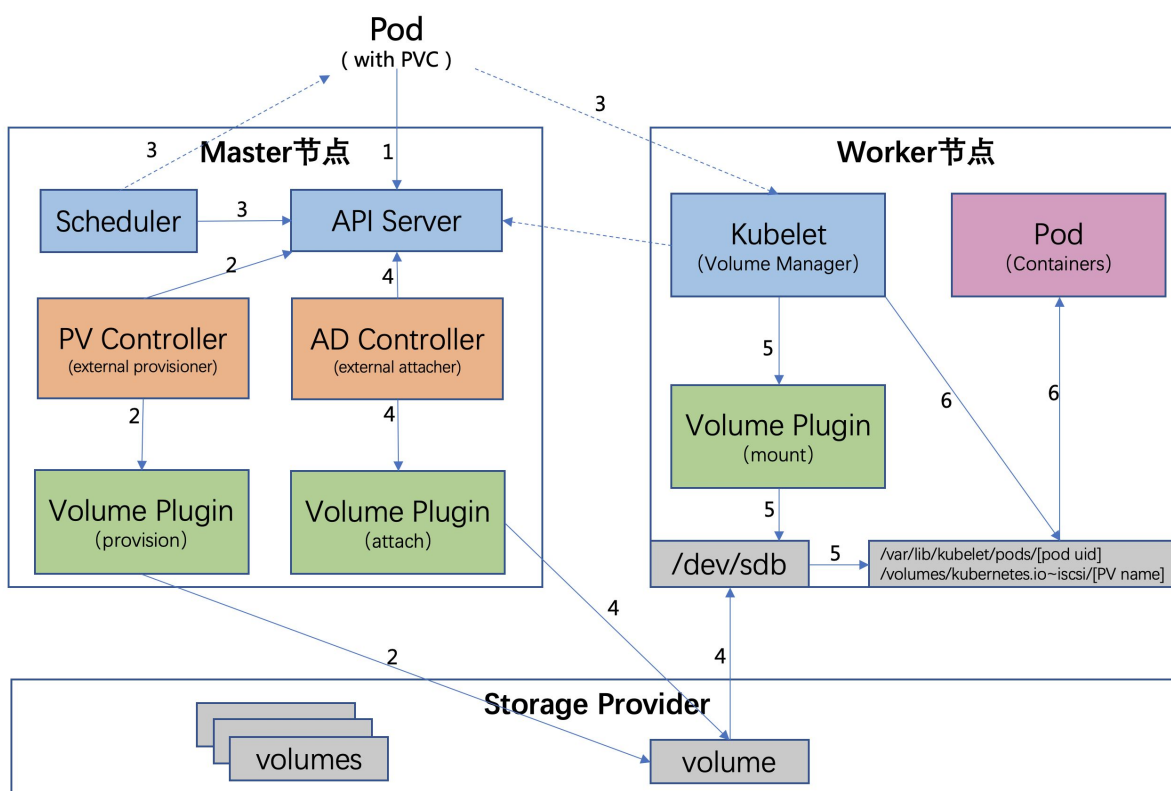
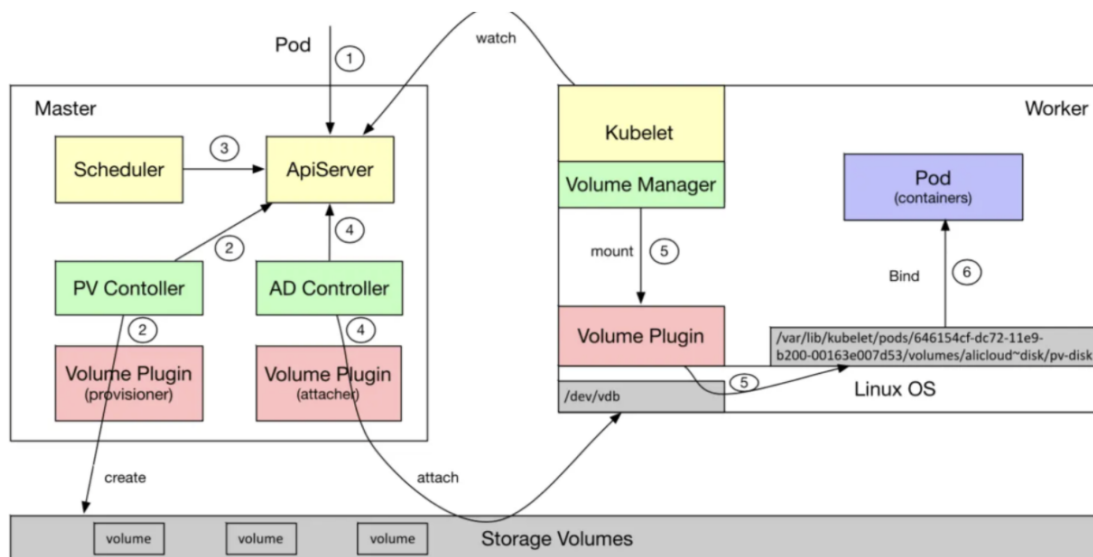
用户需要存储资源的时候:

- 用户根据资源需求创建PVC，由PVC自动匹配(权限、容量)合适的PV对象
- PVC 允许用户按需指定期望的存储特性，并以之为条件，按特定的条件顺序进行PV的过滤
VolumeMode → LabelSelector → StorageClassName → AccessMode → Size
- 在Pod内部通过 PVC 将 PV 绑定到当前的空间，进行使用
- 如果用户不再使用存储资源，解绑 PVC 和 Pod 即可

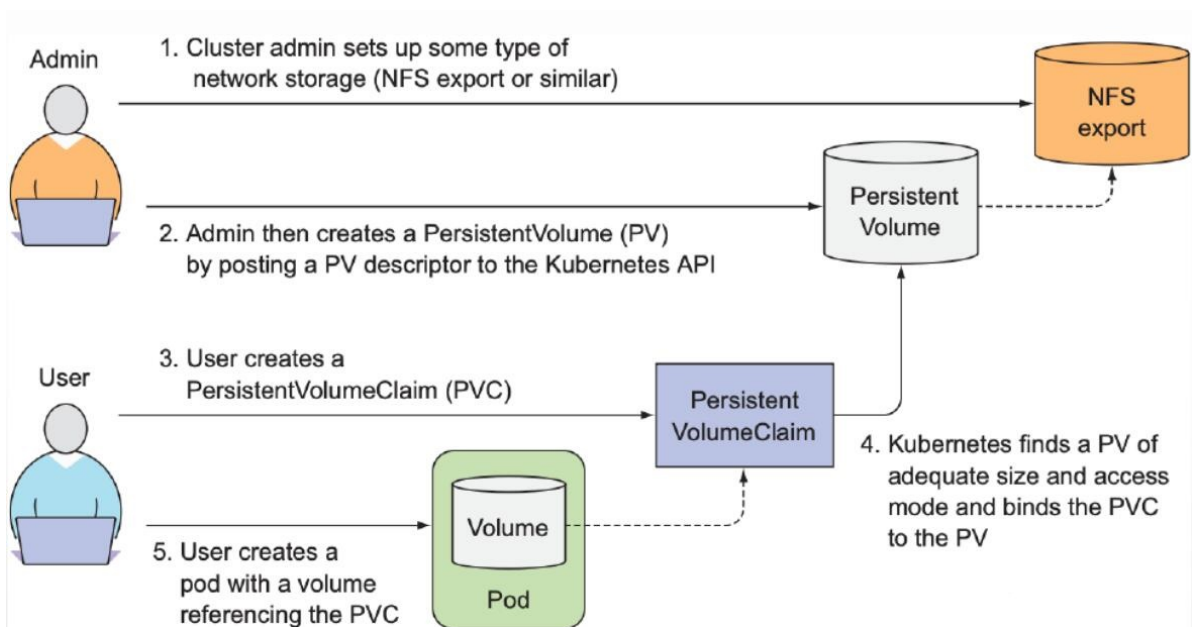
PV和PVC的生命周期



PV和PVC的配置流程



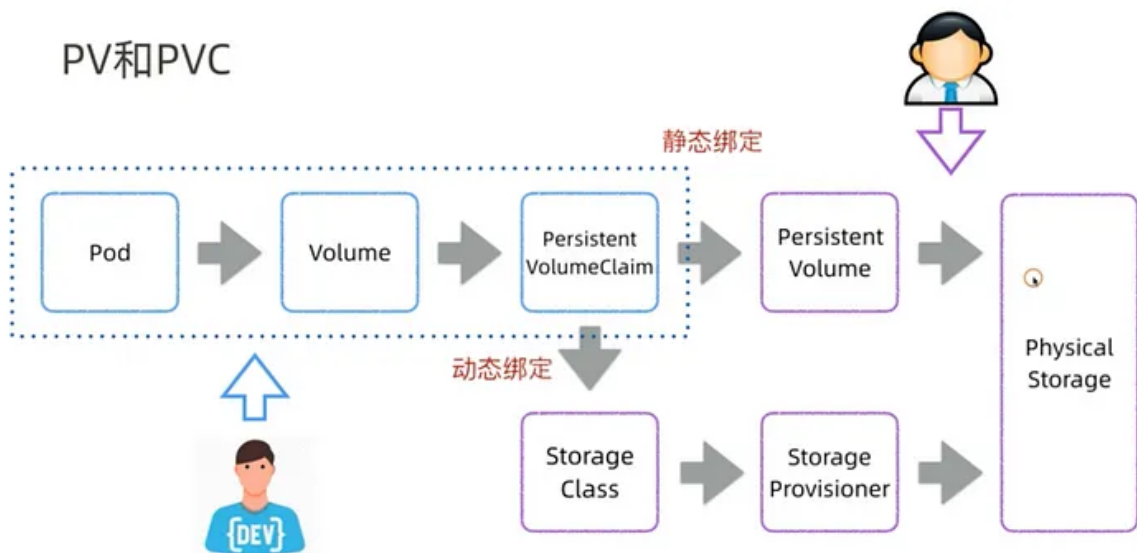
- 用户创建了一个包含 PVC 的 Pod，该 PVC 要求使用动态存储卷
- Scheduler 根据 Pod 配置、节点状态、PV 配置等信息，把 Pod 调度到一个合适的 Worker 节点上
- PV 控制器 watch 到该 Pod 使用的 PVC 处于 Pending 状态，于是调用 Volume Plugin(in-tree)创建存储卷，并创建 PV 对象(out-of-tree 由 External Provisioner 来处理)
- AD 控制器发现 Pod 和 PVC 处于待挂接状态，于是调用 Volume Plugin 挂接存储设备到目标 Worker 节点上
- 在 Worker 节点上，Kubelet 中的 Volume Manager 等待存储设备挂接完成，并通过 Volume Plugin 将设备挂载到全局目录：**/var/lib/kubelet/pods/[pod uid]/volumes/kubernetes.io~iscsi/[PVname]** (以iscsi为例)
- Kubelet 通过 Docker 启动 Pod 的 Containers，用 bind mount 方式将已挂载到本地全局目录的卷映射到容器中



1.5.2 PV 和 PVC 管理

1.5.2.1 基础属性

PV的Provision 置备（创建）方法



- 静态：集群管理员预先手动创建一些 PV。它们带有可供群集用户使用的实际存储的细节。
- 动态：集群尝试根据用户请求动态地自动完成创建卷。此配置基于 StorageClasses：PVC 必须请求存储类，并且管理员必须预先创建并配置该 StorageClasses 才能进行动态创建。声明该类为空字符串 ""，可以有效地禁用其动态配置。

PV 属性

```
#PV 作为存储资源，主要包括存储能力、访问模式、存储类型、回收策略等关键信息,注意:PV的名称不支持大写
#https://kubernetes.io/zh-cn/docs/concepts/storage/persistent-volumes/#persistent-volumes
kubectl explain pv.spec
  capacity      #定义pv使用多少资源，仅限于空间的设定
  accessModes   #访问模式,支持单路读写，多路读写，多路只读，单Pod读写，可同时支持多种模式
  volumeMode    #文件系统或块设备,默认文件系统
```

```
mountOptions    #挂载选项,比如:["ro", "soft"]
persistentVolumeReclaimPolicy    #资源回收策略, 主要三种Retain、Delete、Recycle
存储类型        #每种存储类型的样式的属性名称都是专有的。
storageClassName    #存储类的名称,如果配置必须和PVC的storageClassName相同才能绑定
```

#注意:PersistentVolume 对象的名称必须是合法的 DNS 子域名。

#示例

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0003
  labels:
    release: "stable"    #便签可以支持匹配过滤PVC
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow    #必须和PVC相同
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
```

PVC 属性

#PVC属性信息,与所有空间都能使用的PV不一样, PVC是属于名称空间级别的资源对象,即只有特定的资源才能使用

#<https://kubernetes.io/zh-cn/docs/concepts/storage/persistent-volumes/#persistentvolumeclaims>

#定义PVC

```
kubectl explain pvc.spec
accessModes    #访问模式    *
resources      #资源限制    *
volumeMode     #后端存储卷的模式,文件系统或块,默认为文件系统
volumeName     #指定绑定的卷(pv)的名称
```

#Pod引用PVC

```
kubectl explain pods.spec.volumes.persistentVolumeClaim
claimName    #定义pvc的名称,PersistentVolumeClaim 对象的名称必须是合法的 DNS 子域名。
readOnly     #设定pvc是否只读
storageClassName    #存储类的名称,如果配置必须和PV的storageClassName相同才能绑定
selector     #标签选择器实现选择绑定PV
```

#合法的 DNS 子域名。

<https://kubernetes.io/zh-cn/docs/concepts/overview/working-with-objects/names/#dns-subdomain-names>

#storageClassName类

PVC可以通过为 `storageClassName` 属性设置 `StorageClass` 的名称来请求特定的存储类。只有所请求的类的PV的`storageClassName` 值与 PVC 设置相同,才能绑定

#selector 选择算符

PVC可以设置标签选择算符,来进一步过滤卷集合。只有标签与选择算符相匹配的卷能够绑定到PVC上。选择算符包含两个字段:

`matchLabels` - 卷必须包含带有此值的标签

`matchExpressions` - 通过设定键 (`key`)、值列表和操作符 (`operator`) 来构造的需求。合法的操作符有 `In`、`NotIn`、`Exists` 和 `DoesNotExist`。

来自 `matchLabels` 和 `matchExpressions` 的所有需求都按逻辑与的方式组合在一起。 这些需求都必须被满足才被视为匹配。

#示例

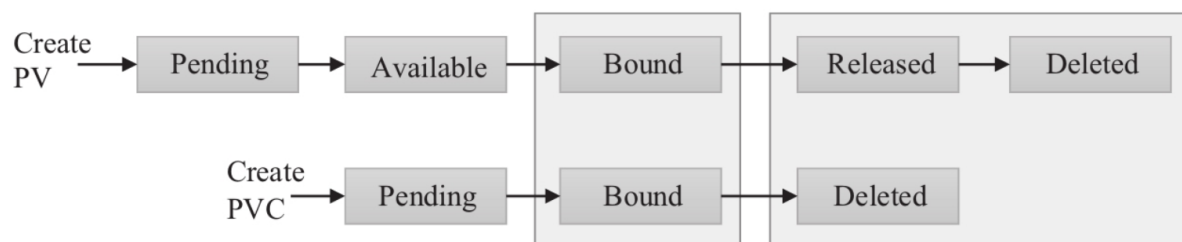
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow #必须和PV相同
  selector:
    matchLabels:
      release: "stable"
    matchExpressions:
      - {key: environment, operator: In, values: [dev]}
```

1.5.2.2 属性进阶

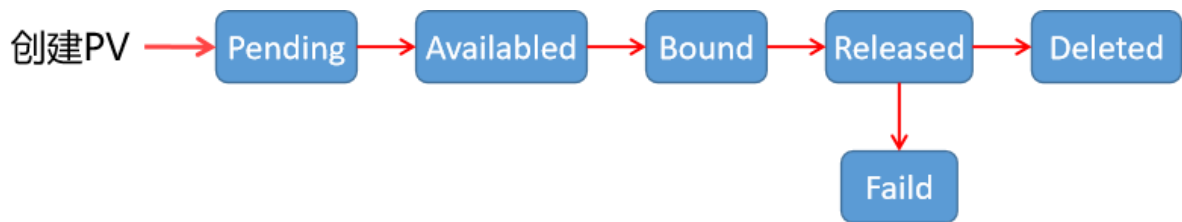
1.5.2.2.1 PV 状态

PV 有生命周期,自然也有自己特定的状态

注意: 这个过程是单向过程,不能逆向



状态	解析
Available	空闲状态, 表示PV没有被其他PVC对象使用
Bound	绑定状态, 表示PV已经被其他PVC对象使用
Released	未回收状态, 表示PVC已经被删除了, 但是资源还没有被回收
Failed	资源回收失败



通过 `kubectl patch` 命令可以直接修改 PV 的状态，使其从 `Released` 状态变为 `Available` 状态。

```
kubectl patch persistentvolume <pv-name> -p '{"spec":{"claimRef": null}}'
```

1.5.2.2 AccessMode 访问模式

AccessModes 是用来对 PV 进行访问模式的设置，用于描述用户应用对存储资源的访问权限，访问权限包括

<https://kubernetes.io/zh-cn/docs/concepts/storage/persistent-volumes/#access-modes>

类型	解析
ReadWriteOnce (RWO)	单节点读写,卷可以被一个节点以读写方式挂载。 ReadWriteOnce 访问模式仍然可以在同一节点上运行的多个 Pod 访问该卷即不支持并行(非并发)写入
ReadOnlyMany (ROX)	多节点只读
ReadWriteMany (RWX)	多节点读写
ReadWriteOncePod(RWOP)	卷可以被单个 Pod 以读写方式挂载。如果你想确保整个集群中只有一个 Pod 可以读取或写入该 PVC，请使用 ReadWriteOncePod 访问模式。单Pod读写,v1.22版以后才支持,v1.29版stable可用

注意：

- 不同的后端存储支持不同的访问模式，所以要根据后端存储类型来设置访问模式。
- 一些 PV 可能支持多种访问模式，但是在挂载的时候只能使用一种访问模式，多种访问模式是不会生效的

卷插件	ReadWriteOnce	ReadOnlyMany	ReadWriteMany	ReadWriteOncePod
AWSElasticBlockStore	✓	-	-	-
AzureFile	✓	✓	✓	-
AzureDisk	✓	-	-	-
CephFS	✓	✓	✓	-
Cinder	✓	-	(如果多次挂接卷可用)	-
CSI	取决于驱动	取决于驱动	取决于驱动	取决于驱动
FC	✓	✓	-	-
FlexVolume	✓	✓	取决于驱动	-
GCEPersistentDisk	✓	✓	-	-
Glusterfs	✓	✓	✓	-
HostPath	✓	-	-	-
iSCSI	✓	✓	-	-
NFS	✓	✓	✓	-
RBD	✓	✓	-	-
VsphereVolume	✓	-	- (Pod 运行于同一节点上时可行)	-
PortworxVolume	✓	-	✓	-

1.5.2.2.3 PV 资源回收策略

PV 三种资源回收策略

当 Pod 结束 volume 后可以回收资源对象并删除PVC，而PVC和PV绑定关系就不存在了，当绑定关系不存在后对应的PV需要怎么处理，而PersistentVolume 的回收策略告诉集群在存储卷声明释放后应如何处理该PV卷。目前，volume 的处理策略有保留、回收或删除。

当PVC被删除后, Kubernetes 会自动生成一个recycler-for-<PV_NAME>的Pod实现回收工作,但Retain策略除外

回收完成后,PV的状态变为Release,如果其它处于Pending状态的PVC和此PV条件匹配,则可以再次此PV进行绑定

类型	解析
Retain	当PVC删除后，会保留对应的PV和存储空间数据，后续数据的删除需要人工干预，一般推荐使用此项，对于手动创建的PV此为默认值
Delete	当PVC删除后，相关的PV和数据都一起删除。需要支持删除功能的存储才能实现，它的优势在于能够自动且快速地完成资源回收，让整个存储资源管理更加自动化，动态存储一般会默认采用此方式
Recycle	当前此项已废弃，保留PV，但清空存储空间的数据，仅支持NFS和hostPath

注意：目前只有 NFS 和 HostPath 支持 Recycle 策略，其他支持 Delete 策略

```
[root@master1 ~]#kubectl explain pv.spec
```

.....

#新版: 提示已废弃

```
persistentVolumeReclaimPolicy <string>
persistentVolumeReclaimPolicy defines what happens to a persistent volume
when released from its claim. Valid options are Retain (default for manually
created PersistentVolumes), Delete (default for dynamically provisioned
PersistentVolumes), and Recycle (deprecated). Recycle must be supported by
the volume plugin underlying this PersistentVolume. More info:
https://kubernetes.io/docs/concepts/storage/persistent-volumes#reclaiming
```

#旧版

```
persistentVolumeReclaimPolicy    <string>
persistentVolumeReclaimPolicy defines what happens to a persistent volume
when released from its claim. Valid options are Retain (default for
manually created PersistentVolumes), Delete (default for dynamically
provisioned PersistentVolumes), and Recycle (deprecated). Recycle must be
supported by the volume plugin underlying this PersistentVolume. More info:
https://kubernetes.io/docs/concepts/storage/persistent-volumes#reclaiming
```

Possible enum values:

- `"Delete"` means the volume will be deleted from Kubernetes on release from its claim. The volume plugin must support Deletion.
- `"Recycle"` means the volume will be recycled back into the pool of unbound persistent volumes on release from its claim. The volume plugin must support Recycling.
- `"Retain"` means the volume will be left **in** its current phase (Released) **for** manual reclamation by the administrator. The default policy is Retain.

1.5.2.3 PV 和 PVC 使用流程

实现方法

- 准备存储
- 基于存储创建PV
- 根据需求创建PVC: PVC会根据capacity和accessModes及其它条件自动找到相匹配的PV进行绑定, 一个PVC对应一个PV
- 创建Pod
 - 在Pod中的 volumes 指定调用上面创建的 PVC 名称
 - 在Pod中的容器中的volumeMounts指定PVC挂载容器内的目录路径

1.5.2.4 故障排错

创建PVC之后, 一直绑定不上PV,导致PVC一直处于Pending状态,主要原因如下:

- PVC 的空间申请大小大于 PV 的大小
- PVC 的 accessModes 和 PV 的不一致
- PVC 的 storageClassName 和 PV的不一致

创建挂载了PVC的Pod之后一直处于Pending状态主要原因如下

- PVC 没有创建
- PVC 创建失败
- PVC 和 Pod 不在同一个Namespace

1.5.3 案例

1.5.3.1 PV 和 PVC 使用

范例: 以NFS类型创建一个3G大小的存储资源对象PV

```
#准备NFS共享存储
[root@master1 ~]#mkdir -p /nfsdata/www
[root@master1 ~]#apt -y install nfs-server

[root@master1 ~]#echo "/nfsdata *(rw,no_root_squash)" >> /etc/exports
[root@master1 ~]#exportfs -r
[root@master1 ~]#exportfs -v
/nfsdata          <world>
(rw,wdelay,no_root_squash,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)

#在所有worker节点安装nfs软件
[root@node1 ~]#apt -y install nfs-common

#准备PV, 定制一个具体空间大小的存储对象
[root@master1 ~]#cat storage-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-test    #注意:不支持大写字母
spec:
  capacity:
    storage: 3Gi
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
    - ReadOnlyMany
  nfs:
    path: /nfsdata/www
    server: nfs.wang.org #需要名称解析

#如果该pv的使用方式支持多个, 可以使用格式: accessModes:
["ReadWriteMany", "ReadWriteOnce"]
#如果nfs提供的目录空间个数多的话, 可以同样的方式写多个。
PV的资源对象类型: PersistentVolume
存储空间大小设定: capacity
存储空间的访问模式: accessModes
    ReadWriteOnce    #该volume只能被单个节点以读写的方式映射
    ReadOnlyMany     #该volume可以被多个节点以只读方式映射
    ReadWriteMany    #该volume只能被多个节点以读写的方式映射
#可以指定回收策略: persistentVolumeReclaimPolicy: Recycle

[root@master1 ~]#kubectl apply -f storage-pv.yaml
persistentvolume/pv-test created
[root@master1 ~]#kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
STORAGECLASS  REASON    AGE
pv-test       3Gi       RWO,ROX,RWX   Retain           Available
7s

#结果显示: 虽然我们在创建pv的时候没有指定回收策略, 而其策略自动帮我们配置了Retain
```

#准备PVC,定义一个资源对象, 请求空间1Gi

```
[root@master1 ~]#cat storage-pvc.yaml
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: pvc-test
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  resources:
```

```
    requests:
```

```
      storage: 1Gi
```

#注意: 请求的资源大小必须在 pv资源的范围内。

```
[root@master1 ~]#kubectl apply -f storage-pvc.yaml
```

#结果显示: 一旦启动pvc会自动去搜寻合适的可用的pv, 然后绑定在一起

#如果pvc找不到对应的pv资源, 状态会一直处于pending

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pv-test	3Gi	RWO,ROX,RWX	Retain	Bound	default/pvc-test
		28s			

```
[root@master1 ~]#kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc-test	Bound	pv-test	3Gi	RWO		5s

#准备pod

```
[root@master1 ~]#cat storage-nginx-pvc.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-nginx
```

```
spec:
```

```
  volumes:
```

```
    - name: volume-nginx
```

```
      persistentVolumeClaim:
```

```
        claimName: pvc-test
```

```
  containers:
```

```
    - name: pvc-nginx-container
```

```
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
```

```
      volumeMounts:
```

```
    - name: volume-nginx
```

```
      mountPath: "/usr/share/nginx/html"
```

#属性解析:

#spec.volumes 是针对pod资源申请的存储资源来说的, 这里使用的主要是pvc的方式。

#spec.containers.volumeMounts 是针对pod资源对申请的存储资源的信息。将pvc挂载的容器目录

```
[root@master1 ~]#kubectl apply -f storage-nginx-pvc.yaml
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES			
pod-nginx	1/1	Running	0	20s	172.16.3.62	node1.wang.org
<none>		<none>				

#默认页面无法访问,如果不准备首页的话,会导致 403 报错

```
[root@master1 ~]#curl 172.16.3.62
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.20.0</center>
</body>
</html>
```

#添加网页文件

```
[root@master1 ~]#echo PV Page > /nfsdata/index.html
[root@master1 ~]#curl 172.16.3.62
PV Page
```

#注意:删除时,要按顺序删除,先删除应用pod,再删除pvc,最后删除pv,否则会出现卡死现象

#删除pod

```
[root@master1 ~]#kubectl delete -f storage-nginx-pvc.yaml
```

#删除PVC

```
[root@master1 ~]#kubectl delete -f storage-pvc.yaml
```

```
[root@master1 ~]#kubectl get pvc
```

No resources found in default namespace.

#显示PV状态Released

```
[root@master1 ~]#kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
STORAGECLASS  REASON    AGE
pv-test       3Gi       RWO           Retain          Released  default/pvc-test
              41s
```

#最后删除PV

```
[root@master1 ~]#kubectl delete -f storage-pv.yaml
```

```
[root@master1 ~]#kubectl get pv
```

No resources found

范例: 创建使用PVC的deployment实现多个Pod共享数据

```
[root@master1 ~]#kubectl apply -f storage-pv.yaml
```

```
[root@master1 ~]#kubectl apply -f storage-pvc.yaml
```

```
[root@master1 ~]#kubectl get pv
```

```
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
STORAGECLASS  REASON    AGE
pv-test       3Gi       RWO           Retain          Bound   default/pvc-test
              7m58s
```

```
[root@master1 ~]#kubectl get pvc
```

```
NAME      STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  AGE
pvc-test  Bound   pv-test  3Gi       RWO           storageclass  7m58s
```

```
[root@master1 ~]#cat storage-deployment-nginx-pvc.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```

metadata:
  name: deployment-nginx-pvc
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pvc
  template:
    metadata:
      labels:
        app: nginx-pvc
    spec:
      volumes:
      - name: volume-nginx
        persistentVolumeClaim:
          claimName: pvc-test
      containers:
      - name: nginx-pvc
        image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
        volumeMounts:
        - name: volume-nginx
          mountPath: "/usr/share/nginx/html"

```

```

apiVersion: v1
kind: Service
metadata:
  name: service-nginx-pvc
spec:
  ports:
  - name: http
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: nginx-pvc
  type: LoadBalancer

```

#创建deployment

```
[root@master1 ~]#kubectl apply -f storage-deployment-nginx-pvc.yaml
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME			READY	STATUS	RESTARTS	AGE	IP
	NODE	NOMINATED NODE	READINESS	GATES			
deployment-nginx-pvc-777d57f5f4-n7hp4	172.16.4.54	node2.wang.org	1/1	Running	0	7s	
deployment-nginx-pvc-777d57f5f4-p7nb5	172.16.3.64	node1.wang.org	1/1	Running	0	7s	
deployment-nginx-pvc-777d57f5f4-pb9lt	172.16.3.65	node1.wang.org	1/1	Running	0	7s	

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service-nginx-pvc	LoadBalancer	10.101.174.54	10.0.0.10	80:32307/TCP

5m57s

```
#查看多个Pod资源实现共享
[root@master1 ~]#curl 172.16.4.54
PV Page
[root@master1 ~]#curl 172.16.3.64
PV Page
[root@master1 ~]#curl 172.16.3.65
PV Page
```

范例: PVC自动绑定相匹配的PV,PVC和 PV 是自动关联的, 而且会匹配容量和权限

```
[root@master1 ~]#mkdir -p /nfsdata/data{1..3}
[root@master1 ~]#cat /etc/exports
/nfsdata *(rw,no_root_squash)
```

#PV清单文件

```
[root@master1 ~]#cat storage-multi-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-1
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: "/nfsdata/data1"
    server: nfs.wang.org
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-2
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadOnlyMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: "/nfsdata/data2"
    server: nfs.wang.org
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-3
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
```

```

persistentVolumeReclaimPolicy: Retain
nfs:
  path:  "/nfsdata/data3"
  server: nfs.wang.org

```

```
[root@master1 ~]#kubectl apply -f storage-multi-pv.yaml
```

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pv-nfs-1	5Gi	RWX	Retain	Available	
	3s				
pv-nfs-2	5Gi	ROX	Retain	Available	
	3s				
pv-nfs-3	1Gi	RWO	Retain	Available	
	3s				

#PVC清单文件

```
[root@master1 ~]#cat storage-multi-pvc.yaml
```

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo-1
  namespace: default
spec:
  accessModes: ["ReadWriteMany"]
  volumeMode: Filesystem
  resources:
    requests:
      storage: 3Gi
    limits:
      storage: 10Gi

```

```
[root@master1 ~]#kubectl apply -f storage-multi-pvc.yaml
```

```
[root@master1 ~]#kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc-demo-1	Bound	pv-nfs-1	5Gi	RWX		5m50s

#自动绑定PVC至容器和权限都匹配的PV

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pv-nfs-1	5Gi	RWX	Retain	Bound	default/pvc-
demo-1					
			2m36s		
pv-nfs-2	5Gi	ROX	Retain	Available	
			2m36s		
pv-nfs-3	1Gi	RWO	Retain	Available	
			2m36s		

案例: 运行一个单实例有状态应用MySQL

<https://kubernetes.io/zh-cn/docs/tasks/run-application/run-single-instance-stateful-application/>

#准备NFS共享存储

```
[root@master1 ~]#mkdir -p /nfsdata/mysql  
[root@master1 ~]#apt -y install nfs-server
```

```
[root@master1 ~]#echo "/nfsdata/mysql *(rw,no_root_squash)" >> /etc/exports  
[root@master1 ~]#exportfs -r  
[root@master1 ~]#exportfs -v  
/nfsdata/mysql          <world>  
(rw,wdelay,no_root_squash,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)
```

#在所有worker节点安装nfs软件

```
[root@node1 ~]#apt -y install nfs-common
```

```
[root@master1 ~]#cat storage-mysql-pv-pvc.yaml
```

```
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: mysql-pv-volume  
  labels:  
    type: local  
spec:  
  storageClassName: manual #用于分类  
  capacity:  
    storage: 20Gi  
  accessModes:  
    - ReadWriteOnce  
  nfs:  
    server: nfs.wang.org #注意:需要宿主机节点能解析此域名,而非Pod解析  
    path: /nfsdata/mysql  
  #hostPath:  
  # path: "/mnt/data"
```

```
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: mysql-pv-claim  
spec:  
  storageClassName: manual  
  accessModes:  
    - ReadWriteOnce  
  resources:  
    requests:  
      storage: 20Gi
```

```
apiVersion: v1  
kind: Service  
metadata:  
  name: mysql  
spec:  
  ports:  
    - port: 3306  
  selector:  
    app: mysql  
clusterIP: None
```

```

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
          name: mysql
          env:
            # 在实际中使用 secret
            - name: MYSQL_ROOT_PASSWORD
              value: "123456"
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pv-claim

```

#准备名称空间

```

[root@master1 ~]#kubectl create ns demo
namespace/demo created

```

```

[root@master1 ~]#kubectl apply -f storage-mysql-pv-pvc.yaml -n demo
persistentvolume/mysql-pv-volume created
persistentvolumeclaim/mysql-pv-claim created
service/mysql created
deployment.apps/mysql created

```

```

[root@master1 ~]#kubectl get all -n demo

```

NAME	READY	STATUS	RESTARTS	AGE
pod/mysql-55dd4c6c84-q5ktt	1/1	Running	0	47s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/mysql	ClusterIP	None	<none>	3306/TCP	47s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/mysql	1/1	1	1	47s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/mysql-55dd4c6c84	1	1	1	47s

```
[root@master1 ~]#kubectl get pv,pvc -n demo
```

NAME		CAPACITY	ACCESS MODES	RECLAIM POLICY
STATUS	CLAIM	STORAGECLASS	REASON	AGE
persistentvolume/mysql-pv-volume		20Gi	RWO	Retain
Bound	demo/mysql-pv-claim	manual		3m8s

NAME		STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	AGE		
persistentvolumeclaim/mysql-pv-claim		Bound	mysql-pv-volume	20Gi
				RWO
	manual			3m8s

#查看Pod运行在node2节点

```
[root@master1 ~]#kubectl get pod -n demo -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED NODE	READINESS GATES				
mysql-55dd4c6c84-q5ktt	1/1	Running	0	44s	10.244.2.59	
node2.wang.org	<none>	<none>				

#在node2节点查看到目录和数据都会自动创建和生成

```
[root@node2 ~]#ls /mnt/data
```

auto.cnf	ca-key.pem	'#ib_16384_0.db1wr'	ib_logfile0	mysql
	public_key.pem	undo_001		
binlog.000001	ca.pem	'#ib_16384_1.db1wr'	ib_logfile1	
mysql.ibd	server-cert.pem	undo_002		
binlog.000002	client-cert.pem	ib_buffer_pool	ibtmp1	
performance_schema	server-key.pem			
binlog.index	client-key.pem	ibdata1	'#innodb_temp'	
private_key.pem	sys			

#测试访问

```
[root@master1 ~]#kubectl exec -n demo -it mysql-55dd4c6c84-q5ktt -- mysql -hmysql -uroot -p123456 -e "status"
```

```
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
mysql Ver 8.0.27 for Linux on x86_64 (MySQL Community Server - GPL)
```

```
Connection id:      12
Current database:
Current user:       root@localhost
SSL:                Not in use
Current pager:      stdout
Using outfile:      ''
Using delimiter:    ;
Server version:     8.0.27 MySQL Community Server - GPL
Protocol version:   10
Connection:         Localhost via UNIX socket
Server characterset: utf8mb4
Db characterset:    utf8mb4
Client characterset: latin1
Conn. characterset: latin1
UNIX socket:        /var/run/mysqld/mysqld.sock
Binary data as:     Hexadecimal
Uptime:             6 min 20 sec
```

```
Threads: 2 Questions: 10 slow queries: 0 Opens: 134 Flush tables: 3 Open
tables: 53 Queries per second avg: 0.026
```

```
#清理环境
```

```
[root@master1 ~]#kubectl delete -f storage-mysql-pv-pvc.yaml -n demo
```

1.5.3.2 强制删除

生产中，对于存储资源的释放，最好按照流程来，即先清空应用，然后在清空pvc，但是生产中，经常遇到应用资源意外终止或者其他情况，导致我们的pvc资源没有使用，而且也没有清空

有多种方式解决，最常用的一种方式就是，在所有的应用pod中增加一个prestop的钩子函数，从而让我们的应用资源合理的清空。

而对于特殊的异常情况，我们还有另外一种策略，即强制清空,但是一般不推荐使用。

范例:

```
#在工作中，由于多方面的原因，会导致我们创建好的pv资源偶尔出现无法删除现象，异常效果：
```

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
	STORAGECLASS	REASON	AGE		
pv-nfs-1	5Gi	RWX	Retain	Bound	default/pvc-demo-1
pv-nfs-2	5Gi	ROX	Retain	Available	
pv-nfs-3	1Gi	RWO	Retain	Available	

```
[root@master1 ~]#kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc-demo-1	Bound	pv-nfs-1	5Gi	RWX		5m50s

```
#正常删除
```

```
[root@master1 ~]#kubectl delete -f storage-multi-pv.yaml
```

```
persistentvolume "pv-nfs-1" deleted
```

```
persistentvolume "pv-nfs-2" deleted
```

```
persistentvolume "pv-nfs-3" deleted
```

```
#它会一直处于这种卡死状态
```

```
#显示正在使用PV无法删除
```

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
	STORAGECLASS	REASON	AGE		
pv-nfs-1	5Gi	RWX	Retain	Terminating	default/pvc-demo-1

```
#新一个窗口执行强制删除
```

```
[root@master1 ~]#kubectl delete pv pv-nfs-1 --grace-period=0 --force=true
```

```
#仍然卡住不动
```

```
#对于这种无论何种方法都无法删除的时候，我们可以通过修改配置属性的方式，从记录中删除该信息
```

```
[root@master1 ~]#kubectl patch pv pv-nfs-1 -p '{"metadata":{"finalizers":null}}'
```

```
persistentvolume/pv-nfs-1 patched
```

```
#finalizers属性：
```

```
[root@master1 ~]#kubectl explain pv.metadata.finalizers
```

```
KIND: PersistentVolume
```

```
VERSION: v1
```

FIELD: finalizers <[]string>

DESCRIPTION:

Must be empty before the object is deleted from the registry. Each entry is an identifier **for** the responsible component that will remove the entry from the list. If the deletionTimestamp of the object is non-nil, entries **in** this list can only be removed. Finalizers may be processed and removed **in** any order. Order is NOT enforced because it introduces significant risk of stuck finalizers. finalizers is a shared field, any actor with permission can reorder it. If the finalizer list is processed **in** order, **then** this can lead to a situation **in** which the component responsible **for** the first finalizer **in** the list is waiting **for** a signal (field value, external system, or other) produced by a component responsible **for** a finalizer later **in** the list, resulting **in** a deadlock. Without enforced ordering finalizers are free to order amongst themselves and are not vulnerable to ordering changes **in** the list.

#该属性的含义就是，当我们确定一个pv资源无用且内容为空的时候，我们可以从注册表中删除该信息

#确认删除

```
[root@master1 ~]#kubectl get pv
```

No resources found

#PVC状态显示为Lost

```
[root@master1 ~]#kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc-demo-1	Lost	pv-nfs-1	0			17m

#如果有Pod使用PVC,先删除Pod,再删除PVC,否则会卡住等待

```
[root@master1 ~]#kubectl delete pvc pvc-demo-1
```

1.5.3.3 subPath

<https://kubernetes.io/docs/concepts/storage/volumes/#using-subpath>

<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#using-subpath>

上面范例中的nginx首页存放在/nfsdata的一级目录中，但是生产中，一个NFS共享资源通常是给多个应用来使用的，比如需要定制每个app的分配单独的子目录存放首页资源，但是如果我们采用PV实现定制的方式，就需要多个PV,此方式有些太繁琐了

可以通过subPath实现针对不同的应用对应同一个PVC下不同子目录的挂载

`volumeMounts.subPath` 属性可用于指定所引用的卷内的子路径，而不是其根路径。

下面例子展示了如何配置某包含 LAMP 堆栈（Linux Apache MySQL PHP）的 Pod 使用同一共享卷。此示例中的 `subPath` 配置不建议在生产环境中使用。PHP 应用的代码和相关数据映射到卷的 `html` 文件夹，MySQL 数据库存储在卷的 `mysql` 文件夹中：

#<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#using-subpath>

apiVersion: v1

kind: Pod

metadata:

 name: my-lamp-site

spec:

 containers:

```

- name: mysql
  image: mysql
  env:
    - name: MYSQL_ROOT_PASSWORD
      value: "rootpasswd"
  volumeMounts:
    - mountPath: /var/lib/mysql
      name: site-data
      subPath: mysql
- name: php
  image: php:7.0-apache
  volumeMounts:
    - mountPath: /var/www/html
      name: site-data
      subPath: html
volumes:
- name: site-data
  persistentVolumeClaim:
    claimName: my-lamp-site-data

```

属性解析

#可以直接在容器内部的属性直接实现，而无需对pv和pvc进行变动，这个属性如下：

```
kubectl explain deployment.spec.template.spec.containers.volumeMounts
subPath <string> Path within the volume from which the container's volume should
be mounted. Defaults to "" (volume's root).
```

#属性解析：

subPath的属性主要的目的是设置数据卷的源地址，如果不设置，表示是源地址的根目录，如果设置，则表示是源地址的子目录

范例: 同一个PVC下基于不同的subpath子目录存储不同Pod数据

```

[root@master1 ~]#kubectl apply -f storage-pv.yaml -f storage-pvc.yaml
#创建清单文件
[root@master1 ~]#cat storage-nginx-pvc-subdir.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-nginx-1
spec:
  volumes:
    - name: nginx-volume
      persistentVolumeClaim:
        claimName: pvc-test
  containers:
    - name: nginx-pv
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      volumeMounts:
        - name: nginx-volume
          mountPath: "/usr/share/nginx/html"
          subPath: web1
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-nginx-2

```

```
spec:
  volumes:
    - name: nginx-volume
      persistentVolumeClaim:
        claimName: pvc-test
  containers:
    - name: nginx-flask
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      volumeMounts:
        - name: nginx-volume
          mountPath: "/usr/share/nginx/html"
          subPath: web2
```

```
[root@master1 ~]#kubectl apply -f storage-nginx-pvc-subdir.yaml
```

#在NFS服务器上自动生成子目录

```
[root@master1 ~]#tree /nfsdata/
/nfsdata/
├── index.html
├── web1
└── web2
```

2 directories, 1 file

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES			
pod-nginx-1	1/1	Running	0	20s	172.16.4.51	node2.wang.org
<none>	<none>					
pod-nginx-2	1/1	Running	0	20s	172.16.3.63	node1.wang.org
<none>	<none>					

#默认无法访问页面

```
[root@master1 ~]#curl 172.16.4.51
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.20.0</center>
</body>
</html>
```

```
[root@master1 ~]#curl 172.16.3.63
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.20.0</center>
</body>
</html>
```

#创建页面

```
[root@master1 ~]#echo /nfsdata/web1/index.html > /nfsdata/web1/index.html
[root@master1 ~]#echo /nfsdata/web2/index.html > /nfsdata/web2/index.html
```

#注意：需要稍等1分钟后，再访问成功

```
[root@master1 ~]#curl 172.16.4.51
/nfsdata/web1/index.html
[root@master1 ~]#curl 172.16.3.63
/nfsdata/web2/index.html
```

1.6 StorageClass

1.6.1 StorageClass

1.6.1.1 说明

<https://kubernetes.io/zh-cn/docs/concepts/storage/storage-classes/>

对于 PV 和 PVC 的使用过程是比较繁琐的，不仅需要自己定义PV和PVC还需要将其与Pod进行关联，而且对于PV和PVC的适配我们也要做好前提规划，而生产环境中，这种繁琐的事情是有悖于我们使用kubernetes的原则，而且这种方式在很大程度上并不能满足我们的需求，而且不同的应用程序对于存储性能的要求可能也不尽相同，比如读写速度、并发性能等，比如有一个应用需要对存储的并发度要求比较高，而另外一个应用对读写速度又要求比较高，特别是对于 StatefulSet 类型的应用简单的来使用静态的 PV 就很不合适了，这种情况下就需要用到动态 PV。

Kubernetes 引入了一个新的资源对象：StorageClass，通过 StorageClass 的定义，管理员可以将存储资源定义为某种类型的资源，比如存储质量、快速存储、慢速存储等，为了满足不同用户的多种多样的需求，用户根据 StorageClass 的描述就可以非常直观的知道各种存储资源的具体特性了，这样就可以根据应用的特性去申请合适的存储资源了。

所以,StorageClass提供了一种资源使用的描述方式，使得管理员能够描述提供的存储的服务质量和等级，进而做出不同级别的存储服务和后端策略。

StorageClass 用于定义不同的存储配置和属性，以供 PersistentVolume (PV) 的动态创建和管理。它为开发人员和管理员提供了一种在不同的存储提供商之间抽象出存储配置的方式。

在 Kubernetes 中，StorageClass 是集群级别的资源，而不是名称空间级别。

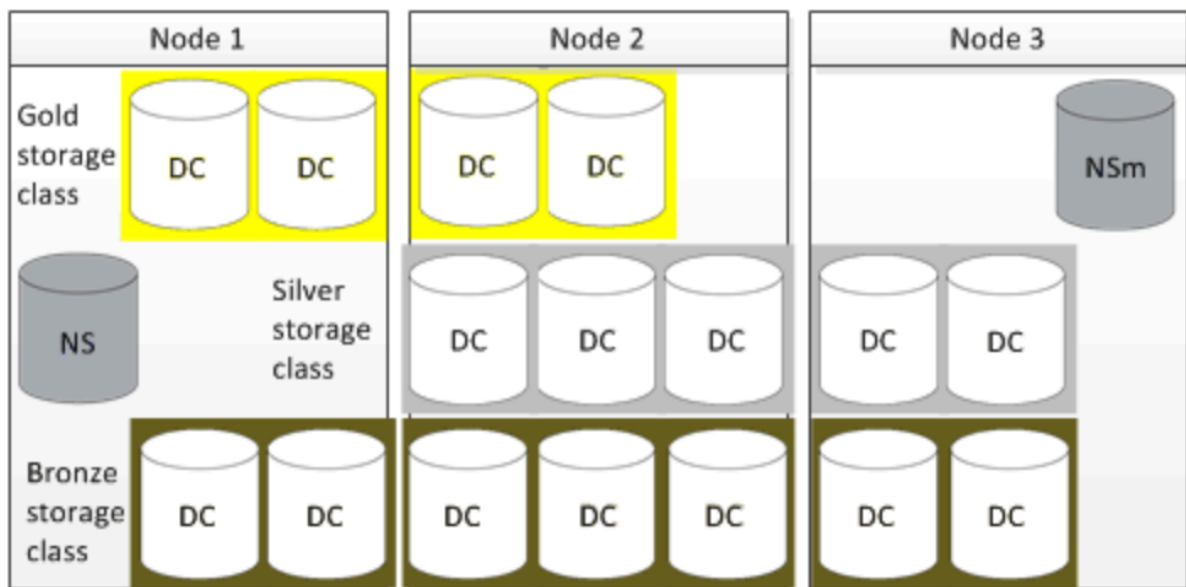
尽管 StorageClass 是集群级别的资源，但是可以在不同的命名空间中使用。

这意味着可以在一个 Kubernetes 集群的不同命名空间中使用相同的 StorageClass 来创建 PersistentVolumeClaim (PVC) 并绑定到 PersistentVolume (PV) 。

PVC和PV可以属于某个SC，也可以不属于任何SC,PVC只能够在同一个storageClass中过滤PV

能建立绑定关系的PVC和PV一定满足如下条件：

- 二者隶属于同个SC
- 二者都不属于任何SC



StorageClass这个API对象可以自动创建PV的机制,即:Dynamic Provisioning

StorageClass对象会定义下面两部分内容:

- PV的属性.比如,存储类型,Volume的大小等
- 创建这种PV需要用到的存储插件

提供以上两个信息,Kubernetes就能够根据用户提交的PVC,找到一个对应的StorageClass,之后Kubernetes就会调用该StorageClass声明的存储插件,进而创建出需要的PV.

要使用 StorageClass, 就得安装对应的自动配置程序, 比如存储后端使用的是 nfs, 那么就需要使用到一个 nfs-client 的自动配置程序, 也称为 Provisioner, 这个程序使用已经配置好的 nfs 服务器, 来自动创建持久卷 PV。

StorageClass API

每个 StorageClass 都包含 `provisioner`、`parameters` 和 `reclaimPolicy` 字段, 这些字段会在 StorageClass 需要动态制备 PersistentVolume 时会使用到。

StorageClass 对象的命名很重要, 用户使用这个命名来请求生成一个特定的类。当创建 StorageClass 对象时, 管理员设置 StorageClass 对象的命名和其他参数。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/aws-ebs #配备器的提供者
parameters:
  type: gp2
reclaimPolicy: Retain
allowVolumeExpansion: true
mountOptions:
  - debug
volumeBindingMode: Immediate | WaitForFirstConsumer (延迟绑定, 只有Pod准备好, 才进行PV和PVC的绑定)

#管理员可以为没有申请绑定到特定 StorageClass 的 PVC 指定一个默认的存储类
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
```

```

accessModes:
  - ReadWriteOnce
volumeMode: Filesystem
resources:
  requests:
    storage: 8Gi
storageClassName: standard
selector:
  matchLabels:
    release: "stable"
  matchExpressions:
    - {key: environment, operator: In, values: [dev]}

```

存储制备器

每个 StorageClass 都有一个制备器（Provisioner），用于提供存储驱动，用来决定使用哪个卷插件制备 PV。该字段必须指定。

<https://kubernetes.io/zh-cn/docs/concepts/storage/storage-classes/>

卷插件	内置制备器	配置示例
AzureFile	✓	Azure File
CephFS	-	-
FC	-	-
FlexVolume	-	-
iSCSI	-	-
NFS	-	NFS
RBD	✓	Ceph RBD
VsphereVolume	✓	vSphere
PortworxVolume	✓	Portworx Volume
Local	-	Local

1.6.1.2 案例

范例：基于 StorageClass 属性字段实现PV和PVC的绑定

```

[root@master1 ~]#cat storage-multi-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-1
spec:
  storageClassName: mystorage      #指定 StorageClass 的名称
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:

```

```
- ReadWriteMany
persistentVolumeReclaimPolicy: Retain
nfs:
  path:  "/nfsdata/data1"
  server: nfs.wang.org
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-2
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path:  "/nfsdata/data2"
    server: nfs.wang.org
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-3
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path:  "/nfsdata/data3"
    server: nfs.wang.org
```

```
[root@master1 ~]#cat  storage-multi-pvc.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo-1
  namespace: default
spec:
  storageClassName: mystorage
  accessModes: ["ReadWriteMany"]
  volumeMode: Filesystem
  resources:
    requests:
      storage: 3Gi
    limits:
      storage: 10Gi
```

#指定 StorageClass 的名称，和上面的PV要

相同，才能绑定

```
[root@master1 ~]#kubectl apply -f storage-multi-pv.yaml -f storage-multi-
pvc.yaml
```

```
[root@master1 ~]#kubectl get pv,pvc
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
persistentvolume/pv-nfs-1	5Gi	RWX	Retain	Bound
default/pvc-demo-1	mystorage	<unset>		13m
persistentvolume/pv-nfs-2	5Gi	RWX	Retain	Available
		<unset>		13m
persistentvolume/pv-nfs-3	1Gi	RWO	Retain	Available
		<unset>		13m

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE		
persistentvolumeclaim/pvc-demo-1	Bound	pv-nfs-1	5Gi	RWX
mystorage	<unset>	13m		


```
#没有创建独立的StorageClass资源
[root@master1 storage]#kubectl get sc
No resources found
```

1.6.2 Local Volume

<https://kubernetes.io/zh-cn/docs/concepts/storage/storage-classes/#local>
<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#local>

1.6.2.1 hostPath 存在的问题

过去我们经常会通过hostPath volume让Pod能够使用本地存储，将Node文件系统中的文件或者目录挂载到容器内，但是hostPath volume的使用是很不方便的，并不适合在生产环境中使用。

- 由于集群内每个节点的差异化，要使用hostPath Volume，我们需要通过NodeSelector等方式进行精确调度，比较繁琐。
- 注意DirectoryOrCreate和FileOrCreate两种类型的hostPath，当Node上没有对应的File/Directory时，你需要保证kubelet有在Node上Create File/Directory的权限。
- 另外，如果Node上的文件或目录是由root创建的，挂载到容器内之后，你通常还要保证容器内进程有权限对该文件或者目录进行写入，比如你需要以root用户启动进程并运行于privileged容器，或者你需要事先修改好Node上的文件权限配置。
- Scheduler并不会考虑hostPath volume的大小，hostPath也不能申明需要的storagesize，这样调度时存储的考虑，就需要人为检查并保证。

1.6.2.2 Local PV 使用场景

Local Persistent Volume 并不适用于所有应用。它的适用范围非常固定，比如：高优先级的系统应用，需要在多个不同节点上存储数据，而且对 I/O 要求较高。Kubernetes 直接使用宿主机的本地磁盘目录，来持久化存储容器的数据。它的读写性能相比于大多数远程存储来说，要好得多，尤其是 SSD 盘。

典型的应用包括：分布式数据存储比如 MongoDB，分布式文件系统比如 GlusterFS、Ceph 等，以及需要在本地磁盘上进行大量数据缓存的分布式应用，其次使用 Local Persistent Volume 的应用必须具备数据备份和恢复的能力，允许你把这些数据定时备份在其他位置。

1.6.2.3 Local PV 的实现

LocalPV 的实现可以理解为我们前面使用的 hostpath 加上 nodeAffinity，比如：在宿主机 NodeA 上提前创建好目录，然后在定义 Pod 时添加 nodeAffinity=NodeA，指定 Pod 在我们提前创建好目录的主机上运行。但是我们绝不应该把一个宿主机上的目录当作 PV 使用，因为本地目录的磁盘随时都可能被应用写满，甚至造成整个宿主机宕机。而且，不同的本地目录之间也缺乏哪怕最基础的 I/O 隔离机

制。所以，一个 Local Persistent Volume 对应的存储介质，一定是一块额外挂载在宿主机的磁盘或者块设备（“额外”的意思是，它不应该是宿主机根目录所使用的主硬盘）。这个原则，我们可以称为“一个 PV 一块盘”。

1.6.2.4 Local PV 和常规 PV 的区别

对于常规的 PV，Kubernetes 都是先调度 Pod 到某个节点上，然后再持久化“这台机器上的 Volume 目录。而 Local PV，则需要运维人员提前准备好节点的磁盘。它们在不同节点上的挂载情况可以完全不同，甚至有的节点可以没这种磁盘。所以调度器就必须能够知道所有节点与 Local Persistent Volume 对应的磁盘的关联关系，然后根据这个信息来调度 Pod。也就是在调度的时候考虑 Volume 分布。

k8s v1.10+以上的版本中推出local pv方案。Local volume 允许用户通过标准 PVC 接口以简单且可移植的方式访问 node 节点的本地存储。PV 的定义中需要包含描述节点亲和性的信息，k8s 系统则使用该信息将容器调度到正确的 node 节点。

`local volume` 所代表的是某个被挂载的本地存储设备，例如磁盘、分区或者目录。

在 Kubernetes 中，HostPath 和 Local Volume 都可以用于将主机上的文件系统挂载到容器内部。虽然它们有一些相似之处，但是它们之间也有一些重要的区别。

HostPath卷类型会直接挂载主机的文件系统到Pod中，这个文件系统可以是一个文件或者是一个目录。当Pod被调度到一个节点上时，该节点上的文件系统就会被挂载到Pod中。这使得可以很容易地在容器内部访问主机上的文件，例如主机上的日志或配置文件。但是，使用 HostPath 卷类型可能会存在安全风险，因为容器可以访问主机上的所有文件和目录，包括其他容器的文件。

相比之下，Local Volume 卷类型只能将节点上的一个目录挂载到容器内部。当Pod被调度到一个节点上时，Kubernetes 会为该节点创建一个唯一的目录，并将该目录挂载到 Pod 中。因为每个 Pod 只能访问其本地的 Local Volume 目录，所以这种卷类型更加安全。但是，如果节点故障或被删除，Local Volume 中的数据将会丢失。因此，使用 Local Volume 卷类型需要谨慎，需要确保有备份机制或持久化存储。

`local volume` 默认不支持动态配置，只能用作静态创建的持久卷。但可以采用第三方方案实现动态配置

`local` 类型的PV是一种更高级的本地存储抽象，它可以允许通过StorageClass来进行管理。

与 `hostPath` 卷相比，`local` 卷能够以持久和可移植的方式使用，而无需手动将 Pod 调度到节点。

同样使用节点上的本地存储，但相比于 `hostPath`，`local` Volume可以声明为动态供应，并且可以利用节点亲和性标签（`nodeAffinity`）实现存储亲和性，确保Pod调度到包含所需数据的节点上。而 `hostPath`卷在Pod重建后可能会调度至新的节点，而导致旧的数据无法使用

可以通过查看 `PersistentVolume` 的节点亲和性配置，就能了解卷的节点约束。

然而，`local` 卷仍然取决于底层节点的可用性，并不适合所有应用程序。如果节点变得不健康，那么 `local` 卷也将变得不可被 Pod 访问。使用它的 Pod 将不能运行。使用 `local` 卷的应用程序必须能够容忍这种可用性的降低，以及因底层磁盘的耐用性特征而带来的潜在的数据丢失风险。

Local PV 优势

- 支持指定PV的存储大小，而HostPath不支持
- Pod调度至哪个节点由PV所在的节点决定，HostPath是由Pod决定HostPath所在节点
- 由存储管理员指定PV所在的节点做为Pod运行的节点，而非由用户决定

1.6.2.5 创建 Local PV

使用 Local 卷流程

- 创建PV，使用 nodeAffinity 指定绑定的节点提供存储
- 创建 PVC，绑定PV的存储条件
- 创建Pod，引用前面的PVC和PV实现Local 存储

下面是一个使用 `local` 卷和 `nodeAffinity` 的持久卷示例：

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner #表示该存储类不使用任何provisioner，即不支持
动态分配持久卷。这意味着管理员需要手动创建并管理持久卷
volumeBindingMode: WaitForFirstConsumer #延迟绑定，只有Pod准备好才绑定PV至PVC，否则
PVC处于Pending状态

---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: example-pv
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd1
  nodeAffinity: #节点亲和性
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - example-node

---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: local-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: local-storage
  #selector: # 支持使用指定的PV
  # matchLabels:
  #   pv: example-pv
```

使用 `local` 卷时，你需要设置 `PersistentVolume` 对象的 `nodeAffinity` 字段。Kubernetes 调度器使用 `PersistentVolume` 的 `nodeAffinity` 信息来将使用 `local` 卷的 Pod 调度到正确的节点。

`PersistentVolume` 对象的 `volumeMode` 字段可被设置为 "Block"（而不是默认值 "Filesystem"），以将 `local` 卷作为原始块设备暴露出来。

使用 `local` 卷时，建议创建一个 `StorageClass` 并将其 `volumeBindingMode` 设置为 `WaitForFirstConsumer`。要了解更多信息，请参考 [local StorageClass 示例](#)。延迟卷绑定的操作可以确保 Kubernetes 在为 `PersistentVolumeClaim` 作出绑定决策时，会评估 Pod 可能具有的其他节点约束，例如：如节点资源需求、节点选择器、Pod 亲和性和 Pod 反亲和性。

1.6.2.6 案例

1.6.2.6.1 范例：基于 `StorageClass` 实现 `Local` 卷

范例：

```
#事先在PV所在的目标节点上准备目录，对于本地存储Kubernetes本身并不会自动创建路径，这是因为
Kubernetes不能控制节点上的本地存储，因此无法自动创建路径
[root@node2 ~]#mkdir -p /data/mysql

[root@master1 ~]#cat storage-sc-local-pv-pvc-mysql-pod.yaml
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer #延迟绑定，只有Pod启动后再绑定PV到Pod所在节点，否则PVC处理Pending状态

---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-sc-local
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete #因为静态置配，所以当PVC删除后，不会删除PV和数据
  storageClassName: local-storage
  local:
    path: /data/mysql
  nodeAffinity:
    required:
      nodeSelectorTerms: #指定节点
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - node2.wang.org

---
```

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-sc-local
spec:
  storageClassName: local-storage
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 100Gi

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
          name: mysql
          env:
            # 在实际中使用 secret
            - name: MYSQL_ROOT_PASSWORD
              value: "123456"
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: pvc-sc-local

```

```
[root@master1 ~]#kubectl apply -f storage-sc-local-pv-pvc-mysql-pod.yaml
```

```
[root@master1 ~]#kubectl get sc
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
local-storage	kubernetes.io/no-provisioner	Delete	
waitForFirstConsumer	false	105s	

```
[root@master1 ~]#kubectl get pod,pvc,pv
```

NAME	READY	STATUS	RESTARTS	AGE
pod/mysql-754f87ddfd-n46wc	1/1	Running	0	57s

NAME	STATUS	VOLUME	CAPACITY	ACCESS
MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE				
persistentvolumeclaim/pvc-sc-local	Bound	pv-sc-local	100Gi	RWO
local-storage <unset>		57s		

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
persistentvolume/pv-sc-local	100Gi	RWO	Delete	Bound
default/pvc-sc-local	local-storage	<unset>		57s

范例

#事先在PV所在的目标节点上准备目录，对于本地存储Kubernetes本身并不会自动创建路径，这是因为Kubernetes不能控制节点上的本地存储，因此无法自动创建路径

```
[root@node2 ~]#mkdir -p /data/www
```

#如果没有准备目录，会出现下面提示错误

```
[root@master1 ~]#kubectl describe pod pod-sc-local-demo
```

Events:

Type	Reason	Age	From	Message
Warning	FailedScheduling	2m2s	default-scheduler	0/4 nodes are available: pod has unbound immediate PersistentVolumeClaims. preemption: 0/4 nodes are available: 4 No preemption victims found for incoming pod..
Normal	Scheduled	2m1s	default-scheduler	Successfully assigned default/pod-sc-local-demo to node2.wang.org
Warning	FailedMount	56s (x8 over 2m)	kubelet	MountVolume.NewMounter initialization failed for volume "example-pv" : path "/data/www" does not exist

#准备清单文件

```
[root@master1 ~]#cat storage-sc-local-pv-pvc-pod.yaml
```

#kubernetes内置了Local的置备器，所以下面StorageClass资源可以不创建

```
---
```

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
  name: local-storage
```

```
provisioner: kubernetes.io/no-provisioner
```

```
volumeBindingMode: WaitForFirstConsumer #延迟绑定，只有Pod启动后再绑定PV到Pod所在节点，否则PVC处理Pending状态
```

```
---
```

```
apiVersion: v1
```

```
kind: PersistentVolume
```

```
metadata:
```

```
  name: pv-sc-local
```

```
spec:
```

```
  capacity:
```

```
    storage: 100Gi
```

```
  volumeMode: Filesystem
```

```
  accessModes:
```

```
  - ReadWriteOnce
```

```
  persistentVolumeReclaimPolicy: Delete
```

```
  storageClassName: local-storage
```

```

local:
  path: /data/www/
nodeAffinity:
  required:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - node2.wang.org

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-sc-local
spec:
  storageClassName: local-storage
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 100Mi

---
apiVersion: v1
kind: Pod
metadata:
  name: pod-sc-local-demo
spec:
  containers:
    - name: pod-sc-local-demo
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      volumeMounts:
        - name: pvc-sc-local
          mountPath: "/usr/share/nginx/html/"
  restartPolicy: "Never"
  volumes:
    - name: pvc-sc-local
      persistentVolumeClaim:
        claimName: pvc-sc-local

```

#应用清单文件

```

[root@master1 ~]#kubectl apply -f storage-sc-local-pv-pvc-pod.yaml
persistentvolume/pv-sc-local created
persistentvolumeclaim/pvc-sc-local created
pod/pod-sc-local-demo created

```

#查看结果

```

[root@master1 ~]#kubectl get sc,pv,pvc,pod
NAME                                PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION  AGE
storageclass.storage.k8s.io/local-storage kubernetes.io/no-provisioner Delete
waitForFirstConsumer false 14s

```

```

NAME      CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
STORAGECLASS  REASON  AGE

```

```

persistentvolume/pv-sc-local 100Gi RWO Delete Bound default/pvc-sc-
local local-storage 2m31s

NAME                                STATUS  VOLUME          CAPACITY  ACCESS
MODES  STORAGECLASS  AGE
persistentvolumeclaim/pvc-sc-local Bound    pv-sc-local  100Gi     RWO
local-storage 2m31s

NAME                                READY  STATUS  RESTARTS
AGE
pod/pod-sc-local-demo              1/1    Running  0
2m31s

[root@master1 ~]#kubectl get pod -o wide
NAME    READY  STATUS  RESTARTS  AGE  IP          NODE          NOMINATED NODE
READINESS GATES
pod-sc-local-demo 1/1    Running  0         117s  10.244.36.1 node2.wang.org
<none>      <none>

#默认没有页面文件，提示如下
[root@master1 ~]#curl 10.244.36.1
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.20.0</center>
</body>
</html>

#在提供存储服务的节点准备页面文件
[root@node2 ~]#hostname > /data/www/index.html

#再次访问
[root@master1 ~]#curl 10.244.36.1
node2.wang.org

#清理环境
[root@master1 ~]#kubectl delete -f storage-sc-local-pv-pvc-pod.yaml
storageclass.storage.k8s.io "local-storage" deleted
persistentvolume "pv-sc-local" deleted
persistentvolumeclaim "pvc-sc-local" deleted
pod "pod-sc-local-demo" deleted

[root@node2 ~]#rm -f /data/www/index.html

```

1.6.2.6.2 案例2

创建数据目录

创建 Local PV 其实应该给宿主机挂载并格式化一个可用的磁盘，这里我们就在宿主机上挂载几个 RAM Disk（内存盘）来模拟本地磁盘。

```
$ mkdir /mnt/disks
```

如果希望其他节点也能支持 Local Persistent Volume 的话，那就需要为它们也执行上述操作，可以创建多个目录供pv挂载。

创建StorageClass

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
```

provisioner 字段定义为 no-provisioner，这是因为 Local Persistent Volume 目前尚不支持 Dynamic Provisioning 动态生成 PV，所以我们需要提前手动创建 PV。

volumeBindingMode 字段定义为 WaitForFirstConsumer，它是 Local Persistent Volume 里一个非常重要的特性，即：延迟绑定。延迟绑定就是在我们提交 PVC 文件时，StorageClass 为我们延迟绑定 PV 与 PVC 的对应关系。

这样做的原因是：比如我们在当前集群上有两个相同属性的 PV，它们分布在不同的节点 Node1 和 Node2 上，而我们定义的 Pod 需要运行在 Node1 节点上，但是 StorageClass 已经为 Pod 声明的 PVC 绑定了在 Node2 上的 PV，这样的话，Pod 调度就会失败，所以我们要延迟 StorageClass 的绑定操作。

也就是延迟到第一个声明使用该 PVC 的 Pod 出现在调度器之后，调度器再综合考虑所有的调度规则，当然也包括每个 PV 所在的节点位置，来统一决定，这个 Pod 声明的 PVC，到底应该跟哪个 PV 进行绑定。

比如上面的 Pod 需要运行在 node1 节点上，StorageClass 发现可以绑定的 PV 后，先不为 Pod 中的 PVC 绑定 PV，而是等到 Pod 调度到 node1 节点后，再为 PVC 绑定当前节点运行的 PV。

所以，通过这个延迟绑定机制，原本实时发生的 PVC 和 PV 的绑定过程，就被延迟到了 Pod 第一次调度的时候在调度器中进行，从而保证了这个绑定结果不会影响 Pod 的正常调度。

现在我们创建 StorageClass

```
$ kubectl create -f localpv-storageclass.yaml
storageclass.storage.k8s.io/local-storage created
```

创建PV资源

我们创建4个pv资源，位于work1和work2节点上各两个，pv分布详情如下

pv名称	所在主机	大小	路径
local-pv1	work1	10Gi	/mnt/data1
local-pv2	work1	10Gi	/mnt/data2
local-pv3	work2	50Gi	/mnt/data1
local-pv4	work2	50Gi	/mnt/data2

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv1
  labels:
    pv: local-pv1
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
```

```

accessModes:
- ReadWriteOnce
persistentVolumeReclaimPolicy: Delete
storageClassName: local-storage
local:
  path: /mnt/data1
nodeAffinity:
  required:
    nodeSelectorTerms:
    - matchExpressions:
      - key: kubernetes.io/hostname
        operator: In
        values:
        - work1
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv2
  labels:
    pv: local-pv2
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/data2
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
          - work1
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv3
  labels:
    pv: local-pv3
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/data1
  nodeAffinity:
    required:

```

```

      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
            - work2
    ---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv4
  labels:
    pv: local-pv4
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/data2
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
            - work2

```

上面定义的 local 字段，指定了它是一个 Local Persistent Volume；而 path 字段，指定的正是这个 PV 对应的本地磁盘的路径，即：/mnt/data。而这个磁盘存在于指定的节点上，也就意味着 Pod 使用这个 PV 就必须运行在指定的节点上。所以 nodeAffinity 字段就指定节点的名字，声明 PV 与节点的对应关系。这正是 Kubernetes 实现“在调度的时候就考虑 Volume 分布”的主要方法。

- 创建PV

```

[root@master1 test]# kubectl apply -f pv.yaml
persistentvolume/local-pv1 created
persistentvolume/local-pv2 created
persistentvolume/local-pv3 created
persistentvolume/local-pv4 created
[root@master1 ~]# kubectl get pv

```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
local-pv1	10Gi	RWO	Delete	Available	
	local-storage		16s		
local-pv2	50Gi	RWO	Delete	Available	
	local-storage		16s		
local-pv3	10Gi	RWO	Delete	Available	
	local-storage		16s		
local-pv4	50Gi	RWO	Delete	Available	
	local-storage		16s		

使用local pv

Pod通过pvc使用

- 创建pvc

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: local-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: local-storage
  selector: # 指定使用local-pv3
    matchLabels:
      pv: local-pv3
```

- 查看pvc

```
[root@master1 ~]# kubectl apply -f pod-pvc.yaml
persistentvolumeclaim/local-pvc created
[root@master1 ~]# kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
local-pvc	Pending				local-storage	3s

```
[root@master1 ~]# kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
local-pv1	10Gi	RWO	Delete	Available	
	local-storage		91s		
local-pv2	50Gi	RWO	Delete	Available	
	local-storage		91s		
local-pv3	10Gi	RWO	Delete	Available	
	local-storage		91s		
local-pv4	50Gi	RWO	Delete	Available	
	local-storage		91s		

可以看到当前 PVC 的状态为 Pending, PV 与 PVC 也没有建立绑定关系。

- 创建pod

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.22.0
      resources:
        limits:
          memory: "128Mi"
          cpu: "500m"
      ports:
        - containerPort: 80
```

```

volumeMounts:
  - name: www
    mountPath: /usr/share/nginx/html
volumes:
  - name: www
    persistentVolumeClaim:
      claimName: local-pvc

```

- [查看资源](#)

```

[root@master1 ~]# kubectl apply -f nginx-pod.yaml
pod/nginx created
[root@master1 ~]# kubectl get pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE
nginx     1/1     Running   0           4s    10.244.4.73   work2     <none>
<none>
[root@master1 ~]# kubectl get pvc
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
local-pvc     Bound    local-pv3  10Gi       RWO             local-storage   108s
[root@master1 ~]# kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
local-pv1     10Gi       RWO            Delete           Available
local-storage 3m15s
local-pv2     50Gi       RWO            Delete           Available
local-storage 3m15s
local-pv3     10Gi       RWO            Delete           Bound
default/local-pvc local-storage 3m15s
local-pv4     50Gi       RWO            Delete           Available
local-storage 3m15s

```

可以看到pod已成功调度至work2节点，因为指定使用local-pvc资源，而local-pvc资源绑定到了local-pv3，local-pv3就位于work2节点上。

StatefulSet通过pvc使用

- 创建pvc

创建的时候注意pvc的名字的构成：pvc的名字 = volume_name-statefulset_name-序号
例如挂载名称为www，statefulset名称为nginx，副本数为2。创建的pvc资源名分别是www-nginx-0、www-nginx-1

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: www-nginx-0
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: local-storage
  selector:
    matchLabels:
      pv: local-pv1

```

```

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: www-nginx-1
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: local-storage
  selector:
    matchLabels:
      pv: local-pv3
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: log-nginx-0
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
  storageClassName: local-storage
  selector:
    matchLabels:
      pv: local-pv2
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: log-nginx-1
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
  storageClassName: local-storage
  selector:
    matchLabels:
      pv: local-pv4

```

- 创建资源

```
[root@master1 ~]# kubectl apply -f pvc.yaml
persistentvolumeclaim/local-pvc-1 created
persistentvolumeclaim/local-pvc-2 created
persistentvolumeclaim/local-pvc-3 created
persistentvolumeclaim/local-pvc-4 created
[root@master1 ~]# kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	AGE	
local-pvc-1	Pending		
		local-storage	13s
local-pvc-2	Pending		
		local-storage	13s
local-pvc-3	Pending		
		local-storage	13s
local-pvc-4	Pending		
		local-storage	13s

可以看到当前 PVC 的状态为 Pending，PV 与 PVC 也没有建立绑定关系。

- 创建statefulset资源

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  ports:
    - port: 80
      name: web
  clusterIP: None
  selector:
    app: nginx
---
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: nginx
spec:
  selector:
    matchLabels:
      app: myapp
  serviceName: nginx
  replicas: 2
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.22.0
          ports:
            - containerPort: 80
              name: web
          volumeMounts:
```

```

- name: www
  mountPath: /usr/share/nginx/html
- name: log
  mountPath: /var/log/nginx
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "local-storage"
    resources:
      requests:
        storage: 10Gi
- metadata:
  name: log
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "local-storage"
    resources:
      requests:
        storage: 50Gi

```

- 创建并查看资源信息

```

[root@master1 ~]# kubectl apply -f nginx.yaml
service/nginx created
statefulset.apps/nginx created
[root@master1 ~]# kubectl get pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-0	1/1	Running	0	6s	10.244.5.77	work1	<none>	
nginx-1	1/1	Running	0	4s	10.244.4.74	work2	<none>	

```

[root@master1 ~]# kubectl get pvc

```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
log-nginx-0	Bound	local-pv2	50Gi	RWO	local-storage	27s
log-nginx-1	Bound	local-pv4	50Gi	RWO	local-storage	27s
www-nginx-0	Bound	local-pv1	10Gi	RWO	local-storage	27s
www-nginx-1	Bound	local-pv3	10Gi	RWO	local-storage	27s

```

[root@master1 ~]# kubectl get pv

```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
local-pv1	10Gi	RWO	Delete	Bound	default/www-
nginx-0	local-storage		114s		
local-pv2	50Gi	RWO	Delete	Bound	default/log-
nginx-0	local-storage		114s		
local-pv3	10Gi	RWO	Delete	Bound	default/www-
nginx-1	local-storage		114s		
local-pv4	50Gi	RWO	Delete	Bound	default/log-
nginx-1	local-storage		114s		

可以看到 Pod 分别调度在work1和work2节点并成功运行后，PVC 和 PV 的状态已经 Bound 绑定。

验证

验证pod与pv节点绑定

- 查看资源信息，nginx-0调度至work1节点，nginx-1调度至work2节点。

```
[root@master1 ~]# kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
nginx-0	1/1	Running	0	4m38s	10.244.5.77	work1	<none>
nginx-1	1/1	Running	0	4m36s	10.244.4.74	work2	<none>

```
[root@master1 ~]# kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
log-nginx-0	Bound	local-pv2	50Gi	RWO	local-storage	5m6s
log-nginx-1	Bound	local-pv4	50Gi	RWO	local-storage	5m6s
www-nginx-0	Bound	local-pv1	10Gi	RWO	local-storage	5m6s
www-nginx-1	Bound	local-pv3	10Gi	RWO	local-storage	5m6s

- 删除pod，让k8s重新调度，看是否能和之前一样调度至原来的节点

```
[root@master1 ~]# kubectl delete pod `kubectl get pod | grep nginx | awk '{ print $1 }'`
```

pod "nginx-0" deleted
pod "nginx-1" deleted

```
[root@master1 ~]# kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-0	1/1	Running	0	4s	10.244.5.79	work1	<none>	
nginx-1	1/1	Running	0	2s	10.244.4.76	work2	<none>	

多次删除后发现pod依然可以调度至先前的节点。

验证文件持久化

现在验证文件是否可以持久化存储，进入当前这个 Pod 的挂载目录新建一个测试文件

```
[root@master1 ~]# kubectl exec -it nginx-0 -- bash
```

root@nginx-0:/# cd /usr/share/nginx/html/
root@nginx-0:/usr/share/nginx/html# echo "hello" > test.html
root@nginx-0:/usr/share/nginx/html# cat test.html
hello

在work1的挂载目录上查看是否创建

```
[root@work1 ~]# cd /mnt/data1/
```

[root@work1 data1]# ls
test.html
[root@work1 data1]# cat test.html
hello

现在我们删除或者重建这个 Pod，进入容器验证文件是否存在

```
[root@master1 ~]# kubectl delete pod nginx-0
pod "nginx-0" deleted
[root@master1 ~]# kubectl exec -it nginx-0 -- bash
root@nginx-0:/# cd /usr/share/nginx/html/
root@nginx-0:/usr/share/nginx/html# ls
test.html
root@nginx-0:/usr/share/nginx/html# cat test.html
hello
```

可以看到文件是依旧存在的，这也说明，像 Kubernetes 这样构建出来的、基于本地存储的 Volume，完全可以提供容器持久化存储的功能。所以，像 StatefulSet 这样的有状态编排工具，也完全可以通过声明 Local 类型的 PV 和 PVC，来管理应用的存储状态。

删除local PV

需要注意的是，我们上面手动创建 PV 的方式，在删除 PV 时需要按如下流程执行操作：

- 删除使用这个 PV 的 Pod
- 从宿主机移除本地磁盘（比如，umount 它）
- 删除 PVC
- 删除 PV

如果不按照这个流程的话，这个 PV 的删除就会失败。

1.6.3 NFS StorageClass

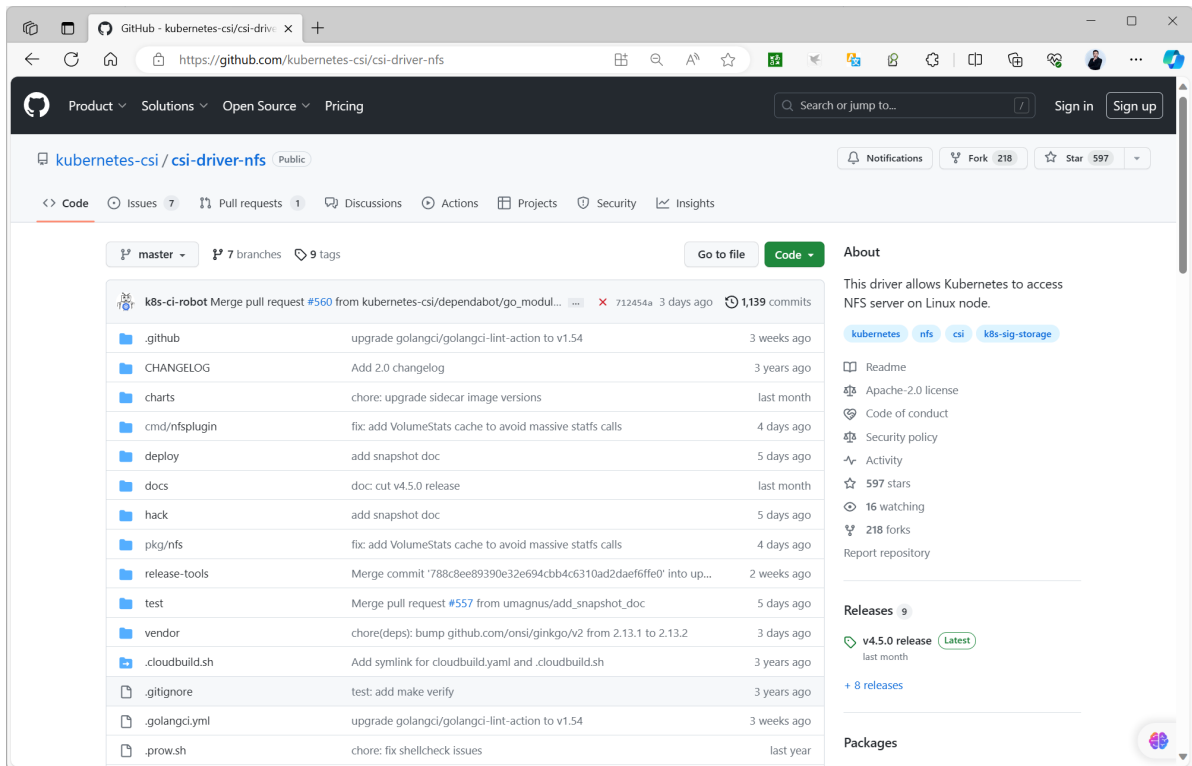
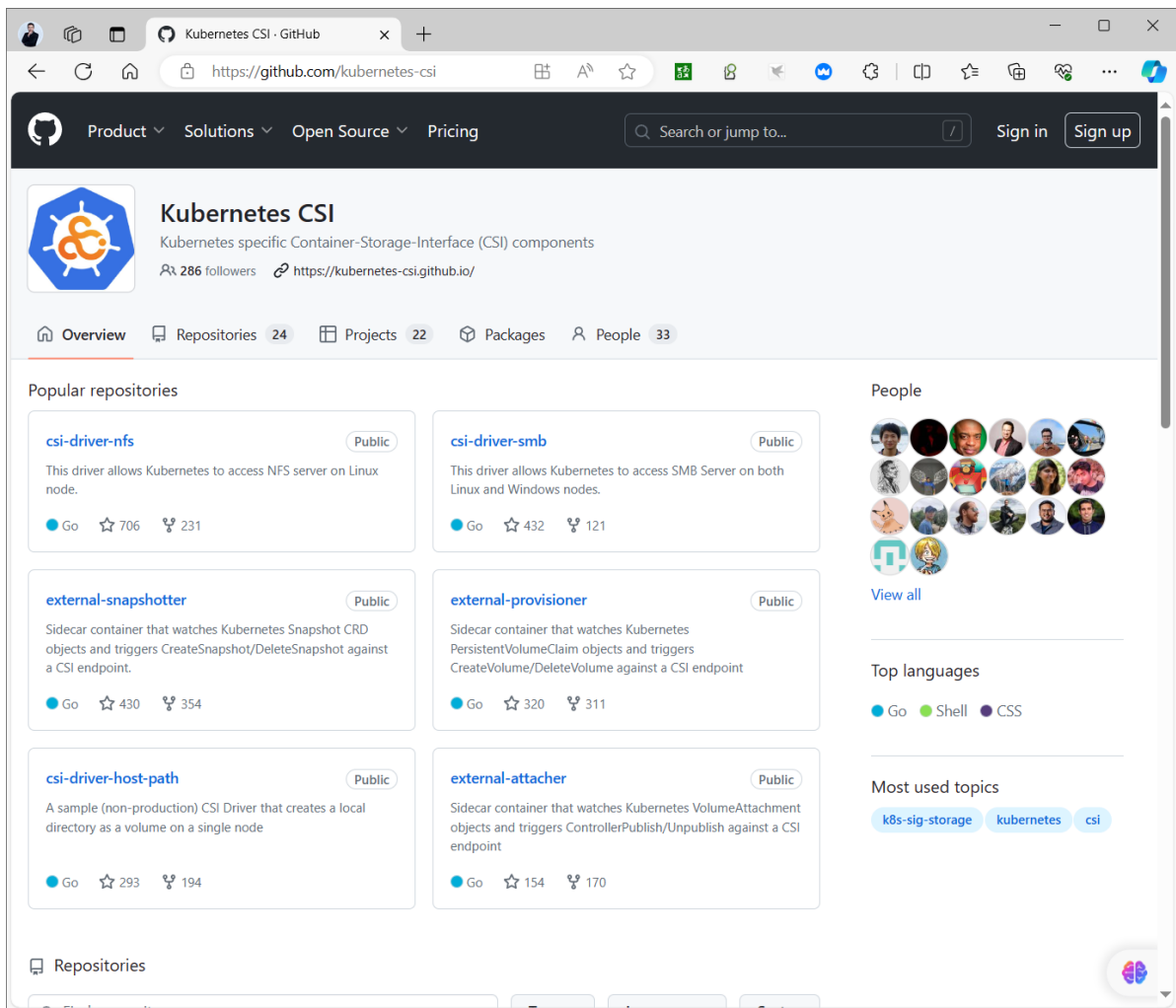
1.6.3.1 NFS的存储制备器方案

NFS 的自动配置程序 Provisioner 可以通过不同的项目实现,比如

```
https://kubernetes.io/zh-cn/docs/concepts/storage/storage-classes/#nfs
```

- csi-driver-nfs

```
https://github.com/kubernetes-csi/csi-driver-nfs
```



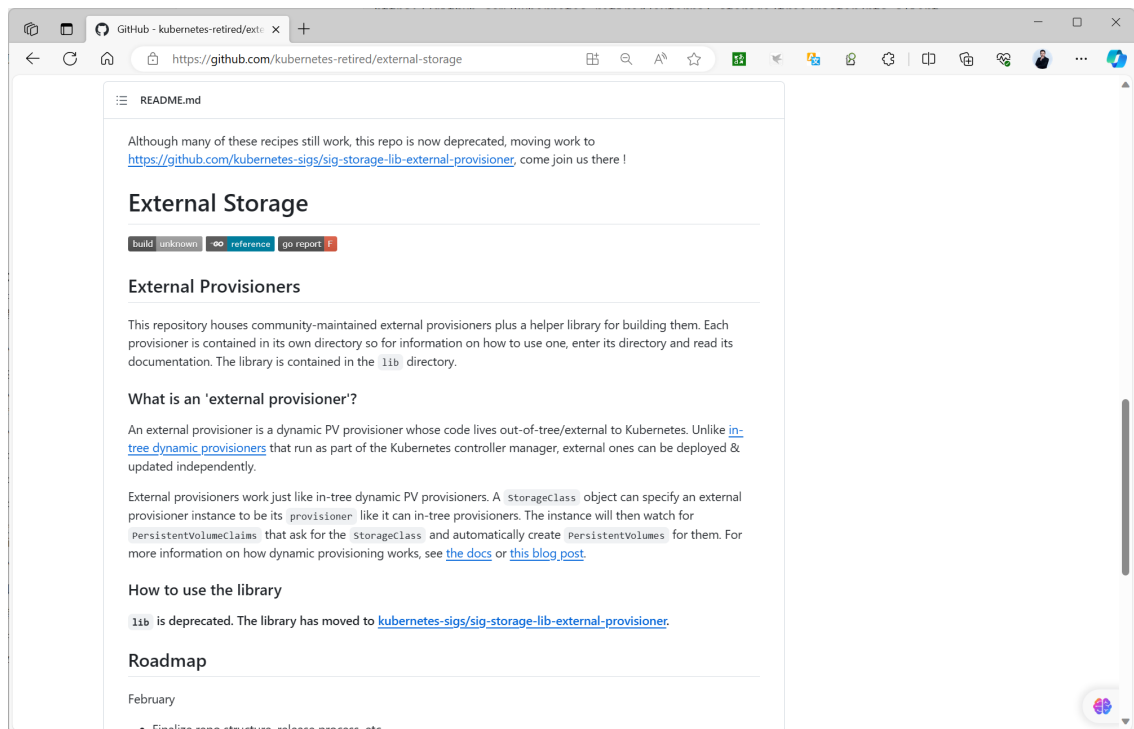
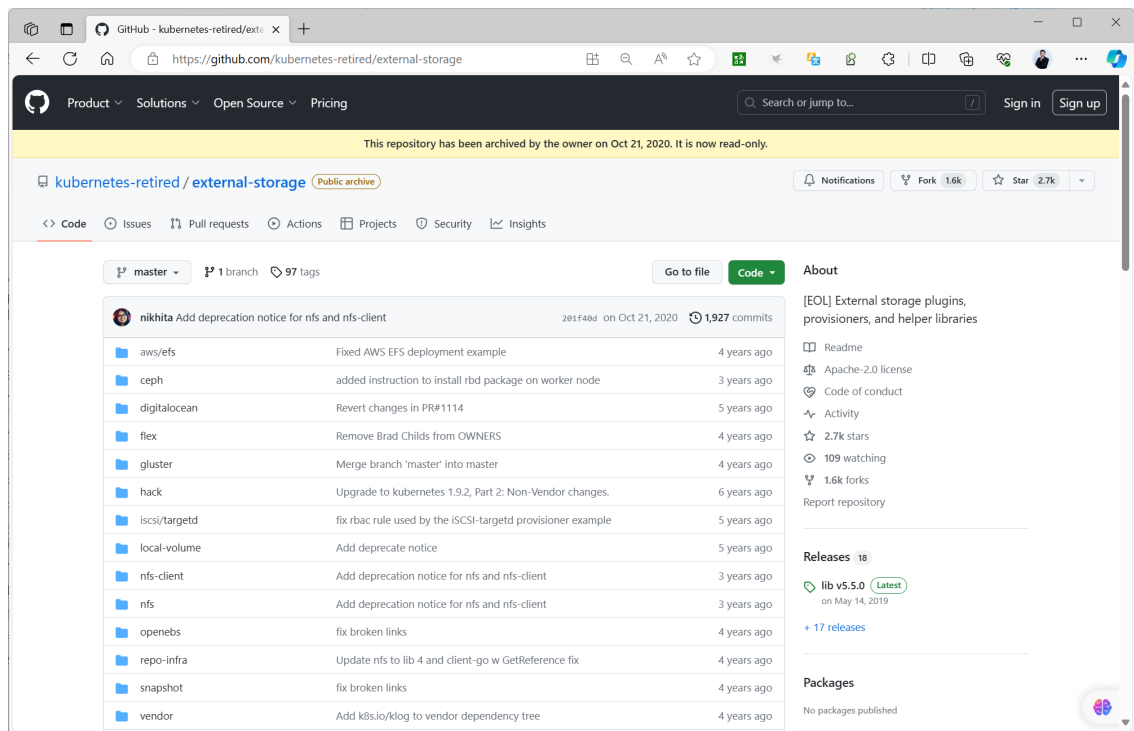
- nfs-client-provisioner

nfs-client-provisioner 是一个自动配置卷程序，它使用现有的和已配置的 NFS 服务器来支持通过 PVC动态配置 PV

nfs-client-provisioner 目前已经不提供更新，nfs-client-provisioner 的 Github 仓库当前已经迁移到 NFS-Subdir-External-Provisioner的仓库

<https://github.com/kubernetes-retired/external-storage/tree/master/nfs-client>

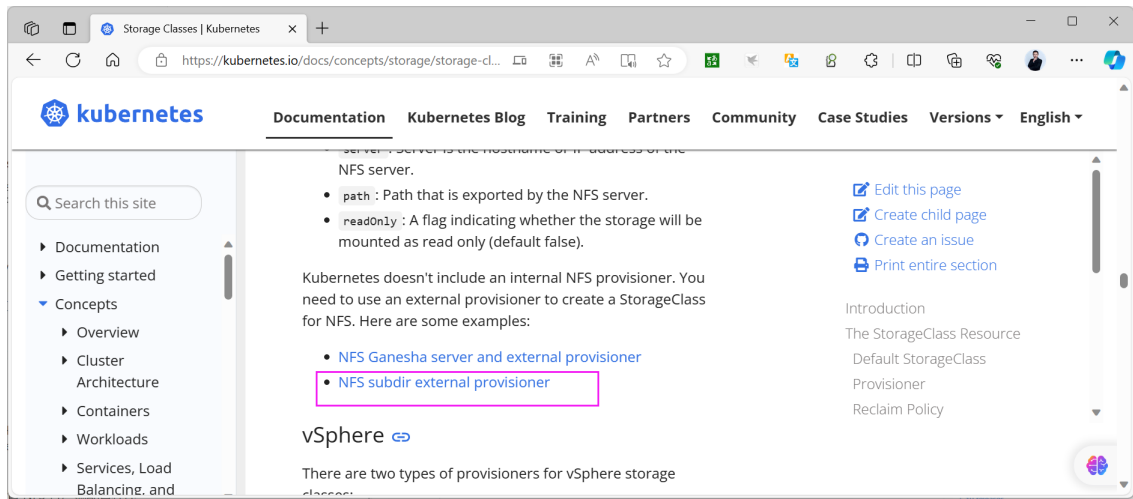
<https://github.com/kubernetes-sigs/sig-storage-lib-external-provisioner>



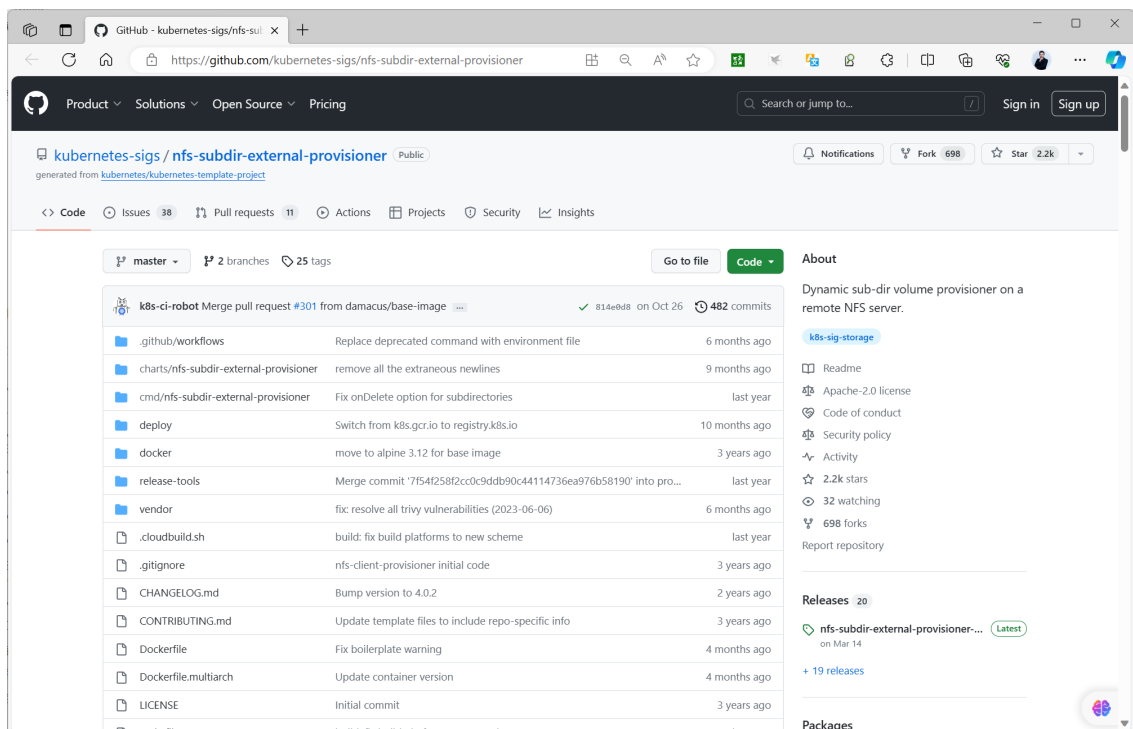
- NFS-Subdir-External-Provisioner

此组件是由Kubernetes SIGs 社区开发,也是Kubernetes官方推荐实现
是对 nfs-client-provisioner 组件的扩展

<https://kubernetes.io/docs/concepts/storage/storage-classes/#nfs>



<https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner>
<https://github.com/kubernetes-sigs/sig-storage-lib-external-provisioner>



NFS-Subdir-External-Provisioner 是一个自动配置卷程序，可以在 NFS 服务器上通过PVC动态创建和配置 Kubernetes 持久卷

PV命名规则如下

自动创建的 PV 以 `${namespace}-${pvcName}-${pvName}` 命名格式创建在 NFS 服务器上的共享数据目录中
当这个 PV 被回收后会以 `archieved-${namespace}-${pvcName}-${pvName}` 命名格式存在 NFS 服务器中

- NFS Ganesha server and external provisioner

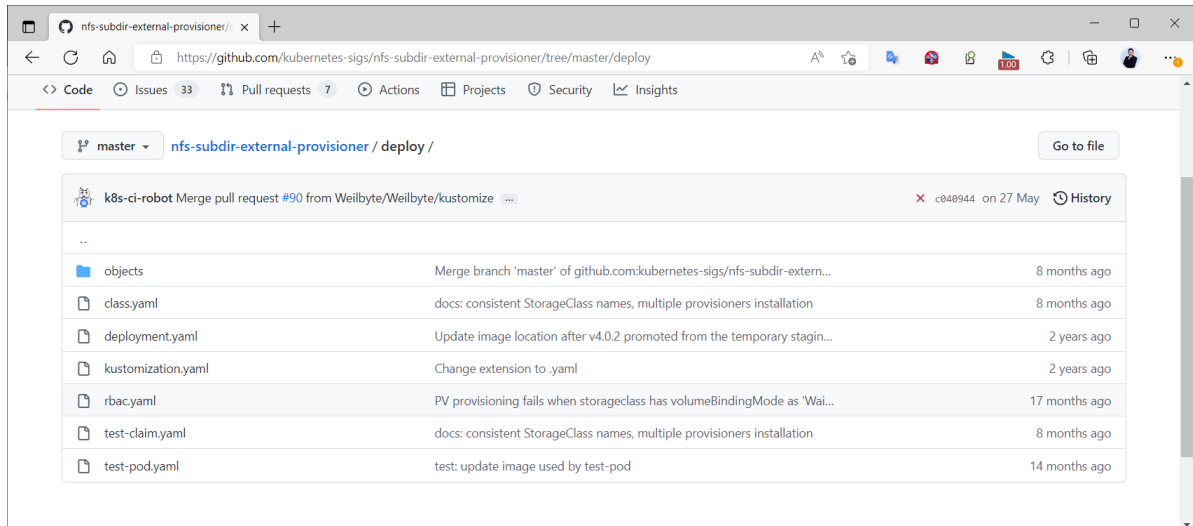
<https://github.com/kubernetes-sigs/nfs-ganesha-server-and-external-provisioner>

nfs-ganesha-server-and-external-provisioner 是 Kubernetes 1.14+ 的树外动态配置程序。
您可以使用它快速轻松地部署几乎可以在任何地方使用的共享存储。

1.6.3.2 案例: 基于 nfs-subdir-external-provisioner 创建 NFS 共享存储的 storageclass

部署相关文件

```
https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner
https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner/tree/master/deploy
https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner?tab=readme-ov-file#manually
```



创建 NFS 共享存储的 storageclass 步骤如下:

- 创建 NFS 共享
- 创建 Service Account 并授予管控 NFS provisioner 在 k8s 集群中运行的权限
- 部署 NFS-Subdir-External-Provisioner 对应的 Deployment
- 创建 StorageClass 负责建立 PVC 并调用 NFS provisioner 进行预定的工作, 并让 PV 与 PVC 建立联系
- 创建 PVC 时自动调用 SC 创建 PV
- 创建 Pod 使用 PVC

1.6.3.2.1 创建 NFS 服务

```
[root@master1 ~]# apt update && apt -y install nfs-server
[root@master1 ~]# systemctl status nfs-server.service
• nfs-server.service - NFS server and services
  Loaded: loaded (/lib/systemd/system/nfs-server.service; enabled; vendor preset: enabled)
  Active: active (exited) since Thu 2021-09-29 09:28:41 CST; 5min ago
  Main PID: 64029 (code=exited, status=0/SUCCESS)
  Tasks: 0 (limit: 2236)
  Memory: 0B
  CGroup: /system.slice/nfs-server.service

9月 29 09:28:40 master1.wang.org systemd[1]: Starting NFS server and services...
9月 29 09:28:41 master1.wang.org systemd[1]: Finished NFS server and services.

[root@master1 ~]# mkdir -pv /data/sc-nfs/

#此步可选, 可以不执行
[root@master1 ~]# chmod 777 /data/sc-nfs
```

```
[root@master1 ~]#vim /etc/exports
#授权worker节点的网段可以挂载
#/data/sc-nfs *(rw,no_root_squash,all_squash,anonuid=0,anongid=0)
/data/sc-nfs *(rw,no_root_squash)

[root@master1 ~]#exportfs -rv
/data/sc-nfs <world>
(sync,wdelay,hide,no_subtree_check,anonuid=0,anongid=0,sec=sys,rw,secure,no_root_squash,all_squash)

#并在所有worker节点安装NFS客户端
[root@nodex ~]#apt update && apt -y install nfs-common 或者 nfs-client
```

1.6.3.2.2 创建 ServiceAccount 并授权

```
[root@master1 ~]#ls
nfs-client-provisioner.yaml  nfs-StorageClass.yaml  pod-test.yaml  pvc.yaml
rbac.yaml

#建议创建独立的名称空间
[root@master1 ~]#kubectl create ns sc-nfs

#指定名称空间
[root@master1 ~]#cat rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  #namespace: default
  namespace: sc-nfs
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: nfs-client-provisioner-runner
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: [""]
  resources: ["persistentvolumeclaims"]
  verbs: ["get", "list", "watch", "update"]
- apiGroups: ["storage.k8s.io"]
  resources: ["storageclasses"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["events"]
  verbs: ["create", "update", "patch"]
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
```

```

    name: run-nfs-client-provisioner
subjects:
  - kind: ServiceAccount
    name: nfs-client-provisioner
    # replace with namespace where provisioner is deployed
    #namespace: default
    namespace: sc-nfs
roleRef:
  kind: ClusterRole
  name: nfs-client-provisioner-runner
  apiGroup: rbac.authorization.k8s.io
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  #namespace: default
  namespace: sc-nfs
rules:
  - apiGroups: [""]
    resources: ["endpoints"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  #namespace: default
  namespace: sc-nfs
subjects:
  - kind: ServiceAccount
    name: nfs-client-provisioner
    # replace with namespace where provisioner is deployed
    #namespace: default
    namespace: sc-nfs
roleRef:
  kind: Role
  name: leader-locking-nfs-client-provisioner
  apiGroup: rbac.authorization.k8s.io

#默认名称空间
[root@master1 ~]#cat rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: nfs-client-provisioner
  # replace with namespace where provisioner is deployed 根据业务需要修改此处名称空间
  namespace: default
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: nfs-client-provisioner-runner
rules:
  - apiGroups: [""]

```

```

    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: [""]
  resources: ["persistentvolumeclaims"]
  verbs: ["get", "list", "watch", "update"]
- apiGroups: ["storage.k8s.io"]
  resources: ["storageclasses"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["events"]
  verbs: ["create", "update", "patch"]
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: run-nfs-client-provisioner
subjects:
- kind: ServiceAccount
  name: nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  namespace: default
roleRef:
  kind: ClusterRole
  name: nfs-client-provisioner-runner
  apiGroup: rbac.authorization.k8s.io
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  namespace: default
rules:
- apiGroups: [""]
  resources: ["endpoints"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  namespace: default
subjects:
- kind: ServiceAccount
  name: nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  namespace: default
roleRef:
  kind: Role
  name: leader-locking-nfs-client-provisioner
  apiGroup: rbac.authorization.k8s.io

[root@master1 ~]#kubectl apply -f rbac.yaml
serviceaccount/nfs-client-provisioner created

```

```
clusterrole.rbac.authorization.k8s.io/nfs-client-provisioner-runner created
clusterrolebinding.rbac.authorization.k8s.io/run-nfs-client-provisioner created
role.rbac.authorization.k8s.io/leader-locking-nfs-client-provisioner created
rolebinding.rbac.authorization.k8s.io/leader-locking-nfs-client-provisioner
created
```

```
[root@master1 ~]#kubectl get sa
NAME                               SECRETS  AGE
default                            0        34d
nfs-client-provisioner             0        9s
```

1.6.3.2.3 部署 NFS-Subdir-External-Provisioner 对应的 Deployment

```
#提定名空间
[root@master1 ~]#cat nfs-client-provisioner.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nfs-client-provisioner
  labels:
    app: nfs-client-provisioner
  # replace with namespace where provisioner is deployed
  #namespace: default
  namespace: sc-nfs
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: nfs-client-provisioner
  template:
    metadata:
      labels:
        app: nfs-client-provisioner
    spec:
      serviceAccountName: nfs-client-provisioner
      containers:
        - name: nfs-client-provisioner
          image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nfs-subdir-
external-provisioner:v4.0.2
          #image: registry.k8s.io/sig-storage/nfs-subdir-external-
provisioner:v4.0.2
          #image: wangxiaochun/nfs-subdir-external-provisioner:v4.0.2
          #image: k8s.gcr.io/sig-storage/nfs-subdir-external-provisioner:v4.0.2
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - name: nfs-client-root
              mountPath: /persistentvolumes
          env:
            - name: PROVISIONER_NAME
              value: k8s-sigs.io/nfs-subdir-external-provisioner #名称确保与 nfs-
StorageClass.yaml文件中的provisioner名称保持一致
            - name: NFS_SERVER
              value: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域
名
            - name: NFS_PATH
```

```

        value: /data/sc-nfs # NFS 共享目录
volumes:
  - name: nfs-client-root
    nfs:
      server: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域
名
      path: /data/sc-nfs # NFS 共享目录

#默认名称空间
[root@master1 ~]#cat nfs-client-provisioner.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nfs-client-provisioner
  labels:
    app: nfs-client-provisioner
  # replace with namespace where provisioner is deployed 根据业务需要修改此处名称空间
  namespace: default
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: nfs-client-provisioner
  template:
    metadata:
      labels:
        app: nfs-client-provisioner
    spec:
      serviceAccountName: nfs-client-provisioner
      containers:
        - name: nfs-client-provisioner
          image: registry.cn-beijing.aliyuncs.com/wangxiaochunnfs-subdir-
external-provisioner:v4.0.2
          #image: k8s.gcr.io/sig-storage/nfs-subdir-external-provisioner:v4.0.2 #
此镜像国内可能无法访问
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - name: nfs-client-root
              mountPath: /persistentvolumes
          env:
            - name: PROVISIONER_NAME
              value: k8s-sigs.io/nfs-subdir-external-provisioner #名称确保与 nfs-
StorageClass.yaml文件中的provisioner名称保持一致
            - name: NFS_SERVER
              value: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域
名
            - name: NFS_PATH
              value: /data/sc-nfs # NFS 共享目录
          volumes:
            - name: nfs-client-root
              nfs:
                server: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域
名
                path: /data/sc-nfs # NFS 共享目录

```

```
[root@master1 ~]#kubectl apply -f nfs-client-provisioner.yaml
```

```
[root@master1 ~]#kubectl get deployments.apps
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nfs-client-provisioner	1/1	1	1	5m26s

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
nfs-client-provisioner-7468dcd749-24lqv	1/1	Running	0	5m34s

#注意:如果失败,检查是否worker节点安装了nfs-client

1.6.3.2.4 创建 NFS 资源的 StorageClass

```
[root@master1 ~]#cat nfs-storageClass.yaml
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sc-nfs
  annotations:
    storageclass.kubernetes.io/is-default-class: "false" # 是否设置为默认的
storageClass
provisioner: k8s-sigs.io/nfs-subdir-external-provisioner # or choose another
name, must match deployment's env PROVISIONER_NAME
parameters:
  archiveOnDelete: "true" # 设置为"false"时删除PVC不会保留数据,"true"则保留数据,基于安全
原因建议设为"true"
```

#应用

```
[root@master1 ~]#kubectl apply -f nfs-storageClass.yaml
```

```
storageClass.storage.k8s.io/sc-nfs created
```

#查看是否创建

```
[root@master1 ~]#kubectl get sc
```

NAME	PROVISIONER	RECLAIMPOLICY
sc-nfs	k8s-sigs.io/nfs-subdir-external-provisioner	Delete
	false	6s

1.6.3.2.5 创建 PVC

范例:

```
[root@master1 ~]#cat pvc.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-nfs-sc
spec:
  storageClassName: sc-nfs #需要和前面创建的storageClass名称相同
  accessModes: ["ReadWriteMany", "ReadOnlyMany"]
  resources:
    requests:
      storage: 100Mi
```

#应用

```
[root@master1 ~]#kubectl apply -f pvc.yaml
```

```
persistentvolumeclaim/pvc-nfs-sc created
```

#验证

```
[root@master1 ~]#kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	AGE	
pvc-nfs-sc	Bound	pvc-eae826dd-6270-445c-8d0f-e06fb0093773	100Mi
ROX,RWX	sc-nfs	2s	

#自动创建PV并绑定PVC

```
[root@master1 ~]#kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM
POLICY	STATUS	CLAIM	STORAGECLASS
		REASON	AGE
pvc-9be97a7b-2d9c-4778-8c2a-519da3266ea4	100Mi	ROX,RWX	Delete
Bound	default/pvc-nfs-sc	sc-nfs	5s

#自动在NFS服务器共享目录创建目录

```
[root@master1 ~]#ls /data/sc-nfs/
```

```
default-pvc-nfs-sc-pvc-9be97a7b-2d9c-4778-8c2a-519da3266ea4
```

```
[root@master1 ~]#ls /data/sc-nfs/default-pvc-nfs-sc-pvc-9be97a7b-2d9c-4778-8c2a-519da3266ea4/
```

1.6.3.2.6 创建 Pod

```
[root@master1 ~]#cat pod-test.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-nfs-sc-test
```

```
spec:
```

```
  containers:
```

```
  - name: pod-nfs-sc-test
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
```

```
    volumeMounts:
```

```
    - name: nfs-pvc
```

```
      mountPath: "/usr/share/nginx/html/"
```

```
  restartPolicy: "Never"
```

```
  volumes:
```

```
  - name: nfs-pvc
```

```
    persistentVolumeClaim:
```

```
      claimName: pvc-nfs-sc #指定前面创建的PVC名称
```

#应用

```
[root@master1 ~]#kubectl apply -f pod-test.yaml
```

```
pod/pod-nfs-sc-test created
```

#验证

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	NODE	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP
nfs-client-provisioner-7468dcd749-24l	10.244.3.9	node3.wang.org	1/1	Running	0	16m	
pod-nfs-sc-test	10.244.1.14	node1.wang.org	1/1	Running	0	41s	

#查看Pod的挂载

```
[root@master1 ~]#kubectl exec -it pod-nfs-sc-test -- sh
# df
Filesystem
1K-blocks    Used Available Use% Mounted on
overlay
101848040 6433408  90194896   7% /
tmpfs
65536      0    65536   0% /dev
tmpfs
999676     0    999676   0% /sys/fs/cgroup
/dev/mapper/ubuntu--vg-ubuntu--lv
101848040 6433408  90194896   7% /etc/hosts
shm
65536      0    65536   0% /dev/shm
10.0.0.101:/data/sc-nfs/default-pvc-nfs-sc-pvc-9be97a7b-2d9c-4778-8c2a-
519da3266ea4 101848064 7129856  89498624   8% /usr/share/nginx/html
tmpfs
1896952    12   1896940   1%
/run/secrets/kubernetes.io/serviceaccount
tmpfs
999676     0    999676   0% /proc/acpi
tmpfs
999676     0    999676   0% /proc/scsi
tmpfs
999676     0    999676   0% /sys/firmware

#在Pod运行的宿主机,可以看到NFS挂载
[root@node1 ~]#df |grep 10.0.0.101
10.0.0.101:/data/sc-nfs/default-pvc-nfs-sc-pvc-9be97a7b-2d9c-4778-8c2a-
519da3266ea4 101848064 7139328  89489152   8% /var/lib/kubelet/pods/eb7a4c14-
ba48-4bf3-87ee-eb8bd6dbae71/volumes/kubernetes.io~nfs/pvc-644ecc87-38f5-4959-
b100-f9a5ff803011

# 查看nginx的网页文件不存在
# ls /usr/share/nginx/html

#由于没有页面文件,访问Pod失败
[root@master1 ~]#curl 10.244.1.14
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.20.0</center>
</body>
</html>

#在NFS服务器创建页面文件
[root@master1 ~]#echo NFS-SC Website > /data/sc-nfs/default-pvc-nfs-sc-pvc-
9be97a7b-2d9c-4778-8c2a-519da3266ea4/index.html

#查看Pod内的文件
[root@master1 ~]#kubectl exec -it pod-nfs-sc-test -- sh
# ls /usr/share/nginx/html
index.html

#访问Pod页面成功
[root@master1 ~]#curl 10.244.1.14
NFS-SC Website
```

1.6.3.2.7 删除 Pod 和 PVC

```
#先删除Pod
[root@master1 ~]#kubectl delete -f pod-test.yaml

#再删除PVC
[root@master1 ~]#kubectl delete -f pvc.yaml

#自动删除对应的PV
[root@master1 ~]#kubectl get pv
No resources found

[root@master1 ~]#kubectl get pvc

#查看对应的数据不会删除,仍保留
[root@master1 ~]#ls /data/sc-nfs/archived-default-pvc-nfs-sc-pvc-9be97a7b-2d9c-4778-8c2a-519da3266ea4
index.html
```

1.6.3.2.8 基于sc-nfs 实现MySQL的动态置备

```
[root@master1 ~]#cat storage-mysql-storage-class-pvc.yaml
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pv-claim
spec:
  storageClassName: sc-nfs
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
  clusterIP: None
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  strategy:
    type: Recreate
```

```

template:
  metadata:
    labels:
      app: mysql
  spec:
    containers:
      - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
        name: mysql
        env:
          # 在实际中使用 secret
          - name: MYSQL_ROOT_PASSWORD
            value: "123456"
        ports:
          - containerPort: 3306
            name: mysql
        volumeMounts:
          - name: mysql-persistent-storage
            mountPath: /var/lib/mysql
    volumes:
      - name: mysql-persistent-storage
        persistentVolumeClaim:
          claimName: mysql-pv-claim

```

1.6.3.3 案例：通过 Helm 部署 nfs-subdir-external-provisioner

<https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner>
<https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner/blob/master/charts/nfs-subdir-external-provisioner/README.md>

1.6.3.3.1 准备 NFS 服务器

Kubernetes 主节点 (Ubuntu 22.04) 上安装 NFS 服务器。登录主节点并运行以下命令：

```

$ sudo apt update
$ sudo apt -y install nfs-kernel-server

```

创建以下文件夹并使用 NFS 共享它：

```

$ sudo mkdir /data/sc-nfs
$ sudo chown -R nobody:nogroup /data/sc-nfs
$ sudo chmod 777 /data/sc-nfs

```

在 `/etc/exports` 文件中添加以下条目：

```

$ sudo vi /etc/exports
/data/sc-nfs 10.0.0.0/24(rw,sync,no_subtree_check)

```

保存并关闭文件。

注意：不要忘记更改导出文件中适合你的部署的网络。

要使上述更改生效，请运行：

```

$ sudo exportfs -a
$ sudo systemctl restart nfs-kernel-server

```

```
$ sudo systemctl status nfs-kernel-server

[root@master1 ~]#systemctl status nfs-kernel-server
• nfs-server.service - NFS server and services
   Loaded: loaded (/lib/systemd/system/nfs-server.service; enabled; vendor preset: enabled)
   Active: active (exited) since Tue 2023-08-15 09:33:05 CST; 4min 0s ago
   Main PID: 148304 (code=exited, status=0/SUCCESS)
      CPU: 4ms

8月 15 09:33:05 master1.wang.org systemd[1]: Starting NFS server and services...
8月 15 09:33:05 master1.wang.org exportfs[148299]: exportfs: can't open /etc/exports for reading
8月 15 09:33:05 master1.wang.org systemd[1]: Finished NFS server and services.
```

在所有工作节点上，使用以下 `apt` 命令安装 `nfs-common` 包。

```
$ sudo apt install nfs-common -y
```

1.6.3.2 安装和配置 NFS 客户端配置程序

NFS 子目录外部配置程序在 Kubernetes 集群中部署 NFS 客户端配置程序。配置程序负责动态创建和管理由 NFS 存储支持的持久卷（PV）和持久卷声明（PVC）。

因此，要安装 NFS 子目录外部配置程序，首先使用以下命令集安装 `helm`：

```
$ curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
$ chmod 700 get_helm.sh
$ ./get_helm.sh
```

运行以下命令来启用 `helm` 仓库：

```
$ helm repo add nfs-subdir-external-provisioner https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner
```

使用以下 `helm` 命令部署配置程序：

```
https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner/blob/master/charts/nfs-subdir-external-provisioner/README.md
```

范例: 创建 SC

```
#默认创建SC名为nfs-client
$ helm install -n nfs-provisioning --create-namespace nfs-subdir-external-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner --set nfs.server=10.0.0.200 --set nfs.path=/data/sc-nfs
#上面的 `helm` 命令将自动创建`nfs-provisioning`命名空间，并安装 NFS 配置程序的pod和 deployment、名称为`nfs-client`的存储类，并将创建所需的rbac。
#删除SC
[root@master1 ~]#helm uninstall -n nfs-provisioning nfs-subdir-external-provisioner

#指定创建SC名为sc-nfs,此步可能因为网络原因失败,可以多试几次
```

```
[root@master1 ~]#helm install -n nfs-provisioning --create-namespace nfs-subdir-external-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
--set nfs.server=10.0.0.200 \
--set nfs.path=/data/sc-nfs \
--set storageClass.name=sc-nfs

NAME: nfs-subdir-external-provisioner
LAST DEPLOYED: Tue Aug 15 10:09:24 2023
NAMESPACE: nfs-provisioning
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

查看

```
$ kubectl get all -n nfs-provisioning
$ kubectl get sc -n nfs-provisioning

[root@master1 ~]#kubectl get all -n nfs-provisioning
```

NAME	READY	STATUS
pod/nfs-subdir-external-provisioner-79756db4bc-qdcvv	0/1	ImagePullBackOff

```
0          54s
```

NAME	READY	UP-TO-DATE	AVAILABLE
deployment.apps/nfs-subdir-external-provisioner	0/1	1	0

```
54s
```

NAME	READY	AGE	DESIRED	CURRENT
replicaset.apps/nfs-subdir-external-provisioner-79756db4bc	0	54s	1	1

```
0          54s
```

```
[root@master1 ~]#kubectl get sc -n nfs-provisioning
```

NAME	PROVISIONER	RECLAIMPOLICY
sc-nfs	cluster.local/nfs-subdir-external-provisioner	Delete

```
Immediate          true          63s
```

上面的输出确认了配置程序容器和存储类已成功创建。

1.6.3.3.3 创建持久卷声明 (PVC)

让我们创建 PVC 来为你的容器或部署请求存储。PVC 将从存储类 sc-nfs 请求特定数量的存储：

```
$ vi demo-pvc.yml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: demo-claim
  namespace: nfs-provisioning
spec:
  storageClassName: sc-nfs
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Mi
```

运行以下 `kubectl` 命令以使用上面创建的 YML 文件创建 PVC:

```
$ kubectl create -f demo-pvc.yml
```

验证 PVC 和 PV 是否创建:

```
$ kubectl get pv,pvc -n nfs-provisioning
```

```
kops@k8s-master:~$
kops@k8s-master:~$ kubectl create -f demo-pvc.yml
persistentvolumeclaim/demo-claim created
kops@k8s-master:~$
kops@k8s-master:~$ kubectl get pv,pvc -n nfs-provisioning
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
persistentvolume/pvc-f4d49dbe-6bcd-4915-a763-98f0f621445e	10Mi	RWX	Delete	Bound	nfs-provisioning/demo-claim	nfs-client		93s

```
NAME
persistentvolumeclaim/demo-claim
STATUS
Bound
VOLUME
pvc-f4d49dbe-6bcd-4915-a763-98f0f621445e
CAPACITY
10Mi
ACCESS MODES
RWX
STORAGECLASS
nfs-client
AGE
93s
kops@k8s-master:~$
kops@k8s-master:~$
```

上面的输出表明 PV 和 PVC 创建成功。

1.6.3.3.4 测试并验证动态 NFS 配置

为了测试和验证动态 NFS 配置, 请使用以下 YML 文件启动测试容器:

```
$ vi test-pod.yml
kind: Pod
apiVersion: v1
metadata:
  name: test-pod
  namespace: nfs-provisioning
spec:
  containers:
    - name: test-pod
      image: busybox:latest
      command:
        - "/bin/sh"
      args:
        - "-c"
        - "touch /mnt/SUCCESS && sleep 600"
      volumeMounts:
        - name: nfs-pvc
          mountPath: "/mnt"
  restartPolicy: "Never"
  volumes:
    - name: nfs-pvc
      persistentVolumeClaim:
```

```
claimName: demo-claim
```

```
kind: Pod
apiVersion: v1
metadata:
  name: test-pod
  namespace: nfs-provisioning
spec:
  containers:
  - name: test-pod
    image: busybox:latest
    command:
    - "/bin/sh"
    args:
    - "-c"
    - "touch /mnt/SUCCESS && sleep 600"
    volumeMounts:
    - name: nfs-pvc
      mountPath: "/mnt"
  restartPolicy: "Never"
  volumes:
  - name: nfs-pvc
    persistentVolumeClaim:
      claimName: demo-claim
```

使用以下 `kubectl` 命令部署容器：

```
$ kubectl apply -f test-pod.yml
```

验证 `test-pod` 的状态：

```
$ kubectl get pods -n nfs-provisioning
```

```
kops@k8s-master:~$
kops@k8s-master:~$ kubectl create -f test-pod.yml
pod/test-pod created
kops@k8s-master:~$
kops@k8s-master:~$ kubectl get pods -n nfs-provisioning
```

NAME	READY	STATUS	RESTARTS	AGE
nfs-subdir-external-provisioner-6456b5c5b5-sqtfr	1/1	Running	0	29m
test-pod	1/1	Running	0	12s

```
kops@k8s-master:~$
kops@k8s-master:~$
```

登录到容器并验证 NFS 卷是否已安装。

```
$ kubectl exec -it test-pod -n nfs-provisioning /bin/sh
```

```
kops@k8s-master:~$ kubectl exec -it test-pod -n nfs-provisioning /bin/sh
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version. Use kubectl exec [POD] -- [COMMAND] instead.
/ # df -h
Filesystem                Size      Used Available Use% Mounted on
overlay                  33.7G    13.6G    18.3G   43% /
tmpfs                     64.0M         0    64.0M    0% /dev
192.168.1.139:/opt/dynamic-storage/nfs-provisioning-demo-claim-pvc-f4d49dbe-6bcd-4915-a763-98f0f621445e 14.6G    17.3G    18.3G   46% /mnt
/dev/sda3                 33.7G    13.6G    18.3G   43% /etc/hosts
/dev/sda3                 33.7G    13.6G    18.3G   43% /dev/termination-log
/dev/sda3                 33.7G    13.6G    18.3G   43% /etc/hostname
/dev/sda3                 33.7G    13.6G    18.3G   43% /etc/resolv.conf
shm                       64.0M         0    64.0M    0% /dev/shm
tmpfs                     3.7G     12.0K     3.7G    0% /var/run/secrets/kubernetes.io/serviceaccount
tmpfs                     1.9G         0     1.9G    0% /proc/asound
tmpfs                     1.9G         0     1.9G    0% /proc/acpi
tmpfs                     64.0M         0    64.0M    0% /proc/kcore
tmpfs                     64.0M         0    64.0M    0% /proc/keys
tmpfs                     64.0M         0    64.0M    0% /proc/timer_list
tmpfs                     1.9G         0     1.9G    0% /proc/scsi
tmpfs                     1.9G         0     1.9G    0% /sys/firmware
/ #
/ # cd /mnt/ && ls
SUCCESS
/mnt #
```

太棒了，上面容器内的输出确认了动态 NFS 卷已安装且可访问。

最后删除Pod和 PVC，查看 PV 是否自动删除。

```
$ kubectl delete -f test-pod.yml
$ kubectl delete -f demo-pvc.yml
$ kubectl get pv,pvc -n nfs-provisioning
```

```
kops@k8s-master:~$
kops@k8s-master:~$ kubectl delete -f test-pod.yml
pod "test-pod" deleted
kops@k8s-master:~$
kops@k8s-master:~$ kubectl delete -f demo-pvc.yml
persistentvolumeclaim "demo-claim" deleted
kops@k8s-master:~$
kops@k8s-master:~$ kubectl get pv,pvc -n nfs-provisioning
No resources found
kops@k8s-master:~$
```

1.6.3.4 案例: 基于 csi-driver-nfs 项目实现

<https://github.com/kubernetes-csi/csi-driver-nfs>

#此项目中的镜像需要科学上网才能下载

1.6.3.4.1 准备 NFS 服务

#方法1:部署至kubernetes

<https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/deploy/example/nfs-provisioner/README.md>

#方法2:手动部署在集群外

```
[root@master1 ~]#apt update && apt -y install nfs-server
```

```
[root@master1 ~]#systemctl status nfs-server.service
```

- nfs-server.service - NFS server and services

```
Loaded: loaded (/lib/systemd/system/nfs-server.service; enabled; vendor preset: enabled)
```

```
Active: active (exited) since Thu 2021-09-29 09:28:41 CST; 5min ago
```

```
Main PID: 64029 (code=exited, status=0/SUCCESS)
```

```
Tasks: 0 (limit: 2236)
```

```
Memory: 0B
```

```
CGroup: /system.slice/nfs-server.service
```

```
9月 29 09:28:40 master1.wang.org systemd[1]: Starting NFS server and services...
9月 29 09:28:41 master1.wang.org systemd[1]: Finished NFS server and services.
```

```
[root@master1 ~]#mkdir -pv /data/sc-nfs

[root@master1 ~]#vim /etc/exports
#授权worker节点的网段可以挂载
#/data/sc-nfs *(rw,no_root_squash,all_squash,anonuid=0,anongid=0)
/data/sc-nfs *(rw,no_root_squash)

[root@master1 ~]#exportfs -rv
/data/sc-nfs <world>
(sync,wdelay,hide,no_subtree_check,anonuid=0,anongid=0,sec=sys,rw,secure,no_root_squash,all_squash)

#并在所有worker节点安装NFS客户端
[root@nodex ~]#apt update && apt -y install nfs-common 或者 nfs-client
```

1.6.3.4.2 Install NFS CSI driver on a kubernetes cluster

```
https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/docs/install-csi-driver-v4.6.0.md
```

Install with kubectl 可能需要科学上网下载相关镜像

1.6.3.4.2.1 方法1: local install

```
git clone https://github.com/kubernetes-csi/csi-driver-nfs.git
cd csi-driver-nfs
./deploy/install-driver.sh v4.6.0 local
```

范例:

```
[root@master1 ~]#git clone https://github.com/kubernetes-csi/csi-driver-nfs.git
Cloning into 'csi-driver-nfs'...

[root@master1 ~]#ls csi-driver-nfs/
CHANGELOG      cmd            deploy         go.mod  LICENSE  OWNERS_ALIASES
RELEASE.md     test
charts         code-of-conduct.md  Dockerfile  go.sum  Makefile  pkg
  release-tools      vendor
cloudbuild.yaml  CONTRIBUTING.md    docs        hack    OWNERS    README.md
  SECURITY_CONTACTS

[root@master1 ~]#ls csi-driver-nfs/deploy/
crd-csi-snapshot.yaml  csi-nfs-node.yaml  install-driver.sh
  uninstall-driver.sh  v4.0.0  v4.3.0  v4.6.0
csi-nfs-controller.yaml  csi-snapshot-controller.yaml  rbac-csi-nfs.yaml
  v3.0.0                v4.1.0  v4.4.0
csi-nfs-driverinfo.yaml  example  rbac-snapshot-controller.yaml
  v3.1.0                v4.2.0  v4.5.0
[root@master1 ~]#cd csi-driver-nfs/deploy/
[root@master1 deploy]#ls v4.6.0/
crd-csi-snapshot.yaml  csi-nfs-driverinfo.yaml  csi-snapshot-controller.yaml
rbac-snapshot-controller.yaml
csi-nfs-controller.yaml  csi-nfs-node.yaml  rbac-csi-nfs.yaml
```

#镜像需要科学上网才能下载

```
[root@master1 deploy]#grep -R image: v4.6.0/
v4.6.0/csi-nfs-controller.yaml:      image: registry.k8s.io/sig-storage/csi-
provisioner:v4.0.0
v4.6.0/csi-nfs-controller.yaml:      image: registry.k8s.io/sig-storage/csi-
snapshotter:v6.3.3
v4.6.0/csi-nfs-controller.yaml:      image: registry.k8s.io/sig-
storage/livenessprobe:v2.12.0
v4.6.0/csi-nfs-controller.yaml:      image: registry.k8s.io/sig-
storage/nfsplugin:v4.6.0
v4.6.0/csi-snapshot-controller.yaml:  image: registry.k8s.io/sig-
storage/snapshot-controller:v6.3.3
v4.6.0/csi-nfs-node.yaml:      image: registry.k8s.io/sig-
storage/livenessprobe:v2.12.0
v4.6.0/csi-nfs-node.yaml:      image: registry.k8s.io/sig-storage/csi-node-
driver-registrar:v2.10.0
v4.6.0/csi-nfs-node.yaml:      image: registry.k8s.io/sig-
storage/nfsplugin:v4.6.0
```

```
[root@master1 deploy]#kubectl apply -f v4.6.0/
```

#自动生成相关CRD

```
[root@master1 ~]#kubectl get crd
```

NAME	CREATED AT
volumesnapshotclasses.snapshot.storage.k8s.io	2024-12-19T09:43:11Z
volumesnapshotcontents.snapshot.storage.k8s.io	2024-12-19T09:43:11Z
volumesnapshots.snapshot.storage.k8s.io	2024-12-19T09:43:11Z

```
[root@master1 ~]#kubectl api-resources --api-group storage.k8s.io
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
csidrivers		storage.k8s.io/v1	false	CSIDriver
csinodes		storage.k8s.io/v1	false	CSINode
csistoragecapacities		storage.k8s.io/v1	true	
CSISStorageCapacity				
storageclasses	sc	storage.k8s.io/v1	false	
StorageClass				
volumeattachments		storage.k8s.io/v1	false	
VolumeAttachment				

#确认相关Pod正常工作

```
[root@master1 ~]#kubectl get pod -A |grep -E 'csi|snapshot'
```

NAME	STATUS	RESTARTS	AGE
kube-system csi-nfs-controller-db9cf5df7-n28nx	Running	0	4/4 3m13s
kube-system csi-nfs-node-8vsbv	Running	0	3/3 4m11s
kube-system csi-nfs-node-nljtc	Running	0	3/3 4m10s
kube-system csi-nfs-node-nnqfg	Running	0	3/3 4m11s
kube-system csi-nfs-node-wztkq	Running	0	3/3 4m7s
kube-system snapshot-controller-796bf74d5f-5fss8	Running	0	1/1 32m
kube-system snapshot-controller-796bf74d5f-f2dp4	Running	0	1/1 32m

```
[root@master1 ~]#kubectl -n kube-system get pod -o wide -l app=csi-nfs-controller
[root@master1 ~]#kubectl -n kube-system get pod -o wide -l app=csi-nfs-node
```

1.6.3.4.2.2 方法2. remote install

```
curl -sSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.6.0/deploy/install-driver.sh | bash -s v4.6.0 --
```

范例:

```
[root@master1 ~]#curl -sSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.6.0/deploy/install-driver.sh | bash -s v4.6.0 --
```

#新创建资源类型

```
[root@master1 ~]#kubectl api-resources |grep storage.k8s.io
csidrivers                                storage.k8s.io/v1
  false          CSIDriver
csinodes                                storage.k8s.io/v1
  false          CSINode
csistoragecapacities                    storage.k8s.io/v1
  true           CSISStorageCapacity
storageclasses                          sc                storage.k8s.io/v1
  false          StorageClass
volumeattachments                       storage.k8s.io/v1
  false          VolumeAttachment
```

#新创建的资源

```
[root@master1 ~]#kubectl get all -n kube-system |grep csi
pod/csi-nfs-controller-57bb8fd754-j9xtb    0/4    ContainerCreating    0
76s
pod/csi-nfs-node-8bg8q                    0/3    ContainerCreating    0
68s
pod/csi-nfs-node-8r784                    0/3    ContainerCreating    0
68s
pod/csi-nfs-node-8wzxk                    0/3    ContainerCreating    0
68s
pod/csi-nfs-node-hvthk                    0/3    ContainerCreating    0
68s
daemonset.apps/csi-nfs-node    4          4          0          4          0
kubernetes.io/os=linux    68s
deployment.apps/csi-nfs-controller    0/1    1          0          76s
replicaset.apps/csi-nfs-controller-57bb8fd754    1          1          0          76s
```

#新建CRD

```
[root@master1 ~]#kubectl get crd
```

1.6.3.4.3 创建 Storage Class

```
https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/deploy/example/README.md
```

Create a storage class

change `server`, `share` with your existing NFS server address and share name

```
#https://github.com/kubernetes-csi/csi-driver-
nfs/blob/master/deploy/example/README.md#storage-class-usage-dynamic-
provisioning
```

```
[root@master1 csi-driver-nfs]#cat storageclass-csi-nfs.yaml
```

```
---
```

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
  name: sc-nfs
```

```
  annotations:
```

```
    storageclass.kubernetes.io/is-default-class: "true"
```

```
provisioner: nfs.csi.k8s.io
```

```
parameters:
```

```
  server: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域名
```

```
  share: /data/sc-nfs
```

```
reclaimPolicy: Delete
```

```
volumeBindingMode: Immediate
```

```
#mountOptions:
```

```
# - hard
```

```
# - nfsvers=4.1
```

```
[root@master1 csi-driver-nfs]#kubectl apply -f storageclass-csi-nfs.yaml
```

```
[root@master1 ~]#kubectl get sc
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
sc-nfs (default)	nfs.csi.k8s.io	Delete	Immediate	false	73s

1.6.3.4.4 PVC (Dynamic Provisioning)

```
wget https://raw.githubusercontent.com/kubernetes-csi/csi-driver-
nfs/master/deploy/example/pvc-nfs-csi-dynamic.yaml
```

```
cat storage-pvc-nfs-csi-dynamic.yaml
```

```
---
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: pvc-nfs-dynamic
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteMany
```

```
  resources:
```

```
    requests:
```

```
      storage: 10Gi
```

```
  storageClassName: sc-nfs #指向前面创建的SC
```

```
#创建PVC,也会自动创建对应的PV
```

```
kubectl create -f pvc-nfs-csi-dynamic.yaml
```

1.6.3.4.5 PV/PVC Usage (Static Provisioning)

- Follow the following command to create `PersistentVolume` and `PersistentVolumeClaim` statically.

```
# create PV
wget https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/deploy/example/pv-nfs-csi.yaml

cat storage-pv-nfs-csi.yaml
---
apiVersion: v1
kind: PersistentVolume
metadata:
  annotations:
    pv.kubernetes.io/provisioned-by: nfs.csi.k8s.io
  name: pv-nfs
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: sc-nfs
  mountOptions:
    - nfsvers=4.1
  csi:
    driver: nfs.csi.k8s.io
    # volumeHandle format: {nfs-server-address}#{sub-dir-name}#{share-name}
    # make sure this value is unique for every share in the cluster
    volumeHandle: nfs-server.default.svc.cluster.local/share##
    volumeAttributes:
      server: nfs.wang.org #指定NFS服务器地址,如果是域名,需要宿主机节点能解析此域名
      share: /data/sc-nfs

kubectl create -f storage-pv-nfs-csi.yaml

# create PVC
wget https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/deploy/example/pvc-nfs-csi-static.yaml

cat storage-pvc-nfs-csi-static.yaml
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-nfs-static
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
  volumeName: pv-nfs
  storageClassName: sc-nfs

kubectl create -f storage-pvc-nfs-csi-static.yaml
```

1.6.3.4.6 Create a deployment

```
wget https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/deploy/example/deployment.yaml

cat storage-deployment.yaml
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-deployment-nfs
spec:
  accessModes:
    - ReadWriteMany # In this example, multiple Pods consume the same PVC.
  resources:
    requests:
      storage: 10Gi
      storageClassName: sc-nfs
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-nfs
spec:
  replicas: 1
  selector:
    matchLabels:
      name: deployment-nfs
  template:
    metadata:
      name: deployment-nfs
      labels:
        name: deployment-nfs
    spec:
      nodeSelector:
        "kubernetes.io/os": linux
      containers:
        - name: deployment-nfs
          image: mcr.microsoft.com/oss/nginx/nginx:1.19.5
          command:
            - "/bin/bash"
            - "-c"
            - set -euo pipefail; while true; do echo $(hostname) $(date) >>
/mnt/nfs/outfile; sleep 1; done
          volumeMounts:
            - name: nfs
              mountPath: "/mnt/nfs"
              readOnly: false
      volumes:
        - name: nfs
          persistentVolumeClaim:
            claimName: pvc-deployment-nfs

kubectl create -f deployment.yaml
```

1.6.3.4.7 clean up NFS CSI driver

```
https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/docs/install-csi-driver-v4.9.0.md
```

- Option#1. remote uninstall

```
curl -sSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.9.0/deploy/uninstall-driver.sh | bash -s v4.9.0 --
```

- Option#2. local uninstall

```
git clone https://github.com/kubernetes-csi/csi-driver-nfs.git
cd csi-driver-nfs
git checkout v4.9.0
./deploy/uninstall-driver.sh v4.9.0 local
```

1.6.4 改变默认存储类

```
https://kubernetes.io/zh-cn/docs/tasks/administer-cluster/change-default-storage-class/
https://kubernetes.io/zh-cn/docs/concepts/storage/storage-classes/#default-storageclass
```

1.6.4.1 为什么要改变默认存储类？

当一个 PVC 没有指定 `storageClassName` 时，会使用默认的 StorageClass。

集群中只能有一个默认的 StorageClass。如果不小心设置了多个默认的 StorageClass，在动态制备 PVC 时将使用其中最新的默认设置的 StorageClass。

如果有多个存储类，需要指定你需要的存储类为默认的

取决于安装模式，你的 Kubernetes 集群可能和一个被标记为默认的已有 StorageClass 一起部署。这个默认的 StorageClass 以后将被用于动态的为没有特定存储类需求的 PersistentVolumeClaims 配置存储。更多细节请查看 [PersistentVolumeClaim 文档](#)。

预先安装的默认 StorageClass 可能不能很好的适应你期望的工作负载；例如，它配置的存储可能太过昂贵。如果是这样的话，你可以改变默认 StorageClass，或者完全禁用它以防止动态配置存储。

删除默认 StorageClass 可能行不通，因为它可能会被你集群中的扩展管理器自动重建。请查阅你的安装文档中关于扩展管理器的细节，以及如何禁用单个扩展。

1.6.4.2 改变默认 StorageClass

1. 列出你的集群中的 StorageClasses:

```
kubectl get storageclass
```

输出类似这样：

NAME	PROVISIONER	AGE
standard (default)	kubernetes.io/gce-pd	1d
gold	kubernetes.io/gce-pd	1d

默认 StorageClass 以 (default) 标记。

2. 标记默认 StorageClass 非默认:

默认 StorageClass 的注解 `storageclass.kubernetes.io/is-default-class` 设置为 `true`。
注解的其它任意值或者缺省值将被解释为 `false`。

要标记一个 StorageClass 为非默认的, 你需要改变它的值为 `false`:

#方法1:交互

```
kubectl edit storageclasses.storage.k8s.io <your-class-name>
metadata:
  annotations:
    .....
    storageclass.kubernetes.io/is-default-class: "false" #修改为false
```

#方法2

```
kubectl patch storageclass standard -p '{"metadata": {"annotations":
{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```

这里的 `standard` 是你选择的 StorageClass 的名字。

3. 标记一个 StorageClass 为默认的:

和前面的步骤类似, 你需要添加/设置注解 `storageclass.kubernetes.io/is-default-class=true`。

#方法1:交互

```
kubectl edit storageclasses.storage.k8s.io <your-class-name>
metadata:
  annotations:
    .....
    storageclass.kubernetes.io/is-default-class: "true" #修改为true
```

#方法2

```
kubectl patch storageclass <your-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

请注意, 最多只能有一个 StorageClass 能够被标记为默认。如果它们中有两个或多个被标记为默认, Kubernetes 将忽略这个注解, 也就是它将表现为没有默认 StorageClass。

4. 验证你选用的 StorageClass 为默认的:

```
kubectl get storageclass
```

输出类似这样:

NAME	PROVISIONER	AGE
standard	kubernetes.io/gce-pd	1d
gold (default)	kubernetes.io/gce-pd	1d

范例: 指定sc-nfs storageClass 是默认的

```
[root@master1 ~]#kubectl get sc
NAME          PROVISIONER          RECLAIMPOLICY
VOLUMEBINDINGMODE  ALLOWVOLUMEEXPANSION  AGE
```

sc-nfs	k8s-sigs.io/nfs-subdir-external-provisioner	Delete	Immediate
	false	17m	

#指定sc-nfs storageClass 是默认的存储类

#方法1

```
[root@master1 ~]#kubectl edit storageclasses.storage.k8s.io sc-nfs
metadata:
  annotations:
    ....
    storageclass.kubernetes.io/is-default-class: "true" #修改为true
```

#方法2

```
[root@master1 ~]#kubectl patch storageclass sc-nfs -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
storageclass.storage.k8s.io/sc-nfs patched
```

#验证已经默认

```
[root@master1 ~]#kubectl get sc
NAME                                PROVISIONER                                RECLAIMPOLICY
VOLUMEBINDINGMODE  ALLOWVOLUMEEXPANSION  AGE
sc-nfs (default)   k8s-sigs.io/nfs-subdir-external-provisioner  Delete
Immediate          false                                17m
```

#取消默认

#方法1

```
[root@master1 ~]#kubectl edit storageclasses.storage.k8s.io sc-nfs
metadata:
  annotations:
    ....
    storageclass.kubernetes.io/is-default-class: "false" #修改为false
```

#方法2

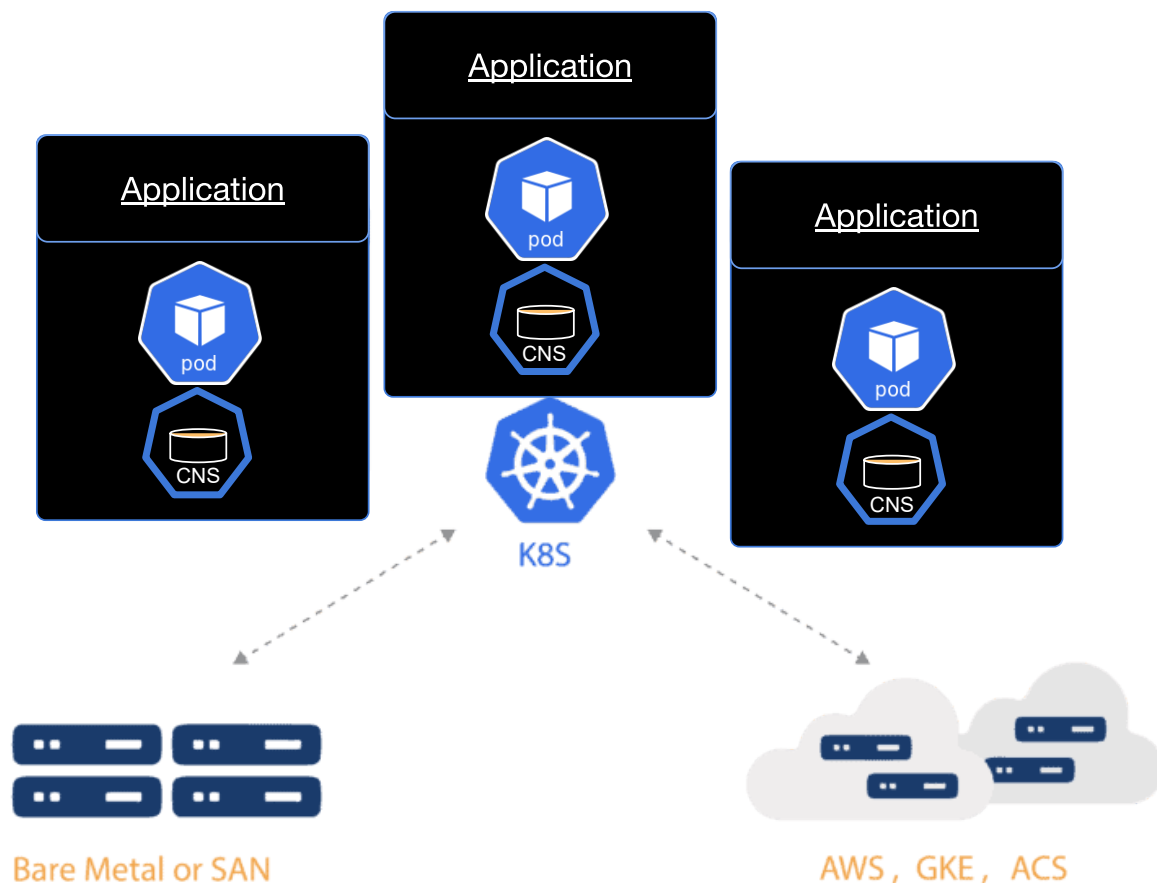
```
[root@master1 ~]#kubectl patch storageclass sc-nfs -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
storageclass.storage.k8s.io/sc-nfs patched
```

#验证取消默认

```
[root@master1 ~]#kubectl get sc
NAME                                PROVISIONER                                RECLAIMPOLICY
VOLUMEBINDINGMODE  ALLOWVOLUMEEXPANSION  AGE
sc-nfs             k8s-sigs.io/nfs-subdir-external-provisioner  Delete
false                                75d
Immediate
```

1.7 CNS 的存储方案 OpenEBS

1.7.1 CNS Container Native Storage



Kubernetes的卷通常是基于外部文件系统或块存储实现，这种存储方案称为共享存储Shared Storage
容器原生存储 CNS (Container Attached Storage，早期称为Container Attached Storage CAS) 则是将存储系统自身部署为Kubernetes集群上的一种较新的存储解决方案

- 存储系统自身(包括存储控制器)在Kubernetes上以容器化微服务的方式运行
- 使得工作负载更易于移植，且更容易根据应用程序的需求改动使用的存储
- 通常基于工作负载或者按集群部署，因此消除了共享存储的跨工作负载甚至是跨集群的爆炸半径
- 存储在CAS中的数据可以直接从集群内的容器访问，从而能显著减少读/写时间

基于CNS的存储解决方案，通常包含两类组件

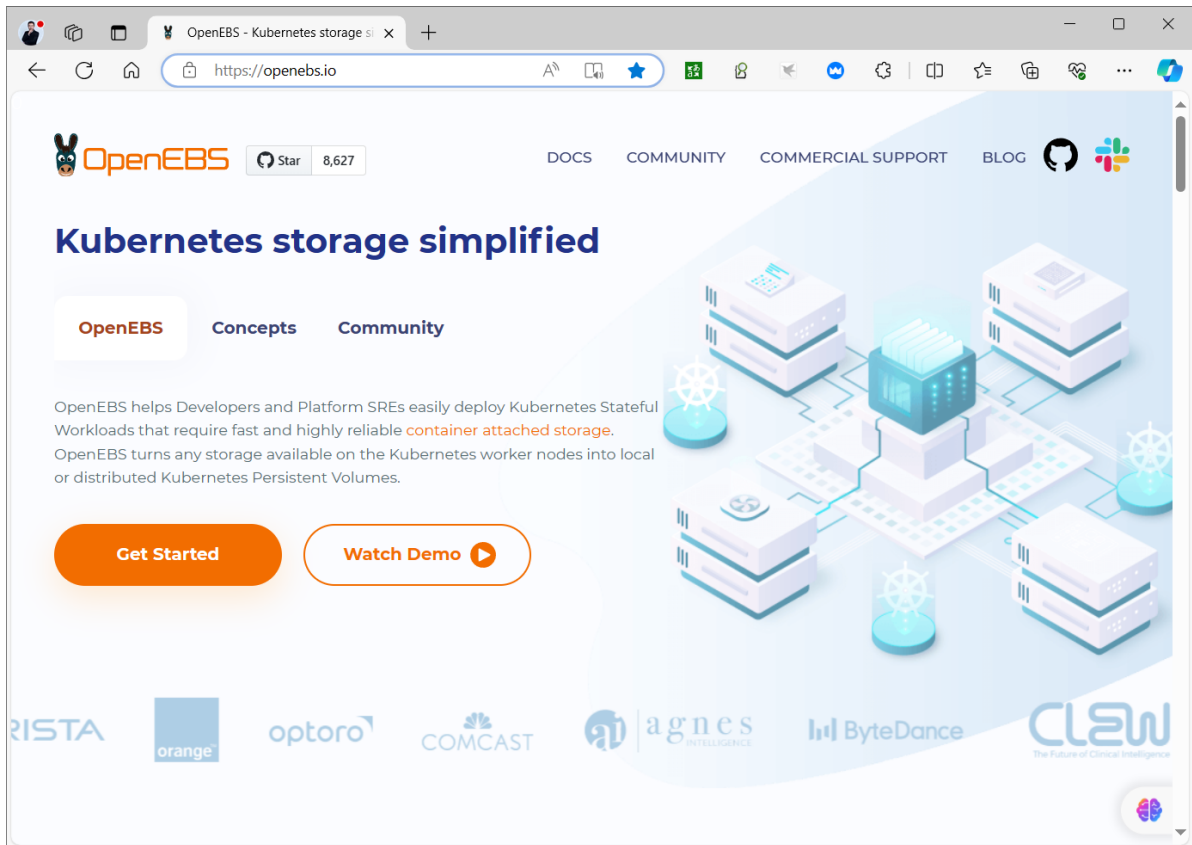
- 控制平面
负责配置卷以及其他同存储相关任务
由存储控制器、存储策略以及如何配置数据平面的指令组成
- 数据平面
接收并执行来自控制平面的有关如何保存和访问容器信息的指令
实现池化存储的存储引擎是数据平面的主要组件
存储引擎本质上负责输入/输出卷路径

1.7.2 OpenEBS 介绍

1.7.2.1 什么是 OpenEBS

OpenEBS是CNS存储机制的著名实现之一，由CNCF孵化

<https://openebs.io/>



<https://openebs.io/docs#what-is-openebs>

OpenEBS 将 Kubernetes 工作线程节点可用的任何存储转换为本地或复制的 Kubernetes 持久卷。

OpenEBS 可帮助应用程序和平台团队轻松部署需要快速、高度持久、可靠且可扩展的容器原生存储的 Kubernetes 有状态工作负载。

OpenEBS 也是基于 NVMe 的存储部署的首选。

OpenEBS 最初由 MayaData 构建并捐赠给云原生计算基金会，现在是 [CNCF 沙盒项目](#)。

1.7.2.2 为什么用户更喜欢 OpenEBS

<https://openebs.io/docs#why-do-users-prefer-openebs>

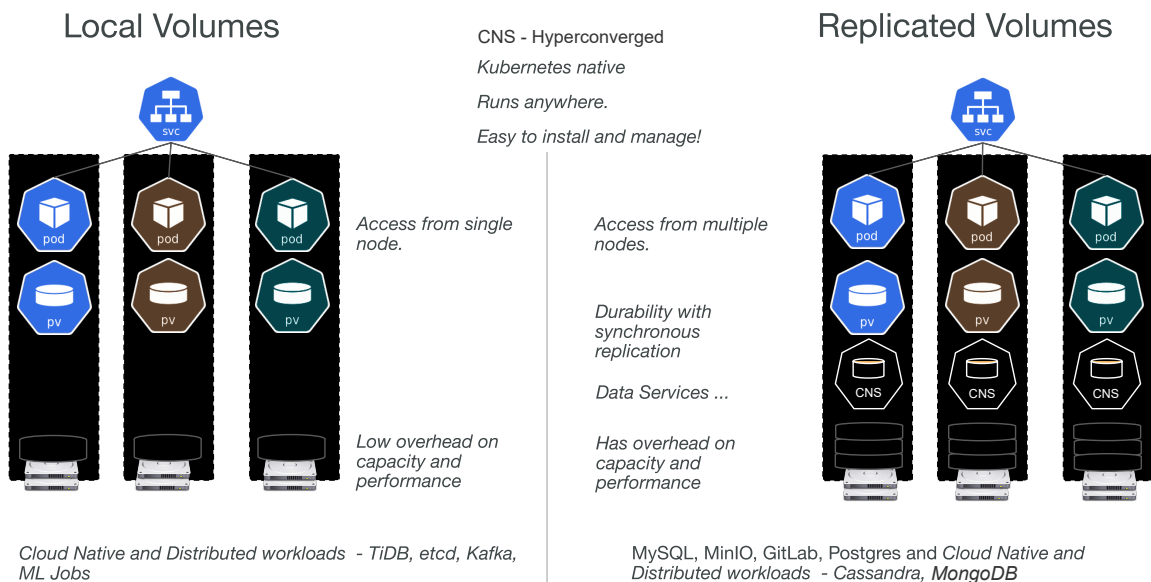
[OpenEBS 采用案例](#)提到了推动用户转向 OpenEBS 的主要原因：

- OpenEBS 在所有 Kubernetes 发行版（本地和云）之间提供一致性。
- OpenEBS with Kubernetes 提高了开发人员和平台 SRE 的工作效率。
- 与其他解决方案相比，OpenEBS 在易用性方面得分很高。设置、安装和配置非常简单。
- OpenEBS 拥有出色的社区支持。
- OpenEBS 是完全开源和免费的。

1.7.2.3 OpenEBS 是做什么的

<https://openebs.io/docs#what-does-openebs-do>

OpenEBS 管理每个 Kubernetes 节点上的可用存储，并使用该存储为有状态工作负载提供本地或复制的持久卷。



对于本地卷：

- OpenEBS 可以在 Hostpaths 上创建持久卷或使用子目录，或者使用本地附加的存储或稀疏文件，或者在现有的 LVM 或 ZFS 上。
- 本地卷直接挂载到 Stateful Pod 中，不会增加数据路径中 OpenEBS 的任何开销，从而减少延迟。
- OpenEBS 为本地卷提供了额外的工具，用于监控、备份/恢复、灾难恢复、由 LVM 或 ZFS 堆栈支持的快照、基于容量的计划等。

对于复制卷：

- OpenEBS 复制存储为每个持久卷创建一个可通过 TCP 访问的 NVMe 目标。
- 有状态 Pod 将数据写入 NVMe-TCP 目标，该目标将数据同步复制到集群中的多个节点。
- OpenEBS 引擎本身部署为 pod，并由 Kubernetes 编排。
- 当运行有状态 Pod 的节点发生故障时，该 Pod 将被重新调度到集群中的另一个节点，OpenEBS 使用其他节点上的可用数据副本提供对数据的访问。
- OpenEBS Replicated Storage 的设计目标是以持久性和性能为目标。它可以有效地管理计算（大页面和内核）和存储（NVMe 驱动器），以提供快速块存储。

1.7.3 OpenEBS 安装和使用

<https://openebs.io/docs/quickstart-guide/installation>

Installation via Helm

<https://openebs.io/docs/quickstart-guide/installation#installation-via-helm>

1. Setup helm repository.

```
helm repo add openebs https://openebs.github.io/openebs
helm repo update
```

`helm repo add openebs https://openebs.github.io/charts` has changed to `helm repo add openebs https://openebs.github.io/openebs`.

OpenEBS provides several options to customize during installation such as:

- Specifying the directory where hostpath volume data is stored or
- Specifying the nodes on which OpenEBS components should be deployed and so forth.

2. Install the OpenEBS helm chart with default values.

```
helm install openebs --namespace openebs openebs/openebs --create-namespace
```

The above commands will install OpenEBS Local PV Hostpath, OpenEBS Local PV LVM, OpenEBS Local PV ZFS, and OpenEBS Replicated Storage components in `openebs` namespace and chart name as `openebs`.

The default OpenEBS helm chart will install both Local Storage and Replicated Storage. If you do not want to install OpenEBS Replicated Storage, use the following command:

```
helm install openebs --namespace openebs openebs/openebs --set engines.replicated.mayastor.enabled=false --create-namespace
```

3. To view the chart and get the output, use the following commands:

```
helm ls -n openebs
```

#Output

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART
	APP VERSION				
openebs	openebs	1	2024-03-25 09:13:00.903321318 +0000 UTC		
deployed	openebs-4.0.1	4.0.1			

4. Verifying OpenEBS Installation

Verify Pods

Default Installation

List the pods in `<openebs>` namespace

```
kubectl get pods -n openebs
```

#In the successful installation of OpenEBS, you should see an example output like below:

NAME	READY	STATUS	RESTARTS
AGE			
openebs-agent-core-674f784df5-7szbm	2/2	Running	0
11m			
openebs-agent-ha-node-nnkmv	1/1	Running	0
11m			
openebs-agent-ha-node-pvcrr	1/1	Running	0
11m			
openebs-agent-ha-node-rqkkk	1/1	Running	0
11m			
openebs-api-rest-79556897c8-b824j	1/1	Running	0
11m			
openebs-csi-controller-b5c47d49-5t5zd	6/6	Running	0
11m			
openebs-csi-node-flq49	2/2	Running	0
11m			
openebs-csi-node-k8d7h	2/2	Running	0
11m			
openebs-csi-node-v7jfh	2/2	Running	0
11m			

openebs-etcd-0	1/1	Running	0
11m			
openebs-etcd-1	1/1	Running	0
11m			
openebs-etcd-2	1/1	Running	0
11m			
openebs-io-engine-7t6tf	2/2	Running	0
11m			
openebs-io-engine-9df6r	2/2	Running	0
11m			
openebs-io-engine-rqph4	2/2	Running	0
11m			
openebs-localpv-provisioner-6ddf7c7978-4fkvs	1/1	Running	0
11m			
openebs-loki-0	1/1	Running	0
11m			
openebs-lvm-localpv-controller-7b6d6b4665-fk78q	5/5	Running	0
11m			
openebs-lvm-localpv-node-mcch4	2/2	Running	0
11m			
openebs-lvm-localpv-node-pdt88	2/2	Running	0
11m			
openebs-lvm-localpv-node-r9jn2	2/2	Running	0
11m			
openebs-nats-0	3/3	Running	0
11m			
openebs-nats-1	3/3	Running	0
11m			
openebs-nats-2	3/3	Running	0
11m			
openebs-obs-callhome-854bc967-5f879	2/2	Running	0
11m			
openebs-operator-diskpool-5586b65c-cwpr8	1/1	Running	0
11m			
openebs-promtail-2vrzk	1/1	Running	0
11m			
openebs-promtail-mwxk8	1/1	Running	0
11m			
openebs-promtail-w7b8k	1/1	Running	0
11m			
openebs-zfs-localpv-controller-f78f7467c-b1r7q	5/5	Running	0
11m			
openebs-zfs-localpv-node-h46m5	2/2	Running	0
11m			
openebs-zfs-localpv-node-svfgq	2/2	Running	0
11m			
openebs-zfs-localpv-node-wm9ks	2/2	Running	0
11m			

Installation with Replicated Storage Disabled

List the pods in `<openebs>` namespace

```
kubectl get pods -n openebs
```

#In the successful installation of OpenEBS, you should see an example output like below:

NAME	READY	STATUS	RESTARTS
openebs-localpv-provisioner-6ddf7c7978-jsstg 3m9s	1/1	Running	0
openebs-lvm-localpv-controller-7b6d6b4665-wfw64 3m9s	5/5	Running	0
openebs-lvm-localpv-node-62lnq 3m9s	2/2	Running	0
openebs-lvm-localpv-node-lhndx 3m9s	2/2	Running	0
openebs-lvm-localpv-node-tlcqv 3m9s	2/2	Running	0
openebs-zfs-localpv-controller-f78f7467c-k7ldb 3m9s	5/5	Running	0
openebs-zfs-localpv-node-5mwbz 3m9s	2/2	Running	0
openebs-zfs-localpv-node-g45ft 3m9s	2/2	Running	0
openebs-zfs-localpv-node-g77g6 3m9s	2/2	Running	0

Verify StorageClasses

List the storage classes to check if OpenEBS has been installed with default StorageClasses.

```
kubectl get sc
```

#In the successful installation, you should have the following StorageClasses created:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION			
mayastor-etcd-localpv waitForFirstConsumer false	openebs.io/local	Delete	
mayastor-loki-localpv waitForFirstConsumer false	openebs.io/local	Delete	
openebs-hostpath waitForFirstConsumer false	openebs.io/local	Delete	
openebs-single-replica true	io.openebs.csi-mayastor	Delete	Immediate

5. PVC和Pod

Hostpath 会生成临时Pod，初始化目录结构，完成后会删除此Pod

默认会创建Pod所在节点的目录/var/openebs/local/pvc-下

```
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: openebs-local-hostpath-pvc
spec:
  storageClassName: openebs-hostpath
```

```
accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 1G

---
apiVersion: v1
kind: Pod
metadata:
  name: redis-with-openebs-local-hostpath
spec:
  containers:
    - name: redis
      image: redis:7-alpine
      ports:
        - containerPort: 6379
          name: redis
      volumeMounts:
        - mountPath: /data
          name: local-storage
  volumes:
    - name: local-storage
      persistentVolumeClaim:
        claimName: openebs-local-hostpath-pvc
```

2 配置管理



2.1 配置说明

2.1.1 配置介绍

生产中的应用程序，一般都会涉及到配置文件，而配置文件经常会有变更，比如：数据库连接、代码版本号、证书更新等

典型场景：

项目经历开发环境、测试环境、预发布环境、线上环境才能完成发布，而每个环境都有定义其独立的各种配置，这些配置手工操作很繁杂，一些流行的开源项目,比如: Nacos, Consul, Apollo等可以实现配置管理，还有大公司专门开发了专用配置管理中心，如百度的disconf等。但通常此方法需要开发者参与，通过开发者写代码实现调用，并且和某种开发语言深度绑定

在容器化大行其道的当前，在为容器化应用提供配置信息方法一般有下面方法

- 制作镜像时，提前将定义好的配置文件拷贝进镜像之中
- 制作镜像时，内置环境变量，在启动容器时通过环境变量向容器传递配置数据，容器启动后将无法传递新的变量
- 启动容器时，基于存储卷向容器传递配置文件,但需要提前准备外部的存储
- 启动容器时，传递自定义的启动的命令，选项，参数

kubernetes作为分布式容器调度平台，同样会遇到配置变更的问题，如果将资源删除，重新修改配置再重新创建，这种方法太繁琐。

kubernetes提供了对 Pod 容器应用可以实现集中式的配置管理功能的相关资源：

- ConfigMap
- Secret
- downwardAPI
- Projected

通过这些组件来实现向pod中的容器应用中注入配置信息的机制，从而避免了开发者参与

注意：对于运行中容器的配置改变，还需要通过应用程序重载相关配置才能生效

2.1.2 配置组件简介

<https://kubernetes.io/zh-cn/docs/concepts/configuration/>

2.1.2.1 Configmap

Configmap是Kubernetes集群中非常重要的一种配置管理资源对象。

借助于ConfigMap API可以向pod中的容器中注入配置信息。

ConfigMap不仅可以保存环境变量或命令行参数等属性，也可以用来保存整个配置文件或者JSON格式的文件。

各种配置属性和数据以 k/v或嵌套k/v 样式 存在到Configmap中

注意：所有的配置信息都是以明文的方式来进行保存，实现资源配置的快速获取或者更新。

2.1.2.2 Secret

Kubernetes集群中，有一些配置属性信息是非常敏感的，所以这些信息在传递的过程中，是不希望其他人能够看到的

Kubernetes提供了一种加密场景中的配置管理资源对象Secret。

它在进行数据传输之前，会对数据进行编码，在数据获取的时候，会对数据进行解码。从而保证整个数据传输过程的安全。

注意：这些数据通常采用Base64机制保存，所以安全性一般

2.1.2.3 DownwardAPI

downwardAPI 为运行在pod中的应用容器提供了一种反向引用。让容器中的应用程序了解所处pod或Node的一些基础外部属性信息。

从严格意义上来说，downwardAPI不是存储卷，它自身就存在。

相较于configmap、secret等资源对象需要创建后才能使用，而downwardAPI引用的是Pod自身的运行环境信息，这些信息在Pod启动的时候就存在。

2.1.2.4 Projected

一个 `projected volumes` 投射卷可以将若干现有的卷源映射到同一个目录之上。

2.2 ConfigMap

2.2.1 ConfigMap 说明

<https://kubernetes.io/zh-cn/docs/concepts/configuration/configmap/>
<https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-pod-configmap/>

Kubernetes提供了对pod中容器应用的集中配置管理组件：ConfigMap。

通过ConfigMap来实现向pod中的容器中注入配置信息的机制。

可以把configmap理解为Linux系统中的/etc目录，专门用来存储配置文件的目录

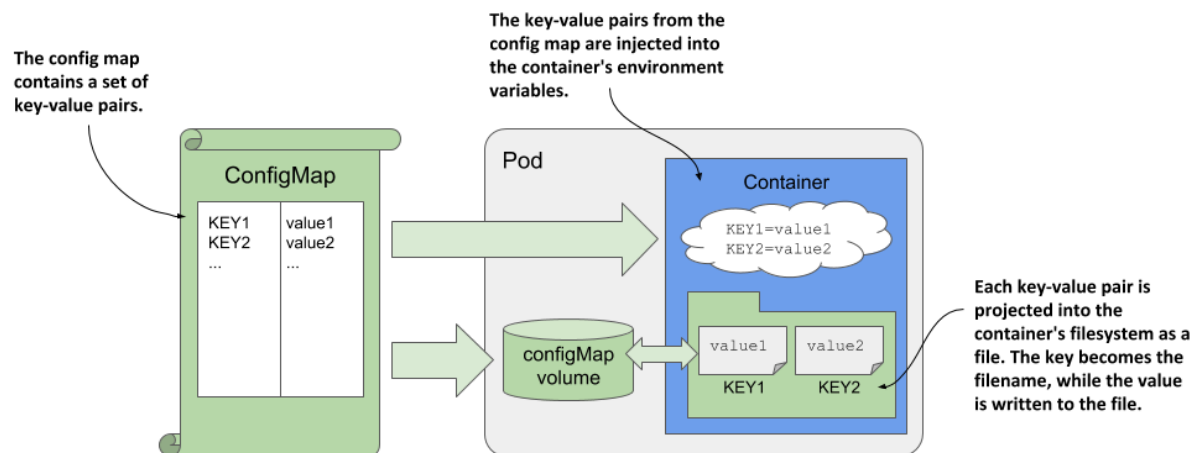
Kubernetes借助于ConfigMap对象实现了将配置信息从容器镜像中解耦，从而增强了工作负载的可移植性、使其配置更易于更改和管理并避免了将配置数据硬编码到Pod配置清单中

ConfigMap不仅仅可以保存单个属性，也可以用来保存整个配置文件。

虽然configmap可以对各种应用程序提供定制配置服务，但是一般不用它来替代专门的配置文件，

从Kubernetes v1.19版本开始，ConfigMap和Secret支持使用immutable字段创建不可变实例，实现不可变基础设施效果

注意: Configmap 属于名称空间级别,只能被同一个名称空间的Pod引用



基本属性

```
#kubectl explain cm
  binaryData      #二进制数据
  data            #文本数据，支持变量和文件
  immutable <boolean> #设为true，不能被修改只能删除，默认为nil可以随时被修改
#注意：基于data的方式传递信息的话，会在pod的容器内部生成一个单独的数据文件
```

数据配置的格式

```
#单行配置数据格式
属性名称key： 属性值value      #单行配置内容，一般保存变量，参数等
文件名：单行内容                #配置文件如果只有一行，也使用此方式，key为文件名，value为文件内容

#多行文件数据格式
文件名称1： |      #注意：| 是"多行键值"的标识符
  文件内容行1      #内容大小不能超过1M
  文件内容行2
  .....
文件名称2： |      #注意：| 是"多行键值"的标识符
  文件内容行1      #内容大小不能超过1M
  文件内容行2
  .....
```

configmap资源类型的创建，与Kubernetes的其他很多资源对象的创建方式一样，主要涉及到两种方式：

- 命令行工具：配置中有大量简单的键值对时建议使用
- 资源定义文件：配置为大量文本内容时建议使用，此方式需要事先准备在资源清单单元文件中加入配置文件内容

虽然 configMap 支持动态更新，但仍可能出现对应的Pod无法自动加载已更新的配置信息而导致无法生效的情况

注意：

- 如果Pod是以subPath 的形式挂载ConfigMap和Secret,那么Pod是不会自动感知到ConfigMap和Secret的更新
- 如果Pod 的变量来自于ConfigMap和Secret中定义的内容,那么ConfigMap和Secret更新后，也不会更新Pod中的变量。

通常为了避免配置更新后没有生效的问题，可以在更新 ConfigMap 之后，手动重建相关的 Pod 或者 Deployment

- 运维方式：可以通过 **重启** 或 **重建** 的方式
 - 使用 `kubectl rollout restart` 命令来重启 Deployment
 - 使用 `kubectl delete pod` 命令来删除 Pod，从而触发 Pod 的重启
- 应用程序方式：不断探测挂载后的 config 的内容变化，程序自身支持进行热加载操作

2.2.2 ConfigMap 创建和更新

2.2.2.1 命令行创建方式

2.2.2.1.1 命令行创建方式说明

命令参数

```
#创建命令格式
kubectl create configmap NAME [--from-file=[key=]source] [--from-literal=key1=value1] [--dry-run=server|client|none] [-n <namespace>] [options]

#参数详解:
--from-literal=key1=value1          #以设置键值对的方式实现变量配置数据
--from-env-file=/PATH/TO/FILE       #以环境变量专用文件的方式实现配置数据
--from-file=[key=]/PATH/TO/FILE     #以配置文件的方式创建配置文件数据，如不指定key，FILE
名称为key名
--from-file=/PATH/TO/DIR            #以配置文件所在目录的方式创建配置文件数据

--dry-run=client -o yaml            #测试运行并显示cm内容

#查看configmap
kubectl create configmap <cm_name> -n <namespace> [-o yaml] --dry-run=client
kubectl get configmap <cm_name> -n <namespace>
kubectl describe configmap <cm_name> -n <namespace>

#删除configmap
kubectl delete configmap <cm_name> [-n <namespace>]
```

2.2.2.1.2 命令行创建方式案例

范例：命令行创建基于key/value形式的变量配置

```
#在使用kubectl创建的时候，通过在命令行直接传递键值对创建
[root@master1 ~]#kubectl create configmap cm-test1 --from-literal=key1='value1' --from-literal=key2='value2'

[root@master1 ~]#kubectl get cm
NAME          DATA   AGE
cm-test1      2       72s
kube-root-ca.crt 1       30d

[root@master1 ~]#kubectl get cm cm-test1 -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2021-03-24T08:35:05Z"
  name: cm-test1
  namespace: default
  resourceVersion: "651484"
  uid: 0fae9ad6-7cf0-448b-a46f-3b4dffa91c23

[root@master1 ~]#kubectl describe cm cm-test1
Name:         cm-test1
Namespace:    default
Labels:       <none>
Annotations:  <none>
```

```

Data
====
key2:
----
value2
key1:
----
value1

BinaryData
====

Events:  <none>

#删除CM
[root@master1 ~]#kubectl delete cm cm-test1

```

范例: 命令行创建基于key/value形式的变量配置

```

#创建CM
[root@master1 ~]#kubectl create configmap pod-test-config --from-
literal=host="127.0.0.1" --from-literal=port="8888"
configmap/pod-test-config created

#验证创建成功
[root@master1 ~]#kubectl get cm
NAME                DATA      AGE
kube-root-ca.crt    1          107d
pod-test-config     2          5s

#显示CM内容
[root@master1 ~]#kubectl get cm pod-test-config -o yaml
apiVersion: v1
data:
  host: 127.0.0.1
  port: "8888"
kind: ConfigMap
metadata:
  creationTimestamp: "2021-02-02T08:23:36Z"
  name: pod-test-config
  namespace: default
  resourceVersion: "46375"
  uid: cef1aeb9-4b7d-4850-9b87-363c534f6e3a

#删除CM
[root@master1 ~]#kubectl delete cm pod-test-config

```

范例: 命令行创建基于环境变量文件的变量配置

```

#如果变量较多，使用上面方式一个个的设定环境变量太繁琐，可以全部添加到环境变量文件中然后基于它来创建CM。
#定制环境变量文件
[root@master1 ~]#cat conf.d/env
key1=value1
key2=value2
#注意: env文件中所有的配置项以 "属性名=属性值" 格式定制

```

```

#将所有环境变量添加到到configmap中
[root@master1 ~]#kubectl create configmap cm-test2 --from-env-file=conf.d/env

[root@master1 ~]#kubectl get cm cm-test2 -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2021-03-24T08:49:42Z"
  name: cm-test2
  namespace: default
  resourceVersion: "653157"
  uid: 9ee7d00f-d2f6-4275-83d6-938bbc898ce5

[root@master1 ~]#kubectl describe cm cm-test2
Name:          cm-test2
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
key1:
----
value1
key2:
----
value2

BinaryData
====

Events:  <none>

#删除CM
[root@master1 ~]#kubectl delete cm cm-test2

```

范例：命令行创建基于配置文件的文件形式CM

```

#直接将多个配置文件创建一个ConfigMap
[root@master1 ~]#ls conf.d/
app1.conf app2.conf app3.conf

[root@master1 ~]#cat conf.d/app1.conf
[app1]
config1

[root@master1 ~]#cat conf.d/app2.conf
[app2]
config2

#文件名自动成为key名
[root@master1 ~]#kubectl create configmap cm-test3 --from-file=./conf.d/app1.conf
--from-file=./conf.d/app2.conf

```

```

[root@master1 ~]#kubectl get cm cm-test3 -o yaml
apiVersion: v1
data:
  app1.conf: |
    [app1]
    config1
  app2.conf: |
    [app2]
    config2
kind: ConfigMap
metadata:
  creationTimestamp: "2021-03-24T08:39:16Z"
  name: cm-test3
  namespace: default
  resourceVersion: "651963"
  uid: 4b1b3836-183b-4648-9c3f-7335046e1d89

[root@master1 ~]#kubectl describe cm cm-test2
Name:          cm-test3
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
app1.conf:
----
[app1]
config1

app2.conf:
----
[app2]
config2

BinaryData
====

Events:  <none>

#删除CM
[root@master1 ~]#kubectl delete cm cm-test3

```

范例: 命令行创建基于目录的CM

```

[root@master1 ~]#ls conf.d/
app1.conf  app2.conf  app3.conf

[root@master1 ~]#cat conf.d/app1.conf
[app1]
config1

[root@master1 ~]#cat conf.d/app2.conf
[app2]
config2

```

```
[root@master1 ~]#cat conf.d/app3.conf
```

```
[app3]
```

```
config3
```

#直接将一个目录下的所有配置文件创建一个ConfigMap

```
[root@master1 ~]#kubectl create configmap cm-test4 --from-file=./conf.d/
```

#结果显示：多个文件之间，属性名是文件名，属性值是文件内容

```
[root@master1 ~]#kubectl get cm cm-test4 -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  app1.conf: |
```

```
    [app1]
```

```
    config1
```

```
  app2.conf: |
```

```
    [app2]
```

```
    config2
```

```
  app3.conf: |
```

```
    [app3]
```

```
    config3
```

```
kind: ConfigMap
```

```
metadata:
```

```
  creationTimestamp: "2021-03-24T08:41:15Z"
```

```
  name: cm-test3
```

```
  namespace: default
```

```
  resourceVersion: "652190"
```

```
  uid: e5453f19-0947-4468-af29-11fb9e88567c
```

#结果显示：多个文件之间，属性名是文件名，属性值是文件内容

```
[root@master1 ~]#kubectl describe cm cm-test4
```

```
Name:          cm-test4
```

```
Namespace:     default
```

```
Labels:        <none>
```

```
Annotations:   <none>
```

```
Data
```

```
====
```

```
app1.conf:
```

```
----
```

```
[app1]
```

```
config1
```

```
app2.conf:
```

```
----
```

```
[app2]
```

```
config2
```

```
app3.conf:
```

```
----
```

```
[app3]
```

```
config3
```

```
BinaryData
```

```
====
```

```
Events:  <none>
```

```
#删除CM
[root@master1 ~]#kubectl delete cm cm-test4
```

2.2.2.2 资源清单文件创建方式

2.2.2.2.1 资源清单文件格式说明

资源清单文件命令格式

```
#清单文件格式
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm_name
  namespace: default
data:
  key: value    #配置信息如果只有一行，也使用此方式，使用卷挂载时，key即为文件名，value为文件
                #内容
  文件名: |      #配置文件有多行使用此方式
  文件内容行1
  文件内容行2
  .....
#注意：CM的清单文件没有spec信息，而是data

#命令式：
kubectl create -f /path/file
#声明式
kubectl apply -f /path/file
```

注意：此方式需要事先将配置文件的内容全部写入清单文件，而且配置文件的格式需要调整才能匹配，所以很不方便，推荐如下方式解决

- 先在命令行执行时指定配置文件的方式创建 CM
- 通过 `kubectl get cm <CM_NAME> -o yaml > cm.yaml` 方式导出资源清单文件
- 或者 `kubectl create configmap NAME --dry-run=client -o yaml > cm.yaml` 方式导出资源清单文件
- 最后调整和修改上面生成的 yaml 资源清单文件

2.2.2.2.2 资源清单文件创建方式案例

范例

```
#资源定义文件
[root@master1 ~]#cat storage-configmap-test.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: config-test
data:
  author: wangxiaochun
  file.conf: |
    [class]
    linux
    go
    java
```

#注释: data: 部分内容是真正的配置项, 有两种格式:

#author: 是key/value格式

#file.conf: | 是文件格式

#创建资源对象

```
[root@master1 ~]#kubectl apply -f storage-configmap-test.yaml
```

#查看

```
[root@master1 ~]#kubectl get cm
```

NAME	DATA	AGE
config-test	2	3s
kube-root-ca.crt	1	30d

```
[root@master1 ~]#kubectl describe cm config-test
```

Name: config-test
Namespace: default
Labels: <none>
Annotations: <none>

Data

====

author:

wangxiaochun

file.conf:

[class]

linux

go

java

BinaryData

====

Events: <none>

#结果显示: 定义了两种类型的配置属性: 文件样式和数据样式, 都是以kv样式展示, 属性名key和属性值value中间使用 "----" 来隔开

#可以在线修改

```
[root@master1 ~]#kubectl edit cm config-test
```

#删除

```
[root@master1 ~]#kubectl delete cm config-test
```

范例: 只读的configmap

```
[root@master1 ~]#cat storage-configmap-immutable-test.yaml
```

apiVersion: v1

kind: ConfigMap

metadata:

name: cm-immutable-test

data:

author: wangxiaochun

file.conf: |

[class]

```

linux
go
java
immutable: true #只读

[root@master1 ~]#kubectl apply -f storage-configmap-immutable-test.yaml

[root@master1 ~]#kubectl get cm cm-immutable-test
NAME          DATA    AGE
cm-immutable-test  2        3m2s

#在线修改configmap提示出错
[root@master1 ~]#kubectl edit cm cm-immutable-test
# * data: Forbidden: field is immutable when `immutable` is set

#可以删除
[root@master1 ~]#kubectl delete cm cm-immutable-test
configmap "cm-immutable-test" deleted

[root@master1 ~]#kubectl get cm
NAME          DATA    AGE
kube-root-ca.crt  1        17d

```

范例：生成redis配置文件对应的cm

```

[root@master1 ~]#kubectl create cm cm-redis --from-file /tmp/redis.conf --dry-run=client -o yaml > cm-redis.yaml

#查看生成的文件内容
[root@master1 config]#head cm-redis.yaml
apiVersion: v1
data:
  redis.conf: "# Redis configuration file example.\n\n Note that in order to
read
the configuration file, Redis must be\n# started with the file path as first
argument:\n#\n#
./redis-server /path/to/redis.conf\n\n# Note on units: when memory size is
needed,
it is possible to specify\n# it in the usual form of 1k 5GB 4M and so
forth:\n#\n#
1k => 1000 bytes\n# 1kb => 1024 bytes\n# 1m => 1000000 bytes\n# 1mb =>
1024*1024
bytes\n# 1g => 1000000000 bytes\n# 1gb => 1024*1024*1024 bytes\n#\n# units
are
case insensitive so 1GB 1Gb 1gb are all the
same.\n\n#####
INCLUDES #####\n\n# Include one or more other
config
.....

```

2.2.2.3 在线更新 configmap

注意：configmap虽然支持在线更新，但是configmap更新后可能不会对已有的Pod的应用生效，可能还需要重建Pod才能生效

```
#创建 configmap
```

```
[root@master1 ~]#kubectl create cm cm-nginx-conf --from-file=nginx.conf

#修改configmap
#方法1,旧版中如果配置内容如果不大,多行内容可以显示在一行,但此方式不方便修改,但如果过大,此方式只能显示大小,而非内容,所以不能修改,新版无此问题
[root@master1 ~]#kubectl edit cm cm-nginx-conf

#方法2
[root@master1 ~]#kubectl get cm cm-nginx-conf -o yaml > cm-config-conf.yaml
[root@master1 ~]#cp cm-config-conf.yaml cm-config-conf.yaml.bak
[root@master1 ~]#vim cm-config-conf.yaml
[root@master1 ~]#kubectl apply -f cm-config-conf.yaml

#方法3
#修改配置
[root@master1 ~]#vim nginx.conf
[root@master1 ~]#kubectl create cm cm-nginx-conf --from-file=nginx.conf --dry-run=client -oyaml | kubectl apply -f -
```

2.2.3 ConfigMap 使用

使用ConfigMap主要有两种方式：

- 通过环境变量的方式直接传递pod
- 使用volume的方式挂载入到pod内的文件中

注意：

- ConfigMap必须在Pod之前创建
- 与ConfigMap在同一个namespace内的pod才能使用ConfigMap，即ConfigMap不能跨命名空间调用。
- ConfigMap通常存放的数据不要超过1M
- CM 可以支持实时更新，在原来的pod里面直接看到效果

2.2.3.1 通过环境变量的方式直接传递 Pod

引用ConfigMap对象上特定的key，以valueFrom赋值给Pod上指定的环境变量

也可以在Pod上使用envFrom一次性导入ConfigMap对象上的所有的key-value,key(可以统一附加特定前缀)即为环境变量,value自动成为相应的变量值

环境变量是容器启动时注入的，容器启动后变量的值不会随CM更改而发生变化,即一次性加载configmap,除非删除容器重建

通过环境变量脚本的方式，预处理这些配置信息

注意：环境变量的名称如果要用横线的话，最好使用下划线 _

环境变量格式

方式1：env 对指定的变量一个一个赋值

```
kubectl explain pod.spec.containers.env
  name          #手工定制环境变量时，设置环境变量的名称，必选字段
  value         #手工定制环境变量时，直接设置环境变量的属性值，不通过CM获取配置，可选字段
  valueFrom     #手工定制环境变量时，设置环境变量的属性来源，可以支持从CM,Secret,downwordAPI获取
```

```
kubectl explain pod.spec.containers.env.valueFrom.configMapKeyRef
  name          #引用指定的configmap
  key            #引用指定的configmap中的具体哪个key
  optional      #如果设置为false，标识该项是必选项，如果设置为true标识这个key是可选的。默认false
```

#此方式实现过程

- 1) 容器中自定义环境变量名
- 2) 根据CM的名称和Key名，找到对应的value
- 3) 再将value赋值给容器的环境变量

方式2: envFrom 使用CM的所有变量实现对变量的批量赋值，此方式生产更为推荐

```
kubectl explain pod.spec.containers.envFrom
  configMapRef    #ConfigMap对象中的所有Key
  secretKeyRef    #Secret对象中的所有Key
  prefix          #为ConfigMap中的每个属性都添加前缀标识
```

#此方实现过程

- 1) 容器中自定义环境变量名，并且和CM的key同名
- 2) 找到指定的CM中所有Key名，将值批量直接赋值给容器中相同名称的变量

2.2.3.2 使用volume的方式挂载入到pod内的文件中

在Pod上将 configMap对象引用为存储卷，而后整体由容器mount至某个目录下，key转为文件名，value即为相应的文件内容

在Pod上定义configMap卷时，仅引用其中的部分key，而后由容器mount至目录下

在容器上仅mount configMap卷上指定的key

容器中挂载的Volume数据可以根据时间戳机制和ConfigMap 同步更新，即configmap变更后会自动加载，但更新时间是不确定的，

而且旧式应用不支持热加载，因此可能不会自动加载更新的配置文件，从而导致配置无法生效

所以一般建议当更新configmap的配置后，可以通过重新Pod使之生效，符合不可变基础设施的理念

推荐滚动升级pod的方式来让ConfigMap内容变化生效

volume 格式

```
#指定卷信息，引用CM
kubectl explain pod.spec.volumes
  name: volume_name #指定volume名
  configMap:
    name: CM_name    #指定CM的名称
    optional: true|false # 表示是否必须存在
    items    <[]Object>
      items if unspecified, each key-value pair in the Data field of the
referenced ConfigMap will be projected into the volume as a file whose name
is the key and content is the value. If specified, the listed keys will be
projected into the specified paths, and unlisted keys will not be present.
If a key is specified which is not present in the ConfigMap, the volume
setup will error unless it is marked optional. Paths must be relative and
may not contain the '..' path or start with '..'.
```

```
#kubectl explain pod.spec.volumes.configMap.items
KIND:      Pod
VERSION:   v1

RESOURCE: items <[]Object>

DESCRIPTION:
  items if unspecified, each key-value pair in the Data field of the
  referenced ConfigMap will be projected into the volume as a file whose name
  is the key and content is the value. If specified, the listed keys will be
  projected into the specified paths, and unlisted keys will not be present.
  If a key is specified which is not present in the ConfigMap, the volume
  setup will error unless it is marked optional. Paths must be relative and
  may not contain the '..' path or start with '..'.

  Maps a string key to a path within a volume.

FIELDS:
  key <string> -required-
    key is the key to project.

  mode <integer>
    mode is Optional: mode bits used to set permissions on this file. Must be
    an octal value between 0000 and 0777 or a decimal value between 0 and 511.
    YAML accepts both octal and decimal values, JSON requires decimal values
    for mode bits. If not specified, the volume defaultMode will be used. This
    might be in conflict with other options that affect the file mode, like
    fsGroup, and the result can be other mode bits set.

  path <string> -required-
    path is the relative path of the file to map the key to. May not be an
    absolute path. May not contain the path element '..'. May not start with
    the string '..'.

#挂载卷到容器内的目录
kubectl explain pod.spec.containers.volumeMounts
volumeMounts:
- name: CM_name      #指定CM的名称
  mountPath; /path   #容器内的挂载点
  readOnly: true    #只读
  subPath <string>   #Path within the volume from which the container's volume
  should be mounted.
  Defaults to "" (volume's root).
```

为了实现不同环境使用不同的配置，可以通过下现方式实现

- 在configmap 对应的配置文件中环境变量，格式：\${VAR_NAME}，本质上configmap的成为了模板文件
- 在deployment等通过env定义变量，并引用configmap实现变量替换

2.2.4 ConfigMap 实战案例

2.2.4.1 范例：env 变量

#资源清单文件

```
[root@master1 ~]#cat storage-configmap-simple-env.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-nginx-config
data:
  port: "10086" #注意:只支持字符串,需要用引号引起来
  user: "www"
---
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-test
spec:
  containers:
  - name: configmap-env-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    env:
      - name: NGINX_HOST
        value: "10.0.0.100" #直接变量赋值
      - name: NGINX_PORT
        valueFrom:
          configMapKeyRef:
            name: cm-nginx-config
            key: port
            optional: true
      - name: NGINX_USER
        valueFrom:
          configMapKeyRef:
            name: cm-nginx-config
            key: user
            optional: false
```

#配置解析：这里面我们可以使用两种方式在pod中传递变量

#资源创建

```
[root@master1 ~]#kubectl apply -f storage-configmap-simple-env.yaml
```

```
[root@master1 ~]#kubectl get cm
```

NAME	DATA	AGE
cm-nginx-config	2	8s
kube-root-ca.crt	1	30d

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
configmap-env-test	1/1	Running	0	29s

```
[root@master1 ~]#kubectl get cm cm-nginx-config -o yaml
```

```
apiVersion: v1
data:
  port: "10086"
  user: www
kind: ConfigMap
metadata:
  annotations:
```

```

kubect1.kubernetes.io/last-applied-configuration: |
  {"apiVersion":"v1","data":
{"port":"10086","user":"www"},"kind":"ConfigMap","metadata":{"annotations":
{},"name":"cm-nginx-config","namespace":"default"}}
  creationTimestamp: "2021-03-24T09:24:43Z"
  name: cm-nginx-config
  namespace: default
  resourceVersion: "657159"
  uid: a446750a-2cd1-4d8b-a672-b83caa1808a7

[root@master1 ~]#kubect1 describe cm cm-nginx-config
Name:          cm-nginx-config
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
port:
----
10086
user:
----
www

BinaryData
====

Events:  <none>

#验证变量
[root@master1 ~]#kubect1 describe pod configmap-env-test |grep -A 5 Environment
Environment:
  NGINX_HOST:  10.0.0.100
  NGINX_PORT:  <set to the key 'port' of config map 'cm-nginx-config'>
Optional: true
  NGINX_USER:  <set to the key 'user' of config map 'cm-nginx-config'>
Optional: false
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-s9hpp
(ro)

[root@master1 ~]#kubect1 exec -it configmap-env-test -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-env-test
TERM=xterm
NGINX_HOST=10.0.0.100
NGINX_PORT=10086
NGINX_USER=www
KUBERNETES_PORT_443_TCP=tcp://192.168.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=192.168.0.1
KUBERNETES_SERVICE_HOST=192.168.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://192.168.0.1:443
NGINX_VERSION=1.20.0

```

```
NJS_VERSION=0.5.3
PKG_RELEASE=1~buster
HOME=/root

#资源删除
[root@master1 ~]#kubectl delete -f storage-configmap-simple-env.yaml
```

2.2.4.2 范例：env 变量

```
#查看Pod默认的变量
[root@master1 ~]#kubectl run pod-test --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
[root@master1 ~]#kubectl exec -it pod-test -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=pod-test
TERM=xterm
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
DEPLOYENV=Production
RELEASE=Stable
PS1=[\u@\h \w]\$
HOME=/root
[root@master1 ~]#kubectl delete pod pod-test

#创建配置文件
[root@master1 ~]#cat storage-configmap-valueFrom-env.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-pod-test
  namespace: default
data:
  host: 0.0.0.0
  port: "8888"

---
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-demo
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    name: pod-test
    env:
    - name: HOST
      valueFrom:
        configMapKeyRef:
          name: cm-pod-test
          key: host
```

optional: true #true时,如果configmap中的key即使不存在,也不会导致容器无法初始化

```
- name: PORT
  valueFrom:
    configMapKeyRef:
      name: cm-pod-test
      key: port
    optional: false #false时,如果configmap中的key不存在,会导致容器无法初始化
```

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-valueFrom-env.yaml
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS	GATES		
configmap-env-demo	1/1	Running	0	6s	10.244.3.4	
node3.wang.org	<none>		<none>			

```
[root@master1 ~]#kubectl exec -it configmap-env-demo -- env
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
HOSTNAME=configmap-env-demo
```

```
TERM=xterm
```

```
HOST=0.0.0.0
```

```
PORT=8888
```

```
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
```

```
KUBERNETES_PORT_443_TCP_PROTO=tcp
```

```
KUBERNETES_PORT_443_TCP_PORT=443
```

```
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
```

```
KUBERNETES_SERVICE_HOST=10.96.0.1
```

```
KUBERNETES_SERVICE_PORT=443
```

```
KUBERNETES_SERVICE_PORT_HTTPS=443
```

```
KUBERNETES_PORT=tcp://10.96.0.1:443
```

```
DEPLOYENV=Production
```

```
RELEASE=Stable
```

```
PS1=[\u@\h \w]\$
```

```
HOME=/root
```

```
[root@master1 ~]#kubectl exec -it configmap-env-demo -- ss -ntl
```

State	Recv-Q	Send-Q	Local Address:Port	Peer
Address:Port			Process	
LISTEN	0	128	0.0.0.0:8888	
0.0.0.0:*				

```
[root@master1 ~]#curl 10.244.3.4:8888
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: configmap-env-demo,
ServerIP: 10.244.3.4!
```

#资源删除

```
[root@master1 ~]#kubectl delete -f storage-configmap-valueFrom-env.yaml
```

2.2.4.3 范例: env 变量

可以基于configmap给pod中的命令执行,以变量的方式,传递一些参数

#清单文件

```
[root@master1 ~]#cat storage-configmap-simple-envargs.yaml
```

```
apiVersion: v1
```

```

kind: ConfigMap
metadata:
  name: cm-command-arg
data:
  time: "3600"
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-cm-command-arg
spec:
  containers:
    - name: pod-cm-command-arg-container
      image: busybox:1.32.0
      command: [ "/bin/sh", "-c", "sleep ${SPECIAL_TIME}" ]
      env:
        - name: SPECIAL_TIME
          valueFrom:
            configMapKeyRef:
              name: cm-command-arg          #引用configmap
              key: time                      #指定CM中的key
        - name: NAME                        #也可以直接对环境变量赋值
          value: "wangxiaochun"
      restartPolicy: Never

```

#应用文件

```
[root@master1 ~]#kubectl apply -f storage-configmap-simple-envargs.yaml
```

```
[root@master1 ~]#kubectl get cm cm-command-arg -o yaml
```

```

apiVersion: v1
data:
  time: "3600"
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"time":"3600"},"kind":"ConfigMap","metadata":
{"annotations":{},"name":"cm-command-arg","namespace":"default"}}
  creationTimestamp: "2021-03-24T09:43:12Z"
  name: cm-command-arg
  namespace: default
  resourceVersion: "659304"
  uid: 670af079-d60f-4ae2-8a63-cee6775d4f61

```

```
[root@master1 ~]#kubectl describe cm cm-command-arg
```

```

Name:          cm-command-arg
Namespace:     default
Labels:        <none>
Annotations:   <none>

```

Data

====

time:

3600

BinaryData

====

Events: <none>

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-cm-command-arg	1/1	Running	0	77s

```
[root@master1 ~]#kubectl exec -it pod-cm-command-arg -- ps aux
```

PID	USER	TIME	COMMAND
1	root	0:00	sleep 3600
7	root	0:00	ps aux

#删除

```
[root@master1 ~]#kubectl delete -f storage-configmap-simple-envargs.yaml
```

2.2.4.4 范例：envFrom 批量导入所有变量

#配置文件资源定义文件

```
[root@master1 ~]#cat storage-configmap-simple-envfrom.yaml
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: cm-nginx
```

```
data:
```

```
  NGINX_PORT: "10086"
```

```
  NGINX_USER: "www"
```

```
---
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: configmap-envfrom-test
```

```
spec:
```

```
  containers:
```

```
  - name: configmap-envfrom-test
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
```

```
    envFrom:          #指定cm
```

```
  - configMapRef:
```

```
    name: cm-nginx  #所有变量从cm中读取
```

#配置解析：envFrom 方式使用cm里面所有的属性信息，而且变量名是默认的

```
[root@master1 ~]#kubectl apply -f storage-configmap-simple-envfrom.yaml
```

```
[root@master1 ~]#kubectl get cm cm-nginx -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  NGINX_PORT: "10086"
```

```
  NGINX_USER: www
```

```
kind: ConfigMap
```

```
metadata:
```

```
  annotations:
```

```
    kubectl.kubernetes.io/last-applied-configuration: |
```

```
      {"apiVersion":"v1","data":
```

```
      {"NGINX_PORT":"10086","NGINX_USER":"www"},"kind":"ConfigMap","metadata":
```

```
      {"annotations":{},"name":"cm-nginx","namespace":"default"}}}
```

```
      creationTimestamp: "2021-03-24T09:33:52Z"
```

```
      name: cm-nginx
```

```

namespace: default
resourceVersion: "658234"
uid: 452f388f-2462-485b-a45c-2d4373eef8f4

[root@master1 ~]#kubectl describe cm cm-nginx
Name:          cm-nginx
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
NGINX_PORT:
----
10086
NGINX_USER:
----
www

BinaryData
====

Events:  <none>

#验证变量
[root@master1 ~]#kubectl exec -it configmap-envfrom-test -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-envfrom-test
TERM=xterm
NGINX_PORT=10086
NGINX_USER=www
KUBERNETES_PORT_443_TCP=tcp://192.168.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=192.168.0.1
KUBERNETES_SERVICE_HOST=192.168.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://192.168.0.1:443
NGINX_VERSION=1.20.0
NJS_VERSION=0.5.3
PKG_RELEASE=1~buster
HOME=/root

#删除
[root@master1 ~]#kubectl delete -f storage-configmap-simple-envfrom.yaml

```

2.2.4.5 范例：volume 生成配置文件并更新生效

```

#configmap资源定义
[root@master1 ~]#cat storage-configmap-simple-volume.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-volume
data:
  author.txt: wangxiaochun

```

```

file.conf: |
  [app]
  config1
  config2
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-volume-test
spec:
  volumes:
    - name: volume-config #指定卷名
      configMap:
        name: cm-volume #指定卷来自cm
  containers:
    - name: nginx
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      volumeMounts:
        - name: volume-config #调用前面定义的卷名
          mountPath: /cmap/ #指定Pod中的挂载点目录

```

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-simple-volume.yaml
```

```
[root@master1 ~]#kubectl get cm cm-volume -o yaml
```

```

apiVersion: v1
data:
  author.txt: wangxiaochun #author成为文件名, wangxiaochun成为文件内容
  file.conf: |
    [app]
    config1
    config2
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"author":"wangxiaochun","file.conf":"
[app]\nconfig1\nconfig2\n"},"kind":"ConfigMap","metadata":{"annotations":
{},"name":"cm-volume","namespace":"default"}}
  creationTimestamp: "2021-03-24T09:59:41Z"
  name: cm-volume
  namespace: default
  resourceVersion: "661232"
  uid: af5aece4-0eb3-46f9-abda-bb0b8441a8ee

```

```
[root@master1 ~]#kubectl describe cm cm-volume
```

```

Name:          cm-volume
Namespace:     default
Labels:        <none>
Annotations:   <none>

```

Data

====

```
file.conf:
```

```

[app]
config1
config2

```

```

author.txt:
----
wangxiaochun

BinaryData
=====

Events:  <none>
[root@master1 ~]#kubectl get pod
NAME                READY   STATUS    RESTARTS   AGE
pod-volume-test     1/1     Running   0           29s

#验证配置文件
[root@master1 ~]#kubectl exec -it pod-volume-test -- sh
# ls /cmap
author.txt  file.conf
# cat /cmap/author.txt
wangxiaochun#
# cat /cmap/file.conf
[app]
config1
config2

#查看一下文件的存储方式
# ls -la /cmap
total 12
drwxrwxrwx 3 root root 4096 Mar 24 10:00 .
drwxr-xr-x 1 root root 4096 Mar 24 10:00 ..
drwxr-xr-x 2 root root 4096 Mar 24 10:00 ..2021_03_24_10_00_38.3037173162
lrwxrwxrwx 1 root root   32 Mar 24 10:00 ..data ->
..2021_03_24_10_00_38.3037173162
lrwxrwxrwx 1 root root   13 Mar 24 10:00 author.txt -> ..data/author
lrwxrwxrwx 1 root root   16 Mar 24 10:00 file.conf -> ..data/file.conf

#结果显示:
#这些文件虽然看起来是在挂载的目录下, 实际上是, 它是经过了两层的软连接才能找到真正挂载的文件
容器内挂载目录的生成的文件file --> ..data/file -->
..YYYY_MM_DD_HH_MM_SS.XXXXXXXX/file
#通过这种双层软连接的方式, 只要容器支持重载技术, 那么只需要更改配置文件就可以实现容器应用的变动

#在线更新配置文件中的内容
[root@master1 ~]#kubectl edit cm cm-volume
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this
file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  author.txt: wang #修改此行
  file.conf: |
    [app]
    config1
    config2
    config3 #加此行
kind: ConfigMap
.....

```

configmap/cm-volume edited

#等一会儿进入pod可以看到配置文件变化

ls -la /cmap

total 12

drwxrwxrwx 3 root root 4096 Mar 24 10:04 .

drwxr-xr-x 1 root root 4096 Mar 24 10:00 ..

drwxr-xr-x 2 root root 4096 Mar 24 10:04 ..2021_03_24_10_04_27.3306965760

lrwxrwxrwx 1 root root 32 Mar 24 10:04 ..data ->

..2022_03_24_10_04_27.3306965760

lrwxrwxrwx 1 root root 13 Mar 24 10:00 author.txt -> ..data/author

lrwxrwxrwx 1 root root 16 Mar 24 10:00 file.conf -> ..data/file.conf

cat /cmap/author.txt

wang# cat /cmap/file.conf

[app]

config1

config2

config3

#结果显示: 无需重启pod对象, 只要等待十几秒后, 配置信息自动发生了变动

#删除

[root@master1 ~]#kubectl delete -f storage-configmap-simple-volume.yaml

2.2.4.6 范例: volume 挂载全部内容

#命令行创建CM, 创建nginx的配置信息

[root@master1 ~]#cat nginx-conf.d/default.conf

```
server {
    listen      8080;
    server_name localhost;
    #access_log /var/log/nginx/host.access.log  main;
    location / {
        root    /usr/share/nginx/html;
        index   index.html index.htm;
    }
    error_page  500 502 503 504 /50x.html;
    location = /50x.html {
        root    /usr/share/nginx/html;
    }
}
```

#命令行创建CM

[root@master1 ~]#kubectl create configmap cm-nginx-conf-files --from-file=nginx-conf.d/default.conf

#清单文件

[root@master1 ~]#cat storage-configmap-nginx-file.yaml

apiVersion: v1

kind: ConfigMap

metadata:

name: cm-nginx-index

data:

index.html: "Nginx Configmap page!" #单行内容的文件生成configmap

apiVersion: v1

kind: Pod

```

metadata:
  name: pod-nginx-conf-configmap
spec:
  volumes:
    - name: nginx-conf
      configMap:
        name: cm-nginx-conf-files
        optional: false
    - name: nginx-index
      configMap:
        name: cm-nginx-index
        optional: false
  containers:
    - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      name: nginx
      volumeMounts:
        - name: nginx-conf
          mountPath: /etc/nginx/conf.d/
          readOnly: true
        - name: nginx-index
          mountPath: /usr/share/nginx/html/
          readOnly: true

```

```
[root@master1 ~]#kubectl get cm
```

NAME	DATA	AGE
cm-nginx-conf-files	1	10m
cm-nginx-index	1	10s
kube-root-ca.crt	1	47h

#查看

```
[root@master1 ~]#kubectl get cm cm-nginx-index -o yaml
```

```

apiVersion: v1
data:
  index.html: Nginx Configmap page!
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"index.html":"Nginx Configmap
page!"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"cm-nginx-
index","namespace":"default"}}
  creationTimestamp: "2021-03-24T11:56:27Z"
  name: cm-nginx-index
  namespace: default
  resourceVersion: "39212"
  uid: 3ec517a4-5bfa-4413-b897-fb735f4b3fe3

```

```
[root@master1 ~]#kubectl get cm cm-nginx-conf-files -o yaml
```

```

apiVersion: v1
data:
  default.conf: |+
    server {
      listen      8080;
      server_name localhost;
      #access_log  /var/log/nginx/host.access.log  main;
      location / {

```

```

        root    /usr/share/nginx/html;
        index   index.html index.htm;
    }
    error_page   500 502 503 504  /50x.html;
    location = /50x.html {
        root    /usr/share/nginx/html;
    }
}

```

kind: ConfigMap

metadata:

```

  creationTimestamp: "2021-03-24T12:00:32Z"
  name: cm-nginx-conf-files
  namespace: default
  resourceVersion: "39713"
  uid: 008efea1-aa21-49aa-9576-002a81237a3b

```

[root@master1 ~]#kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod-nginx-conf-configmap	1/1	Running	0	65s	172.16.3.5	node3.wang.org

#验证配置文件结果

[root@master1 ~]#curl 172.16.3.5:8080

Ngix Configmap page!

#删除

[root@master1 ~]#kubectl delete -f storage-configmap-nginx-file.yaml

[root@master1 ~]#kubectl delete configmap cm-nginx-conf-files

2.2.4.7 范例：volume 挂载 CM 中部分文件并支持动态更新

范例：下面方式指定configmap的指定key对应的文件挂载至Pod中，在Pod目录中只能看到前面特定的配置文件，pod中目录原有的配置文件将被隐藏，支持动态更新

#准备配置文件

[root@master1 ~]#ls nginx-conf.d/

default.conf myserver.conf myserver-gzip.cfg myserver-status.cfg

#配置文件

[root@master1 ~]#cat nginx-conf.d/myserver.conf

```

server {
    listen 8888;
    server_name www.wang.org;
    include /etc/nginx/conf.d/myserver-*.cfg;
    location / {
        root /usr/share/nginx/html;
    }
}

```

#子配置文件,注意:文件是以cfg为后缀,不能以conf文件后缀,会导致冲突

[root@master1 ~]#cat nginx-conf.d/myserver-gzip.cfg

```

gzip on;
gzip_comp_level 5;
gzip_proxied    expired no-cache no-store private auth;

```

```
gzip_types text/plain text/css application/xml text/javascript;
```

```
[root@master1 ~]#cat nginx-conf.d/myserver-status.cfg
```

```
location /nginx-status {  
    stub_status on;  
    access_log off;  
}
```

#创建cm

```
[root@master1 ~]#kubectl create cm cm-nginx-conf-files --from-file=nginx-conf.d/
```

```
[root@master1 config]#kubectl get cm cm-nginx-conf-files -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  default.conf: |  
    server {  
        listen      8080;  
        server_name localhost;  
        #access_log /var/log/nginx/host.access.log main;  
        location / {  
            root /usr/share/nginx/html;  
            index index.html index.htm;  
        }  
        error_page 500 502 503 504 /50x.html;  
        location = /50x.html {  
            root /usr/share/nginx/html;  
        }  
    }  
  myserver-gzip.cfg: |  
    gzip on;  
    gzip_comp_level 5;  
    gzip_proxied expired no-cache no-store private auth;  
    gzip_types text/plain text/css application/xml text/javascript;  
  myserver-status.cfg: |  
    location /nginx-status {  
        stub_status on;  
        access_log off;  
    }  
  myserver.conf: |  
    server {  
        listen 8888;  
        server_name www.wang.org;  
        include /etc/nginx/conf.d/myserver-*.cfg;  
        location / {  
            root /usr/share/nginx/html;  
        }  
    }
```

```
kind: ConfigMap
```

```
metadata:
```

```
  creationTimestamp: "2024-09-30T06:41:55Z"
```

```
  name: cm-nginx-conf-files
```

```
  namespace: default
```

```
  resourceVersion: "251337"
```

```
  uid: 33c08e9b-3c51-4d9b-b02d-a377724d231d
```

#清单文件

```
[root@master1 ~]#cat storage-configmap-nginx-subfile.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-nginx-index
data:
  index.html: "Nginx Sub Configmap Page!\n"
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-cm-nginx-conf
spec:
  volumes:
  - name: nginx-conf
    configMap:
      name: cm-nginx-conf-files
      items:
        - key: myserver.conf          #指定cm中的key
          path: myserver2.conf        #cm中key名称
          mode: 0644                 #Pod中的文件名
        - key: myserver-status.cfg   #Pod中的文件权限
          path: myserver-status2.cfg
          mode: 0644
        - key: myserver-gzip.cfg
          path: myserver-gzip2.cfg
          mode: 0644
      optional: false
  - name: nginx-index
    configMap:
      name: cm-nginx-index
      optional: false
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    name: pod-cm-nginx-conf-container
    volumeMounts:
    - name: nginx-conf
      mountPath: /etc/nginx/conf.d/
      readOnly: true
    - name: nginx-index
      mountPath: /usr/share/nginx/html/
      readOnly: true
```

#创建资源

```
[root@master1 ~]#kubectl apply -f storage-configmap-nginx-subfile.yaml
```

```
[root@master1 ~]#kubectl get cm
```

NAME	DATA	AGE
cm-nginx-conf-files	4	49s
cm-nginx-index	1	47s

```
[root@master1 ~]#kubectl get cm cm-nginx-index -o yaml
```

```
apiVersion: v1
data:
  index.html: |
    Nginx Sub Configmap Page!
kind: ConfigMap
```

```
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"index.html":"Nginx Sub Configmap
      Page!\n"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"cm-nginx-
      index","namespace":"default"}}
    creationTimestamp: "2021-03-24T12:58:31Z"
    name: cm-nginx-index
    namespace: default
    resourceVersion: "46606"
    uid: 475da0b7-b672-4e7f-80d7-b92c89573d32
```

#查看资源

```
[root@master1 ~]#kubectl get cm cm-nginx-conf-files -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  default.conf: |
    server {
      listen      8080;
      server_name localhost;
      #access_log  /var/log/nginx/host.access.log  main;
      location / {
        root      /usr/share/nginx/html;
        index     index.html index.htm;
      }
      error_page   500 502 503 504  /50x.html;
      location = /50x.html {
        root      /usr/share/nginx/html;
      }
    }
  myserver-gzip.cfg: |
    gzip on;
    gzip_comp_level 5;
    gzip_proxied      expired no-cache no-store private auth;
    gzip_types text/plain text/css application/xml text/javascript;
  myserver-status.cfg: |
    location /nginx-status {
      stub_status on;
      access_log off;
    }
  myserver.conf: |
    server {
      listen 8888;
      server_name www.wang.org;
      include /etc/nginx/conf.d/myserver-*.cfg;
      location / {
        root /usr/share/nginx/html;
      }
    }
}
```

```
kind: ConfigMap
```

```
metadata:
```

```
  creationTimestamp: "2021-03-24T12:58:29Z"
  name: cm-nginx-conf-files
  namespace: default
  resourceVersion: "46599"
  uid: f748bda6-da72-42ba-9dc1-157c8d508b55
```

#查看容器的文件

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls -al /etc/nginx/conf.d/
total 12
drwxrwxrwx 3 root root 4096 Mar  9 03:49 .
drwxr-xr-x 1 root root 4096 May 12  2021 ..
drwxr-xr-x 2 root root 4096 Mar  9 03:49 ..2022_03_09_03_49_56.2172224076
lrwxrwxrwx 1 root root  32 Mar  9 03:49 ..data ->
..2022_03_09_03_49_56.2172224076
lrwxrwxrwx 1 root root  24 Mar  9 03:49 myserver-gzip.cfg -> ..data/myserver-
gzip2.cfg
lrwxrwxrwx 1 root root  26 Mar  9 03:49 myserver-status.cfg -> ..data/myserver-
status2.cfg
lrwxrwxrwx 1 root root  20 Mar  9 03:49 myserver.conf -> ..data/myserver2.conf
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod-cm-nginx-conf	1/1	Running	0	7s	172.16.1.15	node1.wang.org

#访问验证

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls /etc/nginx/conf.d/
myserver-gzip.cfg myserver-status.cfg myserver.conf
```

```
[root@master1 ~]#curl 172.16.1.15:8888
```

Nginx Sub Configmap Page!

```
[root@master1 ~]#curl 172.16.1.15:8888/nginx-status
```

Active connections: 1

server accepts handled requests

2 2 2

Reading: 0 Writing: 1 Waiting: 0

```
[root@master1 ~]#curl -I --compressed 172.16.1.15:8888
```

HTTP/1.1 200 OK

Server: nginx/1.20.0

Content-Type: text/html

Connection: keep-alive

ETag: w/"623c6c59-1a"

Content-Encoding: gzip

#更新配置

```
[root@master1 ~]#kubectl edit cm cm-nginx-index
```

apiVersion: v1

data:

index.html: |

Nginx Sub Configmap Page changed! #修改此行

#更新完配置立即生效

```
[root@master1 ~]#curl 172.16.1.15:8888
```

Nginx Sub Configmap Page changed!

#更新配置

```
[root@master1 ~]#kubectl edit cm cm-nginx-conf-files
```

myserver.conf: |

server {

listen 9999; #修改此行

```
server_name www.wang.org;

#在线更新完配置文件的时间戳没有变化，但文件内容变化了
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls -al /etc/nginx/conf.d/

[root@master1 ~]#curl 172.16.1.15:9999
curl: (7) Failed to connect to 172.16.1.15 port 9999: Connection refused
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- nginx -s reload
2022/03/24 13:17:56 [notice] 43#43: signal process started
[root@master1 ~]#curl 172.16.1.15:9999
Nginx Sub Configmap Page changed!

#删除资源
[root@master1 ~]#kubectl delete -f storage-configmap-nginx-subfile.yaml
[root@master1 ~]#kubectl delete cm cm-nginx-conf-files
```

2.2.4.8 范例：volume 基于subpath实现挂载CM部分文件并修改配置文件名称但不支持动态更新

```
kubectl explain pod.spec.volumes.configMap.items

FIELDS:
  key <string> -required-
    key is the key to project.

  mode <integer>
    mode is Optional: mode bits used to set permissions on this file. Must be
    an octal value between 0000 and 0777 or a decimal value between 0 and 511.
    YAML accepts both octal and decimal values, JSON requires decimal values
    for mode bits. If not specified, the volume defaultMode will be used. This
    might be in conflict with other options that affect the file mode, like
    fsGroup, and the result can be other mode bits set.

  path <string> -required-
    path is the relative path of the file to map the key to. May not be an
    absolute path. May not contain the path element '..'. May not start with
    the string '..'.

kubectl explain pod.spec.containers.volumeMounts
FIELDS:
  mountPath <string> -required-
    Path within the container at which the volume should be mounted. Must not
    contain ':'.

  mountPropagation <string>
    mountPropagation determines how mounts are propagated from the host to
    container and the other way around. When not set, MountPropagationNone is
    used. This field is beta in 1.10.

  name <string> -required-
    This must match the Name of a Volume.

  readOnly <boolean>
    Mounted read-only if true, read-write otherwise (false or unspecified).
    Defaults to false.

  subPath <string>
```

Path within the volume from which the container's volume should be mounted. Defaults to "" (volume's root).

subPathExpr <string>

Expanded path within the volume from which the container's volume should be mounted. Behaves similarly to SubPath but environment variable references \$(VAR_NAME) are expanded using the container's environment. Defaults to "" (volume's root). SubPathExpr and SubPath are mutually exclusive.

范例：挂载部分文件并改名，此方式不支持动态更新

下面方式指定configmap的指定key对应的文件挂载至Pod中，在Pod目录中只能看到前面特定的配置文件，pod中目录原有的配置文件不会被隐藏

#准备配置文件同上一样

```
[root@master1 ~]#ls nginx-conf.d/
```

```
default.conf  myserver.conf  myserver-gzip.cfg  myserver-status.cfg
```

#将上面的配置文件都加入cm

```
[root@master1 ~]#kubectl create cm cm-nginx-conf-files --from-file=nginx-conf.d/
```

```
[root@master1 ~]#cat storage-configmap-nginx-usesubfile.yaml
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: cm-nginx-index
```

```
data:
```

```
  index.html: "Nginx Use Sub Configmap Page!\n"
```

```
---
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: pod-cm-nginx-conf
```

```
spec:
```

```
  volumes:
```

```
  - name: nginx-conf
```

```
    configMap:
```

```
      name: cm-nginx-conf-files
```

```
      optional: false
```

```
  - name: nginx-index
```

```
    configMap:
```

```
      name: cm-nginx-index
```

```
      optional: false
```

```
  containers:
```

```
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
```

```
    name: pod-cm-nginx-conf-container
```

```
    volumeMounts:
```

```
    - name: nginx-conf
```

```
      mountPath: /etc/nginx/conf.d/myserver2.conf
```

#修改生成的配置文件名

```
      subPath: myserver.conf
```

#指定nginx-conf中的特定文件，

而非所有文件

```
      readOnly: true
```

```
    - name: nginx-conf
```

```
      mountPath: /etc/nginx/conf.d/myserver-gzip2.cfg
```

#修改生成的配置文件名

```
      subPath: myserver-gzip.cfg
```

#指定nginx-conf中的特定文件，

而非所有文件

```
readOnly: true
- name: nginx-index
  mountPath: /usr/share/nginx/html/
  readOnly: true
```

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-nginx-usesubfile.yaml
```

```
[root@master1 ~]#kubectl get cm
```

NAME	DATA	AGE
cm-nginx-conf-files	4	6m56s
cm-nginx-index	1	4s

```
[root@master1 ~]#kubectl get cm cm-nginx-conf-files -o yaml
```

```
apiVersion: v1
data:
  default.conf: |
    server {
      listen      8080;
      server_name localhost;
      #access_log  /var/log/nginx/host.access.log  main;
      location / {
        root   /usr/share/nginx/html;
        index  index.html index.htm;
      }
      error_page  500 502 503 504  /50x.html;
      location = /50x.html {
        root   /usr/share/nginx/html;
      }
    }
  myserver-gzip.cfg: |
    gzip on;
    gzip_comp_level 5;
    gzip_proxied      expired no-cache no-store private auth;
    gzip_types text/plain text/css application/xml text/javascript;
  myserver-status.cfg: |
    location /nginx-status {
      stub_status on;
      access_log off;
    }
  myserver.conf: |
    server {
      listen 8888;
      server_name www.wang.org;
      include /etc/nginx/conf.d/myserver-*.cfg;
      location / {
        root /usr/share/nginx/html;
      }
    }
kind: ConfigMap
metadata:
  creationTimestamp: "2021-03-24T13:29:38Z"
  name: cm-nginx-conf-files
  namespace: default
  resourceVersion: "50229"
  uid: 1729a0b7-e105-45e5-99c6-d2716b3a93f6
```

```
[root@master1 ~]#kubectl get cm cm-nginx-index -o yaml
apiVersion: v1
data:
  index.html: |
    Nginx Use Sub Configmap Page!
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"index.html":"Nginx Use Sub Configmap
      Page!\n"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"cm-nginx-
      index","namespace":"default"}}
  creationTimestamp: "2021-03-24T13:39:33Z"
  name: cm-nginx-index
  namespace: default
  resourceVersion: "51408"
  uid: 1d7d9eb4-00ad-4b1c-af6d-fa6bcab7dee4
```

#验证结果

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls -al
/etc/nginx/conf.d/
total 28
drwxr-xr-x 1 root root 4096 Mar  9 04:36 .
drwxr-xr-x 1 root root 4096 May 12  2021 ..
-rw-r--r-- 1 root root 1093 Mar  9 04:36 default.conf
-rw-r--r-- 1 root root 149 Mar  9 04:36 myserver-gzip2.cfg
-rw-r--r-- 1 root root 163 Mar  9 04:36 myserver2.conf
```

#结果显示：这种方式不会覆盖容器中的默认文件，除非文件名同名,且此方式没有软链接

```
[root@master1 ~]#kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
NOMINATED NODE   READINESS GATES
pod-cm-nginx-conf 1/1     Running   0           3s    172.16.3.9     node3.wang.org   <none>          <none>
[root@master1 ~]#curl 172.16.3.9:8888
Nginx Use Sub Configmap Page!
```

```
[root@master1 ~]#curl --compressed -I 172.16.3.9:8888
HTTP/1.1 200 OK
Server: nginx/1.20.0
Content-Type: text/html
Connection: keep-alive
ETag: w/"623c7495-1e"
Content-Encoding: gzip
```

#更新文件

```
[root@master1 ~]#kubectl edit cm cm-nginx-conf-files
myserver.conf: |
  server {
    listen 7777;
    server_name www.wang.org;
    .....
  }
```

#没有表现为时间戳的目录形式

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls -al
/etc/nginx/conf.d/
```

```
total 28
drwxr-xr-x 1 root root 4096 Mar  4 12:22 .
drwxr-xr-x 1 root root 4096 May 12 2021 ..
-rw-r--r-- 1 root root 1093 Mar  4 12:22 default.conf
-rw-r--r-- 1 root root 149 Mar  4 12:22 myserver-gzip2.cfg
-rw-r--r-- 1 root root 163 Mar  4 12:22 myserver2.conf

#观察配置文件不变化
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- cat
/etc/nginx/conf.d/myserver2.conf
server {
    listen 8888;
    server_name www.wang.org;
    include /etc/nginx/conf.d/myserver-*.cfg;
    location / {
        root /usr/share/nginx/html;
    }
}

#删除
[root@master1 ~]#kubectl delete -f storage-configmap-nginx-usesubfile.yaml
[root@master1 ~]#kubectl delete cm cm-nginx-conf-files
```

2.2.4.9 范例：volume 基于subPath 挂载CM 部分文件并保留原目录中的其它文件

范例：

```
#查看容器内的文件列表
[root@master1 ~]#docker run --rm --name nginx registry.cn-
beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 ls /etc/nginx/
conf.d fastcgi_params mime.types modules nginx.conf scgi_params uwsgi_params

#导出配置文件
[root@master1 ~]#docker run --rm --name nginx registry.cn-
beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0 cat /etc/nginx/nginx.conf >
nginx.conf

#创建configmap
[root@master1 ~]#kubectl create cm cm-nginx-conf --from-file=nginx.conf

[root@master1 ~]#kubectl get cm cm-nginx-conf
NAME          DATA   AGE
cm-nginx-conf 1       39s

#查看配置内容
[root@master1 ~]#kubectl describe cm cm-nginx-conf
Name:         cm-nginx-conf
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
nginx.conf:
----
```

```

user nginx;
worker_processes auto;

error_log /var/log/nginx/error.log notice;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;
    #tcp_nopush on;

    keepalive_timeout 65;

    #gzip on;

    include /etc/nginx/conf.d/*.conf;
}

```

BinaryData

====

Events: <none>

#准备配置文件

[root@master1 ~]#cat storage-configmap-subPath-nginx-1.yaml

apiVersion: v1

kind: Pod

metadata:

name: pod-cm-nginx-conf

spec:

volumes:

- name: volume-nginx-conf

configMap:

name: cm-nginx-conf

containers:

- image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0

name: pod-cm-nginx-conf-container

command: ["sh", "-c", "sleep 3600"]

volumeMounts:

- name: volume-nginx-conf

mountPath: /etc/nginx/

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-subPath-nginx-1.yaml
```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-cm-nginx-conf	1/1	Running	0	2m

#其它文件都是覆盖无法看到

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls /etc/nginx/
nginx.conf
```

#删除Pod

```
[root@master1 ~]#kubectl delete -f storage-configmap-subPath-nginx-1.yaml
```

#修改配置文件

```
[root@master1 ~]#cat storage-configmap-subPath-nginx-2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-cm-nginx-conf
spec:
  volumes:
  - name: volume-nginx-conf
    configMap:
      name: cm-nginx-conf
      items:
      - key: nginx.conf
        path: etc/nginx/nginx.conf
```

#必须是相对路径，且和下面subPath路径相同

```
    containers:
    - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
      name: pod-cm-nginx-conf-container
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
      - name: volume-nginx-conf
        mountPath: /etc/nginx/nginx.conf
        subPath: etc/nginx/nginx.conf
```

#必须是相对路径，且和volumes的path路径相同

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-subPath-nginx-2.yaml
```

#查看其它文件都在

```
[root@master1 ~]#kubectl exec -it pod-cm-nginx-conf -- ls /etc/nginx/
conf.d      mime.types  nginx.conf  uwsgi_params
fastcgi_params  modules    scgi_params
```

#删除

```
[root@master1 ~]#kubectl delete -f storage-configmap-subPath-nginx-2.yaml
[root@master1 ~]#kubectl delete cm cm-nginx-conf
```

2.2.4.10 范例：利用 configmap volume 实现 Redis 配置

```
#配置
[root@master1 ~]#cat storage-configmap-redis.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```

  name : redis-conf
data:
  redis.conf: |
    appendonly yes
---
apiVersion: v1
kind: Pod
metadata:
  name : cm-redis-demo
spec :
  volumes :
  - name : redis-data
    emptyDir: {}
  - name : redis-config
    configMap:
      name: redis-conf
      items:
      - key: redis.conf
        path: redis.conf
  containers:
  - name: redis
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/redis:6.2.5
    command :
    - redis-server
    - "/conf/redis.conf"
    ports:
    - containerPort : 6379
    volumeMounts :
    - name: redis-data
      mountPath: /data
    - name : redis-config
      mountPath: /conf

```

#应用

```
[root@master1 ~]#kubectl apply -f storage-configmap-redis.yaml
```

#检查

```
[root@master1 ~]#kubectl get cm redis-conf
```

```

NAME      DATA   AGE
redis-conf 1       17s

```

```
[root@master1 ~]#kubectl exec -it cm-redis-demo -- ls /data/ /conf
```

```

/conf:
redis.conf

```

```

/data/:
appendonly.aof

```

```

[root@master1 ~]#kubectl exec -it cm-redis-demo -- cat /conf/redis.conf
appendonly yes

```

#在线修改

```
[root@master1 ~]#kubectl edit cm redis-conf
```

```

apiVersion: v1
data:
  redis.conf: |
    appendonly yes

```

```
requirepass 123456 #添加此行
```

```
#再要等一会儿再查看,发现已同步至配置文件,但是不会生效,还需要重启redis服务才能生效  
[root@master1 ~]#kubectl exec -it cm-redis-demo -- cat /conf/redis.conf  
appendonly yes  
requirepass 123456
```

```
#删除
```

```
[root@master1 ~]#kubectl delete -f storage-configmap-redis.yaml
```

2.3 Secret

2.3.1 Secret 介绍



```
https://kubernetes.io/zh-cn/docs/concepts/configuration/secret/  
https://kubernetes.io/zh-cn/docs/tasks/configmap-secret/
```

Secret 和 Configmap 相似，也可以提供配置数据，但主要用于为Pod提供敏感需要加密的信息

Secret 主要用于存储密码、证书私钥，SSH 密钥，OAuth令牌等敏感信息，这些敏感信息采用base64编码保存，相对明文存储更安全

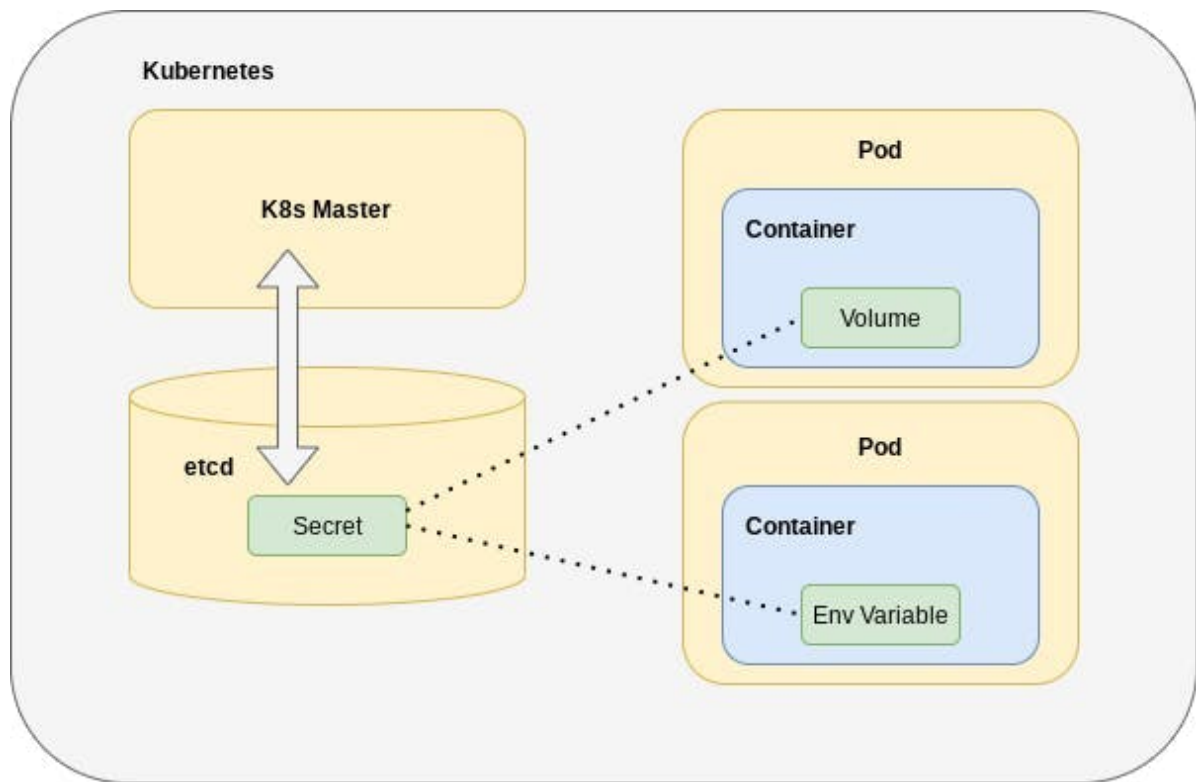
相比于直接将敏感数据配置在Pod的定义或者镜像中，Secret提供了更加安全的机制，将需要共享的数据进行加密，防止数据泄露。

Secret的对象需要单独定义并创建，通常以数据卷的形式挂载到Pod中，Secret的数据将以文件的形式保存，容器通过读取文件可以获取需要的数据。

Secret volume是通过tmpfs（内存文件系统）实现的，所以这种类型的volume不是永久存储的。

每个Secret的数据不能超过1MB，支持通过资源限额控制每个名称空间的Secret的数量

注意: Secret 属于名称空间级别，只能被同一个名称空间的Pod引用



Secret 分成以下常见大的分类

类型	解析
generic	通用类型，基于base64编码用来存储密码，公钥等。常见的子类型有： Opaque,kubernetes.io/service-account-token,kubernetes.io/basic-auth,kubernetes.io/ssh-auth,bootstrap.kubernetes.io/token,kubernetes.io/rbd
tls	专门用于保存tls/ssl用到证书和配对的私钥,常见的子类型:kubernetes.io/tls
docker-registry	专用于让kubelet启动Pod时从私有镜像仓库pull镜像时，首先认证到仓库Registry时使用 常见的子类型:kubernetes.io/dockercfg,kubernetes.io/dockerconfigjson

Secret 细化为的子类型(Type)

大类型	Builtin Type	说明
generic	opaque	arbitrary user-defined data
generic	kubernetes.io/service-account-token	service account token
generic	kubernetes.io/basic-auth	credentials for basic authentication
generic	kubernetes.io/ssh-auth	credentials for SSH authentication
generic	bootstrap.kubernetes.io/token	bootstrap token data 初始化
tls	kubernetes.io/tls	data for a TLS client or server
docker-registry	kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file 新版
docker-registry	kubernetes.io/dockercfg	serialized ~/.dockercfg file 旧版

注意:

不同类型的Secret, 在定义时支持使用的标准字段也有所不同

例如: ssh-auth类型的Secret应该使用ssh-privatekey, 而basic-auth类型的Secret则需要使用username和password等。

另外也可能存在一些特殊的类型, 用于支撑第三方需求, 例如: ceph的keyring信息使用的kubernetes.io/rbd等

Secret 创建方式

- 手动创建: 用户自行创建的Secret 常用来存储用户私有的一些信息
- 自动创建: 集群自动创建的Secret 用来作为集群中各个组件之间通信的身份校验使用

范例: Secret 大的类别

```
[root@master1 ~]#kubectl create secret -h
Create a secret using specified subcommand.

Available Commands:
  docker-registry  创建一个给 Docker registry 使用的 Secret
  generic          Create a secret from a local file, directory, or literal value
  tls              创建一个 TLS secret

Usage:
  kubectl create secret [flags] [options]

Use "kubectl <command> --help" for more information about a given command.
Use "kubectl options" for a list of global command-line options (applies to all commands).
```

范例: 查看 Secret

```
[root@master1 ~]#kubectl get secrets -A
```

NAMESPACE	NAME	TYPE
DATA AGE		
ingress-nginx	ingress-nginx-admission	Opaque
3 3d17h		
kube-system	bootstrap-token-n44d75	bootstrap.kubernetes.io/token
6 47d		

```
[root@master1 ~]#kubectl get secrets -n ingress-nginx ingress-nginx-admission -o
yaml
apiVersion: v1
data:
  ca:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSUJkeNDQVJ5Z0F3SUJBZ01SQU42d3dxOHowWXBQ
MjA1VkZyOD1EMGt3Q2dzSutvWk16
.....
  cert:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSUJlakNDQVdDZ0F3SUJBZ01RYXI3ZDh3aT14TXIw
cnVFVVVzVTZWVFLQmdncWhrak9Q.....
  key:
LS0tLS1CRUdJTiBFQyBQUk1lWQVRFIEtFWS0tLS0tCk1lY0NBUEVFSU1aL0p2K3JuZlF1ZFc4WnVsMmI5
NGpoSUFFVktQTUzM0Q4UW13aS9r.....
kind: Secret
metadata:
  creationTimestamp: "2021-04-20T09:14:08Z"
  name: ingress-nginx-admission
  namespace: ingress-nginx
  resourceVersion: "670798"
  uid: b371e542-44de-4343-893b-268697e1e13b
type: Opaque
```

```
[root@master1 ~]#kubectl get secrets -n kube-system bootstrap-token-n44d75 -o
yaml
apiVersion: v1
data:
  auth-extra-groups:
c3lzdGvbm0Jvb3RzdHJhchB1cnM6a3ViZWFKbTpkZWZhdWx0LW5vZGutdG9rZW4=
  description:
VGhlIGRlZmF1bHQgYm9vdHN0cmFwIHRva2VuIGdlbmVvYXRlZCBieSAna3ViZWFKbSBpbm10Jy4=
  token-id: bjQ0ZDc1
  token-secret: bGFxcmp4aTd6MW93NDZudQ==
  usage-bootstrap-authentication: dHJlZQ==
  usage-bootstrap-signing: dHJlZQ==
kind: Secret
metadata:
  creationTimestamp: "2021-03-07T04:30:23Z"
  name: bootstrap-token-n44d75
  namespace: kube-system
  resourceVersion: "222"
  uid: 4f7f1030-a32e-42ad-a770-03fb6650e8d1
type: bootstrap.kubernetes.io/token
```

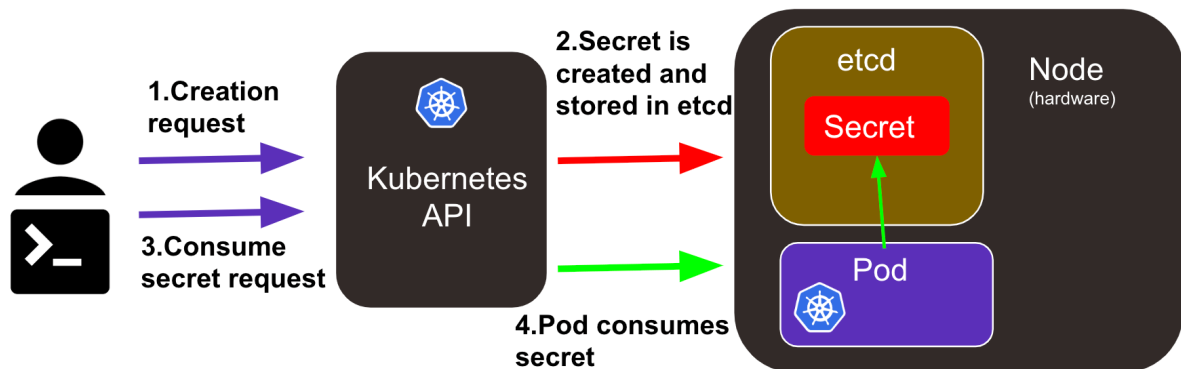
范例: 查看 Secret

```
[root@master1 ~]#kubectl get secret
```

NAME	TYPE	DATA	AGE
default-token-w2gph	kubernetes.io/service-account-token	3	4d13h

```
[root@master1 ~]#kubectl get secrets default-token-w2gph -o yaml
apiVersion: v1
data:
  ca.crt:
LS0tLS1CRudJTtBDRVJUSUZJQ0FURSB0tLS0tCk1JSUMvakNDQWVhZ0F3SUJBZ0lCQURBTkNa3Foa2lH
OXcwQkFRc0ZBREFTVjNd0VRWURWUVFERXdwcMRXSmwKY201bGRHVnpNQjRYRFRJeU1ETXlNakV5TVRV
MU5Gb1hEVE15TURNee9URXlNVFUXtkZvd0ZURVRNqkVHQTFVRQpBeE1L.....
  namespace: ZGVmYXVsdA==
  token:
ZXlKaGJHY2lPaUpTVXpJMU5pSXNjbXRRwWkNjNk1tSTNa0UwU1ZnewEzce1jRlI2U1ZodGRrwnJhRjgy
TUZWnk5HTlNVmjVtTldwRFpuY3Rnb1E1YmswawZRLmV5SnBjM01pT2lKcmRXSmxjbTVsZEdwekwzTmxj
blpwWTJwaFkyTnZkvZUswl3awEzVmlawEp1wlhsbGN5NXBieTl6wlhk.....
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: default
    kubernetes.io/service-account.uid: 6c962755-e74b-433c-b357-faa6376fce5f
  creationTimestamp: "2021-03-22T12:16:16Z"
  name: default-token-w2gph
  namespace: default
  resourceVersion: "408"
  uid: b7a608c7-5fc5-4320-b75b-da203ee619e1
type: kubernetes.io/service-account-token
```

2.3.2 Secret 命令式创建



官方说明

<https://kubernetes.io/zh-cn/docs/tasks/configmap-secret/managing-secret-using-kubectl/>

格式

```
kubectl create secret [flags] [options] [-n <namespace>]
```

帮助信息

```
[root@master ~]# kubectl create secret -h
Create a secret using specified subcommand.
```

Available Commands:

```
docker-registry #创建一个给 Docker registry 使用的 secret
generic         #从本地 file, directory 或者 literal value 创建一个 secret
tls             #创建一个 TLS secret
```

Usage:

```
kubectl create secret [flags] [options]
```

命令格式

#generic类型

```
kubectl create secret generic NAME [--type=string] [--from-file=[key=]source] [--from-literal=key1=value1]
```

```
--from-literal=key1=value1      #以命令行设置键值对的环境变量方式配置数据
--from-env-file=/PATH/FILE      #以环境变量的专用文件的方式配置数据
--from-file=[key=]/PATH/FILE    #以配置文件的方式创建配置数据，如不指定key，FILE名称为key名
--from-file=/PATH/DIR           #以配置文件所在目录的方式创建配置数据
```

#该命令中的--type选项进行定义除了后面docker-registry和tls命令之外的其它子类型，有些类型有key的特定要求

#注意：如果value中有特殊字符，比如:\$,\,*,=,!等，需要用\进行转义或用单引号'引起来

#tls类型

```
kubectl create secret tls NAME --cert=/path/file --key=/path/file
```

#其保存cert文件内容的key名称不能指定自动为tls.crt，而保存private key的key不能指定自动为tls.key

#docker-registry类型

#方式1:基于用户名和密码方式实现

```
kubectl create secret docker-registry NAME --docker-username=user --docker-password=password --docker-email=email [--docker-server=string] [--from-file=[key=]source]
```

#方式2:基于dockerconfig文件方式实现

```
kubectl create secret docker-registry KEYNAME --from-file=.dockerconfigjson=path/to/.docker/config.json
```

#从已有的json格式的文件加载生成的就是dockerconfigjson类型，命令行直接生成的也是该类型

范例:

#创建generic类型

```
kubectl create secret generic my-secret-generic --from-file=/path/bar
kubectl create secret generic my-secret-generic --from-file=ssh-privatekey=~/.ssh/id_rsa --from-file=ssh-publickey=~/.ssh/id_rsa.pub
kubectl create secret generic my-secret-generic --from-literal=username=admin --from-literal=password=123456
kubectl create secret generic my-secret-generic --from-env-file=path/to/bar.env
```

#创建tls类型

```
kubectl create secret tls my-secret-tls --cert=certs/wang.org.cert --key=certs/wang.org.key
```

```
#创建docker-registry类型
#参考示例: https://kubernetesmeetup.github.io/docs/tasks/configure-pod-container/pull-image-private-registry/
#基于私有仓库的用户名和密码
kubectl create secret docker-registry my-secret-docker-registry --docker-
server=harbor.wang.org --docker-username=admin --docker-password=123456 --
docker-email=29308620@qq.com

#先登录并认证到目标仓库server上, 认证凭据自动保存在dockercfg文件中, 基于dockerconfig文件实现
kubectl create secret docker-registry dockerharbor-auth --from-
file=.dockerconfigjson=/root/.docker/config.json

kubectl create secret generic dockerharbor-auth --
type='kubernetes.io/dockerconfigjson' --from-
file=.dockerconfigjson=/root/.docker/config.json
```

2.3.3 Secret 声明式创建

Secret 数据存放在data或stringData字段, 其中data字段中的Key/value必须使用base64编码存放, 而stringData使用明文存放

Secret 资源的元数据: 除了name, namespace之外, 常用的还有labels, annotations

- annotation的名称遵循类似于labels的名称命名格式, 但其数据长度不受限制
- 它不能用于被标签选择器作为筛选条件; 但常用于为那些仍处于Beta阶段的应用程序提供临时的配置接口
- 管理命令: `kubectl annotate TYPE/NAME KEY=VALUE`

范例: 查看 Secret 字段说明

```
[root@master1 ~]#kubectl explain secret
KIND:      Secret
VERSION:   v1

DESCRIPTION:
    Secret holds secret data of a certain type. The total bytes of the values
    in the Data field must be less than MaxSecretSize bytes.

FIELDS:
    apiVersion    <string>
        APIVersion defines the versioned schema of this representation of an
        object. Servers should convert recognized schemas to the latest internal
        value, and may reject unrecognized values. More info:
        https://git.k8s.io/community/contributors/devel/sig-architecture/api-
        conventions.md#resources

    data          <map[string]string>
        Data contains the secret data. Each key must consist of alphanumeric
        characters, '-', '_' or '.'. The serialized form of the secret data is a
        base64 encoded string, representing the arbitrary (possibly non-string)
        data value here. Described in https://tools.ietf.org/html/rfc4648#section-4

    immutable     <boolean>
        Immutable, if set to true, ensures that data stored in the Secret cannot be
        updated (only object metadata can be modified). If not set to true, the
        field can be modified at any time. Defaulted to nil.
```

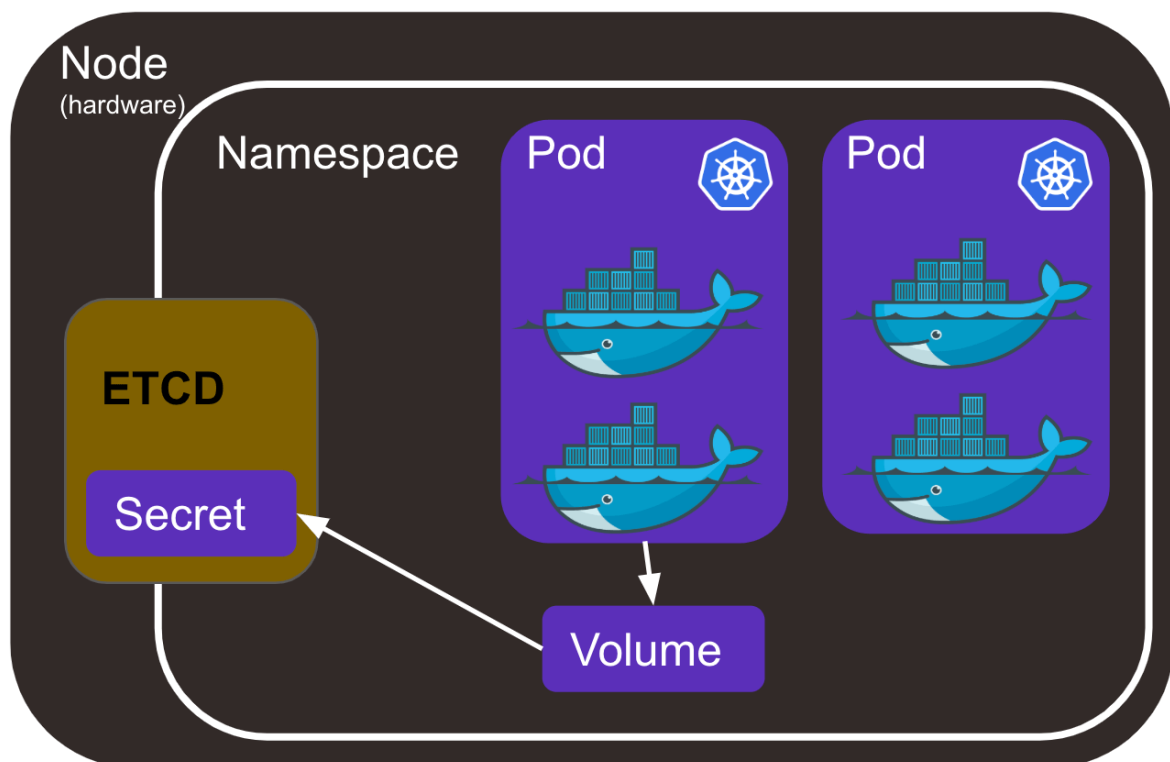
```
kind <string>
  kind is a string value representing the REST resource this object
  represents. Servers may infer this from the endpoint the client submits
  requests to. Cannot be updated. In CamelCase. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-
  conventions.md#types-kinds

metadata <Object>
  standard object's metadata. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-
  conventions.md#metadata

stringData <map[string]string>
  stringData allows specifying non-binary secret data in string form. It is
  provided as a write-only input field for convenience. All keys and values
  are merged into the data field on write, overwriting any existing values.
  The stringData field is never output when reading from the API.

type <string>
  Used to facilitate programmatic handling of secret data. More info:
  https://kubernetes.io/docs/concepts/configuration/secret/#secret-types
```

2.3.4 Secret 引用



Secret资源在Pod中引用的方式有三种

- 环境变量

引用Secret对象上特定的key，以valueFrom赋值给Pod上指定的环境变量

在Pod上使用envFrom一次性导入Secret对象上的所有key-value，key(也可以统一附加特定前缀)即为环境变量名,value自动成为相应的变量值

注意：容器很可能会将环境变量打印到日志中,基于安全考虑不建议以环境变量方式引用Secret中的敏感数据

- secret卷

在Pod上将Secret对象引用为存储卷，而后整体由容器mount至某个目录下,其中key名称转为文件名，value的值转为相应的文件内容

在Pod上定义Secret卷时，也可以仅引用其中的指定的部分key，而后由容器mount至目录下

- 拉取镜像

在Pod 上使用 imagePullSecrets 拉取私有仓库镜像使用

Pod引用Secret的方式：pods.spec.imagePullSecrets

2.3.5 Generic 案例

官方说明

```
https://kubernetes.io/zh-cn/docs/tasks/configmap-secret/managing-secret-using-config-file/
```

generic 主要用于实现用户名和密码的加密保存

注意: generic加密保存要注意换行问题 \n

```
#generic类型
kubectl create secret generic NAME [--type=string] [--from-file=[key=]source] [--from-literal=key1=value1]

--from-literal=key1=value1           #以命令行设置键值对的方式配置数据
--from-env-file=/PATH/TO/FILE       #以环境变量专用文件的方式配置数据
--from-file=[key=]/PATH/TO/FILE     #以配置文件的方式创建配置数据，如不指定key，FILE名称为key名
--from-file=/PATH/TO/DIR            #以配置文件所在目录的方式创建配置数据
#该命令中的--type选项进行定义除了后面docker-registry和tls命令之外的其它子类型，有些类型有key的特定要求
#注意：如果value中有特殊字符,比如:$,\,*,=,!等,需要用\进行转义或用单引号''引起来
```

2.3.5.1 范例：命令式创建 generic

```
#命令式创建
[root@master1 ~]#kubectl create secret generic secret-mysql-root --from-literal=username=root --from-literal=password=123456

#确认创建成功
[root@master1 ~]#kubectl get secrets
NAME                TYPE      DATA  AGE
secret-mysql-root   Opaque    2      7s

#查看Generic使用Base64编码
[root@master1 ~]#kubectl describe secrets secret-mysql-root
Name:                secret-mysql-root
Namespace:           default
Labels:              <none>
Annotations:         <none>

Type: Opaque
```

```

Data
====
password: 6 bytes
username: 4 bytes

[root@master1 ~]#kubectl get secrets secret-mysql-root -o yaml
apiVersion: v1
data:
  password: MTIzNDU2
  username: cm9vdA==
kind: Secret
metadata:
  creationTimestamp: "2021-03-02T14:00:31Z"
  name: secret-mysql-root
  namespace: default
  resourceVersion: "20360"
  uid: 5fa84616-997e-4c86-b4e5-b895516c4c12
type: Opaque

#解码
[root@master1 ~]#echo -n cm9vdA== | base64 -d
root
[root@master1 ~]#echo -n MTIzNDU2 | base64 -d
123456

```

2.3.5.2 范例: Base64 编码数据

```

#将数据转换成base64编码,注意:需要加-n选项去掉换行符
[root@master1 ~]#echo -n 'admin' | base64
YWRTaw4=
[root@master1 ~]#echo -n 'password' | base64
cGFzc3dvcmQ=

#清单文件
[root@master1 ~]#cat storage-secret-Opaque-data.yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-data
  namespace: default
type: Opaque
data:                                     #data表示必须以base64编码存放,stringData表示明文存放数据
  user: YWRTaw4=
  password: cGFzc3dvcmQ=

#应用
[root@master1 ~]#kubectl apply -f storage-secret-Opaque-data.yaml

[root@master1 ~]#kubectl get secrets
NAME          TYPE      DATA   AGE
secret-data   Opaque    2       5m5s

#查看
[root@master1 ~]#kubectl get secrets secret-data -o yaml
apiVersion: v1
data:

```

```

password: CGFzc3dvcmQ=
user: YWRtaW4=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":
{"password":"CGFzc3dvcmQ=","user":"YWRtaW4=","kind":"Secret","metadata":
{"annotations":{},"name":"secret-data","namespace":"default"},"type":"Opaque"}
      creationTimestamp: "2021-09-27T10:19:27Z"
      name: secret-data
      namespace: default
      resourceVersion: "545820"
      uid: d99ced51-1b27-4fcd-ba06-e480f31287be
type: Opaque

```

#如果安装etcdctl工具,可以看到secret实际保存在etcd中

```
[root@master1 ~]#etcdctl get / --keys-only --prefix |grep secret-data
/registry/secrets/default/secret-data
```

#查看secret内容

```
[root@master1 ~]#etcdctl get /registry/secrets/default/secret-data
```

#删除

```
[root@master1 ~]#kubectl delete secrets secret-data
```

2.3.5.3 范例: stringData 明文数据

#清单文件

```
[root@master1 ~]#cat storage-secret-Opaque-stringData.yaml
```

```
apiVersion: v1
```

```
kind: Secret
```

```
metadata:
```

```
  name: secret-stringdata
```

```
  namespace: default
```

```
type: Opaque
```

```
stringData:           #stringData表示明文存放数据,data表示必须以base64编码存放
```

```
  user: 'admin'
```

```
  password: 'password'
```

#应用文件

```
[root@master1 ~]#kubectl apply -f storage-secret-Opaque-stringData.yaml
```

#查看

```
[root@master1 ~]#kubectl get secrets
```

NAME	TYPE	DATA	AGE
secret-stringdata	Opaque	2	2m3s

#查看

```
[root@master1 ~]#kubectl get secrets secret-stringdata -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  password: CGFzc3dvcmQ= #此处显示base64编码形式
```

```
  user: YWRtaW4=
```

```
kind: Secret
```

```
metadata:
```

```
  annotations:
```

```
kubectl.kubernetes.io/last-applied-configuration: |
  {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":
  {},"name":"secret-stringdata","namespace":"default"},"stringData":
  {"password":"password","user":"admin"},"type":"Opaque"}
  creationTimestamp: "2021-09-27T10:22:29Z"
  name: secret-stringdata
  namespace: default
  resourceVersion: "546179"
  uid: 03b3e308-43ff-4f8f-9ac4-a41589272664
type: Opaque

#删除
[root@master1 ~]#kubectl delete -f storage-secret-Opaque-stringData.yaml
```

2.3.5.4 范例: Secret 通过环境变量为提供MySQL环境初始化的密码信息

```
#MySQL容器支持MYSQL_ROOT_PASSWORD变量存放密码,使用环境变量创建MySQL容器
[root@node1 ~]#docker run --name mysql_test -e MYSQL_ROOT_PASSWORD=123456 -d
registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle

#登录验证
[root@node1 ~]#docker exec mysql_test mysql -uroot -h127.0.0.1 -p123456 -e
status
mysql: [Warning] Using a password on the command line interface can be insecure.
-----
mysql Ver 8.0.27 for Linux on x86_64 (MySQL Community Server - GPL)

Connection id:      8
Current database:
Current user:       root@127.0.0.1
SSL:                Cipher in use is TLS_AES_256_GCM_SHA384
Current pager:      stdout
Using outfile:      ''
Using delimiter:    ;
Server version:     8.0.27 MySQL Community Server - GPL
Protocol version:   10
Connection:         127.0.0.1 via TCP/IP
Server characterset: utf8mb4
Db characterset:    utf8mb4
Client characterset: latin1
Conn. characterset: latin1
TCP port:           3306
Uptime:             28 sec

Threads: 2 Questions: 5 slow queries: 0 Opens: 117 Flush tables: 3 Open
tables: 36 Queries per second avg: 0.178

#生成密码的base64编码
[root@master1 ~]#echo -n root | base64
cm9vdA==
[root@master1 ~]#echo -n 123456 | base64
MTIzNDU2

#清单文件, secret 通过环境为提供MySQL环境初始化的密码信息, 但很不安全
[root@master1 ~]#cat storage-secret-mysql-init.yaml
apiVersion: v1
kind: Secret
```

```

metadata:
  name: secret-mysql
type: kubernetes.io/basic-auth
#type: Opaque #也可以用Opaque类型
data:
  username: cm9vdAo= #key名称:username
  password: MTIzNDU2 #key名称:password
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret-mysql-init
spec:
  containers:
  - name: mysql
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
    env:
    - name: MYSQL_ROOT_PASSWORD
      valueFrom:
        secretKeyRef:
          name: secret-mysql #引用指定的secret
          key: password #引用指定的secret中对应的key
          optional: false #必须存在

```

#应用

```
[root@master1 ~]#kubectl apply -f storage-secret-mysql-init.yaml
```

#查看结果

```
[root@master1 ~]#kubectl get secrets
```

NAME	TYPE	DATA	AGE
secret-mysql	kubernetes.io/basic-auth	2	1m8s

#可以看到base64编码的变量信息

```
[root@master1 ~]#kubectl get secrets secret-mysql -o yaml
```

```

apiVersion: v1
data:
  password: MTIzNDU2
  username: cm9vdAo=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":
{"password":"MTIzNDU2","username":"cm9vdAo="},"kind":"Secret","metadata":
{"annotations":{},"name":"secret-
mysql","namespace":"default"},"type":"kubernetes.io/basic-auth"}
  name: secret-mysql
  namespace: default
  resourceVersion: "725930"
  uid: 4089922b-3dc9-4b55-8eb5-61a2756c206a
type: kubernetes.io/basic-auth

```

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
pod-secret-mysql-init	1/1	Running	0	3m30s

#可以通过环境变量查看到密码，很不安全

```
[root@master1 ~]#kubectl exec pod-secret-mysql-init -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=pod-secret-mysql-init
MYSQL_ROOT_PASSWORD=123456 #可以看到此处的环境变量
POD_TEST2_PORT_80_TCP_PROTO=tcp
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
POD_TEST1_SERVICE_HOST=10.101.84.206
POD_TEST2_SERVICE_PORT=80
POD_TEST2_PORT=tcp://10.96.109.255:80
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
POD_TEST1_SERVICE_PORT_80_80=80
POD_TEST2_PORT_80_TCP_PORT=80
POD_TEST2_PORT_80_TCP_ADDR=10.96.109.255
POD_TEST2_SERVICE_HOST=10.96.109.255
POD_TEST2_SERVICE_PORT_80_80=80
POD_TEST2_PORT_80_TCP=tcp://10.96.109.255:80
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
POD_TEST1_SERVICE_PORT=80
POD_TEST1_PORT_80_TCP_PROTO=tcp
KUBERNETES_SERVICE_PORT_HTTPS=443
POD_TEST1_PORT=tcp://10.101.84.206:80
POD_TEST1_PORT_80_TCP=tcp://10.101.84.206:80
POD_TEST1_PORT_80_TCP_PORT=80
POD_TEST1_PORT_80_TCP_ADDR=10.101.84.206
GOSU_VERSION=1.12
MYSQL_MAJOR=8.0
MYSQL_VERSION=8.0.27-1debian10
HOME=/root
```

#验证密码登录MySQL

```
[root@master1 ~]#kubectl exec pod-secret-mysql-init -- mysql -uroot -p123456 -e status
```

```
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
-----
```

```
mysql Ver 8.0.27 for Linux on x86_64 (MySQL Community Server - GPL)
```

```
Connection id:      9
Current database:
Current user:       root@localhost
SSL:                Not in use
Current pager:      stdout
Using outfile:      ''
Using delimiter:    ;
Server version:     8.0.27 MySQL Community Server - GPL
Protocol version:   10
Connection:         Localhost via UNIX socket
Server characterset: utf8mb4
Db characterset:    utf8mb4
Client characterset: latin1
Conn. characterset: latin1
UNIX socket:        /var/run/mysqld/mysqld.sock
Uptime:             33 sec
```

```
Threads: 2 Questions: 5 Slow queries: 0 Opens: 118 Flush tables: 3 Open
tables: 37 Queries per second avg: 0.151
```

```
-----
```

```
[root@master1 ~]#echo -n 654321 | base64
```

```
NjU0MzIx
```

```
[root@master1 ~]#vim storage-secret-mysql-init.yaml
```

```
data:
```

```
  username: cm9vdAo=
```

```
  password: NjU0MzIx #修改密码
```

```
#重新应用
```

```
[root@master1 ~]#kubectl apply -f storage-secret-mysql-init.yaml
```

```
#查看是否变化
```

```
[root@master1 ~]#kubectl get secrets secret-mysql -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  password: NjU0MzIx
```

```
  username: cm9vdAo= #Secret更新
```

```
.....
```

```
#Pod内的变量没有自动更新
```

```
[root@master1 ~]#kubectl exec pod-secret-mysql-init -- env|grep PASSWORD
```

```
MYSQL_ROOT_PASSWORD=123456
```

```
#删除Pod重新创建
```

```
[root@master1 ~]#kubectl delete pod pod-secret-mysql-init
```

```
[root@master1 ~]#kubectl apply -f storage-secret-mysql-init.yaml
```

```
#Pod内变量更新成功
```

```
[root@master1 ~]#kubectl exec pod-secret-mysql-init -- env|grep PASSWORD
```

```
MYSQL_ROOT_PASSWORD=654321
```

```
#清理环境
```

```
[root@master1 ~]#kubectl delete -f storage-secret-mysql-init.yaml
```

2.3.5.5 范例: 通过卷调用 secret

```
#清单文件
```

```
[root@master1 ~]#cat storage-secret-test-pod.yaml
```

```
apiVersion: v1
```

```
kind: Secret
```

```
metadata:
```

```
  name: secret-test
```

```
type: kubernetes.io/basic-auth
```

```
#type: Opaque #也可以用Opaque类型
```

```
data:
```

```
  username: YWRtaW4=
```

```
  password: CGFzc3dvcmQ=
```

```
#说明:
```

```
#type类型指定为 Opaque 或者 kubernetes.io/basic-auth,username和password是加密的信息
```

```
#资源定义文件时, 与命令行创建secret的--from-literal=username=admin 效果是一样的,但命令行的变量无需加密
```

```
---
```

#调用secret的清单文件

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret-volume
spec:
  volumes:
  - name: secret
    secret:
      secretName: secret-test      #指定secret的名称
  containers:
  - name: secret-test-container
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    volumeMounts:                  #卷引用secret
    - name: secret
      mountPath: /secret/
      readOnly: true
```

#创建资源

```
[root@master1 ~]#kubectl apply -f storage-secret-test-pod.yaml
```

#查看

```
[root@master1 ~]#kubectl get secrets
```

NAME	TYPE	DATA	AGE
default-token-w2gph	kubernetes.io/service-account-token	3	4d13h
secret-test	kubernetes.io/basic-auth	2	3s

```
[root@master1 ~]#kubectl get secrets secret-test -o yaml
```

```
apiVersion: v1
data:
  password: CGFzc3dvcmQ=
  username: YWRtaW4=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":
{"password":"CGFzc3dvcmQ=","username":"YWRtaW4=","kind":"Secret","metadata":
{"annotations":{},"name":"secret-
test","namespace":"default"},"type":"kubernetes.io/basic-auth"}
  creationTimestamp: "2021-03-27T02:07:21Z"
  name: secret-test
  namespace: default
  resourceVersion: "81663"
  uid: 7c643f0d-0eac-48b3-95d2-7b11fa374140
type: kubernetes.io/basic-auth
```

#验证

```
[root@master1 ~]#kubectl get pod pod-secret-volume
```

NAME	READY	STATUS	RESTARTS	AGE
pod-secret-volume	1/1	Running	0	5s

```
[root@master1 ~]#kubectl exec pod-secret-volume -- ls /secret
```

```
password
```

```
username
```

#注意:文件内容不会随着Secret更新而自动更新,可以将Pod删除重建来实现更新

```
[root@master1 ~]#kubectl exec pod-secret-volume -- cat /secret/username
```

```

admin
[root@master1 ~]#kubectl exec pod-secret-volume -- cat /secret/password
password

#通过两次软链接实现
[root@master1 ~]#kubectl exec pod-secret-volume -- ls -l /secret -a
total 4
drwxrwxrwt 3 root root 120 Sep 30 07:37 .
drwxr-xr-x 1 root root 4096 Sep 30 07:37 ..
drwxr-xr-x 2 root root 80 Sep 30 07:37 ..2024_09_30_07_37_50.1344469564
lrwxrwxrwx 1 root root 32 Sep 30 07:37 ..data ->
..2024_09_30_07_37_50.1344469564
lrwxrwxrwx 1 root root 15 Sep 30 07:37 password -> ..data/password
lrwxrwxrwx 1 root root 15 Sep 30 07:37 username -> ..data/username

#在Pod运行的宿主机可以看到secret对应的文件
[root@node1 ~]#find /var/lib/kubelet -name password
/var/lib/kubelet/pods/41368cea-bd8e-4c7a-9803-35659f1121e0/volumes/kubernetes.io~secret/secret/password
/var/lib/kubelet/pods/41368cea-bd8e-4c7a-9803-35659f1121e0/volumes/kubernetes.io~secret/secret/..2021_04_24_07_22_12.1876726758/password

[root@node1 ~]#cat /var/lib/kubelet/pods/41368cea-bd8e-4c7a-9803-35659f1121e0/volumes/kubernetes.io~secret/secret/password
password

#清理环境
[root@master1 ~]#kubectl delete -f storage-secret-test-pod.yaml

```

2.3.5.6 范例: 为 Service Account 创建 Secret

```

#k8s-v1.24版本后,创建SA不会自动创建secret,需要手动创建secret
[root@master1 ~]#cat storage-secret-sa.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: testsa

---
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: testsa-secret
  namespace: default
  annotations:
    kubernetes.io/service-account.name: "testsa"

[root@master1 ~]#kubectl apply -f storage-secret-sa.yaml

[root@master1 ~]#kubectl get secrets,sa

```

NAME	TYPE	DATA	AGE
secret/testsa-secret	kubernetes.io/service-account-token	3	36m

NAME	SECRETS	AGE
------	---------	-----

```
serviceaccount/default 0 41d
serviceaccount/testsa 0 36m
```

#查看对应的token

#方法1

```
[root@master1 ~]#kubectl get secrets testsa-secret -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  ca.crt:
```

```
LS0tLS1CRudJTtBDRVJUSUZJQ0FURS0tLS0tCk1JSUMvakNDQWVhZ0F3SUJBZ0lCQURBTklna3Foa2lH
OXcwQkFRc0ZBREFTVjND0VRWURWUVFERXdwcmlRSmwKY201oc1k1aTZRdUtuTkZQbVV5WEVtYmV5Qmt
VTFFxQjVUK3lYalJqNmRJR3ZITUo4OEJpcTl3bTdrdUhtRTlkKRUNZPQotLS0tLUVORCBDRVJUSUZJQ0F
URS0tLS0tCg==
```

```
  namespace: ZGVmYXVsdA==
```

```
  token:
```

```
ZXlkaGJHY2lPaUpTVXpJMU5pSXNjbXRwWkNjNk1rWkpTWEJSUZE5RVNUSlVRamxhVlhadmEzwnJha2c0
YjI5QmIwZ3RSa1ZrZDI0eWJxdHRVWGRGY0RBawZRLmV5SnBjTFRwWFh6aEgxU3ZNaGRxOVJZWXBBSmdM
QQ==
```

```
kind: Secret
```

```
metadata:
```

```
  annotations:
```

```
    kubectl.kubernetes.io/last-applied-configuration: |
```

```
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":
```

```
{"kubernetes.io/service-account.name":"testsa"},"name":"testsa-
secret","namespace":"default"},"type":"kubernetes.io/service-account-token"}
```

```
    kubernetes.io/service-account.name: testsa
```

```
    kubernetes.io/service-account.uid: d2abc1df-9d0b-40e9-a592-78fe50b3c6ee
```

```
  creationTimestamp: "2022-04-18T03:10:17Z"
```

```
  name: testsa-secret
```

```
  namespace: default
```

```
  resourceVersion: "578846"
```

```
  uid: c7efbc9d-de09-4442-a159-9ecd5933a890
```

```
type: kubernetes.io/service-account-token
```

#获取上面的Token值并解码生成变量TOKEN

```
[root@master1 ~]#TOKEN=$(echo
```

```
ZXlkaGJHY2lPaUpTVXpJMU5pSXNjbXRwWkNjNk1rWkpTWEJSUZE5RVNUSlVRamxhVlhadmEzwnJha2c0
YjI5QmIwZ3RSa1ZrZDI0eWJxdHRVWGRGY0RBawZRLmV5SnBjTFRwWFh6aEgxU3ZNaGRxOVJZWXBBSmdM
QQ==|base64 -d)
```

#使用上面TOKEN访问，验证用户身份，但权限不足

```
[root@master1 ~]#curl -s --cacert /etc/kubernetes/pki/ca.crt -H "Authorization:
Bearer ${TOKEN}" https://k8sapi.wang.org:6443|head{
```

```
  "kind": "Status",
```

```
  "apiVersion": "v1",
```

```
  "metadata": {},
```

```
  "status": "Failure",
```

```
  "message": "forbidden: User \"system:serviceaccount:default:testsa\" cannot
get path \"/\"",
```

```
  "reason": "Forbidden",
```

```
  "details": {},
```

```
  "code": 403
```

```
}
```

#方法2:此方式无需base64解码

```
[root@master1 ~]#kubectl describe secrets testsa-secret
```

#清理环境

```
[root@master1 ~]#kubectl delete -f storage-secret-sa.yaml
```

2.3.6 TLS 案例

TLS 类型的 Secret 主要用于对https场景的证书和密钥文件来进行加密传输

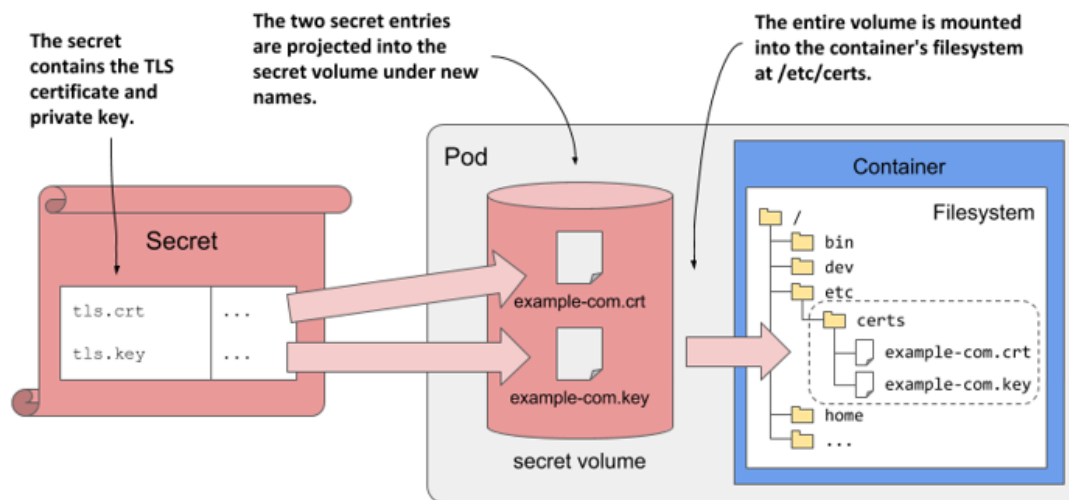
#tls类型格式

```
kubectl create secret tls NAME --cert=/path/file --key=/path/file
```

#注意:

#保存cert文件内容的key名称不能指定, 自动为tls.crt, 卷挂载后生成的文件名也为tls.crt

#保存private key文件内容的key不能指定, 自动为tls.key, 卷挂载后生成的文件名也为tls.key



下面案例实现一个基于https的nginx的web服务

2.3.6.1 创建 TLS 证书文件

#生成私钥匙

```
[root@master1 ~]#mkdir nginx-certs
```

```
[root@master1 ~]#openssl genrsa -out nginx-certs/www.wang.org.key 2048
```

#生成自签名证书

```
[root@master1 ~]#openssl req -new -x509 -key nginx-certs/www.wang.org.key -days 3650 -out nginx-certs/www.wang.org.crt -subj /C=CN/ST=Beijing/L=Beijing/O=DevOps/CN=www.wang.org
```

#注意: CN指向的域名必须是nginx配置中使用的域名信息

#查看文件

```
[root@master1 ~]#ls nginx-certs/
wang.org.crt  wang.org.key
```

2.3.6.2 基于 TLS 证书文件创建对应的 Secret

```
[root@master1 ~]#kubectl create secret tls secret-nginx-ssl --cert=nginx-certs/www.wang.org.crt --key=nginx-certs/www.wang.org.key
```

#查看结果

```
[root@master1 ~]#kubectl get secrets
```

NAME	TYPE	DATA	AGE
------	------	------	-----

```
secret-nginx-ssl      kubernetes.io/tls      2      2m53s
```

#查看内容

```
[root@master1 ~]#kubectl get secrets secret-nginx-ssl -o yaml
apiVersion: v1
data:
  tls.crt: #自动生成的名为tls.crt的key名称
LS0tLS1CRudJTtBDRVJUSUZJQ0FURS0tLS0tCk1JSURrekNDQW51Z0F3SUJBZ0lVU2RkV09tczhwQWNC
Z3BRUMMlRk9na1dGUvg0d0RRWUpL.....
  tls.key: #自动生成的名为tls.key的key名称
LS0tLS1CRudJTtBQUklwQVRFIEtFWS0tLS0tCk1JSUV2UUlCQURBTk1na3Foa2lHOXcwQkFRRUZBQVND
Qktjd2dnU2pBZ0VhQW9JQkFRQzg2
.....
kind: Secret
metadata:
  name: secret-nginx-ssl
  namespace: default
  resourceVersion: "717070"
  uid: 5dde10b9-56ff-4136-b77e-a6eaabd9865d
type: kubernetes.io/tls
```

2.3.6.3 准备 Nginx 配置文件

#nginx主配置文件 myserver.conf

```
[root@master1 nginx-ssl-conf.d]#ls
myserver.conf myserver-gzip.cfg myserver-status.cfg
[root@master1 nginx-ssl-conf.d]#cat myserver.conf
server {
    listen 80;
    server_name www.wang.org;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name www.wang.org;

    ssl_certificate /etc/nginx/certs/tls.crt;
    ssl_certificate_key /etc/nginx/certs/tls.key;

    ssl_session_timeout 5m;

    ssl_protocols TLSv1 TLSv1.1 TLSv1.2;

    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256:HIGH:!aNULL:!MD5:!RC4:!DHE;
    ssl_prefer_server_ciphers on;

    include /etc/nginx/conf.d/myserver-*.cfg;

    location / {
        root /usr/share/nginx/html;
    }
}
```

#nginx的压缩配置文件:myserver-gzip.cfg

```
[root@master1 nginx-ssl-conf.d]#cat myserver-gzip.cfg
gzip on;
```

```

gzip_comp_level 5;
gzip_proxied      expired no-cache no-store private auth;
gzip_types text/plain text/css application/xml text/javascript;

#nginx的状态页配置文件myserver-status.cfg
[root@master1 nginx-ssl-conf.d]#cat myserver-status.cfg
location /status {
    stub_status on;
    access_log off;
}

```

2.3.6.4 创建配置文件对应的 Configmap

```

[root@master1 ~]#kubectl create configmap cm-nginx-ssl-conf --from-file=nginx-ssl-conf.d/

[root@master1 ~]#kubectl get cm

```

NAME	DATA	AGE
cm-nginx-ssl-conf	3	15m
kube-root-ca.crt	1	4d15h

2.3.6.5 创建引用Secret和configmap资源配置文件

```

#创建资源配置文件
[root@master1 ~]#cat storage-secret-nginx-ssl.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-nginx-ssl
  namespace: default
spec:
  volumes:
  - name: nginx-certs
    secret:
      secretName: secret-nginx-ssl
  - name: nginx-confs
    configMap:
      name: cm-nginx-ssl-conf
      optional: false
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.20.0
    name: nginx-ssl-server
    volumeMounts:
    - name: nginx-certs
      mountPath: /etc/nginx/certs/      #其保存cert文件自动命名为tls.crt，而保存private
key文件的自动命名为tls.key
      readOnly: true
    - name: nginx-confs
      mountPath: /etc/nginx/conf.d/
      readOnly: true

#创建pod
[root@master1 ~]#kubectl apply -f storage-secret-nginx-ssl.yaml

```

2.3.6.6 验证结果

```
[root@master1 ~]#kubectl get pod pod-nginx-ssl -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
	NOMINATED	NODE	READINESS	GATES		
pod-nginx-ssl	1/1	Running	0	3m33s	172.16.3.13	
node3.wang.org	<none>		<none>			

#查看生成的证书文件，表示为关联时间戳目录下的两个双层软链接文件tls.crt和tls.key，保存证书的原始文件，而非base64格式

```
[root@master1 ~]#kubectl exec -it pod-nginx-ssl -- ls -l /etc/nginx/certs/
total 0
```

```
lrwxrwxrwx 1 root root 14 Mar 27 06:21 tls.crt -> ..data/tls.crt
lrwxrwxrwx 1 root root 14 Mar 27 06:21 tls.key -> ..data/tls.key
```

#访问Pod

```
[root@master1 ~]#curl -I http://172.16.3.13
```

```
HTTP/1.1 301 Moved Permanently
Server: nginx/1.20.0
Date: Sun, 27 Mar 2022 04:08:17 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://172.16.3.13/
```

```
[root@master1 ~]#curl -Lkv http://172.16.3.13
```

```
* Trying 172.16.3.13:80...
* Connected to 172.16.3.13 (172.16.3.13) port 80 (#0)
> GET / HTTP/1.1
> Host: 172.16.3.13
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 301 Moved Permanently
< Server: nginx/1.20.0
< Date: Sun, 27 Mar 2022 04:08:17 GMT
< Content-Type: text/html
< Content-Length: 169
< Connection: keep-alive
< Location: https://172.16.3.13/
<
* Ignoring the response-body
* Connection #0 to host 172.16.3.13 left intact
* Clear auth, redirects to port from 80 to 443
* Issue another request to this URL: 'https://172.16.3.13/'
* Trying 172.16.3.13:443...
* Connected to 172.16.3.13 (172.16.3.13) port 443 (#1)
* ALPN, offering h2
* ALPN, offering http/1.1
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
```

```

* TLSv1.2 (IN), TLS handshake, Server key exchange (12):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Server finished (14):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.2 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Certificate Status (22):
* TLSv1.2 (OUT), TLS handshake, Finished (20):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Finished (20):
* SSL connection using TLSv1.2 / ECDHE-RSA-AES128-GCM-SHA256
* ALPN, server accepted to use http/1.1
* Server certificate:
*  subject: C=CN; ST=Beijing; L=Beijing; O=DevOps; CN=www.wang.org
*  issuer: C=CN; ST=Beijing; L=Beijing; O=DevOps; CN=www.wang.org
*  SSL certificate verify result: self-signed certificate (18), continuing
anyway.
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
> GET / HTTP/1.1
> Host: 172.16.3.13
> User-Agent: curl/7.81.0
> Accept: */*
>
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: nginx/1.20.0
< Date: Sun, 27 Mar 2022 04:08:17 GMT
< Content-Type: text/html
< Content-Length: 50
< Last-Modified: Tue, 20 Apr 2021 13:35:47 GMT
< Connection: keep-alive
< ETag: "624abf6a-32"
< Accept-Ranges: bytes
<
Welcome to Nginx Web Site ! Nginx Version: 1.20.0
* Connection #1 to host 172.16.3.13 left intact

[root@master1 ~]#curl -Lk http://172.16.3.13/status
Active connections: 2
server accepts handled requests
 8 8 8
Reading: 0 Writing: 1 Waiting: 1

#验证证书信息
[root@master1 ~]#openssl s_client -connect 172.16.3.13:443
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 C = CN, ST = Beijing, L = Beijing, O = DevOps, CN = www.wang.org
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = CN, ST = Beijing, L = Beijing, O = DevOps, CN = www.wang.org
verify return:1
---
Certificate chain
 0 s:C = CN, ST = Beijing, L = Beijing, O = DevOps, CN = www.wang.org

```

```
i:C = CN, ST = Beijing, L = Beijing, O = DevOps, CN = www.wang.org
.....

#清理环境
[root@master1 ~]#kubectl delete -f storage-secret-nginx-ssl.yaml
[root@master1 ~]#kubectl delete cm cm-nginx-ssl-conf
[root@master1 ~]#kubectl delete secret tls secret-nginx-ssl
```

2.3.6.7 更新证书

以上颁发的证书是有一定的有效期限的，在过期前需要提前更新证书

在线更新证书流程

- 生成新的证书和私钥文件
- 将生成的新的证书文件和私钥文件的全部内容分别先进行base64编码并删除空行
- 利用编码后的内容分别替换以下命令kubectl edit secret secret-nginx-ssl 中原有的证书信息即可

证书的有效期一般为1年,到期前需要提前更新证书

范例: 证书更新

```
#重新颁发证书
[root@master1 ~]#(umask 077; openssl genrsa -out wang.key 2048)
[root@master1 ~]#openssl req -new -x509 -key wang.key -out wang.crt -subj
/C=CN/ST=Beijing/L=Beijing/O=SRE/CN=www.wang.org -days 3650

#方法1:
#在线修改证书配置,需要提前先将新证书文件用base64编码并删除换行符
[root@master1 ~]#cat wang.crt |base64 | tr -d '\n'
[root@master1 ~]#cat wang.key |base64 | tr -d '\n'
#上面生成的内容替换下面命令的内容,立即生效
[root@master1 ~]#kubectl edit secrets secret-nginx-ssl

#方法2
#删除旧证书配置
[root@master1 ~]#kubectl delete secrets secret-nginx-ssl
#创建新证书配置
[root@master1 ~]#kubectl create secret tls secret-nginx-ssl --cert=./wang.crt --
key=./wang.key

#集群外验证证书是否生效
[root@master1 ~]#echo 172.16.3.13 www.wang.org >> /etc/hosts

#nginx不支持热加载,需要手动加载
[root@master1 ~]#kubectl exec pod-nginx-ssl -- nginx -s reload
[root@master1 ~]#openssl s_client www.wang.org:443
CONNECTED(00000003)
depth=0 C = CN, ST = Beijing, L = Beijing, O = SRE, CN = www.wang.org
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = CN, ST = Beijing, L = Beijing, O = SRE, CN = www.wang.org
verify return:1
---
Certificate chain
0 s:C = CN, ST = Beijing, L = Beijing, O = SRE, CN = www.wang.org
i:C = CN, ST = Beijing, L = Beijing, O = SRE, CN = www.wang.org
a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
```

```
v:NotBefore: Jun 15 03:18:44 2021 GMT; NotAfter: Jun 12 03:18:44 2031 GMT
.....
```

2.3.7 Docker-registry 案例

2.3.7.1 Secret 实现 Docker 私有仓库的镜像访问说明

有时Docker的仓库是私有的,如果想下载镜像,就需要在下载镜像的主机上登录仓库并认证后才能实现
但是在Kubernetes集群使用私有仓库镜像就比较繁琐,因为运行Pod 的宿主机并不固定,如果在每个worker节点登录验证显然是不方便的

dockercfg及dockerconfigjson类型的Secret 适用于实现让kubelet从私有Image Registry中下载容器镜像

其引用定义在pod.spec.imagePullSecrets字段上的列表,即支持多个docker-registry类型的Secret,从上至下逐个进行匹配

也支持pod.spec.serviceAccountName引用sa, 再在serviceAccount.imagePullSecrets间接引用Docker-registry的secret

创建此种secret 可以通过下面方式实现

方法1: 通过命令创建

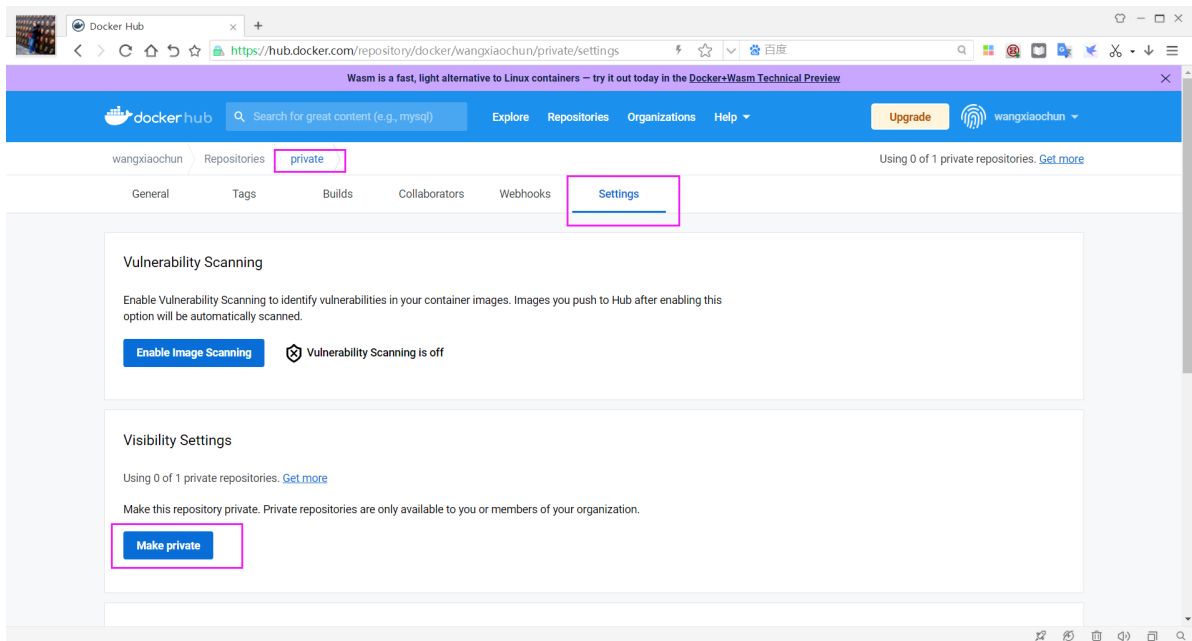
```
kubectl create secret docker-registry KEYNAME --docker-  
server=DOCKER_REGISTRY_SERVER --docker-username=DOCKER_USER --docker-  
password=DOCKER_PASSWORD --docker-email=EMAIL
```

方式二: 通过docker认证文件创建

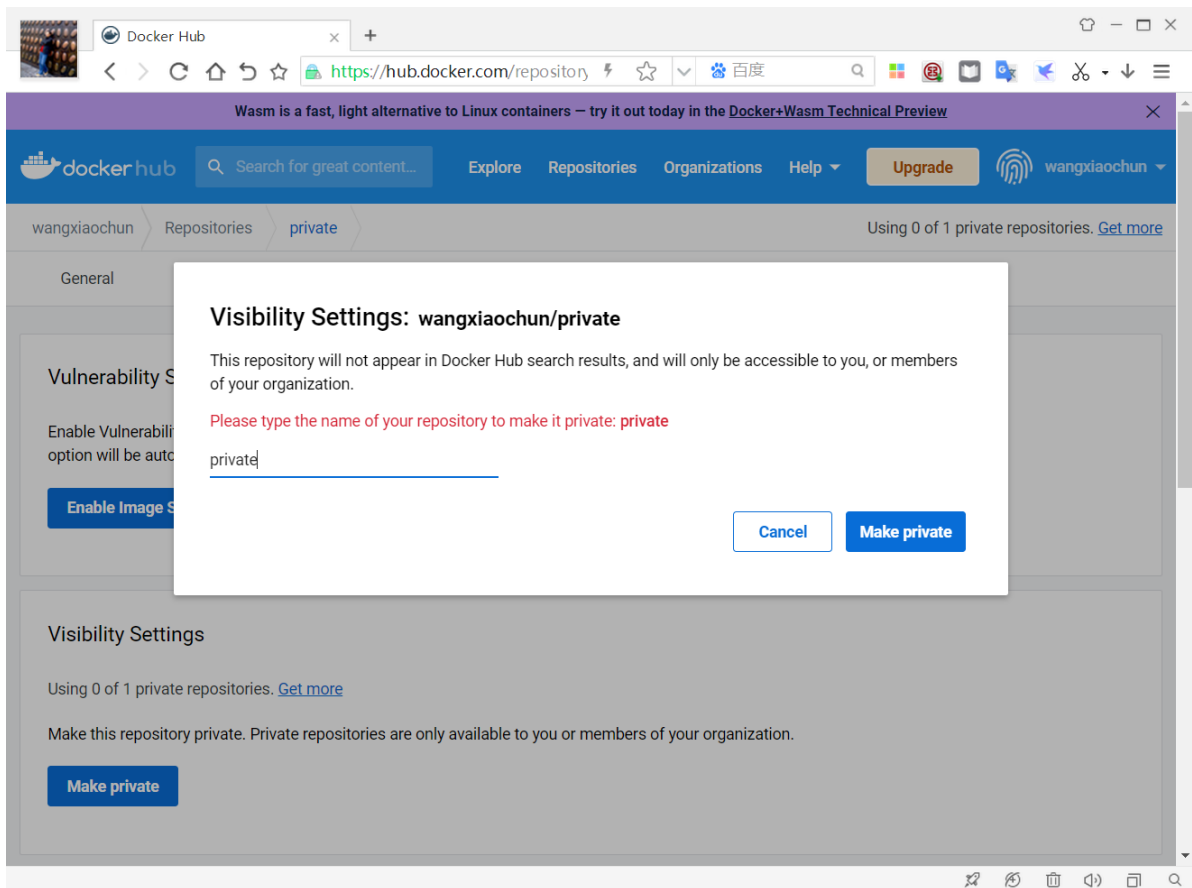
```
#先登录并认证到目标仓库server上, 认证凭据自动保存在 ~/.docker/config.json 文件中  
docker login --username=DOCKER_USER DOCKER_REGISTRY_SERVER  
  
#基于 config.json 文件生成 secret  
kubectl create secret docker-registry KEYNAME --from-  
file=.dockerconfigjson=path/to/.docker/config.json  
  
kubectl create secret generic KEYNAME --type='kubernetes.io/dockerconfigjson' --  
from-file=.dockerconfigjson=/root/.docker/config.json  
  
#示例:  
kubectl create secret docker-registry my-secret --from-  
file=.dockerconfigjson=path/to/.docker/config.json  
  
kubectl create secret generic dockerharbor-auth --  
type='kubernetes.io/dockerconfigjson' --from-  
file=.dockerconfigjson=/root/.docker/config.json
```

范例: 将dockerhub指定的 repository 变为私有

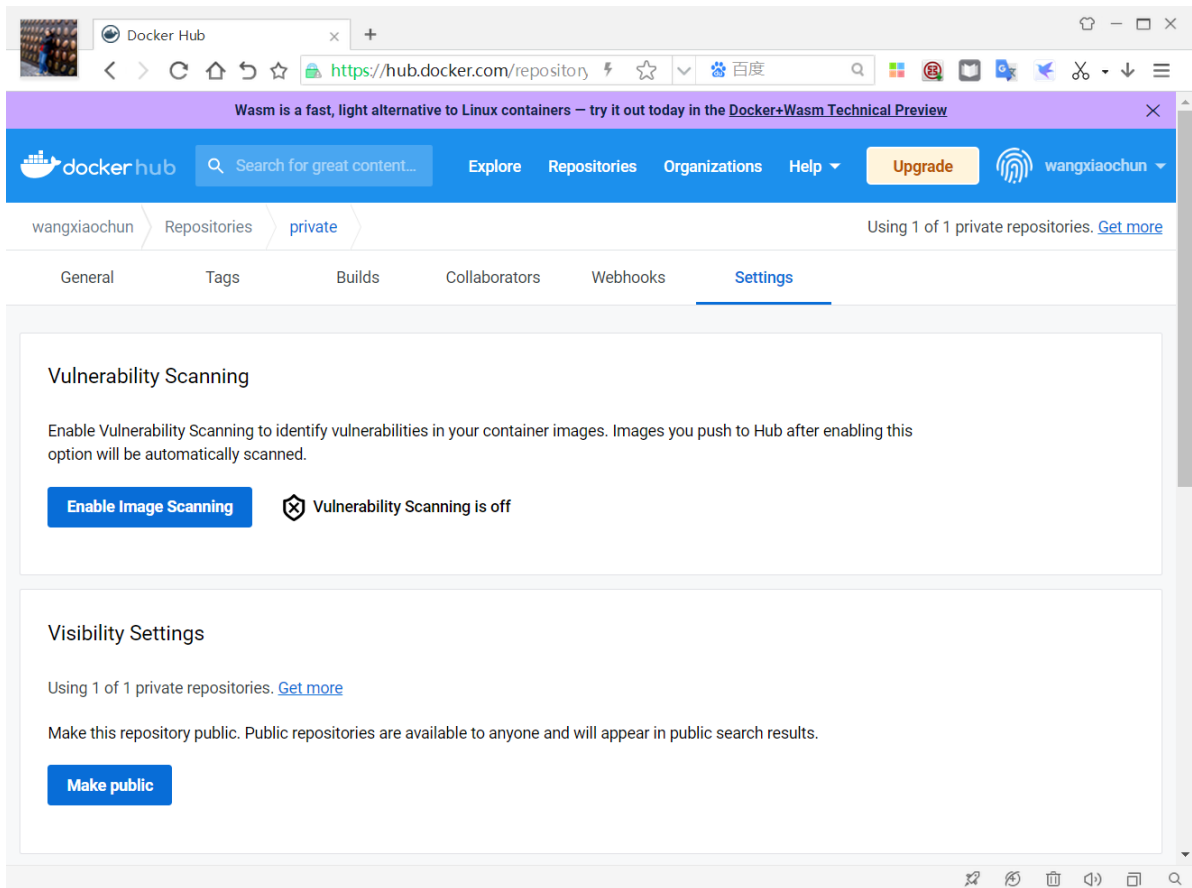
默认dockerhub的镜像都是公开的,个人用户可以将一个仓库变为私有



输入仓库的名称



查看状态变为private



范例: 测试用docker拉取上面的私有镜像

#不登录验证无法拉取镜像

```
[root@ubuntu2204 ~]#docker pull wangxiaochun/private:nginx-1.20.0
Error response from daemon: pull access denied for wangxiaochun/private,
repository does not exist or may require 'docker login': denied: requested
access to the resource is denied
```

#登录

```
[root@ubuntu2204 ~]#docker login -u wangxiaochun -p 123456
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
```

Login Succeeded

```
[root@ubuntu2204 ~]#docker pull wangxiaochun/private:nginx-1.20.0
```

```
[root@ubuntu2204 ~]#docker images wangxiaochun/private:nginx-1.20.0
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
wangxiaochun/private	nginx-1.20.0	7145ef2ae727	12 months ago	133MB

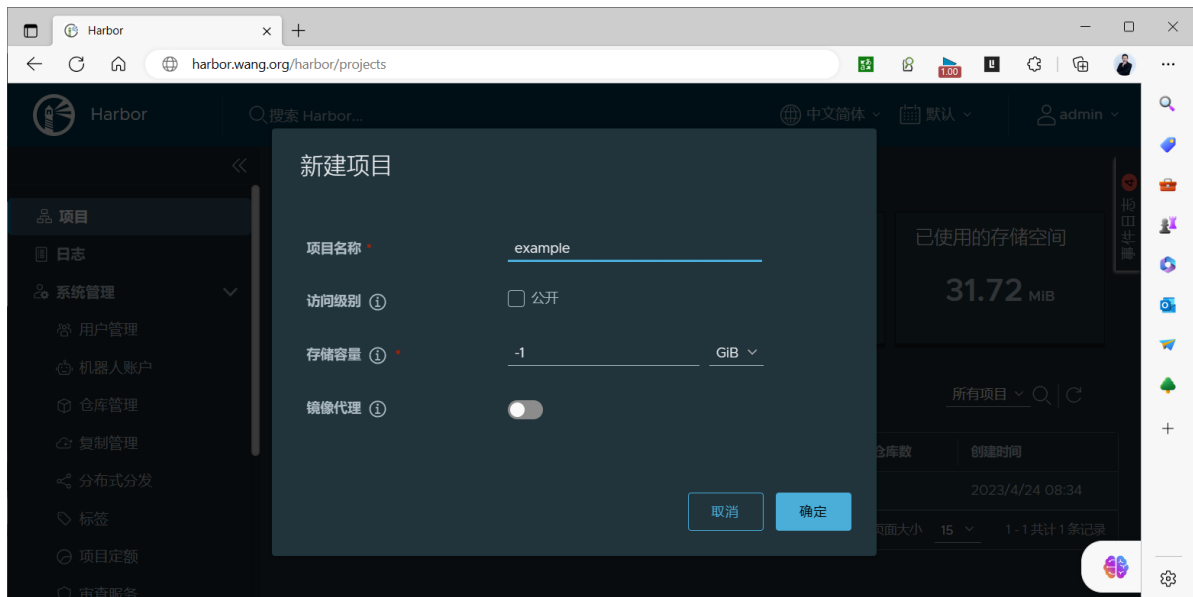
```
[root@ubuntu2204 ~]#cat .docker/config.json
{
  "auths": {
    "https://index.docker.io/v1/": {
      "auth": "d2FuZ3hpYW9jaHVuOjE5MzQ1Ng=="
    }
  }
}
```

2.3.7.2 通过命令创建私有仓库登录认证信息

2.3.7.2.1 命令创建私有仓库harbor登录认证信息

范例: 命令创建私有仓库harbor登录认证信息

安装并创建Harbor的私有仓库和项目



#修改所有集群节点的Docker配置支持私有仓库

```
[root@node1 ~]#vim /etc/docker/daemon.json
{
  "registry-mirrors": [
    "https://si7y70hh.mirror.aliyuncs.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com"
  ],
  "insecure-registries":["harbor.wang.org"], #添加此行,确保docker节点能够域名解析,而非Pod解析
  "exec-opts": ["native.cgroupdriver=systemd"]
}
[root@node1 ~]#systemctl restart docker
```

#拉取镜像

```
[root@ubuntu2204 ~]#docker pull wangxiaochun/pod-test:v0.1
[root@ubuntu2204 ~]#docker tag wangxiaochun/pod-test:v0.1
harbor.wang.org/example/pod-test:v0.1
```

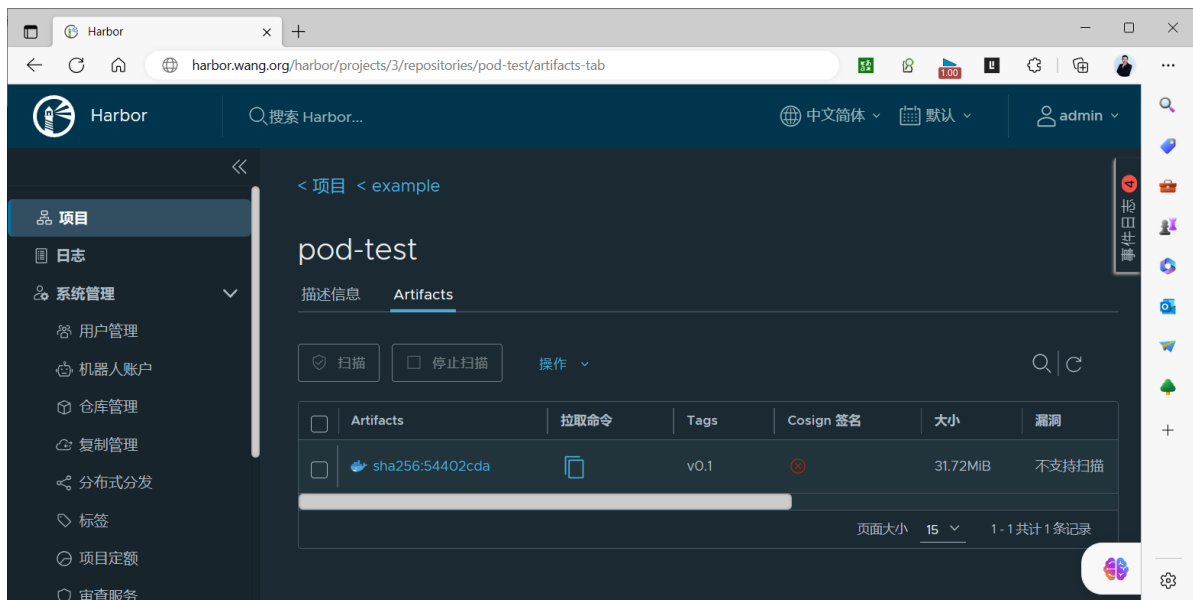
#登录

```
[root@ubuntu2204 ~]#docker login -u admin -p 123456 harbor.wang.org
```

#上传镜像到私有仓库

```
[root@ubuntu2204 ~]#docker push harbor.wang.org/example/pod-test:v0.1
```

查看镜像



#方法1：命令行创建Secret

```
root@master1 ~]# kubectl create secret docker-registry harbor-docker-registry-  
secret --docker-server=harbor.wang.org --docker-username=admin --docker-  
password=123456
```

#方法2

```
[root@master1 ~]#kubectl create secret docker-registry harbor-docker-registry-secret --from-file=.dockerconfigjson=/root/.docker/config.json
```

```
[root@master1 ~]# kubectl get secrets harbor-docker-registry-secret
```

NAME	TYPE	DATA	AGE
harbor-docker-registry-secret	kubernetes.io/dockerconfigjson	1	10m

```
[root@master1 ~]# kubectl get secrets harbor-docker-registry-secret -o yaml
```

```
apiVersion: v1
```

data:

`.dockerconfigjson:`

eyJhdXNlcm5hbWUoIjZhZG1pbSIpInBhc3N3bjkiMTIzNDU2IiwiaXV0aCI6IlxUnRhvZQ2TVRJek5EVTIiOjE5

kind: Secret

```
metadata:
```

```
creationTimestamp: "2022-02-24T00:51:43Z"
```

```
name: harbor-docker-registry-secret
```

```
namespace: default
```

resourceVersion: "682806"

uid: d5bc9bc6-6a28-4f8f-ab19-b4c52725c8f3

```
type: kubernetes.io/dockerconfigjson
```

#查看Secret内容

```
[root@master1 ~]#echo
```

```
eyJhdXNlcm5hbWUoIjZG1pbSI6bnB3b3JkIjoiaMTizNDU2IiwiaXYxOAcCI6IlxunRhyZ0TVRjek5EVTIifX19 |base64 -d
```

```
{"auths":{"harbor.wang.org":{"username":"admin","password":"123456","auth":"YWRTaw46MTizNDU2"}}
```

#准备清单文件

```
[root@master1 ~]#cat storage-secret-docker-registry-harbor.yaml
```

```
apiVersion: apps/v1
```

kind: Deployment

```
metadata:
```

```

name: deployment-harbor-docker-registry-secret
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pod-test-harbor-docker-registry-secret
  template:
    metadata:
      labels:
        app: pod-test-harbor-docker-registry-secret
    spec:
      containers:
        - name: pod-test-harbor-docker-registry-secret
          image: harbor.wang.org/example/pod-test:v0.1
          ports:
            - containerPort: 80
          imagePullSecrets:
            - name: harbor-docker-registry-secret #指定docker-registry类型的secret,如果有多个会逐个验证

```

#应用清单

```
[root@master1 ~]#kubectl apply -f storage-secret-docker-registry-harbor.yaml
```

#验证Pod启动成功

```
[root@master1 ~]#kubectl get pod |grep harbor
deployment-harbor-docker-registry-secret-648986d7d-4pnbv 1/1 Running
0 3s
deployment-harbor-docker-registry-secret-648986d7d-66ppw 1/1 Running
0 3s
deployment-harbor-docker-registry-secret-648986d7d-tvf24 1/1 Running
0 3s
```

#测试访问Pod成功

```
[root@master1 ~]#curl 10.244.2.39
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: deployment-harbor-docker-registry-secret-648986d7d-4pnbv, ServerIP: 10.244.2.39!
```

2.3.7.2.2 命令行创建阿里云的私有仓库登录认证信息

范例: 通过命令创建阿里云的私有仓库登录认证信息

#注意:如果不是docker官方仓库,必须指定--docker-server 选项

```
[root@master1 ~]#kubectl create secret docker-registry aliyun-docker-registry-secret --docker-server=registry.cn-beijing.aliyuncs.com --docker-username=29308620@qq.com --docker-password=123456
```

```
[root@master1 ~]#cat storage-secret-docker-registry-aliyun.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-aliyun-docker-registry-secret
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-aliyun-docker-registry-secret
  template:
```

```

metadata:
  labels:
    app: nginx-aliyun-docker-registry-secret
spec:
  containers:
  - name: nginx-aliyun-docker-registry-secret
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/private/nginx-
1.20.0
    ports:
    - containerPort: 80
  imagePullSecrets:
  - name: aliyun-docker-registry-secret #调用上面创建的secret名称，列表方式，多
个容器遍历多个secret访问

```

```
[root@master1 ~]#kubectl apply -f storage-secret-docker-registry.aliyun.yaml
deployment.apps/deployment-aliyun-docker-registry-secret created
[root@master1 ~]#kubectl get pod
```

NAME		READY	STATUS	
deployment-aliyun-docker-registry-secret-54b9c6766f-95szd	1/1	Running	0	
deployment-aliyun-docker-registry-secret-54b9c6766f-kbtth	1/1	Running	0	
deployment-aliyun-docker-registry-secret-54b9c6766f-rxx6b	1/1	Running	0	

```
[root@master1 ~]#kubectl get secrets aliyun-docker-registry-secret -o yaml
apiVersion: v1
data:
  .dockerconfigjson: eyJhdXNldyI6eyJyZWdpYyZpc3RyeS5jbG9jaWwqaw5nLmFsaXllbmNzLnVbSI6eyJlc2VybmFtZSI6IjE5MZA4NjIwQHFxLnVbSisInBhc3N3b3JkIjoimTIzNDU2IiwiaXYvOAcI6Ik1qa3pNRGcyTwpCQWNyRXVzMjl0T2pFeU16UTFOZZ09In19fq==
kind: Secret
metadata:
  creationTimestamp: "2020-11-08T08:28:53Z"
  name: aliyun-docker-registry-secret
  namespace: default
  resourceVersion: "824679"
  uid: dadd0676-49de-40be-aeb4-e87249b523ba
type: kubernetes.io/dockerconfigjson
```

#解析secret信息,可以看到密码

```
[root@master1 ~]# echo eyJhdXNpY2Vudpc3RyeS5jb2lnZlqW5nLmFsaX11bmNzLmNvbSI6eyJlc2VybmFtZSI6IjI5MZA4NjIwQHFxLmNvbSIsInBhc3N3b3JkIjoIMTIzNDU2IiwiaXV0aCI6Ikk1a3pNRGcyTwpCQWNYRXVzMj10T2pFeU16UTFOZz09In19fQ== |base64 -d
{"auths":{"registry.cn-beijing.aliyuncs.com":{"username":"29308620@qq.com","password":"123456","auth":"MjZkdGd2MjBACxEuY29tojEyMzQ1Ng=="}}}
```

2.3.7.3 通过 Docker 认证文件实现

范例: 通过docker认证文件实现Harbor私有仓库访问

```
#登录Harbor仓库生成dockerconfig文件
[root@master1 ~]#docker login harbor.wangq.org -u admin -p 123456
```

```
[root@master1 ~]#cat .docker/config.json
```

```
{
  "auths": {
    "harbor.wang.org": {
      "auth": "YWRtaW46MTIzNDU2"
    }
  }
}
```

#准备镜像并上传到私用仓库

```
[root@master1 ~]#docker pull wangxiaochun/pod-test:v0.2
```

```
[root@master1 ~]#docker tag wangxiaochun/pod-test:v0.2
```

```
harbor.wang.org/example/pod-test:v0.2
```

```
[root@master1 ~]#docker push harbor.wang.org/example/pod-test:v0.2
```

#在worker节点上访问私有仓库不经过验证,无法下载镜像

```
[root@node1 ~]#docker pull harbor.wang.org/example/pod-test:v0.2
```

```
Error response from daemon: unauthorized: unauthorized to access repository:
example/pod-test, action: pull: unauthorized to access repository: example/pod-
test, action: pull
```

#基于dockerconfig生成secret

```
[root@master1 ~]#kubectl create secret docker-registry secret-docker-registry-
harbor-dockerconfig --from-file=.dockerconfigjson=/root/.docker/config.json
```

```
[root@master1 ~]#kubectl get secrets
```

NAME	TYPE
DATA AGE	
secret-docker-registry-harbor-dockerconfig	kubernetes.io/dockerconfigjson 1 10s

```
[root@master1 ~]#kubectl get secrets secret-docker-registry-harbor-dockerconfig -
o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  .dockerconfigjson:
```

```
ewoJImFldGhZIjogewoJCSJoYXJib3Iud2FuZy5vcmcioiB7CgkJCSJhdXRoIjogI1lXUnRhVzQ2TVRJ
ek5EVTIiCgkJfQoJfQp9
```

```
kind: Secret
```

```
metadata:
```

```
  name: secret-docker-registry-harbor-dockerconfig
```

```
  namespace: default
```

```
  resourceVersion: "734984"
```

```
  uid: f039e6d5-c30a-4412-98d8-ca2c8a7cd532
```

```
type: kubernetes.io/dockerconfigjson
```

#查看data内容

```
[root@master1 ~]#echo
```

```
ewoJImFldGhZIjogewoJCSJoYXJib3Iud2FuZy5vcmcioiB7CgkJCSJhdXRoIjogI1lXUnRhVzQ2TVRJ
ek5EVTIiCgkJfQoJfQp9 |base64 -d
```

```
{
  "auths": {
    "harbor.wang.org": {
      "auth": "YWRtaW46MTIzNDU2"
    }
  }
}
```

#准备清单文件

```
[root@master1 ~]#cat storage-secret-docker-registry-harbor-dockerconfig.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-secret-docker-registry-harbor-dockerconfig
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pod-test-secret-docker-registry-harbor-dockerconfig
  template:
    metadata:
      labels:
        app: pod-test-secret-docker-registry-harbor-dockerconfig
    spec:
      containers:
      - name: pod-test-harbor-docker-registry-secret
        image: harbor.wang.org/example/pod-test:v0.2
        ports:
        - containerPort: 80
      imagePullSecrets:
      - name: secret-docker-registry-harbor-dockerconfig
```

#应用

```
[root@master1 ~]#kubectl apply -f storage-secret-docker-registry-harbor-dockerconfig.yaml
```

#查看

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	RESTARTS	AGE	IP	NODE	NOMINATED	READY	STATUS
deployment-secret-docker-registry-harbor-dockerconfig-6cf59fmd8						1/1	
Running	0	67s	10.244.1.42	node1.wang.org	<none>		
deployment-secret-docker-registry-harbor-dockerconfig-6cf59h8vx						1/1	
Running	0	67s	10.244.3.32	node3.wang.org	<none>		
deployment-secret-docker-registry-harbor-dockerconfig-6cf5k5r22						1/1	
Running	0	67s	10.244.2.43	node2.wang.org	<none>		

#查看到Pod的镜像下载成功

```
[root@master1 ~]#kubectl describe pod deployment-secret-docker-registry-harbor-dockerconfig-6cf5k5r22 |tail
```

```
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	6m23s	default-scheduler	Successfully assigned default/deployment-secret-docker-registry-harbor-dockerconfig-6cf5k5r22 to node2.wang.org

```

Normal Pulling 6m23s kubelet Pulling image
"harbor.wang.org/example/pod-test:v0.2"
Normal Pulled 6m23s kubelet Successfully pulled image
"harbor.wang.org/example/pod-test:v0.2" in 74.107717ms (74.117064ms including
waiting)
Normal Created 6m23s kubelet Created container pod-test-harbor-
docker-registry-secret
Normal Started 6m23s kubelet Started container pod-test-harbor-
docker-registry-secret

#在上面pod运行的节点上仍然无法用docker拉取镜像
[root@node1 ~]#docker pull harbor.wang.org/example/pod-test:v0.2
Error response from daemon: unauthorized: unauthorized to access repository:
example/pod-test, action: pull: unauthorized to access repository: example/pod-
test, action: pull

#环境清理
[root@master1 ~]#kubectl delete -f storage-secret-docker-registry-harbor-
dockerconfig.yaml
[root@master1 ~]#kubectl delete secrets secret-docker-registry-harbor-
dockerconfig

```

范例: 通过docker认证文件实现阿里云私有仓库访问

```

#尝试无登录直接拉取私有仓库的镜像失败
[root@node1 ~]#docker push registry.cn-
beijing.aliyuncs.com/wangxiaochun/private:nginx-1.20.0
The push refers to repository [registry.cn-
beijing.aliyuncs.com/wangxiaochun/private]
An image does not exist locally with the tag: registry.cn-
beijing.aliyuncs.com/wangxiaochun/private

[root@master1 ~]#docker login --username=29308620@qq.com registry.cn-
beijing.aliyuncs.com
Password:

[root@master1 ~]#cat /root/.docker/config.json
{
  "auths": {
    "registry.cn-beijing.aliyuncs.com": {
      "auth": "MjkzMDg2MjBACXEY29tOjEyMzQ1Ng=="
    }
  }
}
[root@master1 ~]#

#创建secret
[root@master1 ~]#kubectl create secret generic aliyun-registry-secret-auth --
type='kubernetes.io/dockerconfigjson' --from-
file=.dockerconfigjson=/root/.docker/config.json

[root@master1 ~]#kubectl get secrets

```

NAME	TYPE	DATA	AGE
aliyun-registry-secret-auth	kubernetes.io/dockerconfigjson	1	48s

```

[root@master1 ~]#kubectl get secrets aliyun-registry-secret-auth -o yaml

```

```

apiVersion: v1
data:
  .dockerconfigjson:
    ewoJImFldGhziJogewoJCSJyZWdpY3RyeS5jbiliZWlqaw5nLmFsaxl1bmNzLmNvbSI6IHsKCQkKImF1
    dGgiOiAiTWprek1EZzJNakJBYS1hFdvkyOXRPbmRoYm1jd016QTAiCgkKJfQoJfQp9
kind: Secret
metadata:
  creationTimestamp: "2021-02-28T15:38:48Z"
  name: aliyun-registry-secret-auth
  namespace: default
  resourceVersion: "568129"
  uid: 70605fce-35ef-4b21-9a41-3fa09a58c2c6
type: kubernetes.io/dockerconfigjson

```

#创建deployment引用secret

```
[root@master1 ~]#cat storage-secret-docker-registry-dockerconfig.yaml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-aliyun-registry-secret-auth
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-aliyun-registry-secret
  template:
    metadata:
      labels:
        app: nginx-aliyun-registry-secret
    spec:
      containers:
        - name: nginx-aliyun-registry-secret
          image: registry.cn-beijing.aliyuncs.com/wangxiaochun/private/nginx-
1.20.0
          ports:
            - containerPort: 80
          imagePullSecrets:
            - name: aliyun-registry-secret-auth

```

```
[root@master1 ~]#kubectl apply -f storage-secret-docker-registry-
dockerconfig.yaml
```

```
deployment.apps/deployment-aliyun-registry-secret-auth created
```

#查看Pod成功创建在node1节点

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME				READY	STATUS	
RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS
GATES						
deployment-aliyun-registry-secret-auth-5c76cb9d99-drz8r	1/1	Running	0			
43s	10.244.1.52	node1.wang.org	<none>		<none>	

#成功下载镜像

```
[root@master1 ~]#kubectl describe pod deployment-aliyun-registry-secret-auth-
5c76cb9d99-drz8r
```

```
.....
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----

```

Normal Scheduled 27s default-scheduler Successfully assigned
default/deployment-aliyun-registry-secret-auth-5c76cb9d99-drz8r to
node1.wang.org
Normal Pulling 26s kubelet Pulling image "registry.cn-
beijing.aliyuncs.com/wangxiaochun/private:nginx-1.20.0"
Normal Pulled 2s kubelet Successfully pulled image
"registry.cn-beijing.aliyuncs.com/wangxiaochun/private:nginx-1.20.0" in
24.153465179s
Normal Created 2s kubelet Created container nginx-aliyun-
registry-secret
Normal Started 2s kubelet Started container nginx-aliyun-
registry-secret

```

#Pod正常访问

```

[root@master1 ~]#curl 10.244.1.52
Welcome to Nginx Web Site ! Nginx Version: 1.20.0

```

#在node1节点实时观察日志如下

```

[root@node1 ~]#docker events
2021-02-28T16:40:02.660893707Z container create
4b107a1940e673148cf2e4a6f7c3a280ccdfd0070d91cdf8690809ff454b9135
(annotation.kubernetes.io/config.seen=2021-02-28T16:40:02.290472349Z,
annotation.kubernetes.io/config.source=api, app=nginx-aliyun-registry-secret,
image=registry.aliyuncs.com/google_containers/pause:3.7,
io.kubernetes.container.name=POD, io.kubernetes.docker.type=podsandbox,
io.kubernetes.pod.name=deployment-aliyun-registry-secret-auth-5c76cb9d99-x6lf9,
io.kubernetes.pod.namespace=default, io.kubernetes.pod.uid=bf09c67d-4318-4a17-
b745-a048106afe2e, name=k8s_POD_deployment-aliyun-registry-secret-auth-
5c76cb9d99-x6lf9_default-bf09c67d-4318-4a17-b745-a048106afe2e_0, pod-template-
hash=5c76cb9d99)
2021-02-28T16:40:02.673345012Z network connect
39a693c74c1dd533cd09c9fe81a21098b15f135150251f3500bb3f12947d1726
(container=4b107a1940e673148cf2e4a6f7c3a280ccdfd0070d91cdf8690809ff454b9135,
name=none, type=null)
2021-02-28T16:40:03.113171612Z container start
4b107a1940e673148cf2e4a6f7c3a280ccdfd0070d91cdf8690809ff454b9135
(annotation.kubernetes.io/config.seen=2021-02-28T16:40:02.290472349Z,
annotation.kubernetes.io/config.source=api, app=nginx-aliyun-registry-secret,
image=registry.aliyuncs.com/google_containers/pause:3.7,
io.kubernetes.container.name=POD, io.kubernetes.docker.type=podsandbox,
io.kubernetes.pod.name=deployment-aliyun-registry-secret-auth-5c76cb9d99-x6lf9,
io.kubernetes.pod.namespace=default, io.kubernetes.pod.uid=bf09c67d-4318-4a17-
b745-a048106afe2e, name=k8s_POD_deployment-aliyun-registry-secret-auth-
5c76cb9d99-x6lf9_default-bf09c67d-4318-4a17-b745-a048106afe2e_0, pod-template-
hash=5c76cb9d99)

```

```
2021-02-28T16:40:03.224076848Z container create
26cd765096a6ac461474822b26da7348d2a7a1e5de8ed44a1d81a11c2d3391ea
(annotation.io.kubernetes.container.hash=80100196,
annotation.io.kubernetes.container.ports=
[{"containerPort":80,"protocol":"TCP"}],
annotation.io.kubernetes.container.restartCount=0,
annotation.io.kubernetes.container.terminationMessagePath=/dev/termination-log,
annotation.io.kubernetes.container.terminationMessagePolicy=File,
annotation.io.kubernetes.pod.terminationGracePeriod=30,
image=sha256:7145ef2ae7275df3a3f51bcfb81936451c334a6511d13473f97e7a520f4670f6,
io.kubernetes.container.logpath=/var/log/pods/default_deployment-aliyun-
registry-secret-auth-5c76cb9d99-x61f9_bf09c67d-4318-4a17-b745-a048106afe2e/nginx-
aliyun-registry-secret/0.log, io.kubernetes.container.name=nginx-aliyun-
registry-secret, io.kubernetes.docker.type=container,
io.kubernetes.pod.name=deployment-aliyun-registry-secret-auth-5c76cb9d99-x61f9,
io.kubernetes.pod.namespace=default, io.kubernetes.pod.uid=bf09c67d-4318-4a17-
b745-a048106afe2e,
io.kubernetes.sandbox.id=4b107a1940e673148cf2e4a6f7c3a280ccdfd0070d91cdf8690809f
f454b9135, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>,
name=k8s_nginx-aliyun-registry-secret_deployment-aliyun-registry-secret-auth-
5c76cb9d99-x61f9_default_bf09c67d-4318-4a17-b745-a048106afe2e_0)
2021-02-28T16:40:03.338513035Z container start
26cd765096a6ac461474822b26da7348d2a7a1e5de8ed44a1d81a11c2d3391ea
(annotation.io.kubernetes.container.hash=80100196,
annotation.io.kubernetes.container.ports=
[{"containerPort":80,"protocol":"TCP"}],
annotation.io.kubernetes.container.restartCount=0,
annotation.io.kubernetes.container.terminationMessagePath=/dev/termination-log,
annotation.io.kubernetes.container.terminationMessagePolicy=File,
annotation.io.kubernetes.pod.terminationGracePeriod=30,
image=sha256:7145ef2ae7275df3a3f51bcfb81936451c334a6511d13473f97e7a520f4670f6,
io.kubernetes.container.logpath=/var/log/pods/default_deployment-aliyun-
registry-secret-auth-5c76cb9d99-x61f9_bf09c67d-4318-4a17-b745-a048106afe2e/nginx-
aliyun-registry-secret/0.log, io.kubernetes.container.name=nginx-aliyun-
registry-secret, io.kubernetes.docker.type=container,
io.kubernetes.pod.name=deployment-aliyun-registry-secret-auth-5c76cb9d99-x61f9,
io.kubernetes.pod.namespace=default, io.kubernetes.pod.uid=bf09c67d-4318-4a17-
b745-a048106afe2e,
io.kubernetes.sandbox.id=4b107a1940e673148cf2e4a6f7c3a280ccdfd0070d91cdf8690809f
f454b9135, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>,
name=k8s_nginx-aliyun-registry-secret_deployment-aliyun-registry-secret-auth-
5c76cb9d99-x61f9_default_bf09c67d-4318-4a17-b745-a048106afe2e_0)
```

#node1节点仍然无法直接下载镜像

```
[root@node1 ~]#docker push registry.cn-
beijing.aliyuncs.com/wangxiaochun/private:nginx-1.20.0
The push refers to repository [registry.cn-
beijing.aliyuncs.com/wangxiaochun/private]
0544a0706b26: Preparing
272bc57d3405: Preparing
f7141923aaa3: Preparing
9b63e6289fbe: Preparing
a2f4f809e04e: Preparing
1839f9962bd8: waiting
02c055ef67f5: waiting
denied: requested access to the resource is denied
```

2.4 downwardAPI

2.4.1 downwardAPI 介绍

<https://kubernetes.io/zh-cn/docs/concepts/storage/volumes/#downwardapi>

在Kubernetes集群场景中，对于运行中的pod来说，可以通过 configmap、secret、volume等方式为pod提供相应的配置属性或者配置文件等信息。

但是对于pod中的应用程序来说，有时候还需要获取其所在的宿主机提供的外在环境信息。

比如：集群节点主机本身是有资源容量的，可以通过resources的limits或者requests对pod内部的容器进行资源限制，如果能够将宿主机本地的相关资源信息，传递到pod内部，便于其在进行资源调度的时候，进行充分的考量。

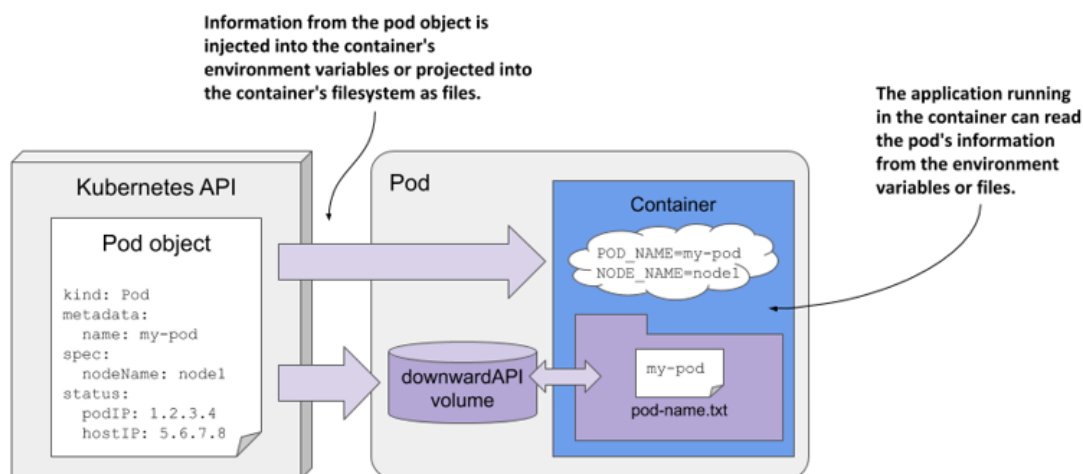
在Kubernetes中可以基于一种 downwardAPI 的对象，将宿主机相关的信息以存储卷的样式加载到pod内部。

相较于configmap、secret等资源对象需要创建后才能使用，downwardAPI 不是存储卷，无需人为创建，它自身一直就存在。

downwardAPI 不是一种独立的API资源类型，只是一种引用Pod自身的运行环境信息

downwardAPI 包括Pod的metadata,spec或status字段值，将这些信息注入到容器内部的方式

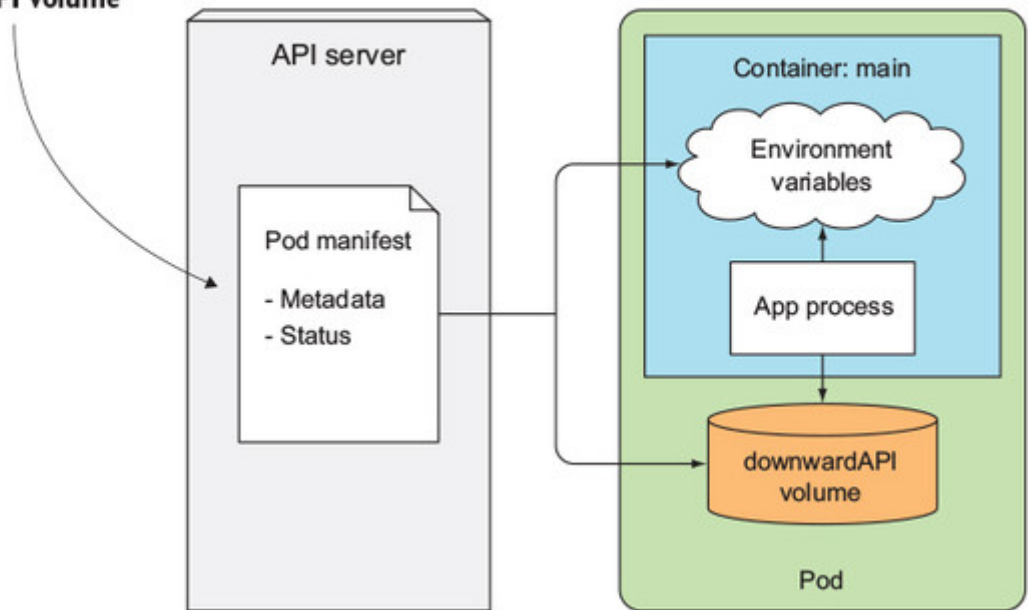
downwardAPI 为运行在pod中的应用容器提供了一种反向引用。让容器中的应用程序了解所处pod或Node的一些基础属性信息。



不过，通常只有常量类型的属性才能够通过环境变量注入到容器中，毕竟，在进程启动完成后无法再向其告知变量值的变动，于是，环境变量也就不支持中途的更新操作。

DownwardAPI提供了两种方式用于将 Pod 的信息注入到容器内部

Used to initialize environment variables and files in the downwardAPI volume



- 环境变量：用于单个变量，可以将 Pod 信息和容器信息直接注入容器内部
- Volume挂载：将 Pod 信息生成为文件，再挂载到容器内部中

类似于ConfigMap或Secret资源，容器能够在环境变量中valueFrom字段中基于两个字段来引用其所属Pod对象的元数据信息

- fieldRef：引用常规性的元数据
- resourceFieldRef：引用和资源限制和资源请求相关的元数据

Downward API fields injected via the fieldRef field

Field	Description	Allowed in env	Allowed in volume
<code>metadata.name</code>	The pod's name.	Yes	Yes
<code>metadata.namespace</code>	The pod's namespace.	Yes	Yes
<code>metadata.uid</code>	The pod's UID.	Yes	Yes
<code>metadata.labels</code>	All the pod's labels, one label per line, formatted as <code>key="value"</code> .	No	Yes
<code>metadata.labels['key']</code>	The value of the specified label.	Yes	Yes
<code>metadata.annotations</code>	All the pod's annotations, one per line, formatted as <code>key="value"</code> .	No	Yes
<code>metadata.annotations['key']</code>	The value of the specified annotation.	Yes	Yes
<code>spec.nodeName</code>	The name of the worker node the pod runs on.	Yes	No
<code>spec.serviceAccountName</code>	The name of the pod's service account.	Yes	No
<code>status.podIP</code>	The pod's IP address.	Yes	No
<code>status.hostIP</code>	The worker node's IP address.	Yes	No

Table 9.6 Downward API resource fields injected via the resourceFieldRef field

Resource field	Description	Allowed in env	Allowed in vol
<code>requests.cpu</code>	The container's CPU request.	Yes	Yes
<code>requests.memory</code>	The container's memory request.	Yes	Yes
<code>requests.ephemeral-storage</code>	The container's ephemeral storage request.	Yes	Yes
<code>limits.cpu</code>	The container's CPU limit.	Yes	Yes
<code>limits.memory</code>	The container's memory limit.	Yes	Yes
<code>limits.ephemeral-storage</code>	The container's ephemeral storage limit.	Yes	Yes

容器规范中可在环境变量配置中的valueFrom通过内嵌字段fieldRef引用的信息包括如下信息：

```
metadata.name: Pod对象的名称
metadata.namespace: Pod对象隶属的名称空间
metadata.uid: Pod对象的UID
metadata.labels['<KEY>']: Pod对象标签中的指定键的值, 例如metadata.labels['mylabel']
metadata.annotations['<KEY>']: Pod对象注解信息中的指定键的值
```

容器上的计算资源需求和资源限制相关的信息, 以及临时存储资源需求和资源限制相关的信息可通过容器规范中的resourceFieldRef字段引用, 相关字段包括requests.cpu、limits.cpu、requests.memory和limits.memory等。

另外, 可通过环境变量引用的信息有如下几个:

```
status.podIP: Pod对象的IP地址
spec.serviceAccountName: Pod对象使用的ServiceAccount资源名称
spec.nodeName: 节点名称
status.hostIP: 节点IP地址
```

容器上的当前容器的资源请求及资源限额的定义, 可以通过resourceFieldRef字段引用, 因此它们包括6项

```
requests.cpu
requests.memory
requests.ephemeral-storage
limits.cpu
limits.memory
limits.ephemeral-storage
```

2.4.2 downwardAPI 案例

范例: 获取基本的变量信息通过变量方式引用

```
#定制的资源对象
[root@master1 ~]#cat storage-downwardapi-env-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: downwardapi-env-test
  labels:
    app: downwardapi-env
spec:
  containers:
  - name: downwardapi-env-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    resources:
      requests:
        memory: "32Mi"
        cpu: "250m"
      limits:
        memory: "64Mi"
        cpu: "500m"
    env:
    - name: THIS_POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
```

```

- name: THIS_POD_NAMESPACE
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace
- name: THIS_APP_LABEL
  valueFrom:
    fieldRef:
      fieldPath: metadata.labels['app']
- name: THIS_CPU_LIMIT
  valueFrom:
    resourceFieldRef:
      resource: limits.cpu
- name: THIS_MEM_REQUEST
  valueFrom:
    resourceFieldRef:
      resource: requests.memory
      divisor: 1Mi
- name: VAR_REF
  value: $(THIS_POD_NAMESPACE).wang.org #变量引用格式:$(VAR_NAME)

```

#应用资源对象

```
[root@master1 ~]# kubectl apply -f storage-downwardapi-env-test.yaml
```

#查看效果

```
[root@master1 ~]# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
downwardapi-env-test	1/1	Running	0	7s

```

[root@master1 ~]# kubectl exec -it downwardapi-env-test -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=downwardapi-env-test
TERM=xterm
THIS_POD_NAMESPACE=default
THIS_APP_LABEL=downwardapi-env
THIS_CPU_LIMIT=1 #向
THIS_MEM_REQUEST=32
THIS_POD_NAME=downwardapi-env-test
VAR_REF=default.wang.org

```

范例: 存储卷方式使用

```

[root@master1 ~]# cat storage-downwardapi-volume-test.yaml
kind: Pod
apiVersion: v1
metadata:
  name: downwardapi-volume-test
  labels:
    zone: Beijing
    rack: zhongguancun
    app: redis-master
  annotations:
    region: Asia-China
spec:
  volumes:
  - name: podinfo
    downwardAPI:
      defaultMode: 0420 #文件权限，默认0644

```

```

items:
- fieldRef:
    fieldPath: metadata.namespace
  path: "pod_namespace"
- fieldRef:
    fieldPath: metadata.labels
  path: "pod_labels"
- fieldRef:
    fieldPath: metadata.annotations
  path: "pod_annotations"
- resourceFieldRef:
    containerName: downwardapi-volume-test
    resource: limits.cpu
  path: "cpu_limit"
- resourceFieldRef:
    containerName: downwardapi-volume-test
    resource: requests.memory
    divisor: "1Mi"
  path: "mem_request"
containers:
- name: downwardapi-volume-test
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
  resources:
    requests:
      memory: "32Mi"
      cpu: "250m"
    limits:
      memory: "64Mi"
      cpu: "500m"
  volumeMounts:
  - name: podinfo
    mountPath: /etc/podinfo/
    readOnly: false

```

#属性解析: `defaultMode` 表示挂载文件后, 文件的权限限制, `items` 定制自己的一些名称

#应用资源对象

```
[root@master1 ~]# kubectl apply -f storage-downwardapi-volume-test.yaml
```

#查看效果

```
[root@master1 ~]# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
downwardapi-volume-test	1/1	Running	0	2s

```
[root@master1 ~]# kubectl exec -it downwardapi-volume-test -- ls /etc/podinfo/
```

```
cpu_limit      pod_annotations pod_namespace
```

```
mem_request    pod_labels
```

```
[root@master1 ~]# kubectl exec -it downwardapi-volume-test -- cat
```

```
/etc/podinfo/pod_labels
```

```
app="redis-master"
```

```
rack="zhongguancun"
```

```
zone="Beijing"
```

```
root@master1:~/secret# kubectl exec -it downwardapi-volume-test -- cat
```

```
/etc/podinfo/cpu_limit
```

```
1
```

2.5 Projected

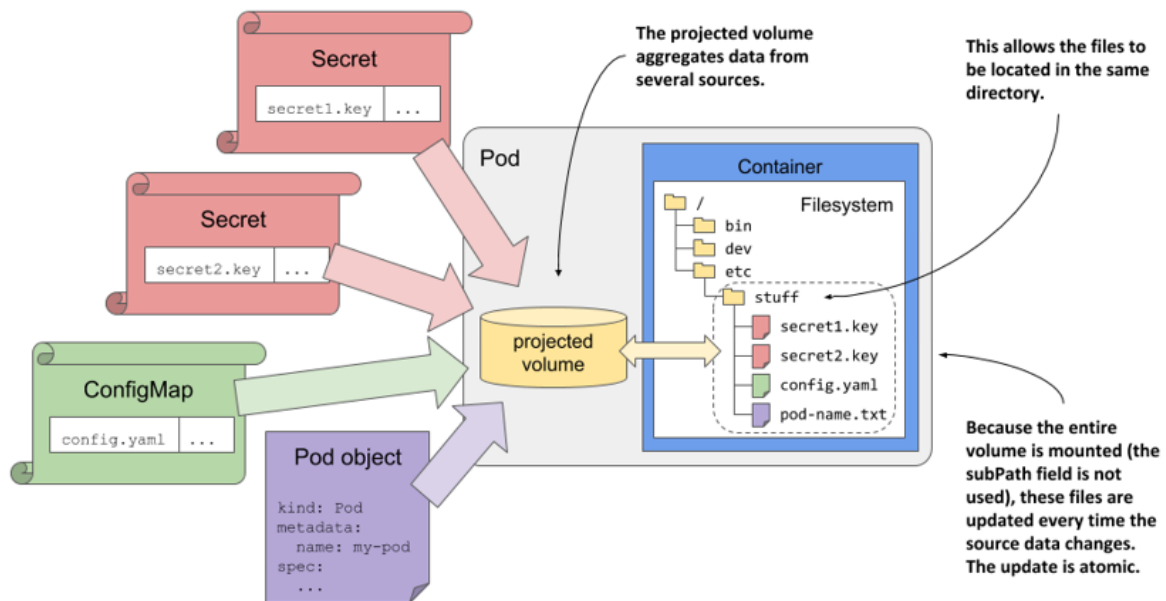
<https://kubernetes.io/docs/concepts/storage/projected-volumes/>
<https://kubernetes.io/zh-cn/docs/concepts/storage/projected-volumes/>

之前的CM,Secret等卷资源在Pod内的一个目录同时只能挂载一个卷,而我们有时希望在一个目录内生成来自多个卷的多个文件

Projected volumes 是一种特殊的卷类型,支持同时投射多个卷至同一个挂载点

Projected 不是一个独立的API 资源类型,但其可以引用现有的configmap、secret资源对象,或 downwardAPI中的元数据信息

此类的卷一般用于为容器提供预先定义好的数据



Projected Volume仅支持对如下四种类型的卷（数据源）进行投射操作

- Secret: 投射Secret对象
- ConfigMap: 投射ConfigMap对象
- DownwardAPI: 投射Pod元数据
- ServiceAccountToken: 投射ServiceAccountToken

范例: 默认每一个Pod都有Projected卷

```
[root@master1 ~]#kubectl run myapp --image registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1

[root@master1 ~]#kubectl get pod myapp -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2022-04-17T15:38:19Z"
  labels:
    run: myapp
  name: myapp
  namespace: default
  resourceVersion: "185818"
  uid: 3059fd59-1a6e-47d0-af1b-28d015f2b3bb
spec:
  containers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
```

```
name: myapp
resources: {}
terminationMessagePath: /dev/termination-log
terminationMessagePolicy: File
volumeMounts:
- mountPath: /var/run/secrets/kubernetes.io/serviceaccount
  name: kube-api-access-cl7p6
  readOnly: true
dnsPolicy: ClusterFirst
enableServiceLinks: true
nodeName: node1.wang.org
preemptionPolicy: PreemptLowerPriority
priority: 0
restartPolicy: Always
schedulerName: default-scheduler
securityContext: {}
serviceAccount: default
serviceAccountName: default
terminationGracePeriodSeconds: 30
tolerations:
- effect: NoExecute
  key: node.kubernetes.io/not-ready
  operator: Exists
  tolerationSeconds: 300
- effect: NoExecute
  key: node.kubernetes.io/unreachable
  operator: Exists
  tolerationSeconds: 300
volumes:
- name: kube-api-access-cl7p6
  projected:
    defaultMode: 420
    sources:
    - serviceAccountToken:
        expirationSeconds: 3607
        path: token
    - configMap:
        items:
        - key: ca.crt
          path: ca.crt
        name: kube-root-ca.crt
    - downwardAPI:
        items:
        - fieldRef:
            apiVersion: v1
            fieldPath: metadata.namespace
          path: namespace
status:
  conditions:
  - lastProbeTime: null
    lastTransitionTime: "2022-04-17T15:38:19Z"
    status: "True"
    type: Initialized
  - lastProbeTime: null
    lastTransitionTime: "2022-04-17T15:38:19Z"
    message: 'containers with unready status: [myapp]'
    reason: ContainersNotReady
    status: "False"
```

```

    type: Ready
  - lastProbeTime: null
    lastTransitionTime: "2022-04-17T15:38:19Z"
    message: 'containers with unready status: [myapp]'
    reason: ContainersNotReady
    status: "False"
    type: ContainersReady
  - lastProbeTime: null
    lastTransitionTime: "2022-04-17T15:38:19Z"
    status: "True"
    type: PodScheduled
containerStatuses:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imageID: ""
    lastState: {}
    name: myapp
    ready: false
    restartCount: 0
    started: false
    state:
      waiting:
        reason: ContainerCreating
hostIP: 10.0.0.201
phase: Pending
qosClass: BestEffort
startTime: "2022-04-17T15:38:19Z"

```

范例：Example configuration with a secret, a downwardAPI, and a configMap

<https://kubernetes.io/docs/concepts/storage/projected-volumes/#example-configuration-secret-downwardapi-configmap>

```
[root@master1 ~]#cat storage-projected-demo.yaml
```

```

apiVersion: v1
data:
  username: d2FuZ3hpYW9jaHVu
kind: Secret
metadata:
  name: mysecret
  namespace: default
type: Opaque

```

```

apiVersion: v1
data:
  myconfig: Hello,Myconfig
kind: ConfigMap
metadata:
  name: myconfigmap
  namespace: default

```

```

apiVersion: v1
kind: Pod
metadata:
  name: projected-volume-demo
spec:
  containers:

```

```

- name: container-test
  image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
  volumeMounts:
  - name: all-in-one
    mountPath: "/projected-volume"
    readOnly: true
volumes:
- name: all-in-one
  projected:
    sources:
    - secret:
        name: mysecret
        items:
        - key: username
          path: my-group/my-username
    - downwardAPI:
        items:
        - path: "labels"
          fieldRef:
            fieldPath: metadata.labels
        - path: "cpu_limit"
          resourceFieldRef:
            containerName: container-test
            resource: limits.cpu
    - configMap:
        name: myconfigmap
        items:
        - key: myconfig
          path: my-group/my-config

```

```

[root@master1 ~]#kubectl apply -f storage-projected-demo.yaml
secret/mysecret created
configmap/myconfigmap created
pod/projected-volume-demo created

```

```

[root@master1 ~]#kubectl exec -it projected-volume-demo -- sh
[root@projected-volume-demo /]# ls -l /projected-volume/
total 0
lrwxrwxrwx    1 root    root          16 Apr 17 16:07 cpu_limit ->
..data/cpu_limit
lrwxrwxrwx    1 root    root          13 Apr 17 16:07 labels -> ..data/labels
lrwxrwxrwx    1 root    root          15 Apr 17 16:07 my-group -> ..data/my-
group
[root@projected-volume-demo /]# cat /projected-volume/cpu_limit
2
[root@projected-volume-demo /]# cat /projected-volume/labels
[root@projected-volume-demo /]# ls /projected-volume/my-group/
my-config    my-username
[root@projected-volume-demo /]# cat /projected-volume/my-group/my-config
Hello,Myconfig
[root@projected-volume-demo /]# cat /projected-volume/my-group/my-username
wangxiaochun

```

范例：Example configuration: secrets with a non-default permission mode set

<https://kubernetes.io/docs/concepts/storage/projected-volumes/#example-configuration-secrets-nondefault-permission-mode>

```

apiVersion: v1
kind: Pod
metadata:
  name: volume-test
spec:
  containers:
  - name: container-test
    image: busybox:1.28
    volumeMounts:
    - name: all-in-one
      mountPath: "/projected-volume"
      readOnly: true
  volumes:
  - name: all-in-one
    projected:
      sources:
      - secret:
          name: mysecret
          items:
          - key: username
            path: my-group/my-username
      - secret:
          name: mysecret2
          items:
          - key: password
            path: my-group/my-password
            mode: 511

```

范例: serviceAccountToken projected volumes

[#https://kubernetes.io/docs/concepts/storage/projected-volumes/#serviceaccounttoken](https://kubernetes.io/docs/concepts/storage/projected-volumes/#serviceaccounttoken)

```

apiVersion: v1
kind: Pod
metadata:
  name: sa-token-test
spec:
  containers:
  - name: container-test
    image: busybox:1.28
    command: ["sleep", "3600"]
    volumeMounts:
    - name: token-vol
      mountPath: "/service-account"
      readOnly: true
  serviceAccountName: default
  volumes:
  - name: token-vol
    projected:
      sources:
      - serviceAccountToken:
          audience: api
          expirationSeconds: 3600
          path: token

```

范例:

```
#https://livebook.manning.com/book/kubernetes-in-action-second-edition/chapter-9/v-13/371
```

```
apiVersion: v1
kind: Pod
metadata:
  name: kiada-ssl
spec:
  volumes:
  - name: etc-envoy
    projected:
      sources:
      - configMap:
          name: kiada-envoy-config
      - secret:
          name: kiada-tls
          items:
          - key: tls.crt
            path: example-com.crt
          - key: tls.key
            path: example-com.key
            mode: 0600
  containers:
  - name: kiada
    image: luksa/kiada:1.2
    env:
    ...
  - name: envoy
    image: envoyproxy/envoy:v1.14.1
    volumeMounts:
    - name: etc-envoy
      mountPath: /etc/envoy
      readOnly: true
    ports:
    ...
```

3 综合案例: 使用持久卷和CM等部署 WordPress 和 MySQL

在kubernetes部署 wordpress, 要满足以下要求:

基本需求

- 独立部署两个wordpress Pod实例, 它们使用NFS StorageClass 存储卷存储用户上传的图片或文件等数据; 以ConfigMap和Secret提供必要的配置
- 部署一个MySQL数据库; 以ConfigMap和Secret提供必要的配置
- service 暴露wordpress 的服务

扩展需求

- 部署一个独立的ingress或者nginx Pod实例, 为wordpress提供反向代理
- 同时提供https和http虚拟主机, 其中发往http的请求都重定向给https; 以ConfigMap和Secret提供必要的配置

- 动态的蓝绿发布和滚动发布
 - 对于wordpress 来说，没有本质的区别
 - nginx的更新，依赖configmap和secret的内容

示例

```
https://kubernetes.io/zh-cn/docs/tutorials/stateful-application/mysql-wordpress-persistent-volume/
```

3.1 MySQL的相关资源

```
#提前准备名为sc-nfs的storage Class
[root@master1 ~]#cat > storage-wordpress-mysql.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysql-pass
type: kubernetes.io/basic-auth
#type: Opaque #也可以用Opaque类型
data:
  password: MTIzNDU2 #key名称:password,value为123456

---
apiVersion: v1
kind: Service
metadata:
  name: wordpress-mysql
  labels:
    app: wordpress
spec:
  ports:
    - port: 3306
  selector:
    app: wordpress
    tier: mysql
  clusterIP: None

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pv-claim
  labels:
    app: wordpress
spec:
  storageClassName: sc-nfs #需要和前面创建的storageClass名称相同
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-mysql
  labels:
```

```

    app: wordpress
spec:
  selector:
    matchLabels:
      app: wordpress
      tier: mysql
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: wordpress
        tier: mysql
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
          name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-pass
                  key: password
            - name: MYSQL_DATABASE
              value: wordpress
            - name: MYSQL_USER
              value: wordpress
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-pass
                  key: password
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pv-claim

```

3.2 wordpress 相关资源

```

[root@master1 ~]#cat > storage-wordpress-wordpress.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  ports:
    - port: 80
  selector:
    app: wordpress

```

```

    tier: frontend
    type: LoadBalancer
    #type: ClusterIP #如何部署了ingress可以使用此项
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wp-pv-claim
  labels:
    app: wordpress
spec:
  storageClassName: sc-nfs #需要和前面创建的storageClass名称相同
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
      tier: frontend
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: wordpress
        tier: frontend
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
          name: wordpress
          env:
            - name: WORDPRESS_DB_HOST
              value: wordpress-mysql
            - name: WORDPRESS_DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-pass
                  key: password
            - name: WORDPRESS_DB_USER
              value: wordpress
          ports:
            - containerPort: 80
              name: wordpress
          volumeMounts:
            - name: wordpress-persistent-storage
              mountPath: /var/www/html

```

```
volumes:
- name: wordpress-persistent-storage
  persistentVolumeClaim:
    claimName: wp-pv-claim
```

3.3 创建 Ingress

```
[root@master1 ~]#cat ingress-wordpress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-test
  namespace: wordpress
  #annotations:
  #  kubernetes.io/ingress.class: "nginx" ,k8s-v1.32.0不支持
spec:
  ingressClassName: nginx #新版建议使用此项指定controller类型
  rules:
  - host: www.wang.org
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: wordpress
            port:
              number: 80

[root@master1 wordpress-mysql]#kubectl apply -f ingress-wordpress.yaml

[root@master1 wordpress-mysql]#kubectl get ingress -n wordpress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ingress-test	nginx	www.wang.org	10.0.0.10,10.0.0.99	80	25s

3.4 应用并测试

```
#创建独立的名称空间
[root@master1 ~]#kubectl create ns wordpress

#应用
[root@master1 ~]#kubectl apply -f storage-wordpress-mysql.yaml -f storage-
wordpress-wordpress.yaml -n wordpress
secret/mysql-pass created
service/wordpress-mysql created
persistentvolumeclaim/mysql-pv-claim created
deployment.apps/wordpress-mysql created
service/wordpress created
persistentvolumeclaim/wp-pv-claim created
deployment.apps/wordpress created

#可以正常访问
[root@master1 ~]#kubectl get all -n wordpress
```

NAME	READY	STATUS	RESTARTS	AGE
pod/wordpress-9c8c6d676-xghzw	1/1	Running	0	95s

```
pod/wordpress-mysql-6d86b54f56-4gbjt 1/1 Running 0 2m35s
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
service/wordpress	LoadBalancer	10.99.25.42	10.0.0.10	
80:32734/TCP 8m38s				
service/wordpress-mysql	ClusterIP	None	<none>	3306/TCP
8m38s				

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/wordpress	1/1	1	1	8m38s
deployment.apps/wordpress-mysql	1/1	1	1	8m38s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/wordpress-9c8c6d676	1	1	1	8m38s
replicaset.apps/wordpress-mysql-6d86b54f56	1	1	1	8m38s

#通过SVC直接访问

10.0.0.10

[root@master1 ~]#kubectl get pod -n wordpress

NAME	READY	STATUS	RESTARTS	AGE
wordpress-9c8c6d676-xghzw	1/1	Running	0	68s
wordpress-mysql-6d86b54f56-4gbjt	1/1	Running	0	2m8s

#删除Pod, 自动创建新的Pod, 仍可以继续访问

[root@master1 ~]#kubectl delete pod --all -n wordpress

pod "wordpress-9c8c6d676-xghzw" deleted

pod "wordpress-mysql-6d86b54f56-4gbjt" deleted

[root@master1 ~]#kubectl get pod -n wordpress

NAME	READY	STATUS	RESTARTS	AGE
wordpress-9c8c6d676-5cqrk	1/1	Running	0	33s
wordpress-mysql-6d86b54f56-s8gw8	1/1	Running	0	33s

#清理环境

[root@master1 ~]#kubectl delete ns wordpress

namespace "wordpress" deleted

